

All That AI · Vol.03

AI 휴먼 해부학

얼굴·목소리·두뇌·기억 — 네 층을 조립하고 실측하는 법
사진 한 장에서 실시간 대화 아바타까지,
픽셀이 사람이 되는 여정

이석창 (Seokchang Lee) 지음

서문 — 얼굴을 만든다는 것

1. 하늘이가 말을 했습니다

사진은 한 장뿐이었습니다. 세상을 떠난 반려동물이 정면을 보고 입을 다문, 특별할 것 없는 사진. 그 사진이 한국어로 말하기까지 나흘이 걸렸습니다. 나흘 중 결과가 나온 것은 마지막 4분이고, 나머지는 전부 실패였습니다.

실패는 이런 모양이었습니다. 동물의 얼굴에 사람의 입술과 이가 그려졌습니다. 프레임을 음량에 맞춰 재배열했더니 머리가 좌우로 90도씩 흔들렸습니다. 시간을 워핑했더니 영상이 끊겼습니다. 스티칭을 컷더니 왼쪽 귀가 잘렸습니다. 그리고 마침내 입이 제대로 움직였을 때는 — 뒤로 갈수록 소리보다 입이 늦었습니다.

마지막 문제의 원인은 모델이 아니었습니다. LivePortrait가 29fps로 영상을 뱉는데 드라이버가 29.97fps였습니다. 3.3%의 차이. 10초짜리 영상의 끝에서 3분의 1초가 밀립니다. 사람은 그 3분의 1초를 정확히 알아챱니다.

이 책은 그 나흘에서 시작했습니다.

2. AI 휴먼은 하나의 모델이 아닙니다

강의에서 가장 자주 받는 질문은 이것입니다. *"AI 휴먼 만들려면 어떤 모델을 써야 하나요?"*

질문 자체가 틀렸습니다. AI 휴먼은 하나의 모델이 아니라 **네 층의 조립** 이기 때문입니다.

얼굴 이 있습니다. 화면에 보이는 것, 입이 움직이고 눈을 깜빡이는 층입니다. **목소리** 가 있습니다. 텍스트를 소리로 바꾸는 층이고, 감정의 80%가 여기서 결정됩니다. **머리** 가 있습니다. 무엇을 말할지 정하는 층입니다. 그리고 **기억** 이 있습니다. 어제 무슨 얘기를 했는지, 이 캐릭터만 아는 것이 무엇인지를 쥐고 있는 층입니다.

네 층은 서로 독립입니다. 얼굴을 2D 파츠에서 3D VRM으로 바꿔도 목소리는 그대로입니다. LLM을 Gemini에서 로컬 모델로 바꿔도 얼굴은 모릅니다. 이 독립성이 좋은 소식입니다. **한 번에 하나씩 올릴 수 있다** 는 뜻이기 때문입니다.

그래서 이 책의 1장 결과물은 GPU 0장, 모델 다운로드 0GB로 나옵니다. 거대 언어모델은 받지 않습니다. 그런데도 화면 속의 무언가가 여러분에게 말을 겁니다.

3. 4분과 2초 사이

10.67초짜리 영상 하나를 만드는 데 230초가 걸립니다. RTX 4070 SUPER 한 장을 다 쓰고도 그렇습니다. 실시간의 21배입니다.

같은 날 오후, 브라우저에서 도는 3D 아바타는 말을 걸면 **2초 안에** 대답합니다. GPU는 쓰지 않습니다.

이 두 숫자 사이의 거리가 이 책의 무게중심입니다.

흔한 계획은 이렇습니다 — *"일단 품질 좋게 만들고, 나중에 최적화해서 실시간으로 만들면 되지."* 그런 일은 일어나지 않습니다. 21배는 최적화로 메울 수 있는 거리가 아닙니다. **목표 지연이 아키텍처를 결정합니다.** 나중에 바꾸는 것이 아니라 처음에 고르는 것입니다.

그래서 이 책은 트랙 셋을 병렬로 놓았습니다. 어느 하나가 다른 하나의 상위 호환이 아니기 때문입니다.

4. 실패의 목록이 곧 커리큘럼입니다

한글 파일명을 쓰면 OpenCV가 Windows에서 파일을 못 읽습니다. 학생들은 이런 데서 막힙니다. 가르치면서 배운 것이 그것입니다 — 막히는 곳은 논문이 설명하는 곳이 아닙니다.

막히는 자리는 더 있습니다. 드라이버 영상이 미디엄샷이면 입 모션이 약하게 전사되고, 그 약한 모션을 사용자는 "싱크가 안 맞는다"고 느낍니다. 배율을 함부로 올리면 허가 나옵니다. 눈이 안 깜빡여서 부자연스럽다고 하면 region을 바꿔야 하는데, 그러면 싱크 선명도가 희석됩니다.

그리고 가장 무서운 실패는 — **잘못된 지표를 쫓는 것**입니다.

저는 한동안 "입 벌림 정도와 음량의 상관계수"를 립싱크 품질 지표로 썼습니다. 그럴듯합니다. 그러나 진짜 립싱크는 음소에 맞추지 음량에 맞추지 않습니다. 상관계수는 원래 0에 가깝습니다. 저는 0에 가까운 그 숫자를 올리려고 프레임을 흔들고 시간을 늘리는 꼼수를 만들었습니다. 결과물은 전부 쓰레기였습니다.

그래서 이 책은 실패를 부록 F에 50종 그대로 실었습니다. **재현 금지 목록**입니다. 그리고 각 트랙의 마지막 장에는 반드시 **검증하는 법**이 들어갑니다. 만드는 법만 있고 검증하는 법이 없는 교재는, 학생을 제 실수 위로 그대로 걸어가게 합니다.

5. 800시간의 교실에서

KDT AI 휴먼 과정은 800시간입니다. 이 책의 구성은 책상이 아니라 그 800시간에서 나왔습니다.

Python으로 시작해 전처리·머신러닝을 지나, 음성 데이터와 TTS·STT로 96시간과 64시간을 쓰고, 거대 언어모델과 프롬프트와 LangChain과 RAG를 지나, 세 개의 프로젝트로 끝납니다. 나만의 음성 모델, 답변 기능 구현, 그리고 사전 데이터 기반 답변 커스텀.

한 기수를 운영하면서 순서를 한 번 바꿨습니다. 원래 계획은 프로젝트-2를 먼저 하고 LangChain을 나중에 배우는 것이었습니다. 그러면 학생들이 도구를 모르는 채로 프로젝트를 시작합니다. 그래서 LangChain을 프로젝트-2 앞으로 당겼습니다. 총 시수도 종료일도 그대로였지만, 그 한 번의 스왑으로 프로젝트의 완성도가 달라졌습니다.

핵심 도구는 그것을 쓸 프로젝트 바로 앞에 와야 합니다. 이 책의 목차도 같은 원리로 짜였습니다. Part 1의 지연 이야기가 트랙보다 앞에 있는 이유, 페르소나가 멀티 아바타보다 앞에 있는 이유가 전부 그것입니다.

28차시 강의 자료 역시 이 책의 뼈대입니다. 4차시가 아바타 립싱크였고, 10차시가 얼굴 이미지 생성, 11차시가 음성 클로닝, 12차시가 실시간 대화의 끼어들기, 26차시가 덤페이크 윤리였습니다. 이 책은 그 자료들을 한 권의 서사로 다시 켜는 것입니다. 부록 D와 E가 그 매핑표입니다.

6. 같은 카메라, 이번엔 반대 방향

전작 Vol.01 *테슬라처럼 만드는 비전 자율주행과 퍼지컬 AI*의 입장은 "**픽셀이 행동이 된다**"였습니다. Vol.02 *우리집 AI 의사·수 의사·헬스코치*는 그것을 집으로 옮겨 "**픽셀이 진단이 된다**"로 갔습니다.

두 권 모두 카메라가 **들어오는** 방향이었습니다. 세상을 보고, 이해하고, 판단합니다.

이번 책은 반대입니다. **나가는** 방향입니다. 시스템이 이해한 것을 사람에게 돌려주는 층 — 얼굴과 목소리. 아무리 좋은 판단을 해도 그것을 전할 얼굴이 없으면 사람은 믿지 않습니다. 어머니가 스마트워치를 차지 않으시는 이유와, 사람들이 챗봇 대신 사람에게 전화하는 이유는 같습니다.

픽셀이 사람이 되는 여정입니다.

7. 이 책을 다 읽으면 갖게 되는 것

말하는 영상 파이프라인 한 벌. 사진 한 장과 대본을 넣으면 mp4가 나옵니다. 사람 얼굴도, 동물 얼굴도, 캐릭터도.

실시간 대화 아바타 두 종. 브라우저에서 도는 2D 파츠 마스코트와 3D VRM 휴먼. 둘 다 GPU 없이, 첫 소리까지 2초.

인격 한 벌. 페르소나 프롬프트, 기억 3층, 캐릭터 전용 지식 RAG, 그리고 그 일관성을 지키는 평가 하네스.

멀티 아바타 앱 한 개. 서로 다른 인격이 한 화면에서 대화하고 추론하는 구조.

통역 아바타 한 벌. 듣고, 문장 단위로 옮기고, 다른 언어의 목소리로 말합니다. 수어 인식기의 단어열을 문장으로 엮고, 반대로 문장을 손동작 태그로 내보내는 브리지까지.

배포와 책임. FastAPI·도커·원격 GPU 잡·터널로 내보내는 법, 그리고 동의·워터마킹·거절 기준.

GPU는 트랙 A와 두 곳(실사 실시간 아바타 Ch08·Ch11, Ch03A 서빙)에서만 필요하고, 나머지는 노트북 한 대로 끝납니다.

8. 이 책이 아닌 것

논문 해설서가 아닙니다. 확산모델의 수식을 유도하지 않습니다. 필요한 만큼의 원리와, 필요한 전부의 실행을 씁니다.

모델 카탈로그가 아닙니다. 최신 모델을 나열하지 않습니다. 대신 어느 상황에서 무엇을 고르는가를 씁니다. 모델은 6개월이면 바뀌지만 결정 트리는 남습니다.

그래서 모델 이름은 그것이 방법의 이름이 된 세 장(Ch10~12)의 제목에만 남기고, 후보 목록은 온라인 부록 K로 보냈습니다. 본문에는 모델 이름이 적게 나옵니다. 대신 **온라인 부록 K 한 곳에 그 시점의 지형도**를 모았고, 그 부록만 6개월마다 갱신합니다. 몇 년 뒤에 이 책을 펴는 분은 본문을 그대로 읽고, 온라인 부록 K만 저장소에서 최신판을 받으세요.

딥페이크 제작 가이드가 아닙니다. 이 책은 동意的한 얼굴과 목소리만 다룹니다. 29장은 그 경계를 다루고, 그 경계는 협상 대상이 아닙니다.

아바타 책도 아닙니다. 아바타는 얼굴 한 층입니다. 이 책은 네 층 전부를 다룹니다.

9. 감사의 말

강의실에서 첫 립싱크 영상이 나왔을 때 "선생님 이거 무섭게 잘 되는데요"라고 말해 준 학생들에게 빛을 쬔습니다. 그 반응이 이 책의 29장(윤리)을 쓰게 했습니다.

800시간 과정의 운영을 함께 조정해 주신 분들 덕분에 커리큘럼 순서에 대한 §5의 결론이 나왔습니다. 실패를 실패로 기록하도록 밀어붙여 준 저장소의 커밋 로그에도 감사합니다. 롤백할 수 있기 때문에 무모하게 시도할 수 있었습니다.

그리고 하늘이. 이 책의 첫 번째 얼굴이었습니다. 다시 목소리를 들려주고 싶었던 그 마음이, 이 기술을 어디까지 써도 되는가에 대한 이 책의 모든 문장에 남아 있습니다.

2026년 가을, 이석창

이 책의 사용법

0. 먼저 다섯을 소개합니다

이 책의 예제는 다섯입니다. 전부 저자가 실제로 만들어 돌려 본 것이고, 각각이 한 트랙의 얼굴입니다.

하늘이 는 세상을 떠난 반려동물입니다. 사진 한 장으로 말하게 만듭니다. **홈런이** 는 야구 용어를 설명하는 2D 마스코트인데, 그림 네 조각이 전부입니다. **코치** 는 브라우저에서 도는 3D 캐릭터로, 말하면서 실제로 스퀴트를 합니다. **바텐더** 는 이 책에서 유일하게 GPU를 상시 물고 있는 실사 아바타입니다. **마을 사람들** 은 서로의 비밀을 모른 채 추리하는 여럿입니다.

상세는 [00 등장인물.md](#). 지금은 이름만 기억하셔도 됩니다.

1. 먼저 자기 자리를 찾으세요

만들려는 것이 어느 트랙인지 먼저 정하면 읽을 양이 3분의 1로 줄어듭니다. 이 책은 트랙 셋이 병렬로 놓여 있고, 어느 하나가 다른 하나의 상위 버전이 아닙니다.

만들려는 것이 영상이라면 Track A 입니다. 결과물은 mp4 파일입니다. 만드는 데 몇 분이 걸려도 상관없고, 대신 최대한 사실적이어야 합니다. 추모 영상, 인터뷰 재현, 캐릭터 홍보 영상이 여기 속합니다. GPU가 필요합니다.

만들려는 것이 대화 상대라면 Track B 입니다. 사람이 말을 걸면 즉시 반응해야 하고, 완벽한 실사보다 *끊기지 않음* 이 중요합니다. 키오스크, 학습 도우미, 상담 프론트, 버추얼 스트리머가 여기 속합니다. GPU는 필요 없습니다(실사 실시간 아바타 한 종만 예외 — Ch08·Ch11).

만들려는 것이 인격이라면 Track C 입니다. 얼굴은 A든 B든 상관없고, 무엇을 말하는가와 어제 한 얘기를 기억하는가가 본질입니다. 캐릭터 챗, 게임 NPC, 도메인 상담이 여기 속합니다.

대부분의 실제 제품은 **B + C** 입니다. A는 콘텐츠 제작에 쓰입니다.

만들려는 것이 통역이라면 Ch23A 입니다. 아바타가 자기 말을 하는 것이 아니라 *남의 말을 옮기* 는 경우입니다 — 관광·민원 창구, 병원 접수, 그리고 수어 통역 보조. 새 모델을 쓰지 않고 B와 C의 부품을 다른 순서로 잇습니다. 수어 인식기와 아바타를 잇는 브리지도 그 장에 있습니다 (§ 6).

2. 어느 트랙이든 Part 1과 Part 2는 읽으세요

Part 1 (Ch01~05, Ch03A 포함)은 4층 스택과 결정 트리입니다. 여기를 건너뛰면 트랙 안에서 "왜 이걸 쓰지"라는 질문이 계속 생깁니다.

Part 2 (Ch06~09)는 지연입니다. 이 책에서 한 파트만 읽어야 한다면 Part 2 입니다. 여기에는 모델 이름이 거의 나오지 않습니다. 대신 왜 4분이 2초가 되지 않는지, 지연을 어떻게 숨기는지, 립싱크가 맞았다는 것을 어떻게 증명하는지가 있습니다. 모델은 바뀌지만 이 내용은 남습니다.

3. 준비물 — 공통은 노트북 한 대뿐입니다

공통 (모든 트랙)

노트북 한 대, Python 3.10 이상(GPU 트랙은 3.10 고정), ffmpeg, 그리고 최신 브라우저면 됩니다. Windows를 기준으로 썼고 macOS·Linux는 각 장의 각주에 차이를 적었습니다.

LLM은 API 키 하나로 시작합니다. 로컬 모델은 Ch22에서 선택지로 다룹니다. TTS도 무료 무제한 옵션(edge-tts)으로 시작하세요. 품질이 필요해지면 그때 Ch03의 결정표를 보고 알아합니다.

Track A 추가

NVIDIA GPU 한 장이 필요합니다. 본문의 실측은 RTX 4070 SUPER 12GB 기준이고, 8GB에서도 배치 크기를 줄이면 됩니다. conda 환경은 두 개 만듭니다 — LivePortrait와 MuseTalk는 의존성이 충돌하므로 반드시 분리 합니다. 상세는 부록 A.

동물 모드를 쓰려면 X-Pose 커스텀 연산자를 빌드해야 합니다. Windows에서는 MSVC와 CUDA 11.8이 버전 충돌을 일으켜 우회가 필요합니다. 그 스크립트를 부록 A에 실었습니다.

Track B 추가

없습니다. 브라우저와 Python이면 끝입니다. VRM 모델은 무료로 배포되는 것을 쓰거나 직접 만듭니다(Ch18).

Track C 추가

임베딩 모델과 벡터 저장소가 Ch25에서 필요합니다. 한국어는 bge-m3 계열을 기본으로 씁니다.

4. 학습 경로 — 48시간부터 800시간까지

6주 미니코스 — 하나만 골라 끝내기

Part 1 (1주) → Part 2 (1주) → 트랙 1개 (3주) → Part 4 (1주). 한 학기 교양 한 과목 분량이고, 결과물은 완성된 데모 하나입니다.

14주 풀코스 — 셋 다

Part 1 (1주) → Part 2 (2주) → Track A (3주) → Track B (3주) → Track C (3주) → Part 4 (1주) → 발표 (1주).

해커톤 48시간 코스

Ch03(TTS 고르기) + Ch18~21(VRM 실시간 아바타) + Ch22(페르소나). 이틀 안에 말이 통하는 아바타가 나옵니다. GPU를 기다리지 않는 것이 핵심입니다.

졸업작품 12주 코스

Part 1 + Part 2 + Track B + Track C + Ch28(배포) + Ch29(윤리). 심사에서 윤리 항목이 배점에 있는 경우가 늘고 있습니다.

KDT 800시간 과정

부록 D의 매핑표를 보세요. 정규교과 4·5(음성)가 Ch03·Ch23에, 정규교과 6·7·8(LLM·프롬프트·LangChain)이 Track C에, 정규교과 9(RAG)가 Ch25에 붙습니다. 프로젝트 1·2·3은 각각 Ch03·Ch23, Ch22~24, Ch25~27을 결과물로 씁니다.

5. 코드는 어디에 있나

저장소는 여기입니다.

```
git clone https://github.com/leelang7/talking-ai-human-book.git
cd talking-ai-human-book
pip install -r requirements.txt
python scripts/gates.py          # 회귀 테스트 전부 - GPU 없이 돛니다
```

종료 코드 0이면 전부 통과입니다. 책 본문은 **저장소에 없습니다** — 코드와, 그 코드가 만든 실측 근거만 있습니다.

무거운 모델(MuseTalk · LivePortrait · Wav2Lip)을 부르는 장은 저장소·환경 경로가 기계마다 다릅니다. 그 경로는 코드에 박혀 있지 않습니다. `book.config.example.json` 을 복사해 여러분의 경로를 적으세요. 못 찾으면 스크립트가 조용히 넘어가지 않고 **무엇을 설정해야 하는지 말하고 멈춥니다**(Ch02 §3의 원칙 그대로입니다).

각 장은 `code/chXX_*/` 한 폴더에 대응합니다. 폴더 안에는 실행 가능한 스크립트와 README가 있고, README에는 그 장에서 재현해야 할 결과와 **예상 소요 시간** 이 적혀 있습니다.

예상 소요 시간을 반드시 확인하세요. Track A의 어떤 장은 한 번 실행에 4분이 걸립니다. 그것이 정상입니다.

6. 모델 이름을 찾는다면 — 온라인 부록 K

본문은 어느 상황에서 무엇을 고르는가 만 씁니다. 구체적인 모델 이름과 그 시점의 후보 목록은 온라인 부록 K(모델 지형도) 한 곳에 모여 있습니다.

온라인 부록 K에는 기준일이 적혀 있고, 6개월마다 갱신됩니다. 책을 산 시점과 부록의 기준일이 많이 벌어졌다면 저장소에서 최신판을 받으세요. 본문의 결정 트리는 그대로 쓰시면 됩니다 — 바뀌는 것은 각 칸에 들어가는 이름이지 칸 자체가 아닙니다.

7. 실패하면 — 부록 F를 먼저 여세요

먼저 부록 F 실패 카탈로그 를 보세요. 저자가 실제로 밟은 50종이 증상별로 정리되어 있습니다. 입술이 이상하게 그려진다, 머리가 흔들린다, 뒤로 갈수록 싱크가 밀린다, 귀가 잘린다, 눈을 안 깜빡인다 — 대부분 여기 있습니다.

Windows 특유의 문제는 부록 H 입니다. 한글 파일명, cp949 인코딩, fps 정수/실수 타입, PowerShell 파싱 오류. 이 넷이 전체 환경 문제의 대부분입니다.

그래도 안 되면 저장소 Issues에 남겨 주세요. 정오표에 누적됩니다.

8. 먼저 알아 둘 말 — 열여덟

본문에서 설명이 나오기 전에 먼저 마주치는 낱말들입니다. 여기서 한 줄로 보고, 자세한 것은 표시된 장에서 보세요.

표 먼저 알아 둘 말 열여덟

말	한 줄 뜻	어디서
드라이버(영상)	표정·입 모양을 빌려 오는 영상. 결과물에는 안 나오고 움직임만 옮겨진다	Ch04 · 부록 B
소스(이미지)	말하게 만들 대상 얼굴. 사진 한 장	Ch04
립싱크	음성에 맞춰 입 모양을 다시 그리는 일	Ch10~11
리타게팅	한 얼굴의 움직임을 다른 얼굴로 옮기는 일	Ch12
파트(2D)	그림 한 장을 몸통·눈·입 조각으로 쪼갠 것. 조각을 움직여 표정을 만든다	Ch16
리깅	뼈대나 조각에 움직일 손잡이를 다는 일	Ch16 · Ch18
VRM	브라우저에서 도는 3D 사람 모델 규격. 뼈 이름이 정해져 있다	Ch18
블렌드셰이프	얼굴을 통째로 다시 그리지 않고 섞어서 표정을 만드는 값	Ch18
TTS · STT	글자→소리 · 소리→글자	Ch03 · Ch23

말	한 줄 뜻	어디서
TTFA (첫 소리까지)	사용자가 말을 끝낸 순간부터 아바타의 첫 소리가 나기까지	Ch07
VAD (발화 종료 판정)	사용자가 말을 끝냈는지 소리로 판단하는 일	Ch23
끼어들기(barge-in)	아바타가 말하는 중에 사람이 말을 걸면 아바타가 멈추는 것	Ch23

세 트랙을 부르는 이름도 미리 적어 둡니다. **Track A** 는 사진 한 장을 오래 걸려 잘 만드는 길(분 단위), **Track B** 는 GPU 없이 브라우저에서 바로 움직이는 길(초 단위), **Track C** 는 그 얼굴에 인격과 기억을 넣는 길입니다.

접침 경로 — 언어모델의 첫 문장이 나오는 즉시 TTS로 넘기고, TTS의 첫 조각이 나오는 즉시 재생하는 경로. 단계를 순차로 기다리지 않아 첫 소리가 두 배 빨라집니다(Ch07).

아이들 루프 — 아무도 말을 걸지 않을 때 얼굴이 도는 대기 영상. 스피너 대신 얼굴이 살아 있게 하는 장치(Ch08).

폴든셋 — 정답과 함께 고정해 둔 질문 묶음. 고칠 때마다 다시 채점해 회귀를 잡습니다(Ch27).

게이트 — 사람이 기억해 지키는 규칙이 아니라, 통과하지 못하면 다음 단계가 돌지 않는 자동 검사(Ch09·Ch27·Ch28).

p95 — 100번 중 95번이 이보다 빠른 값. 평균은 느린 5%를 숨깁니다(Ch07·Ch28A).

체크 / 조각 — 스트리밍에서 문장을 자른 단위는 체크, 검색용으로 문서를 자른 단위는 조각으로 부릅니다(Ch07·Ch25).

9. 마지막 조언 — 결말부터 읽으세요

이 책의 각 트랙 마지막 장(Ch15, Ch21, Ch27)은 **완성** 장입니다. 앞의 다섯~여섯 장을 하나로 합칩니다.

먼저 그 완성 장을 읽고 오세요. 목적지를 알고 나면 중간 장들의 선택 하나하나가 왜 필요했는지 보입니다. 특히 Ch09(싱크 검증)와 Ch27(평가 하네스)은 먼저 읽으면 나머지를 만드는 태도 자체가 달라집니다.

등장인물 — 이 책에서 실제로 만드는 다섯

이 책에는 예제가 다섯 있습니다. 전부 저자가 실제로 만들어 돌려 본 것이고, 각각이 한 트랙의 얼굴입니다. 33장을 지나는 동안 이 다섯이 계속 다시 나옵니다.

하늘이 — 사진 한 장이 말한다



그림 왼쪽이 소스 이미지(입을 다문 정면), 오른쪽이 그 사진이 한국어로 말하는 순간. 오른쪽은 얼굴을 잘라 리타게팅한 결과라 화각이 다르지만 같은 사진이다. 소스는 LivePortrait 예제 이미지(오픈소스), 드라이버는 스톡 영상

Track A · Ch13·Ch15

세상을 떠난 반려동물입니다. 남은 것은 정면을 보고 입을 다문 사진 한 장.

이 책에 실린 사진에 대하여 — Track A를 시작하게 만든 것은 저자의 반려동물이지만, 이 책의 예제와 모든 실측은 공개된 **예제 이미지(오픈소스 립싱크·리타게팅 저장소에 들어 있는 고양이 사진)** 로 다시 만들었습니다. 드라이버 영상도 라이선스가 확인된 스톡 영상입니다. 여러분이 같은 파일로 같은 결과를 재현할 수 있어야 하고, 남의 얼굴·남의 반려동물 사진을 예제로 돌리는 습관을 이 책이 먼저 들이면 안 되기 때문입니다(Ch29 · 부록 G).

이 얼굴이 이 책에서 가장 어려운 조건을 전부 갖고 있습니다. **사람이 아니고, 사진이 한 장뿐이고, 다시 찍을 수 없고, 결과를 보는 사람에게 감정적으로 중요합니다.** 여기서 통하는 방법은 다른 데서는 다 통합니다.

하늘이를 말하게 하려면 사람의 입을 빌려야 합니다. 왜 그런지가 Ch13이고, 어떻게 하는지가 Ch15입니다.

민트 — 조건을 재는 고양이

저자의 고양이입니다. 이 책의 실측 소스 이미지 대부분은 공개 예제 이미지(하늘이 역의 §39)지만, 소스 사진의 조건을 재는 실험(Ch04 §2·§3)은 민트의 스마트폰 스냅으로 했습니다 — 화면 한가운데 작게 찍힌 사진을 잘라 소스로 만들고, 드라이버 셋을 갈아 끼우며 입이 얼마나 열리는지를 켜 그 고양이입니다. Ch13 §5에는 입을 벌린 채 찍힌 공개 예제의 **줄무늬 고양이**가 한번 더 나옵니다 — 벌린 입은 다물어지지 않는다는 조건을 보여 주는 역할입니다.

홈런이 — 그림 한 장이 움직인다



그림 홈런이 — 그림 한 장을 몸통·원눈·오른눈·입 네 조각으로 쪼갬다(오른눈은 원눈과 같아 그림에서 생략). 몸통은 눈·입을 잘라낸 자리를 주변 색으로 메운 것이라 표정이 지워진 채로 보인다. 원본 그림은 저자의 수업에서 지도 학생이 생성

Track B 1단 · Ch16~17

야구 용어를 설명하는 2D 마스크입니다. 그림 파일 하나를 몸통 · 원눈 · 오른눈 · 입 네 조각으로 쪼갬 것이 전부입니다.

신경망은 한 줄도 쓰지 않습니다. 눈을 감기는 것은 눈 이미지를 중앙 기준으로 눌러 찌그러뜨리는 일이고, 입은 재생 중인 오디오의 음량이 엽니다. GPU 0장, 모델 다운로드 0GB.

홈런이에게는 비밀 무기가 하나 있습니다. **언어모델이 죽어도 볼넷과 버트를 설명할 수 있습니다.** 왜 그런 장치가 필요한지는 Ch07에서 이야기합니다.

코치 — 3D가 손을 흔든다

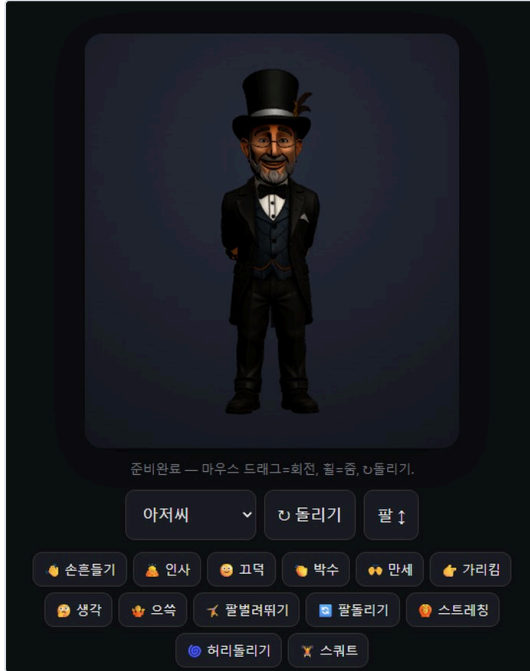


그림 코치 — 저자의 브라우저 뷰어 화면. 3D 캐릭터(프리셋 이름은 '아저씨') 아래에 동작 버튼 열셋이 있고, 팔벌려뛰기·스쿼트·스트레칭이 그중 셋이다

Track B 2단 · Ch18~27

브라우저에서 도는 VRM 3D 캐릭터입니다. 말을 걸면 **첫 소리까지 2초**, 그 사이 눈을 깜빡이고 숨을 쉬고 이쪽을 봅니다.

그림의 동작 버튼이 이 캐릭터의 어휘입니다. 손흔들기·인사·고덕임 같은 대화 동작과 **팔벌려뛰기·팔돌리기·스트레칭·허리돌리기·스쿼트** 같은 운동 동작이 섞여 있고, 전부 코드가 만듭니다(Ch20). 말에서 동작을 고르는 **텍스트투모션** 이 이 얼굴에서 돕니다 — "같이 팔벌려뛰기"라고 하면 실제로 뛸니다(Ch20).

코치는 운동을 시킵니다. "같이 스트레칭할까요"라고 말하면서 **실제로 팔을 위로 뻗습니다**. 팔벌려뛰기·팔돌리기·허리돌리기·스쿼트 — 위 화면의 아래 두 줄입니다. 말과 몸이 따로 놀지 않게 만드는 방법이 Ch20입니다.

역시 GPU를 쓰지 않습니다. 사용자가 백 명이면 백 대의 브라우저가 각자 렌더합니다.

바텐더 — 실사가 실시간으로



그림 바텐더 — 3D 렌더로 만든 얼굴 한 장. 실사 트랙의 신경망 립싱크가 이 얼굴 위에서 실시간으로 돈다. 이미지는 저자의 수업에서 지도 학생이 생성

Ch08·Ch11

얼굴 이미지 한 장 위에서 실사 립싱크 모델이 도는 실시간 대화 아바타입니다. 이 책에서 유일하게 GPU를 상시 물고 있는 얼굴입니다.

바텐더의 진짜 기술은 말할 때가 아니라 **말이 없을 때** 있습니다. 대기 중에도 눈을 깜빡이고 미세하게 움직입니다. 로딩 스피너가 없습니다. 그 아이들 루프를 어떻게 만드느냐가 Ch08이고, 왜 그것이 스피너보다 중요한지도 Ch08입니다.

바텐더 한 명이 GPU 한 장을 씁니다. 이 문장이 원가 구조의 전부입니다.

마을 사람들 — 여럿이 서로의 비밀을 모른다



그림 마을 사람들 — 서로 다른 인격을 받는 네 얼굴. 이미지는 저자의 수업에서 지도 학생이 생성

Track C · Ch26

마피아 게임, 살인 미스터리, 방탈출 — 서로 다른 인격의 아바타 여럿이 한 화면에서 대화하고 추론합니다.

여기서 지켜야 할 단 하나의 규칙이 있습니다. **시민은 마피아가 누구인지 몰라야 합니다.** 각 캐릭터에게 무엇을 보여줄지는 진행자가 분배합니다. 그 진행자는 언어모델이 아니라 코드입니다. 왜 코드여야 하는지가 Ch26입니다.

이 다섯 중 유일하게 얼굴이 본질이 아닌 예제입니다. 여기서 중요한 것은 **무엇을 아는가**입니다.

다섯을 한 줄로

표 다섯 얼굴 — 트랙 · 만드는 것 · GPU

이름	트랙	만드는 것	GPU
하늘이	A	사진 1장 + 대본 → 말하는 영상	필요
홈런이	B 1단	그림 4조각 → 움직이는 마스코트	불필요
코치	B 2단	VRM → 2초 응답 + 제스처	불필요
바텐더	심화	실사 + 실시간	상시 점유
마을 사람들	C	인격 여럿 + 비밀 격리	불필요

하늘이는 **품질** 을, 홈런이와 코치는 **지연** 을, 바텐더는 **원가** 를, 마을 사람들은 **일관성** 을 가르칩니다. 이 네 가지가 이 책의 전부입니다.

차례

Part 1. AI 휴먼의 해부

Ch1 왜 지금 AI 휴먼인가	21
1.1 질문이 틀렸습니다 · 1.2 거대 모델은 필요 없습니다 · 1.3 세 개의 시장, 세 개의 트랙 · 1.4 이 책의 다섯 가지 입 장 · 1.5 왜 하필 지금인가 · 1.6 이 장에서 기억할 것	
Ch2 네 층을 세우는 30분	27
2.1 길림길이 하나 있습니다 · 2.2 무GPU 트랙 — 30분 · 2.3 GPU 트랙 — 반나절 · 2.4 네 층을 각각 세웁니다 · 2.5 첫 번째 결과물 — LLM 없이 말을 시킵니다 · 2.6 이 장에서 기억할 것	
Ch3 목소리부터 만든다	31
3.1 감정의 80%는 목소리입니다 · 3.2 TTS를 고르는 네 가지 축 · 3.3 네 가지 선택지 — 무엇을 언제 쓰나 · 3.4 대 본 쓰는 법 — 엔진보다 먼저 고칠 것 · 3.5 감정을 넣는 두 개의 경로 · 3.6 검증 — 세 가지만 확인하세요 · 3.7 이 장에서 기억할 것	
Ch3A 사례 — 사투리 다섯 개를 만든 이야기	42
3A.1 무엇을 만들었나 · 3A.2 이 사례가 증명하는 것 셋 · 3A.3 어댑터 핫스왑 — 이 사례의 핵심 ★ · 3A.4 학습 절 차 — 먼저 실패를 확인한다 · 3A.5 감정 노브 프리셋 · 3A.6 서비스로 내보낼 때 · 3A.7 이 장에서 기억할 것	
Ch4 얼굴의 조건	51
4.1 얼굴은 돌입니다 — 하나는 보이고 하나는 안 보입니다 · 4.2 보이는 얼굴 — 좋은 소스 이미지의 조건 · 4.3 안 보이는 얼굴 — 좋은 드라이버 영상의 조건 · 4.4 얼굴이 없으면 만듭니다 · 4.5 이 장에서 기억할 것	
Ch5 어느 사다리를 오를 것인가	62
5.1 사다리는 다섯 — 그리고 이 책 밖의 둘 · 5.2 세 질문으로 고릅니다 · 5.3 결정표 — 다섯 칸을 나란히 놓고 · 5.4 혼한 시나리오별 답 · 5.5 사다리를 섞을 수도 있습니다 · 5.6 이 장에서 기억할 것	

Part 2. 지연의 기초

Ch6 왜 4분이 걸리는가	69
6.1 숫자부터 봅시다 · 6.2 시간이 어디로 가는가 · 6.3 왜 립싱크가 그렇게 느린가 · 6.4 그래서 무엇을 할 수 있나 · 6.5 21배는 왜 매울 수 없는가 · 6.6 그렇다면 품질 트랙은 어떻게 쓰나 · 6.7 이 장에서 기억할 것	
Ch7 지연 예산 2초	74
7.1 사용자가 재는 시간은 하나뿐입니다 · 7.2 예산을 쪼갭니다 · 7.3 가장 큰 절약 — 문장 단위 청킹 · 7.4 두 번째 절약 — 짧게 대답하게 만들기 · 7.5 세 번째 절약 — 추론 예산 끄기 · 7.6 네 번째 절약 — 모델을 작게, 풀백을 길 게 · 7.7 다섯 번째 절약 — 모델을 상시 로드 · 7.8 측정하지 않으면 개선되지 않습니다	
Ch8 지연을 숨기는 법	85
8.1 스피너는 패배입니다 · 8.2 아이들 루프 — 살아 있는 대기 상태 · 8.3 왜 무음을 넣어야 하나 · 8.4 크로스페이드 — 전환을 감추기 · 8.5 터블 버퍼링 — 깜빡임을 없애기 · 8.6 선점 — 기다림을 앞당기기 · 8.7 실패도 자연스럽게 · 8.8 이 장에서 기억할 것	
Ch9 싱크를 어떻게 증명하는가	90
9.1 저자가 며칠을 버린 이야기 · 9.2 왜 음량 상관은 틀린 지표인가 · 9.3 나머지 나쁜 지표들 · 9.4 올바른 검증 — 발화구간과 활발한 프레임 · 9.5 다른 목소리로 검증하기 · 9.6 눈으로 보는 검증도 규칙이 있습니다 · 9.7 검증을 파 이프라인에 넣으세요 · 9.8 이 장에서 기억할 것	

Part 3 · Track A. 사진 한 장이 말한다

Ch10 Wav2Lip — 가장 빠른 첫 성공	97
10.1 왜 여기서 시작하나 · 10.2 구조 — 입만 지우고 다시 그린다 · 10.3 실행은 명령 한 줄입니다 · 10.4 실행보다 설치가 어렵습니다 ★ · 10.5 첫 성공 뒤에 보이는 네 가지 한계 · 10.6 눈 문제를 어떻게 풀 것인가 · 10.7 그럼에도	

이 모델을 쓰는 경우 · 10.8 이 장에서 기억할 것

Ch11 MuseTalk v1.5 — 실사 립싱크의 기준선 103

11.1 무엇이 달라졌나 · 11.2 이 모델은 이미지가 아니라 영상을 받습니다 · 11.3 손델 파라미터는 셋뿐입니다 · 11.4 전처리 캐시 — 두 번째부터 빨라집니다 · 11.5 실시간 모드 — GPU 한 장에 사용자 한 명 · 11.6 흔한 실패 셋 · 11.7 이 장에서 기억할 것

Ch12 LivePortrait — 모션만 옮긴다 109

12.1 이 모델이 푸는 문제 · 12.2 왜 이것이 필요한가 · 12.3 조절해야 할 파라미터 넷 — 그리고 다섯째 · 12.4 다섯째 파라미터 — driving-option ★ · 12.5 옵션이 조용히 무시되는 세 경우 ★ · 12.6 실제로 출하한 조합 — 배울 3.0의 사연 · 12.7 사람 모드와 동물 모드 · 12.8 비교 영상을 활용하세요

Ch13 사람이 아닌 얼굴 116

13.1 하늘이에게 사람 입술이 그려졌습니다 · 13.2 시도했다가 버린 것 넷 · 13.3 정답 — 층을 나눈다 · 13.4 실패했다고 한 것을 다시 봅니다 ★ · 13.5 동물 얼굴에 붙는 조건 넷 · 13.6 캐릭터와 3D 렌더 얼굴 · 13.7 이 장에서 기억할 것

Ch14 fps 드리프트와 mux 125

14.1 앞은 맞는데 뒤가 밀립니다 · 14.2 원인은 29와 29.97입니다 · 14.3 잘못된 해결책 — setpts로 늘리기 · 14.4 올바른 해결책 — fps를 재해석한다 · 14.5 원칙 — 파이프라인 전체에서 fps를 하나로 · 14.6 mux 단계에서 명시할 넷 · 14.7 이 장에서 기억할 것

Ch15 트랙 A 완성 — 추모 영상 파이프라인 130

15.1 만들 것 · 15.2 전체 구조 · 15.3 파라미터 기본값 · 15.4 파이프라인을 스크립트로 묶기 · 15.5 원격 GPU로 보내기 · 15.6 검증과 리뷰 · 15.7 실측 성능 — 영상 1초당 21초 · 15.8 사람 얼굴은 순서가 뒤집힙니다 — 여섯 번의 실험 ★

Part 3 · Track B. GPU 없이 움직이는 아바타

Ch16 그림 한 장을 파츠로 쪼갬다 138

16.1 신경망을 한 줄도 쓰지 않는 얼굴 · 16.2 최소 파츠 넷 · 16.3 직접 그리거나, 오려 내거나 · 16.4 변형의 문법 · 16.5 파츠 대신 손잡이 넷을 잡으세요 · 16.6 팔이 없어도 몸은 말합니다 · 16.7 이 장이 만든 것은 Live2D가 아닙니다 ★ · 16.8 이 방식의 장단점

Ch17 음량이 입을 연다 146

17.1 Ch09가 버린 신호를 다시 씁니다 · 17.2 왜 음량으로 충분인가 · 17.3 브라우저에서 음량 읽기 · 17.4 날것의 음량은 쓸 수 없습니다 · 17.5 열린 입 이미지를 따로 두세요 · 17.6 분석이 실패해도 입은 움직여야 합니다 ★ · 17.7 말이 끝나면 확실히 닫으세요 · 17.8 서버가 할 일은 거의 없습니다

Ch18 VRM — 브라우저에서 도는 3D 휴먼 152

18.1 왜 3D로 올라가나 · 18.2 VRM — 뼈 이름이 정해져 있는 포맷 · 18.3 라이브러리를 로컬에 두세요 · 18.4 세 가지 제어 통로 · 18.5 렌더 루프의 구조 · 18.6 카메라와 조명 · 18.7 모델을 바꿔도 코드가 그대로인 이유 ★ · 18.8 더 사실적인 3D는 없나 — 있습니다, 그런데

Ch19 코드로 만드는 생명감 160

19.1 완벽하게 규칙적인 것은 죽어 보입니다 · 19.2 먼저 — 절대 각도를 쓰지 마세요 ★ · 19.3 깜빡임 — 가장 중요한 하나 · 19.4 호흡 — 눈에 보이지 않지만 느껴집니다 · 19.5 미세 움직임 — 정지하지 않기 · 19.6 시선 — 존재감의 핵심 · 19.7 아이들 모션 — 심심하지 않게 · 19.8 층으로 쌓기

Ch20 감정 태그와 제스처 167

20.1 말과 몸이 따로 놀면 · 20.2 태그를 앞에 붙이게 합니다 · 20.3 목록을 좁게 유지하세요 · 20.4 파싱은 관대하게 · 20.5 동작을 어떻게 만드나 · 20.6 동작 실행의 규칙 · 20.7 몸을 만지다 배운 것 셋 · 20.8 이모지와 특수문자 제거

Ch21 트랙 B 완성 — 2초 응답 대화 아바타 176

21.1 만들 것 · 21.2 전체 구조 · 21.3 서버 — 한 파일로 충분합니다 · 21.4 프론트엔드 — 세 개의 루프 · 21.5 실측 지연 ★ · 21.6 첫 호출이 유독 느린 이유 · 21.7 여러 캐릭터 지원 · 21.8 확인해야 할 것들

Part 3 · Track C. 인격을 넣는다

- Ch22 페르소나 — 인격의 90%는 프롬프트다 185
 22.1 파인튜닝 전에, 한 줄 고치고 새로고침 · 22.2 좋은 시스템 프롬프트의 뼈대 — 딱 다섯 부분 · 22.3 말투를 고정하는 기법 — 예시 · 금지 · 호칭 · 어미 · 22.4 페르소나가 무너지는 네 자리 · 22.5 온도 0.7~0.9, 그리고 반복 방지 · 22.6 폴백도 캐릭터여야 합니다 · 22.7 캐릭터는 코드가 아니라 데이터입니다 · 22.8 언제 파인튜닝으로 넘어가나
- Ch23 귀를 붙인다 — STT와 끼어들기 192
 23.1 타이핑에서 음성으로 · 23.2 언제 말이 끝났는가 — 침묵을 재는 일 · 23.3 STT를 어디서 돌릴 것인가 · 23.4 끼어들기 — barge-in · 23.5 STT는 틀립니다 — 오인식 다루기 · 23.6 상태 기계로 관리하세요 · 23.7 이 장을 건너뛰게 되는 경우 · 23.8 이 장에서 기억할 것
- Ch23A 실시간 통역 — 듣고, 옮기고, 다른 목소리로 말한다 199
 23A.1 통역은 대화가 아닙니다 · 23A.2 동시통역이 아니라 순차 통역입니다 · 23A.3 지연 예산 — 어디서부터 재는가 · 23A.4 번역기에 거는 네 가지 규칙 · 23A.5 겹쳐 말하지 않기 — 화자가 항상 이깁니다 · 23A.6 수어 브리지 — 손이 입력이고 손이 출력입니다 · 23A.7 실측 — 문장 하나가 첫 소리가 되기까지 · 23A.8 이 장의 경계
- Ch24 기억 3층 — 그리고 원장 206
 24.1 기억이 없으면 인격도 없습니다 · 24.2.1층 — 단기 기억 (대화 컨텍스트) · 24.2.2층 — 요약 기억 · 24.2.3층 — 장기 기억 (백터 저장소) · 24.5 기억은 대화 로그가 아니라 이벤트 원장입니다 ★ · 24.6 잊는 것도 설계입니다 · 24.7 기억이 만드는 인격 · 24.8 기억에 없는 사실은 물어봅니다 — 도구 호출 ★
- Ch25 이 캐릭터만 아는 것 — RAG 213
 25.1 기억과 지식은 다릅니다 · 25.2 왜 프롬프트에 다 넣으면 안 되나 · 25.3 한국어 청킹 — 품질의 절반 · 25.4 임베딩과 검색 · 25.5 대화 아바타에서의 특수 사정 · 25.6 언제 검색할 것인가 · 25.7 근거를 다루는 법 · 25.8 이 장에서 기억할 것
- Ch26 여러 아바타가 함께 217
 26.1 하나에서 여럿으로 · 26.2 진행자가 필요합니다 · 26.3 비밀 정보를 격리하세요 · 26.4 누가 말할 차례인가 · 26.5 비용과 지연 · 26.6 화면에 여러 배치하기 · 26.7 무엇이 막혀도 게임은 끝나야 합니다 · 26.8 이 장에서 기억할 것
- Ch27 트랙 C 완성 — 평가 하네스 224
 27.1 프롬프트 한 줄을 고쳤을 뿐인데 · 27.2 골든셋 — 물어볼 것들의 목록 · 27.3 무엇을 채점하나 — 여섯 항목 · 27.4 LLM 채점의 요령 · 27.5 회귀 게이트 · 27.6 콘텐츠를 생성한다면 — 정답을 LLM에게 맡기지 마세요 ★ · 27.7 지연도 함께 재세요 · 27.8 사람의 눈이 여전히 필요합니다

Part 4. 세상에 내보내기

- Ch28 배포 — FastAPI · 도커 · 원격 GPU 잡 232
 28.1 두 개를 따로 내보냅니다 · 28.2 실시간 서비스 — 한 프로세스면 끝납니다 · 28.3 밖에서 접속하게 하기 — 터널의 함정 셋 · 28.4 렌더 잡은 컨테이너에 통째로 갑니다 · 28.5 잡 큐 — 순번과 회수 · 28.6 GPU를 빌리거나, 빌려주거나 · 28.7 웹을 앱으로 감싸기 · 28.8 이 장에서 기억할 것
- Ch28A 운영자 콘솔 — 만든 다음에 시작되는 일 245
 28A.1 배포는 끝이 아니라 시작입니다 · 28A.2 콘솔은 다섯 개의 방입니다 · 28A.3 ① 지표 — 한 번의 호출로 · 28A.4 ② 품질 — 이 방이 이 장의 핵심입니다 · 28A.5 ③ 사람 — 파괴적 작업에는 미리보기를 · 28A.6 ④ 콘텐츠 — DB만 고치면 안 됩니다 · 28A.7 ⑤ 자동화 — 운영자를 루프에서 빼기 · 28A.8 여섯째 방 — 화면에서는 ⑥번 ★
- Ch29 책임 — 덤페이크 · 음성 복제 · 동의 254
 29.1 학생이 한 말 · 29.2 이 기술이 실제로 하는 일 · 29.3 선은 어디인가 — 세 가지 기준 · 29.4 개발자가 실제로 해야 할 것들 · 29.5 대화 아바타에만 있는 문제들 · 29.6 배우는 사람에게 · 29.7 이 장에서 기억할 것
- Ch30 커리큘럼으로 — 800시간과 28차시 259

30.1 순서가 완성도를 바꿉니다 · 30.2 이 책을 가르치는 세 가지 방식 · 30.3 800시간 과정에 얹기 · 30.4 28차시 강의 자료와의 관계 · 30.5 가르치면서 배운 것들 · 30.6 평가 루브릭 제안 · 30.7 이 장에서 기억할 것 · 30.8 마지막 한 줄

부록

부록 A — 환경 구축 (무GPU 트랙 / GPU 트랙)	264
부록 B — 드라이버 영상 카탈로그	268
부록 C — 실측 벤치마크	271
부록 D — KDT AI 휴먼 800시간 커리큘럼 ↔ 본책 매핑	279
부록 E — 28차시 강의 자료 ↔ 본책 매핑	285
부록 F — 실패 카탈로그 (재현 금지 목록)	289
부록 G — 윤리 · 동의서 · 워터마킹 템플릿	299
부록 H — Windows 트러블슈팅	306
부록 L — 아바타 UI/UX 가이드	310
부록 N — 원가 모형	315
온라인 부록	319
찾아보기	320

Part 1

AI 휴먼의 해부

"AI 휴먼은 하나의 모델이 아니라 네 층의 조립이다. 얼굴 · 목소리 · 머리 · 기억."

Ch01	왜 지금 AI 휴먼인가	4층 스택의 정의. 거대 모델 없이 시작한다
Ch02	네 층을 세우는 30분	무GPU 트랙과 GPU 트랙의 갈림길, 환경 두 벌
Ch03	목소리부터 만든다	TTS 4종 결정표. 감정의 80%는 목소리다
Ch03A	사례 — 사투리 다섯 개를 만든 이야기	하루에 LoRA 다섯 개. 어댑터 핫스왑이 15.0GB를 3.2GB로
Ch04	얼굴의 조건	좋은 소스 이미지와 좋은 드라이버 영상은 다르다
Ch05	어느 사다리를 오를 것인가	2D 파츠 · 3D VRM · 신경망 립싱크 · 원샷 오디오→영상 결정 트리

CHAPTER 1

왜 지금 AI 휴먼인가

1.1 질문이 틀렸습니다

강의를 하다 보면 반드시 나오는 질문이 있습니다.

"AI 휴먼 만들려면 어떤 모델 쓰면 되나요?"

이 질문에는 답이 없습니다. 질문이 틀렸기 때문입니다. "자동차를 만들려면 어떤 부품을 쓰면 되나요"와 같은 구조의 질문입니다. 엔진인가요, 바퀴인가요, 핸들인가요.

AI 휴먼은 하나의 모델이 아닙니다. **네 개의 층**입니다.

얼굴 — 화면에 보이는 것. 입이 움직이고 눈이 깜빡이는 층입니다.

목소리 — 텍스트를 소리로 바꾸는 층. 감정의 대부분이 여기서 결정됩니다.

머리 — 무엇을 말할지 정하는 층. 보통 LLM(언어모델)입니다.

기억 — 어제 무슨 얘기를 했는지, 이 캐릭터만 아는 것이 무엇인지를 쥐고 있는 층.

이 넷은 **서로 독립**입니다. 얼굴을 2D 그림에서 3D 캐릭터로 바꿔도 목소리 층은 아무것도 모릅니다. LLM을 클라우드 API에서 로컬 모델로 갈아끼워도 얼굴은 그대로 움직입니다. 목소리를 무료 TTS(글자를 소리로 바꾸는 엔진)에서 프리미엄 TTS로 올려도 나머지 세 층은 코드 한 줄 바뀌지 않습니다.

이 독립성이 이 책 전체를 관통하는 좋은 소식입니다. **한 번에 한 층씩 올릴 수 있습니다.**



그림 1.1 한 층을 바꿔도 나머지 셋은 모른다 — 그래서 한 번에 하나씩 올릴 수 있다

휴먼의 네 층

실물로 보겠습니다. 이 책에서 가장 먼저 만들 **휴먼**이 는 야구 용어를 설명하는 마스크트입니다. 네 층이 이렇게 생겼습니다.

얼굴 은 그림 파일 하나입니다. 정확히는 그림 하나를 **몸통·원눈·오른눈·입** 네 조각으로 쪼갠 것. 눈을 감기는 것은 눈 이미지를 세로로 눌러 찌그러뜨리는 일입니다.

목소리 는 무료 클라우드 TTS 한 줄입니다. 텍스트를 넣으면 mp3가 나옵니다.

머리 는 API 키 하나입니다. "볼넷이 뭐야"를 받으면 문장을 만듭니다.

기억 은 이 단계에서 없습니다. 빈 덩어리 하나입니다.

합쳐서 파이썬 파일 한 개, GPU 0장, 모델 다운로드 0GB. 그런데 휴먼이는 눈을 깜빡이고, 입을 움직이고, 볼넷을 설명합니다.

1.2 거대 모델은 필요 없습니다

수업 첫 시간에 학생들이 가장 놀라는 문장이 이것입니다. 말하는 아바타를 만드는 데 **7B 언어모델은 필요 없습니다**. 모델 가중치를 수십 기가바이트 내려받을 필요도 없습니다.

아바타의 정의는 이렇게 단순합니다.

아바타 = 얼굴 이미지 + 음성 + 립싱크

여기 어디에도 언어모델이 없습니다. 언어모델은 **무엇을 말할지**를 정하는 층이고, 그것은 나중 문제입니다. 대본을 직접 쓰면 언어모델 없이도 아바타는 말합니다.

이 순서가 중요한 이유는 실무적입니다. 거대 모델부터 시작하면 첫 결과물을 보기까지 몇 시간이 걸립니다. 다운로드가 끊기고, VRAM(그래픽카드 메모리)이 모자라고, 양자화 설정이 틀리고, 그러다 수업이 끝납니다. 반대로 목소리와 얼굴부터 시작하면 **30분 안에 화면 속의 무언가가 말을 합니다**. 그때부터 학생은 남은 열 시간을 스스로 씁니다.

그래서 이 책의 순서는 **목소리 → 얼굴 → 머리 → 기억**입니다. 대부분의 자료와 반대 방향입니다.

홍련이의 비밀 무기

홍련이에게는 언어모델 말고 하나가 더 있습니다. **야구 용어 열두 개의 답이 코드에 그냥 박혀 있습니다**.

볼넷, 번트, 도루, 안타, 홈런, 삼진, 병살, 타율, 방어율, 스트라이크, 이닝, 타점. 각각에 대해 감정과 동작과 한 문장짜리 설명이 디렉터리에 들어 있습니다.

우스위 보이지만 이것이 여러 번 시연을 살렸습니다. 무료 한도가 끝나거나, API가 잠깐 응답하지 않거나, 강의실 와이파이가 죽었을 때 — **홍련이는 여전히 볼넷을 설명합니다**. 조금 덜 똑똑한 대답이 침묵보다 낫기 때문입니다.

층이 독립이라는 말이 실무에서 갖는 의미가 이것입니다. 머리 층이 죽어도 얼굴과 목소리는 살아 있습니다. 그 사이에 다른 답을 끼워 넣을 수 있습니다. Ch07에서 이 구조를 제대로 만듭니다.

1.3 세 개의 시장, 세 개의 트랙

AI 휴먼이라는 말은 지금 최소한 세 가지 다른 물건을 가리킵니다. 제약은 서로 완전히 다른데 이름만 같습니다. 혼란은 거기서 옵니다.

첫째, 만들어 두는 얼굴

결과물이 영상 파일입니다. 사람이 한 번 만들고, 여러 사람이 여러 번 봅니다. 광고, 교육 콘텐츠, 뉴스 앵커, 추모 영상, 캐릭터 IP 홍보물이 여기 속합니다.

이 시장의 제약은 **품질**입니다. 만드는 데 4분이 걸리든 40분이 걸리든 상관없습니다. 대신 확대해서 봐도 어색하지 않아야 하고, 소리와 입이 한 프레임도 어긋나면 안 됩니다. 이것이 Track A입니다.

둘째, 응답하는 얼굴

사람이 말을 걸면 반응해야 합니다. 키오스크, 학습 도우미, 상담 프론트, 버추얼 스트리머, 게임 NPC.

이 시장의 제약은 **지연**입니다. 아무리 사실적이어도 5초를 기다리게 하면 사용자는 화면을 떠납니다. 반대로 그림체가 만화 같아도 즉시 반응하면 대화가 성립합니다. 사실감과 지연을 바꾸는 거래가 이 트랙의 본질이고, 이것이 Track B입니다.

셋째, 기억하는 얼굴

얼굴은 A든 B든 상관없습니다. 무엇을 말하는가 이고, *어제 한 얘기를 기억하는가*입니다. 캐릭터 챗, 도메인 상담, 페르소나 서비스.

이 시장의 제약은 **일관성**입니다. 어제는 존댓말을 쓰던 캐릭터가 오늘 반말을 하면, 그것은 같은 인격이 아닙니다. 이것이 Track C입니다.

세 트랙은 상하 관계가 아닙니다. Track A를 마스터해도 Track B를 만들 수 없습니다. 서로 다른 물건이기 때문입니다.

어디에 쓰이나 — 도메인과 트랙

같은 "말하는 얼굴" 이라도 쓰이는 자리에 따라 요구가 다릅니다. 이 책의 예제 다섯을 실제 도메인에 대응시키면 이렇습니다.

표 1.1 도메인별 요구와 맞는 트랙

도메인	요구	맞는 트랙 · 이 책의 예제
추모·기념 영상	사진 한 장 · 최고 품질 · 실시간 불필요	A — 하늘이(Ch10~15)
교육·강의 아바타	긴 대본 · 배치 생성 · 자막	A — 실사 립싱크(Ch11) + Ch15 파이프라인
키오스크·민원 안내	소음 환경 · 짧은 응답 · 정해진 답 · GPU 없는 단말	B — 코치의 구조(Ch18~21) + Ch07 § 6 오프라인 답변 · Ch23 소음 보정
고객 상담·콜센터 화면	실시간 · 지식 검색 · 거절 규칙	B + C — 코치 + RAG(Ch25) + 페르소나 경계(Ch22)
박물관·전시 도슨트	인격 · 기억 · 여러 관람객	C — 마을 사람들(Ch26)
게임·인터랙티브 스토리	여러 캐릭터 · 비밀 · 진행자(누가 말할 차례인지 정하는 코드, Ch26)	C — 마을 사람들(Ch26~27)
실시간 통역 — 관광·민원·의료 창구	문장 단위 순차 통역 · 용어 정확 · 겹쳐 말하지 않기	B + C — Ch23A (Ch23 STT + Ch21 TTS + Ch22 규칙)
접근성 — 수어 통역 보조	손·상체 동작이 핵심 · 카메라 입력	B의 3D 뼈대(Ch18~20)가 출력 쪽, 인식은 별도 모델 — Ch23A § 6 브리지

마지막 줄이 이 책의 경계입니다. 저자는 한국수어(KSL) 단어 인식 MVP(MediaPipe Holistic + BiLSTM)를 따로 만들었습니다. 그 **입력 쪽** — 카메라로 손을 읽는 인식이 — 은 이 책 밖입니다.

이 책이 만드는 것은 *브리지* 입니다. 인식된 단어열을 문장으로 엮어 말하고, 반대로 문장을 손을 쓰는 3D 아바타 의 수어 단어 태그로 내보냅니다. Ch23A § 6에 있고, Ch20의 제스처 태그가 수어 단어 태그로 바뀔 뿐입니다.

키오스크·민원 창구도 마찬가지입니다. 새 모델이 아니라 **소음 보정(Ch23 § 2) · 정해진 답의 오프라인 사전(Ch07 § 6) · 짧은 응답의 페르소나 규칙(Ch22)** 을 조합하는 문제입니다.

1.4 이 책의 다섯 가지 입장

첫째, AI 휴먼은 4층의 조립입니다. 각 층은 갈아 끼울 수 있는 부품입니다. 층을 섞어서 생각하면 문제가 풀리지 않습니다. 입이 안 움직이는 것은 얼굴 층의 문제이지 LLM의 문제가 아닙니다.

둘째, 거대 모델 없이 시작합니다. 1장의 결과물은 GPU 0장, 모델 다운로드 0GB로 나옵니다.

셋째, 품질 트랙과 실시간 트랙은 다른 물건입니다. 목표 지연이 아키텍처를 결정합니다. 나중에 최적화해서 실시간으로 만들 수는 없습니다. Part 2 전체가 이 이야기입니다.

넷째, 사다리는 넷 — 2D 파츠 · 3D VRM · 신경망 립싱크 · 원샷 오디오→영상(2.5D 메시를 넣으면 다섯: Ch05). 위로 갈수록 사실적이고, 아래로 갈수록 빠르고 싼다. 선택은 사실감이 아니라 *지연 예산과 얼굴의 종류*로 갈립니다.

다섯째, 만들 수 있게 되면 만들지 않을 책임도 배웁니다. 이 책은 동의한 얼굴과 목소리만 다룹니다. 29장의 기준선은 협상 대상이 아닙니다.

1.5 왜 하필 지금인가

세 가지가 최근 몇 년 사이에 동시에 바뀌었습니다.

첫째, 립싱크가 쓸 만해졌습니다. 몇 년 전까지 오픈소스 립싱크는 입 주변이 뭉개져서 확대하면 바로 티가 났습니다. 잠재공간에서 입·턱 영역을 인페인팅하는 — 그 부분만 주변에 맞춰 다시 그려 넣는 — 방식이 자리 잡으면서 실사 품질이 실용 수준에 들어왔습니다.

둘째, 리타게팅이 분리되었습니다. 예전에는 "이 얼굴로 이 소리를 말하게 한다"가 하나의 통짜 작업이었습니다. 지금은 *소리에 입을 맞추는 단계*와 *그 모션을 다른 얼굴로 옮기는 단계*를 분리할 수 있습니다. 이 분리가 이 책 Track A의 핵심 트릭이고, 사람이 아닌 얼굴을 말하게 하는 유일한 실용적 경로입니다.

셋째, GPU 없는 경로가 생겼습니다. 브라우저에서 3D 캐릭터를 60fps로 렌더하고, 오디오 음량으로 입을 열고, 코드로 깜빡임과 제스처를 만드는 데 필요한 것은 자바스크립트 라이브러리 두 개 뿐입니다. 몇 년 전이라면 상상하기 어려웠던 품질이 이제 노트북에서 돕니다.

세 번째가 특히 중요합니다. 이 분야의 진입 장벽이 돈에서 *지식*으로 옮겨갔다는 뜻이기 때문입니다. 그리고 지식의 장벽은 책이 낮출 수 있습니다.

1.6 이 장에서 기억할 것

AI 휴먼은 하나의 모델이 아니라 **얼굴 · 목소리 · 머리 · 기억**의 네 층입니다. 네 층은 독립이고, 한번에 하나씩 올릴 수 있습니다.

아바타를 만드는 데 거대 언어모델은 필요 없습니다. **아바타 = 얼굴 + 음성 + 립싱크**입니다.

세 트랙은 상하 관계가 아니라 서로 다른 제약을 가진 다른 물건입니다. 품질(A) · 지연(B) · 일관성(C).

다음 장에서는 네 층을 세울 환경을 만듭니다. 갈림길이 하나 있는데, 그 갈림길이 이 책 전체에서 여러분이 내리는 가장 중요한 결정입니다.

실습 코드: `code/ch01_stack/` — 4층 스택 최소 예제 (LLM 없이 대본만으로 말하는 아바타)

예상 소요: 30분

관련 부록: N(원가 모델) · L(아바타 UI/UX)

CHAPTER 2

네 층을 세우는 30분

2.1 갈림길이 하나 있습니다

강의 첫 시간에 CUDA 버전 충돌과 싸우다 종이 치면, 그 반은 회복되지 않습니다.

여러 번 꺾고 나서 순서를 바꿨습니다. 첫 시간에는 GPU를 아예 켜지 않습니다. 대신 30분 만에 홈런이가 말을 하게 만듭니다. 무거운 환경은 그 다음 주에 세웁니다.

이 장의 구조가 그 순서입니다.

환경 구축을 시작하기 전에 결정할 것이 하나 있습니다. 이 결정이 이후 열 장의 내용을 절반으로 줄여 줍니다.

GPU를 쓸 것인가.

써야 한다면 conda 환경 두 벌과 CUDA 툴킷과 커스텀 연산자 빌드가 기다립니다. 반나절이 걸릴 수도 있습니다. 쓰지 않는다면 Python 하나와 브라우저면 끝이고, 30분 뒤에 아바타가 말합니다.

많은 사람이 "일단 GPU 환경부터 만들어 두자" 고 생각합니다. 저는 반대를 권합니다. **먼저 무 GPU 트랙으로 끝까지 한 번 가 보세요.**

목소리가 나고, 입이 움직이고, 대답이 돌아오는 전체 루프를 눈으로 본 다음에 GPU 트랙을 여는 편이 훨씬 빠릅니다. 전체 그림을 모르는 채로 CUDA 버전 충돌과 싸우는 것은 학습이 아니라 소모입니다.

2.2 무GPU 트랙 — 30분

필요한 것은 셋입니다. Python 3.10 이상, ffmpeg, 그리고 최신 브라우저.

Python 은 가상환경 하나면 됩니다. 이 트랙에서 설치할 패키지는 웹 서버(FastAPI·uvicorn)와 HTTP 클라이언트, TTS 클라이언트 정도입니다. 전부 합쳐 수십 메가바이트입니다.

ffmpeg 는 오디오 포맷 변환과 길이 측정에 씁니다. Windows에서는 패키지 매니저로 설치한 뒤, 실행 파일 경로를 프로세스의 PATH 앞에 직접 붙이세요. 셸을 새로 열 때마다 PATH가 사라지는 문제를 코드 안에서 미리 막는 것입니다.

```
FFBIN = r"...\\ffmpeg\\bin"
os.environ["PATH"] = FFBIN + os.pathsep + os.environ["PATH"]
```

브라우저 는 실제로 얼굴을 렌더하는 곳입니다. 이 트랙에서 얼굴 층은 서버가 아니라 브라우저에 있습니다. 서버는 텍스트와 오디오 파일만 만들어 보내고, 입을 움직이고 눈을 깜빡이고 손을 흔드는 일은 전부 클라이언트에서 일어납니다. 이 구조가 GPU 없이 실시간이 가능한 이유입니다.

여기까지가 30분입니다. 이 위에서 Track B 전부와 Track C 전부가 돕니다.

이 책의 저장소(컴패니언)에 있는 `code/ch02_setup/doctor.py` 가 이 트랙에 필요한 것을 한 번에 점검합니다. 저자 기계의 출력을 그대로 옮깁니다 — 여러분 화면과 대조하세요.

```
■ 무GPU 트랙 – Track B·C 는 이것만 있으면 됩니다
[OK] Python 3.12.10
[OK] fastapi      (웹 서버)
[OK] uvicorn      (ASGI 실행기)
[OK] edge_tts     (무료 TTS)
[OK] ffmpeg       ffmpeg version 8.1.1-full_build
[OK] PYTHONUTF8=1
[OK] 작업 경로가 ASCII
준비 완료.
```

일곱 줄 중 마지막 둘이 이 책이 굳이 넣은 검사입니다. 한글 경로와 인코딩은 이 뒤 31개 장에서 나는 오류 대부분의 원인입니다(부록 H·부록 F). 도구가 잘 깔렸는지보다 먼저 봐야 합니다.

2.3 GPU 트랙 — 반나절

Track A를 하려면 GPU가 필요합니다. 본문의 모든 실측은 **RTX 4070 SUPER 12GB** 기준이고, 8GB 카드에서도 배치 크기를 줄이면 동작합니다.

환경은 반드시 두 벌로 분리하세요

이 책에서 쓰는 두 모델 계열 — 립싱크 계열과 리타게팅 계열 — 은 의존성이 충돌합니다. 같은 환경에 넣으면 한쪽이 반드시 깨집니다. **conda 환경 두 개** 를 만들고 이름을 각각 붙이세요. 둘 다 Python 3.10 · CUDA 11.8로 맞추되, PyTorch는 계열이 요구하는 버전(립싱크 2.0.1 · 리타게팅 2.3.0)을 그대로 씁니다.

한 환경에서 다른 환경의 스크립트를 부를 때는 conda를 활성화하는 대신 **환경의 python 실행 파일을 절대 경로로 직접 호출** 하는 방식을 권합니다. 셸 상태에 의존하지 않아 자동화가 쉽고, 파이프라인을 한 스크립트에서 순서대로 돌릴 수 있습니다.

커스텀 연산자 빌드

리타게팅 모델의 동물 모드는 커스텀 CUDA 연산자를 빌드해야 합니다. Windows에서는 최신 MSVC와 CUDA 11.8이 서로를 지원하지 않는다고 주장하며 빌드를 거부합니다. 컴파일러에 우회 플래그를 주입해 통과시키세요. 배치 스크립트 한 벌로 정리해 두었습니다(부록 A).

빌드가 필요한 시점은 환경을 새로 만들 때뿐입니다. 한 번 성공하면 다시 불 일이 없습니다.

인코딩 설정

Windows 한국어 환경에서 모델 다운로드 도구가 이모지를 출력하다 인코딩 오류로 죽는 경우가 있습니다. 환경변수 하나로 예방합니다.

```
PYTHONUTF8=1
```

이 한 줄이 없어서 반나절을 잃는 사람을 여러 번 봤습니다.

doctor.py --gpu 가 이 트랙까지 점검합니다. 저자 기계의 출력입니다.

```
■ GPU 트랙 - Track A 에만 필요합니다
[OK]  CUDA 사용 가능 (torch 2.6.0+cu124)
      NVIDIA GeForce RTX 4070 SUPER · 12.0GB
■ 격리 환경 두 별 - 립싱크 / 리타게팅 (Ch02 §3)
[OK]  MuseTalk      torch 2.0.1+cu118 · CUDA True
[OK]  LivePortrait  torch 2.3.0+cu118 · CUDA True
```

한 기계에 **torch**가 셋 입니다 — 기본 파이썬 2.6, 립싱크 환경 2.0.1, 리타게팅 환경 2.3.0. 이것이 §3이 "환경을 반드시 두 벌로" 나누라고 하는 이유입니다. 점검기가 기본 파이썬의 CUDA만 보고 "준비 완료"를 내면 안 되는 이유이기도 합니다.

처음 만든 점검기가 정확히 그렇게 했습니다. 환경 두 벌의 python을 절대 경로로 불러 직접 문도록 고쳤습니다(Ch10 §4의 격리 원칙).

2.4 네 층을 각각 세웁니다

목소리 층

먼저 세웁니다. 무료 무제한 옵션 하나를 기본으로 깔고 시작합니다. 텍스트를 넣으면 mp3가 나오지만 확인하면 이 층은 끝입니다. 품질을 올리는 이야기는 Ch03에서 합니다.

검증은 단순합니다. 짧은 한국어 문장 하나를 넣고, 나온 파일을 재생해서 들어 보세요. 그리고 **길이를 초 단위로 측정** 해 두세요. 이 길이가 나중에 립싱크와 영상 길이를 맞출 때 기준이 됩니다.

얼굴 층

무GPU 트랙이라면 브라우저에 캐릭터 하나를 띄우는 것으로 끝납니다. 아직 말하지 않아도 됩니다. 눈을 깜빡이기만 하면 이 층은 살아 있는 것입니다.

GPU 트랙이라면 각 모델의 공식 예제를 그대로 한 번 돌려서 결과 파일이 나오는지 확인합니다. 이것을 **스모크 테스트** 라고 부릅니다. 잘 도는지가 아니라 일단 도는지만 보는 시험이고, 환경을 새로 만들 때마다 가장 먼저 합니다. 자기 데이터로 시작하면 실패했을 때 원인이 환경인지 데이터인지 구분할 수 없습니다.

머리 층

API 키 하나로 시작합니다. 요청을 보내고 응답이 오지만 볼입니다.

여기서 한 가지 습관을 들이세요. **모델 이름을 리스트로 두고 순서대로 폴백** 하는 구조입니다. 무료 한도와 잠깐의 무응답은 실제로 자주 일어납니다 — 폴백 리스트를 처음부터 넣어 두세요(완결 서술은 Ch07 § 6).

기억 층

이 단계에서는 세우지 않습니다. 빈 딕셔너리 하나면 충분합니다. Ch24에서 제대로 만듭니다.

2.5 첫 번째 결과물 — LLM 없이 말을 시킵니다

네 층이 서 있으면 이제 연결합니다. 가장 단순한 연결은 이렇습니다.

대본 문자열을 하나 준비합니다. 목소리 층에 넣어 오디오를 만듭니다. 얼굴 층에 그 오디오를 재생시키면서 입을 움직이게 합니다. 끝입니다.

LLM은 쓰지 않았습니다. 그런데도 화면 속의 무언가가 여러분이 쓴 문장을 소리 내어 말합니다. 이것이 1장에서 말한 *아바타* = 얼굴 + 음성 + 립싱크의 실물입니다.

머리 층을 연결하는 것은 다음 한 줄입니다. 대본 문자열 자리에 LLM의 응답을 넣기만 하면 됩니다. 층이 독립이라는 말의 의미가 여기서 드러납니다.

2.6 이 장에서 기억할 것

갈림길은 GPU입니다. 무GPU 트랙은 30분, GPU 트랙은 반나절. 먼저 무GPU로 전체 루프를 눈으로 보세요.

GPU 환경은 두 벌로 분리 하고, 서로 부를 때는 실행 파일 절대 경로로 직접 호출하세요.

층은 하나씩 세우고 하나씩 검증 합니다. 자기 데이터가 아니라 공식 예제로 스모크 테스트를 먼저 합니다.

LLM 폴백 리스트 를 처음부터 넣으세요. 데모가 멈추는 사고의 절반이 여기서 예방됩니다.

다음 장은 목소리입니다. 네 층 중에서 가장 과소평가되고, 결과물의 인상을 가장 크게 좌우하는 층입니다.

실습 코드: `code/ch02_setup/` — 무GPU 환경 검증 스크립트 + GPU 스모크 테스트

예상 소요: 무GPU 30분 / GPU 3~5시간 (빌드 포함)

관련 부록: A(환경 구축), H(Windows 트러블슈팅)

CHAPTER 3

목소리부터 만든다

3.1 감정의 80%는 목소리입니다

같은 문장을 표정만 바꿔 들려주면 아무도 감정을 못 맞히고, 목소리만 바꾸면 대부분 맞습니다 — 저자의 청취 비교에서 체감상 8할이 목소리였습니다 (§ 5).

점심을 먹고 왔더니 무료 한도가 끝나 있었습니다.

프리미엄 TTS 하나를 붙잡고 대본을 다듬던 중이었습니다. 문장 하나 고치고 다시 뽑고, 쉼표 하나 옮기고 다시 뽑고. 그렇게 오전에 열 번 남짓 호출하고 점심을 먹으러 갔는데, 돌아와 보니 그날 치가 없었습니다.

그날 배운 것이 이 장의 결론 중 하나입니다. **개발용 엔진과 최종 출력용 엔진은 달라야 합니다.**

립싱크를 아무리 정교하게 맞춰도, 목소리가 기계적이면 결과물은 기계적입니다. 반대로 목소리가 좋으면 입 모양이 조금 어긋나도 사람들은 알아채지 못합니다.

이유는 단순합니다. **입은 소리를 따라가기 때문**입니다. 슬프고 느린 목소리를 넣으면 입도 느리게 움직이고, 영상 전체의 호흡이 느려집니다. 밝고 빠른 목소리를 넣으면 반대가 됩니다. 목소리가 영상의 템포를 결정하고, 템포가 감정의 대부분을 전달합니다.

그래서 이 책은 목소리를 얼굴보다 먼저 다룹니다. 결과물의 품질을 한 단계 올리고 싶을 때 가장 싸게 먹히는 곳도 여기입니다.

3.2 TTS를 고르는 네 가지 축

품질

한국어에서 특히 갈립니다. 영어에서 훌륭한 모델이 한국어에서는 끝음절을 늘어뜨리거나 조사를 뭉개는 경우가 흔합니다. **반드시 한국어 문장으로 직접 들어보고 판단하세요.** 데모 페이지의 영어 샘플은 아무것도 말해주지 않습니다.

저는 로컬 음성 클로닝 모델 하나를 한국어로 시도했다가 폐기한 적이 있습니다. 영어 평판은 좋았습니다. 그런데 한국어에서는 문장 끝이 늘어지고 발음이 뭉개져서, 립싱크를 걸어도 입 모양이 무엇을 말하는지 알 수 없었습니다. 며칠을 쓰고 버렸습니다. 이 판단은 빠를수록 좋습니다.

덧붙일 것이 하나 있습니다. 저는 같은 모델을 강의에서는 "한국어 됩니다"라고 가르치고 있었습니다.

거짓말을 한 것이 아닙니다. 강의 실습에서는 실제로 됩니다. 문장을 넣으면 한국어가 나오고, 목소리도 참조와 비슷합니다. 학생들이 처음 자기 목소리로 합성된 문장을 들으면 탄성이 나옵니다.

그런데 그 결과물을 **추모 영상의 최종 음성**으로 쓰려니 못 쓰겠던 것입니다. 문장 끝이 늘어지고, 그 늘어짐이 립싱크로 그대로 넘어가 입 모양이 무너졌습니다.

"된다"와 "쓸 수 있다"는 다릅니다. 데모에서 되는 것과 최종 산출물에 넣을 수 있는 것 사이에는 한 칸이 더 있고, 그 칸은 용도가 정합니다. 학습용으로는 훌륭한 모델이 제품에서는 탈락할 수 있습니다. 그 반대도 마찬가지입니다.

그 모델 이름으로 결론을 내리지 않는 이유가 있습니다. **이 판단은 그 시점의 것이기 때문**입니다. 한국어 TTS 지형은 반년 단위로 바뀌고, 그때 버린 계열이 지금은 쓸 만해졌을 수도 있습니다.

가져가야 할 것은 결론이 아니라 절차입니다 — **한국어로 직접 듣고, 끝음절과 조사를 확인하고, 아니면 빠르게 버린다.** 현재 후보 목록은 온라인 부록 K §5에 있고, 그 부록은 6개월마다 갱신됩니다.

한도와 비용

무료 무제한인 것, 하루 몇 번으로 제한되는 것, 유료지만 저렴한 것이 섞여 있습니다. 개발 중에는 호출을 수백 번 합니다. **개발용과 최종 출력용을 다른 엔진으로 나누는 것이 정석**입니다. 개발은 무제한 무료로, 최종 결과물만 프리미엄으로.

하루 열 번 남짓 쓸 수 있는 프리미엄 엔진을 개발 중에 쓰면, 점심 먹고 오면 한도가 끝나 있습니다.

감정 제어

세 가지 방식이 있습니다.

파라미터 방식은 속도와 음높이를 숫자로 조절합니다. 슬픔은 대체로 *느리게 + 낮게*로 근사됩니다. 표현력은 제한적이지만 예측 가능하고 재현됩니다.

실행에서 반드시 걸리는 함정 하나 — *낮게*는 음수 값입니다. 그런데 명령줄 도구에 `--pitch -8Hz` 처럼 띄어 쓰면, `-8Hz`가 **값이 아니라 옵션으로 해석되어 실패**합니다. `--pitch=-8Hz`로 붙여 쓰세요.

웃기게도 **이 책의 조언을 그대로 따를 때만 걸립니다.** 밝은 톤(양수)은 잘 되고 슬픈 톤(음수)만 안 되니, 원인을 찾기까지 한참 걸립니다. 컴패니언 코드에서 실제로 겪었습니다.

자연어 지시 방식은 프롬프트로 톤을 지시합니다. 하늘이의 추모 대본에 실제로 붙였던 문장은 이것입니다.

"잔잔하고 따뜻하게, 조금 슬프지만 다정한 목소리로 천천히 읽어줘:"

이 한 줄을 붙이고 안 붙이고의 차이가 결과물의 절반이었습니다. 톤과 속도가 실제로 반영되고, 입이 그 속도를 따라가므로 **영상 전체의 호흡이 바뀝니다**. 표현력이 가장 좋은 방식이지만, 같은 지시에 같은 결과가 나온다는 보장은 약합니다. 최종본은 서너 번 뽑아 놓고 고르게 됩니다.

참조 음성 방식은 몇 초짜리 샘플을 주고 그 목소리와 감정을 흉내내게 합니다. 가장 강력하고, 가장 위험합니다. 이 방식은 Ch29의 동의 문제와 직결됩니다.

외울 단어는 두 개입니다 — 스피커 임베딩과 제로샷

참조 음성 방식의 원리는 단어 두 개로 끝납니다.

스피커 임베딩 — 목소리를 숫자 지문으로 뽑는 것입니다. 누가 말하는지를 벡터 하나로 요약합니다. 무슨 말을 하는지와는 분리되어 있어서, 그 지문만 가져와 다른 문장에 입힐 수 있습니다.

제로샷 — 재학습 없이, 샘플 하나로 즉시 복제하는 것입니다.

이 둘의 대안은 **파인튜닝**입니다. 두 방식의 거리는 이렇습니다.

표 3.1 제로샷 · 파인튜닝

	제로샷	파인튜닝
참조 음성	6~30초	수십 분~몇 시간
재학습	없음	있음
결과까지	1~2초	시간·비용 큼
품질	"아주 비슷" 수준	훨씬 높고 안정적
쓰는 곳	대부분의 경우	성우 전용 음성 · 오디오북

이 책은 제로샷으로 갑니다. 대부분의 프로젝트에 그것으로 충분하기 때문입니다.

그리고 여기서 잠깐 멈춰야 합니다. **"샘플 하나만 주면 즉시 그 목소리"**라는 이 편리함이 곧 이 기술의 진입 장벽이 무너진 이유이고, **동시에 위험한 이유**입니다. 6초짜리 음성은 어디에 나 있습니다. Ch29가 이 문장에서 시작합니다.

스피커 임베딩이 성립하는 이유 — 목소리는 둘로 나뉩니다

"무슨 말을 하는지와는 분리되어 있다"고 썼는데, 이 분리가 이 분야 전체의 뼈대입니다.

한 문장의 소리에는 두 가지가 섞여 있습니다.

내용 — 어떤 말인가. 음소의 배열, 발음.

음색 — 누가 말하는가. 성대의 특성, 공명, 말버릇.

이 둘을 떼어낼 수 있다는 것 이 음성 클로닝의 전제입니다. 떼어낸 뒤 음색만 바꿔 끼우면 같은 말을 다른 사람 목소리로 얻습니다.

그리고 이 관점이 두 갈래 구조를 설명합니다.

표 3.2 구조 · 하는 일 · 성격

구조	하는 일	성격
통합형 (zero-shot TTS)	텍스트 + 참조 음성 → 소리를 처음부터 만든다	한 모델이 다 한다
분리형 (voice conversion)	일단 소리를 만들고, 음색만 갈아 끼운다	기본 TTS + 변환기

분리형은 톤 컬러 컨버터 같은 이름으로 불리는 층을 따로 둡니다. 이미 있는 TTS의 출력을 받아 음색만 바꾸므로, 기본 TTS를 자유롭게 고를 수 있다는 것이 장점입니다. 한국어가 약한 클로닝 모델을 만났을 때 — 한국어가 좋은 TTS + 음색 변환기로 우회할 수 있습니다.

이 우회로를 아는 것이 실무에서 중요합니다. 모델 하나가 한국어와 클로닝을 동시에 잘하기를 기다릴 필요가 없습니다.

제로샷과 파인튜닝 사이에 한 칸이 더 있습니다

위 표는 양 끝만 보여줍니다. 실제로는 셋입니다.

표 3.3 제로샷 · 소량 파인튜닝 · 전체 파인튜닝

	제로샷	소량 파인튜닝	전체 파인튜닝
데이터	6~30초	몇 분	수십 분~몇 시간
학습	없음	가볍게 (LoRA 등)	본격적
언제	대부분	제로샷이 "살짝 아쉬울" 때	성우 전용·상용 배포

가운데 칸이 실무에서 가장 자주 쓰이는데 가장 덜 알려져 있습니다. 제로샷 결과가 쓸 만한데 어딘가 어색할 때 — 특정 발음이 뭉개지거나 억양이 미묘하게 다를 때 — 몇 분짜리 데이터로 가볍게 얹으면 상당히 올라갑니다.

Ch03A의 사투리 다섯 개가 이 칸의 학습 방식(LoRA)입니다 — 데이터는 그보다 훨씬 큼니다. 베이스 모델은 그대로 두고 22.5MB짜리 어댑터만 학습했습니다. 전체 파인튜닝이었다면 팔도에 15.0GB가 필요했을 것입니다.

감정을 넣는 방법은 셋입니다

앞에서 지시문 방식과 참조 음성 방식을 봤는데, 정리하면 이렇습니다.

표 3.4 방식 · 어떻게 · 특징

방식	어떻게	특징
참조 기반	그 감정으로 말한 샘플을 준다	가장 자연스럽다 · 샘플을 구해야 한다
레이블 기반	기쁨 같은 값을 따로 넣는다	재현성이 좋다 · 모델이 지원해야 한다
텍스트 지시	태그나 SSML로 지시한다	손이 가장 덜 간다 · 결과가 흔들린다

재현성이 필요하면 레이블, 자연스러움이 필요하면 참조입니다. 태그 방식은 편하지만 같은 지시에 같은 결과가 나오지 않아서 최종본은 여러 번 뽑아 고르게 됩니다(§ 2 감정 제어에서 본 그대로입니다).

그리고 태그 방식에는 함정이 하나 더 있습니다 — **모델이 태그를 지시로 안 읽고 그냥 읽어 버리면 소리가 납니다.** § 4의 정규화가 그래서 필요합니다.

좋은 참조가 좋은 결과입니다

참조 음성을 고를 때 지킬 것 셋입니다.

깨끗해야 합니다. 배경 소음, 음악, 다른 사람의 목소리가 섞이면 지문이 흐려집니다.

합성할 언어와 같은 언어로 녹음된 것 을 쓰세요. 영어 참조로 한국어를 합성하면 발음이 어색해집니다.

너무 짧지 않게. 6초면 된다고들 하지만, 20~30초 쪽이 안정적입니다.

지연

실시간 트랙에서만 문제가 됩니다. 첫 소리가 나기까지의 시간이 곧 사용자가 느끼는 반응 속도입니다. 로컬 모델이 API보다 항상 빠른 것은 아닙니다. 모델 로딩과 추론이 네트워크 왕복보다 느린 경우가 흔합니다.

3.3 네 가지 선택지 — 무엇을 언제 쓰나

표 3.5 방식 · 품질(한국어) · 한도 · 감정 제어 · 지연

방식	품질(한국어)	한도	감정 제어	지연	쓰는 곳
클라우드 무료 TTS	보통	사실상 무제한	속도·음높이	0.7초/문장(첫 청크 0.5초 — 중앙값, 실측 0.24~0.69초)	개발 기본값 · 실시간 트랙
프리미엄 LLM TTS	매우 좋음	무료는 하루 십여 회	자연어 지시	중간	최종 출력 · 감정이 핵심일 때
로컬 다국어 TTS	보통~좋음	무제한	제한적	1.4초/문장(RTF 0.62 · 적재 39초)	오프라인 · 외부 전송 금지 환경
음성 클로닝	모델마다 편차 큼	무제한	참조 음성	높음	동의 확보 후에만

지연 칸의 두 숫자는 같은 한국어 문장 다섯으로 켄 중앙값입니다(부록 C § 6). 나머지 둘은 정성 표기입니다.

로컬 클로닝을 살리기까지 — 시행착오와 운영 규칙 ★

폐기와 채택 사이에 있었던 일을 순서대로 적습니다. 목소리를 만드는 일은 모델 선택보다 운영 규칙에서 같았습니다.

첫 비교는 감정 4종 매트릭스였습니다. 2026년 6월, 중립·기쁨·분노·슬픔 네 대사를 XTTS 제로샷 · XTTS 파인튜닝 · OpenVoice 세 구성으로 각각 합성해 들었습니다. 판정은 **귀로** 내렸습니다 — 한국어에서 끝음질이 늘어지고 호흡이 붙는다는 것. 영어 평판이 좋은 모델이라도 한국어는 따로 들어 봐야 합니다.

그리고 여기서 제가 한 번 속았습니다. 두 모델의 출력 파일 크기가 2.2배 차이 나기에 *"XTTS가 그만큼 길게 늘어 읽는다"*고 적었습니다. 나중에 길이를 재 보니 **14.48초와 14.49초 — 같았습니다.** 2.2배는 길이가 아니라 형식이었습니다. XTTS는 32비트 실수 24kHz, OpenVoice는 16비트 정수 22.05kHz로 저장하고 있었습니다(4바이트 대 2바이트). **파일 크기로 길이를 추정하지 마세요.** 그리고 그 32비트 실수 WAV가 이 절 뒤에서 다시 나옵니다 — 일부 브라우저가 재생하지 못하는 바로 그 형식입니다.

채택한 다국어 클로닝 모델은 버전이 아니라 커밋으로 고정했습니다. 패키지 저장소판은 구형 체크포인트만 읽어 **한국어 입력에 영어로 말했습니다.** 저장소의 특정 커밋 + 새 텍스트 모델(v3)에서만 한국어가 정상이었고, 그 조합을 요구 파일에 커밋 해시로 박았습니다.

그런데 설치기가 "이미 설치됨"으로 속아 새 버전을 안 깔았습니다. 그래서 서버가 뜰 때 함수 서명을 검사해, 다르면 강제로 재설치하고 스스로 재시작하게 했습니다(Ch10 § 4의 격리 원칙이 서버에서 취하는 형태).

폭주 감지. 클로닝 모델은 가끔 한 문장을 끝없이 늘어 읽습니다. 규칙은 하나입니다 — 생성된 오디오가 $\max(6\text{초}, \text{글자 수} \times 0.45\text{초})$ 를 넘으면 그 문장을 한 번 다시 만듭니다. 한국어 발화 속도의 상한을 글자 수로 잡은 것이고, 실시간 스트리밍 경로(Ch21)에서 이 한 줄이 "혼자 떠드는 문장"을 막았습니다.

감정 노브는 청취 A/B로 네 값만 남겼습니다. 모델의 두 노브(감정 강도 · 표현력 $\leftrightarrow$ 정확성)를 뉴스 0.25/0.6 · 대화 0.55/0.4 · 광고 0.95/0.3 · 드라마 1.4/0.25로 고정하고 이름을 붙였습니다. 사용자는 숫자가 아니라 *뉴스처럼 · 드라마처럼* 을 고릅니다(Ch03A §5의 프리셋과 같은 사상).

목소리 유출 방지. 참조 음성으로 만든 화자 조건은 해시로 캐시합니다(같은 참조면 재계산 없음). 그런데 그 캐시가 *다음* 호출에 남으면, 참조 음성 없이 부른 사람이 **직전 사용자의 목소리**로 답을 듣게 됩니다. 참조가 없는 호출은 반드시 기본 목소리로 되돌리도록 못박았습니다. 부록 G §5의 요건 중 하나입니다.

동의와 워터마크는 코드가 강제합니다. 클로닝 프리셋은 동의 플래그가 없으면 요청을 거절하고, 출력에는 모델 내장 워터마크를 엽니다. 다만 워터마크 패키지가 깨진 환경에서는 조용히 *무표시*로 떨어져 **워터마크 없는 출력이 나옵니다** — 워커를 새로 세우면 워터마크가 실제로 박히는지 먼저 확인해야 합니다(Ch29 §4).

서비스로 내보낼 때의 인터페이스는 이렇게 됩니다. 저자의 서비스에서 음성 기능이 취한 모양입니다. 사용자는 문장을 넣습니다. 목소리 파일(5~15초)을 함께 넣으면 그 목소리로 복제됩니다. 생성은 문장 단위로 흘러나옵니다. 코드 0줄(주소창), 3줄(파이썬), 실시간(웹소켓·초당 과금) 세 층입니다.

GET /tts/{배포id}?text=안녕하세요	← 주소창에 쳐도 소리가 난다
from atl_voice import Voice	
Voice().say("안녕하세요!")	← 문장 단위 스트리밍 재생
Voice(voice="내목소리.mp3").say("내 목소리로!")	← 참조 음성 5~15초로 클로닝
Voice().chat("오늘 뭐 먹을까?")	← LLM 답을 만들어 그대로 음성으로
wss://.../ws/talk/{배포id}	← 대화형 세션 · 초당 과금

세 층의 요점은 **가장 쉬운 경로가 0줄**이라는 것입니다. 클로닝을 쓰는 사람 대부분은 파이썬을 모릅니다. 그리고 실시간 층만 초당 과금인 이유는 Ch28 §6에 있습니다 — 세션이 GPU를 연속으로 점유하기 때문입니다.

출력은 16비트 PCM으로. float32 WAV는 일부 브라우저·플레이어에서 재생되지 않고 파이썬의 wave 모듈도 못 읽습니다. 서버가 뭘 내보내든 마지막에 16비트로 바꾸세요.

그리고 사고 하나. 이미지 생성 프리셋을 위해 붙여 둔 한→영 자동 번역이 TTS 프리셋의 대사가까지 번역했습니다. 한국어를 넣었는데 영어로 말하는 "미스터리"가 하루를 잡아먹었습니다. TTS·읽기형 프리셋은 번역 제외 목록에 등록하고, 이름 접두로 자동 제외되게 구조를 바꿨습니다 — 같은 사고가 방언 프리셋을 새로 만들 때 한 번 더 났기 때문입니다(부록 F 45).

라이선스를 함께 보세요. 오픈소스 음성 클로닝 계열은 연구용 라이선스가 흔합니다. 성능이 좋아서 골랐다가 상업 배포 단계에서 못 쓰게 되는 일이 자주 있습니다. 상업 제품이라면 **라이선스가 명확한 것을 먼저 후보에 두세요.** 라이선스와 별개로, 실존 인물의 목소리를 복제한다면 Ch29의 동의 요건이 그대로 적용됩니다.

여기에 다섯 번째 축이 하나 더 붙었습니다 — 텍스트를 거치는가. 음성을 직접 받아 음성을 내는 모델(S2S)이 있고, 지연이 크게 줄어듭니다. 대신 중간에 텍스트가 없어서 이 책의 여러 장치가 동작하지 않습니다. Ch07 §9에서 자세히 다룹니다. **얼굴이 있는 아바타를 만든다면 일단 이 장의 일반 TTS로 시작하세요.**

권장 조합은 이렇습니다. 개발 내내 클라우드 무료 TTS로 수백 번 돌리고, 최종 렌더 한 번만 프리미엄으로 뽑습니다. 실시간 트랙은 처음부터 끝까지 무료 TTS로 갑니다 — 실시간에서는 품질보다 끊기지 않는 것이 압도적으로 중요합니다.

상용으로 내보낼 때 — 측정용 목소리와 판매용 목소리는 다릅니다 ★

이 책의 실시간 실측(Ch21 · 부록 C)은 `edge-tts` 로 했습니다. 빠르고 무료이고 한국어 목소리가 좋아서 **측정과 수업에는 최선**입니다.

그런데 이것은 브라우저의 '소리 내어 읽기' 기능을 흉내 내는 **비공식 클라이언트**입니다. 판매하는 제품에 넣으면 어느 날 조용히 끊기거나, 약관 문제가 됩니다. 저자는 개발·측정에만 쓰고 출하 물에는 넣지 않습니다.

표 3.6 같은 목소리, 다른 경로 — 출하할 때 바꾸는 것

단계	쓰는 것	이유
개발·측정·수업	<code>edge-tts</code>	설치 0, 무료, 첫 체크 0.24~0.69초(Ch21 §5)
출하 — 클라우드	같은 목소리의 공식 API(Azure Speech) 또는 Gemini TTS	약관과 SLA가 있다. 목소리 이름이 같아 코드는 어댑터 한 장만 바뀐다
출하 — 셀프호스트	Chatterbox(§3의 클로닝 서버) 또는 한국어 되는 오픈 모델	초당 과금이 없다. 대신 GPU 상주(부록 N §5)

어댑터 한 장으로 갈아 끼우게 만들어 두세요. 컴패니언 `ch21_realtime/server.py` 의 `tts()` 가 그 자리입니다 — 함수 하나가 목소리 엔진을 감싸고 있어서, 출하 때 바뀌는 것은 그 함수 안쪽뿐입니다. 측정에 쓴 부품을 그대로 출하하는 실수는 TTS에서 가장 자주 납니다. 무료라서요.

3.4 대본 쓰는 법 — 엔진보다 먼저 고칠 것

TTS 품질의 절반은 엔진이 아니라 대본에서 나옵니다.

문장을 짧게 쓰세요. 긴 문장은 어느 엔진에서든 호홉이 무너집니다. 실시간 트랙이라면 두 문장을 넘기지 않는 것이 좋습니다.

숫자와 영어를 한글로 바꿔 두세요. 엔진마다 읽는 방식이 다릅니다. 다국어 클로닝 모델(Chatterbox 계열)에는 이 앞단 — 글자를 읽는 대로 고쳐 주는 텍스트 정규화(TN) — 이 아예 없어서 "AI"를 "아이"로, "3.5"를 제멋대로 읽습니다.

저자의 음성 서비스에서 그 앞에 붙여 운영하던 규칙을 컴패니언 `normalize_ko.py` 로 옮겼습니다. 순서가 중요한 다섯 단계입니다.

표 3.7 텍스트 정규화 다섯 단계 — 실제로 틀리게 읽었던 것들(test_normalize.py가 전부 검사)

단계	규칙	예
① 십표·기호	천 단위 십표 제거, % → 퍼센트	1,000원 → 천원 · 50% → 오십퍼센트
② 단위 사전	km·kg·cm·mm·ml·kb·mb·gb·tb	5km → 오 킬로미터
③ 로마자 약어	대문자 연속은 낱자 이름으로	AI → 에이아이 · USB → 유에스비
④ 수분류사 분기	개·명·마리·살·시간… 앞 1~99는 고유어 , 나머지는 한자어	10개 → 열 개 · 20살 → 스무 살 · 15분 → 십오분
⑤ 남은 숫자	만 단위 묶음, 소수는 '점', 1만 은 '만'	3.5배 → 삼점오배 · 1만 명 → 만 명

④ 가 한국어 특유의 함정입니다. 같은 숫자가 뒤에 오는 낱말에 따라 **열 개**도 되고 **십 분**도 됩니다. 수분류사 목록이 곧 정확도입니다. 목록에 없는 낱말(원·년·월·배·초)은 한자어로 떨어지게 두고, **개월**·**번지**처럼 앞글자가 겹치는 것은 부정 전방탐색으로 걸러 둡니다(10개월 → 십개월).

영단어(cloud → 클라우드)까지 음차하려면 g2pk의 english 모듈이 필요합니다. 그런데 풀체인이 형태소 분석기(mecab)에 묶여 있어 Windows 위커에는 못 갑니다 — mecab 없는 모듈만 쓰고, 없으면 건너뛩니다.

이모지와 특수문자를 제거 하세요. 엔진이 읽어버리거나 침묵으로 처리하는데, 어느 쪽인지 예측할 수 없습니다. LLM이 대본을 만드는 구조라면 시스템 프롬프트에 "이모지나 특수문자는 쓰지 마세요"를 명시하고, 그래도 나오면 코드에서 한 번 더 걸러냅니다.

```
t = re.sub(r"^[가-힣a-zA-Z0-9 .,!~]", " ", t)
t = re.sub(r"\s+", " ", t).strip()
```

이 한 줄만 쓰면 태그가 소리로 새어 나갑니다

위 필터는 **허용 문자만 남기는** 방식입니다. 그래서 `[excited][wave]` 안녕하세요 를 넣으면 대괄호만 사라지고 이렇게 됩니다.

```
excited wave 안녕하세요
```

TTS는 이것을 "익사이티드 웨이브 안녕하세요" 라고 읽습니다. Ch20 §4가 경고한 태그 유출의 변종 인데, 여기 적힌 정규화로는 막히지 않습니다.

순서를 하나 더 뒤야 합니다.

```
t = re.sub(r"^s*(?:[\\w+\\s*]+", "", t)    # ① 맨 앞 태그를 내용째 제거
t = re.sub(r"^[^\\]{1,20}\\]", " ", t)      # ② 중간에 긴 것까지
t = re.sub(r"^[가-힣a-zA-Z0-9 .,!?~]", " ", t) # ③ 그 다음 문자 필터
```

태그는 대괄호가 아니라 통째로 지워야 합니다. 문자 필터를 먼저 돌리면 이미 늦습니다 — 대괄호가 사라진 뒤에는 태그였는지 본문이었는지 구분할 방법이 없습니다.

문장 부호로 호흡을 만드세요. 쉼표 하나가 실제로 짧은 멈춤을 만듭니다. 마침표는 더 긴 멈춤입니다. 추모 영상처럼 호흡이 중요한 대본은 문장 부호로 리듬을 설계할 수 있습니다.

3.5 감정을 넣는 두 개의 경로

감정은 두 경로로만 들어갑니다. 이 두 개가 전부입니다.

첫째, 목소리 — 전체의 80%입니다. 자연어 지시가 되는 엔진이라면 스타일 문장을 앞에 붙입니다. 파라미터만 되는 엔진이라면 속도를 15% 정도 늦추고 음높이를 낮추는 것으로 슬픔을 근사합니다.

둘째, 얼굴 표정 — 나머지 20%입니다. 여기에는 중요한 한계가 있습니다. **얼굴에는 "슬픔 강도 다이얼"이 없습니다.** 립싱크 모델은 소리에 맞춰 입을 움직일 뿐, 슬프게 움직이지 않습니다.

그래서 Track A에서는 표정을 *드라이버 영상 선택* 으로 간접 제어합니다. 차분한 화자를 드라이버로 쓰면 결과도 차분해지고, 밝은 화자를 쓰면 결과도 밝아집니다. Track B에서는 반대입니다. **표정이 완전히 코드 제어** 라서 감정 파라미터를 직접 줄 수 있습니다. 이것이 두 트랙의 중요한 성격 차이입니다.

3.6 검증 — 세 가지만 확인하세요

들어보세요. 당연한 말 같지만, 파이프라인을 짜다 보면 파일이 생겼는지만 보고 넘어가게 됩니다. 무음 파일도 아무 오류 없이 잘 생성됩니다.

길이를 재세요. 초 단위로 기록해 둡니다. 이 값이 영상 길이와 맞지 않으면 뒤로 갈수록 싱크가 밀립니다. Ch14 전체가 이 문제를 다룹니다.

샘플레이트와 채널을 확인하세요. 립싱크 모델은 대부분 16kHz 모노를 기대합니다. 44.1kHz 스테레오를 넣으면 조용히 이상한 결과가 나옵니다 — 에러가 나지 않아서 더 위험합니다. ffmpeg으로 한 번 정규화해서 넘기는 습관을 들이세요.

왜 하필 16kHz인가

이 책에서 16kHz가 계속 나오는데, 임의로 정한 숫자가 아닙니다.

사람 목소리의 중요한 성분은 대부분 8kHz 아래에 있습니다. 그리고 **표본화 정리(나이퀴스트)**는 **샘플레이트의 절반까지** 표현할 수 있다고 말합니다. 그러니 8kHz를 담으려면 16kHz로 뜨면 됩니다.

표현 가능한 최대 주파수 = 샘플레이트 ÷ 2
 16,000 ÷ 2 = 8,000Hz ← 음성에 필요한 만큼

44.1kHz는 음악용입니다. 심벌즈의 15kHz를 담으려고 그렇게 뜨는 것이고, 음성 인식·립싱크에는 **그 위쪽이 아무 일도 하지 않습니다**. 파일만 세 배 무거워집니다.

여기서 따라 나오는 실무 지침이 둘입니다.

올려도 없던 정보는 안 생깁니다. 전화 녹음(8kHz)을 16kHz로 업샘플해도 4kHz 위는 여전히 비어 있습니다. 숫자만 맞고 내용은 그대로입니다. 참조 음성이나 드라이버 오디오를 고를 때 **원본이 몇 kHz로 녹음됐는지** 를 보세요.

내려가는 것은 안전합니다. 48kHz 스튜디오 녹음을 16kHz로 줄이는 것은 잃을 것이 없는 작업입니다 — 버리는 대역에 음성이 없기 때문입니다.

3.7 이 장에서 기억할 것

목소리가 감정의 80%이고, 입은 소리를 따라갑니다. 그래서 목소리를 먼저 만듭니다.

개발용과 최종 출력용 엔진을 분리 하세요. 프리미엄 엔진의 무료 한도는 개발 중에 만나절이면 소진됩니다.

한국어 품질은 **직접 한국어로 들어보고** 판단합니다. 영어 데모는 근거가 아닙니다.

대본은 짧게, 숫자·영어는 한글로, 이모지는 제거. TTS 품질의 절반은 대본에서 나옵니다.

오디오는 **16kHz 모노로 정규화** 해서 다음 층에 넘기고, **길이를 초 단위로 기록** 해 둡니다.

다음 장은 얼굴입니다. 좋은 소스 이미지와 좋은 드라이버 영상은 완전히 다른 조건을 요구하는데, 이 둘을 혼동하는 것이 초심자의 가장 흔한 실수입니다.

실습 코드: code/ch03_tts/ — TTS 4종 비교 스크립트 + 대본 정규화 + 오디오 검증

예상 소요: 1시간

관련 부록: C(벤치마크), G(음성 복제 윤리)

CHAPTER 3A

사례 — 사투리 다섯 개를 만든 이야기

앞 장에서 목소리 층을 세웠습니다. 이 장은 그 층으로 실제로 무엇을 만들 수 있는지 를 하나의 사례로 보여 줍니다. 저자가 2026-07-29 저녁부터 다음 날 저녁까지 하루에 학습해 배포한 것이고, 수치는 전부 실측·실운영 값입니다.

뒤의 여러 장을 미리 건드립니다 — 파인튜닝 시점(Ch22) · 원가(Ch05) · 분산 GPU(Ch28 §6).

지금은 "이런 것이 가능하다"만 보고 넘어가도 됩니다. 각 주제는 해당 장에서 다시 만납니다.

3A.1 무엇을 만들었나

한국어 TTS에 지역 사투리 를 입혔습니다. 표준어 문장을 넣으면 그 지역의 억양과 어미 로 말합니다.

표 3A.1 코드 · 지역 · 어댑터 · 학습 데이터

코드	지역	어댑터	학습 데이터
gs	경상도	cbx_gyeongsang_0730_0020	AI-Hub 방언 15,296 발화
jl	전라도	cbx_jeolla_0730_1147	AI-Hub 방언 12,025 발화
gw	강원도	cbx_gangwon_0730_1345	AI-Hub 방언 데이터
cc	충청도	cbx_chungcheong_0730_1536	AI-Hub 방언 데이터
jj	제주도	cbx_jeju_0730_1704	AI-Hub 방언 데이터

어댑터 이름의 뒤 네 자리가 학습 완료 시작입니다. 00:20부터 17:04까지, 하루에 다섯 개 를 뽑았습니다.

베이스 모델은 MIT 라이선스의 오픈소스 TTS 이고, 그 위에 LoRA 파인튜닝 — 원본은 건드리지 않고 얇은 층 하나만 새로 학습하는 방식 — 을 엮었습니다. 라이선스 표기는 MIT + 자체 LoRA(AI-Hub 방언 데이터 학습 — 지능형 서비스 영리활용 허용) 입니다. 온라인 부록 K §5의 "라이선스가 성능보다 먼저"가 실제로 적용된 모습입니다.

3A.2 이 사례가 증명하는 것 셋

① 사투리는 파인튜닝이 필요한 대표 사례다

Ch22 § 8은 프롬프트의 한계가 또렷해지는 자리 셋을 들면서 그 예로 사투리를 꼽았고, 그 예로 사투리를 들었습니다. 이 사례가 그 이유를 보여줍니다.

사투리는 어휘의 문제가 아니라 억양의 문제입니다. LLM에게 "경상도 사투리로 말해"라고 지시하면 *어미* 는 바뀌 줍니다. 그런데 그 텍스트를 표준어 TTS에 넣으면 **표준어 억양으로 사투리 단어를 읽습니다**. 아무도 경상도 사람이라고 듣지 않습니다.

억양은 음성 모델 안에 있습니다. **텍스트 층에서 고칠 수 없습니다**. 그래서 음성 모델을 건드려야 하고, 그것이 파인튜닝입니다.

층을 기억하세요(Ch01). 사투리 문제는 *머리* 층이 아니라 *목소리* 층의 문제입니다. 층을 잘못 짚으면 아무리 프롬프트를 고쳐도 해결되지 않습니다.

② 파인튜닝이 생각보다 싸다

Ch03 § 2의 제로샷/파인튜닝 표에서 파인튜닝을 "시간·비용 큼"으로 적었습니다. 전체 모델을 다시 학습한다면 그렇습니다.

LoRA는 다릅니다. 베이스 모델(Chatbox 다국어, 실측 3.0GB — 파라미터 2.99GB · 적재 VRAM 3.00GB)은 그대로 두고 그 위에 얇은 층만 학습합니다. 산출물은 **어댑터 22.5MB** — 베이스의 **0.7%** 입니다.

그래서 클라우드 GPU 한 대를 하루 빌려 **다섯 지역을 전부** 뽑을 수 있었습니다.

주의 — 주석과 실물이 다를 때 학습 스크립트 주석에는 산출물이 ~4MB로 적혀 있습니다. 그건 초기 설계 목표였고, **실제로 배포된 어댑터는 22.5MB** 입니다(§ 4에서 설정을 키웠기 때문). 코드 주석과 실물이 다를 때는 **실물을 믿으세요**.

주석이 낡은 이유까지 확인했습니다. 저장소에 남아 있는 콜랩 학습 스크립트는 **아직 $r=16$, $\alpha=32$, $q_proj \cdot v_proj$** 입니다 — § 4에서 설정을 키운 뒤에도 스크립트를 되돌려 고치지 않은 것입니다. 주석은 **코드와 함께 늙습니다**. 그리고 그 초기 설정의 실제 산출물조차 4MB가 아니라 **7.5MB** 였습니다(§ 4 표). 주석은 두 번 틀렸습니다.

③ 데이터를 로컬로 내리지 않는다

AI-Hub 방언 데이터는 원천만 수십 기가바이트입니다(경상도 Validation 원천 약 3.1GB — 2026-09 디스크 실측 3.16GiB, 본학습용 Training 원천 23.2GB).

이걸 로컬로 받으면 안 됩니다. 학습 VM이 API로 직접 받고, 세션이 끝나면 원본은 함께 사라집니다. **밖으로 나오는 것은 22.5MB 어댑터뿐**입니다.

압축본만 드라이브에 캐시해 두면 세션이 재시작돼도 재다운로드가 0입니다. 소파일 I/O는 느리므로 **압축본만** 두고 실행 때 로컬에 풀었습니다.

3A.3 어댑터 핫스왑 — 이 사례의 핵심 ★

지역별로 배포를 다섯 개 띄우면 어떻게 될까요.

경상 3.0GB + 전라 3.0GB + 강원 3.0GB + 충청 3.0GB + 제주 3.0GB = 15.0GB

12GB 카드 한 장에 다섯을 다 올릴 수 없습니다 — 세 지역에서 한계입니다. Ch05 결정표의 "동시 사용자 = GPU 대수"가 여기서 그대로 나타납니다.

해법은 베이스 모델 하나에 어댑터 다섯을 동시에 얹는 것 입니다. 요청에 담긴 `dialect` 값으로 활성 어댑터만 전환합니다.

기본 모델 1개(3.0GB) + LoRA 5개(각 22.5MB = 112.5MB) ≈ 3.1GB 가중치
실제 VRAM 은 생성 중 피크로 3.2GB(실측, reserved 기준)

표 3A.2 개별 배포 · 핫스왑

	개별 배포	핫스왑
VRAM	15.0GB	3.2GB
12GB 카드	셋이 한계	다섯 지역 전부
전환 비용	배포 교체	즉시 · 오버헤드 거의 0

4.7배 절감이고, 12GB 카드 한 장에 다섯 지역이 전부 올라갑니다.

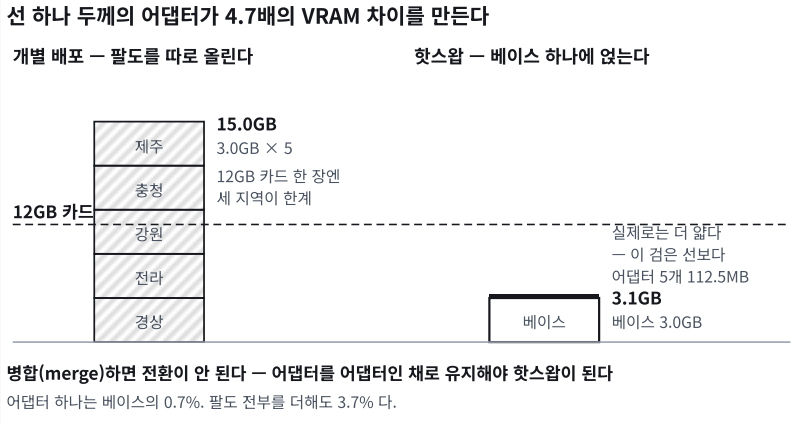


그림 3A.1 어댑터 다섯을 다 더해도 다섯을 합쳐도 베이스의 3.7% — 그 차이가 12GB 카드 한 장 안에 들어가는냐를 가른다

구현에서 딱 하나 조심할 것

LoRA를 쓸 때 흔히 어댑터를 베이스에 병합(merge) 합니다. 추론이 조금 빨라지기 때문입니다.

병합하면 전환이 안 됩니다. 병합은 어댑터를 가중치에 녹여 없애는 동작이라, 다시 떼어내거나 다른 것으로 바꿀 수 없습니다.

어댑터를 어댑터인 채로 유지해야 핫스왑이 됩니다. 이 한 줄이 15.0GB와 3.2GB를 가릅니다.

실제 배포 코드가 하는 일

로드는 두 단계입니다. 첫 어댑터는 모델을 감싸면서 이름을 붙이고, 나머지는 그 위에 이름만 추가했습니다.

```
첫 번째 : t3 = PeftModel.from_pretrained(t3, dir, adapter_name="gs")
나머지 : t3.load_adapter(dir, adapter_name="j1") # cc, gw, jj ...
```

그리고 요청마다 한 줄입니다.

```
t3.set_adapter(요청의_dialect)
```

여기에 실무적으로 중요한 장치가 셋 더 있습니다.

어댑터 하나가 실패해도 나머지는 서빙됩니다. 로드를 개별적으로 감싸고, 실패한 것만 로그를 남기고 넘어갑니다. 제주 어댑터가 깨져도 경상·전라는 계속 돕니다. Ch28A §3의 항목별 격리 원칙과 같은 원칙입니다.

적용된 어댑터를 응답에 되돌려줍니다. 요청한 방언과 실제 적용된 방언이 다를 수 있기 때문입니다 — 없는 코드를 보냈거나 전환이 실패했으면 기본 목소리로 나갑니다. 클라이언트가 그것을 알 수 있어야 합니다.

응답에는 **생성 소요 시간과 오디오 길이가 함께 들어 있습니다.** 둘의 비가 곧 실시간 배율(Ch06)이고, 이 값이 API 응답에 있으면 별도 계측 없이 지연을 추적할 수 있습니다. Ch28A §4의 품질 방에 그대로 꽂힙니다.

관측을 나중에 붙이지 마세요. 어느 어댑터가 적용됐는지, 얼마나 걸렸는지를 **응답에 처음부터 넣어 두면** 운영 콘솔은 그것을 모으기만 하면 됩니다.

3A.4 학습 절차 — 먼저 실패를 확인한다

학습 스크립트의 1차 실행은 **모델을 만들지 않습니다.** 파이프라인이 도는지만 봅니다.

```
표본 50개 → 스피치 토큰 추출 → LoRA 100스텝 과적합 → 손실이 내려가는지 확인
```

이것을 **샤크다운(shakedown)** 이라고 부릅니다. 일부러 과적합시키는 이유는, 손실이 안 내려가면 데이터나 파이프라인이 잘못된 것 이기 때문입니다. 50개로 100스텝을 돌려서 손실이 내려가지 않으면, 전체 데이터(만 단위)로 며칠을 돌려도 내려가지 않습니다.

몇 분짜리 실패를 먼저 확보하는 것 이 며칠을 아깁니다. Ch02 § 4의 스모크 테스트와 같은 사상입니다 — 자기 데이터로 시작하지 말고, 되는지부터 확인하세요.

레시피는 첫 지역에서만 찾습니다 ★

어댑터 파일에 남은 기록이 학습 과정을 그대로 보여줍니다.

표 3A.3 시각 · r · alpha · target modules · 크기

시각	r	alpha	target modules	크기
7/29 20:52	16	32	q_proj, v_proj	7.5MB
7/29 21:00	16	32	q_proj, v_proj	7.5MB
7/29 22:16	16	32	q_proj, v_proj	7.5MB
7/30 00:20	32	64	q_proj, v_proj, o_proj	22.5MB

이 표는 산수로 계산됩니다. r 을 16에서 32로 두 배, 대상 모듈을 둘에서 셋으로 늘렸으니

$$7.5\text{MB} \times (32/16) \times (3/2) = 22.5\text{MB}$$

정확히 맞습니다. 자기가 기록한 숫자를 산수로 한 번 맞춰 보는 습관 이 표를 신뢰할 수 있게 만듭니다. 맞지 않으면 둘 중 하나가 틀린 것이고, 그때 알아채는 편이 낫습니다.

경상도 하나에 네 번 돌렸습니다. 낮은 랭크로 세 번 시도한 뒤 랭크를 두 배로 올리고 출력 프로젝션을 추가 해서 본학습을 했습니다.

그리고 그 설정으로 나머지 넷을 갖습니다.

7/30 11:47 전라 · 13:45 강원 · 15:36 충청 · 17:04 제주

측정 가능한 세 구간이 1시간 28분~1시간 58분(평균 1시간 46분), 전부 한 번에. 재시도가 없습니다.

비용 구조가 이렇습니다. 첫 지역은 레시피를 찾는 비용 이고, 나머지는 복제 비용 입니다. 첫 하나에 네 번을 쓰는 것이 낭비처럼 보이지만, 그 덕에 나머지 넷이 한 번에 끝났습니다. 파인 튜닝 프로젝트의 견적을 낼 때 이 비대칭을 반영하세요 — "다섯 개니까 다섯 배"가 아닙니다.

확정된 설정

```
PEFT LoRA · r=32 · alpha=64 · dropout=0
target: q_proj, v_proj, o_proj      (어텐션 프로젝트션)
대상 모듈: 텍스트→스피치토큰 모델
```

보코더가 아니라 텍스트→토큰 단계에 겁니다. 사투리의 억양은 어떤 소리를 낼지 정하는 단계에서 결정되기 때문입니다. 소리를 파형으로 바꾸는 단계는 건드리지 않습니다.

$\alpha = 2r$ 는 관행적인 기본값이고, dropout 0은 데이터가 충분할 때의 선택입니다.

레시피 안의 두 결함 — 손실 함수와 화자 조건 ★

설정표보다 중요한 것이 이 둘이었습니다. 둘 다 들어 보고 찾았습니다.

첫째, 손실 함수가 상류에서 틀려 있었습니다. 텍스트→스피치토큰 모델의 손실 계산에 두 가지가 빠져 있었습니다. 로짓을 (배치, 길이, 어휘) 모양 그대로 교차 엔트로피에 넣어 축이 어긋났습니다. 그리고 타깃을 한 칸 밀지 않아 모델이 다음 토큰이 아니라 같은 자리를 베끼도록 학습됐습니다.

손실은 내려가는데 추론에서 폭주하고 웅얼거리는 이유가 그것이었습니다. 로짓을 전치하고 `logits[:, :-1]` 대 `tokens[:, 1:]` 로 맞춘 뒤에야 억양이 잡혔습니다. 손실은 스피치 손실 + 0.2 × 텍스트 손실로 두었습니다 — 발음·억양이 목표이지 텍스트 복원이 아니기 때문입니다.

둘째, 화자 조건을 한 사람 것으로 돌려줬습니다(v3). 8,000쌍을 한 화자의 조건으로 학습하자 억양이 뭉개졌습니다. 181명 화자마다 조건을 따로 만들어 캐시하고(v4) 15,296발화로 다시 돌리자 손실이 5.16 → 3.95로 내려갔고, 귀로도 달랐습니다. 아래가 그 뒤 지역별 결과입니다.

표 3A.4 지역별 학습 — 같은 레시피, 손실 시작 → 끝 (2026-07-30 학습 로그)

지역	데이터	손실
경상 (v4)	15,296발화 · 181화자 · 45,000스텝	5.16 → 3.95
전라	12,025발화	— (자동 수집이 502로 끊겨 수동 진행)
강원	AI-Hub 71517	5.03 → 3.57
충청	AI-Hub 71558	4.81 → 3.18
제주	AI-Hub 71558	5.31 → 2.91

A/B 청취 파일을 남겼습니다. `ab_train_base.wav` 대 `ab_train_lora.wav` (학습 문장), `ab_novel_base.wav` 대 `ab_novel_lora.wav` (처음 보는 문장). 손실 숫자는 두 번째 쌍이 다르게 들릴 때만 뜻이 있습니다.

실전 메모

처음 계획은 콜랩이었습니다. 그런데 AI-Hub가 해외 IP를 막아(국외 반출 정책) 내려받기 자체가 안 됐습니다. 그래서 로컬 4070에서 샤크다운을 돌리고 레시피를 확정했습니다 — 100스텝에 약 9초.

데이터셋 목록 API도 502로 자주 죽었습니다. 데이터셋 키를 기본값으로 고정 하고 자동 탐색은 폴백으로 돌렸습니다. 외부 API에 의존하는 자동화는 반드시 수동 지정 경로를 남겨 두세요.

3A.5 감정 노브 프리셋

같은 엔진의 감정 제어는 노브 두 개입니다. **exaggeration** (감정 강도, 0~2, 기본 0.5)과 **cfg_weight** (표현력↔정확성, 낮을수록 표현적).

A/B 청취로 확정한 조합입니다.

표 3A.5 프리셋 · exaggeration · cfg_weight · 쓰는 곳

프리셋	exaggeration	cfg_weight	쓰는 곳
news	0.25	0.60	뉴스·브리핑 — 차분·정확
talk	0.55	0.40	일상 대화 — 친근
ad	0.95	0.30	광고·홍보 — 밝은 에너지
drama	1.40	0.25	드라마·긴박 — 격정

두 값이 반대 방향으로 움직인다 는 점을 보세요. 감정을 올릴수록 정확성 가중치를 낮춥니다. 표현적일수록 원문에서 멀어지는 것을 허용한다는 뜻입니다.

자연어 지시보다 재현성이 좋습니다. 같은 값을 넣으면 같은 톤이 나옵니다(Ch03 §2의 파라미터 방식). 최종 렌더를 여러 번 뽑아 고르는 작업에서 이 차이가 큼니다.

3A.6 서비스로 내보낼 때

이 어댑터들은 판매 상품으로 등록 되어 있습니다. 최소 VRAM과 라이선스 문구가 프리셋에 함께 정의됩니다.

표 3A.6 항목 · 개별 지역 · 핫스압

항목	개별 지역	핫스압
최소 VRAM(카드 한 장)	5GB(지역 하나)	6GB(다섯 지역)

(최소 VRAM은 §3의 실측 피크(3.2GB)에 헤드룸을 더한 상품 정의값입니다.)

라이선스	MIT + 자체 LoRA	동일
------	---------------	----

최소 VRAM을 상품 정의에 넣는 것 이 실무 포인트입니다. 분산 GPU 환경(Ch28 §6)에서는 아무 워크에나 배정하면 안 되고, 이 값이 배정 조건이 됩니다.

모델 카드가 비어 있습니다

배포된 어댑터 안의 README.md 를 열어 보면, 학습 도구가 자동 생성한 빈 템플릿 그대로입니다. 학습 데이터도, 라이선스도, 한계도, 편향도 전부 [More Information Needed] 입니다.

기술적으로는 아무 문제가 없습니다. 모델은 잘 돌립니다.

그런데 이걸 Ch29의 문제입니다. 이 어댑터는 **상품으로 판매** 되고, 사는 사람은 그것이 무엇으로 학습됐는지 알 권리가 있습니다. 특히 사투리 모델은 **특정 지역 화자들의 목소리** 로 만들어졌습니다. AI-Hub 데이터는 영리 활용이 허용되지만, 그 사실을 **명시하는 것** 은 별개의 의무입니다.

최소한 이 다섯 줄은 채워야 합니다.

```
학습 데이터 : AI-Hub 중·노년층 한국어 방언 (dataSetSn 71517)
베이스 모델 : Chatterbox 다국어 (MIT)
라이선스 : MIT + 자체 LoRA - 지능형 서비스 영리활용 허용
용도 밖 사용 : 실존 인물 사칭 · 특정 지역 화자 희화화
알려진 한계 : 중·노년층 화자 중심 데이터 → 청년층 역양 재현 제한
```

마지막 줄이 특히 중요합니다. **학습 데이터가 중·노년층이면 모델도 중·노년층입니다.** 그것을 모르고 쓰면 "왜 우리 서비스 사투리는 다 나이 들어 보이지"라는 질문에 답할 수 없습니다.

모델 카드는 형식이 아니라 **인수인계 문서** 입니다. 반년 뒤의 자신도 사용자입니다.

3A.7 이 장에서 기억할 것

사투리는 목소리 층의 문제입니다. 프롬프트로 어미는 바뀌도 억양은 못 바꿉니다.

LoRA 산출물은 베이스의 0.7% 입니다. 파인튜닝이 비싸다는 통념은 전체 학습 기준입니다.

첫 지역은 레시피 비용, 나머지는 복제 비용입니다. 다섯 개라고 다섯 배가 아닙니다.

대용량 데이터는 학습 VM이 직접 받고, 밖으로는 어댑터만 나옵니다.

어댑터 핫스왑이 VRAM을 4.7배 줄입니다. 단, 병합하면 전환이 안 됩니다.

샷크다운을 먼저 하세요. 50개·100스텝으로 손실이 안 내려가면 전체 데이터로도 안 내려갑니다.

감정 노브는 A/B로 프리셋을 확정해 두세요. 재현 가능한 파라미터가 자연어 지시보다 실무에서 강합니다.

코드 주석과 실물이 다르면 실물을 믿으세요. 주석의 ~4MB와 배포된 22.5MB가 그 예입니다.

모델 카드를 비워 두지 마세요. 특히 판매하는 모델이라면. 학습 데이터의 성격이 곧 모델의 한계입니다.

관련 — Ch01(층) · Ch03(목소리·제로샷/파인튜닝) · Ch05(원가) · Ch22 § 8(파인튜닝 시점) · Ch28 § 6(분산 GPU) · 온라인 부록 K § 5(TTS 지형)

실습 코드: `code/ch03plus_dialect/` — `dialect` 라우팅(`router.py`) + 핫스왑 상태 검사(병합하면 전환 거절) · 베이스·가중치는 GPU 필요

예상 소요: 2시간 (학습 제외 · 서버 구성만)

관련 부록: C(실측) · N(원가)

CHAPTER 4

얼굴의 조건

4.1 얼굴은 둘입니다 — 하나는 보이고 하나는 안 보입니다

드라이버 영상을 고르는 가장 빠른 요령부터 알려 드립니다. 스톡 사이트에서 "의사" 나 "상담사"로 검색하세요.

농담이 아닙니다. 직업적으로 차분하게 정면을 보며 또박또박 말하는 사람들이기 때문입니다. 반대로 "행복한 사람"이나 "웃는 여성"으로 검색하면 표정이 과장된 영상이 나옵니다. 그런 영상을 쓰면 하늘이의 입꼬리가 좌우로 비틀립니다.

이 요령이 왜 통하는지가 이 장의 핵심입니다. **드라이버 영상은 결과물에 나오지 않습니다.**

Track A 파이프라인에는 얼굴이 둘 나옵니다. 처음 하는 사람이 가장 많이 헛갈리는 지점입니다. 여기서 헛갈리면 결과는 아무리 손봐도 좋아지지 않습니다.

소스 이미지는 결과물에 보이는 얼굴입니다. 여러분이 말하게 만들고 싶은 바로 그 얼굴입니다.

드라이버 영상은 움직임을 빌려주는 얼굴입니다. **결과물에는 전혀 보이지 않습니다.** 여기서 뽑아내는 것은 **움직임의 궤적** 뿐입니다 — 입이 어떻게 벌어지고, 눈이 언제 깜빡이고, 머리가 어떻게 기울는가.

이 한 줄이 실무를 바꿉니다. 드라이버 영상은 **외모도 배경도 성별도 인종도 상관없습니다.** 움직임만 중요합니다. 잘생긴 사람을 찾을 필요가 없습니다. 결과물의 얼굴과 닮은 사람을 찾을 필요는 더더욱 없습니다.

4.2 보이는 얼굴 — 좋은 소스 이미지의 조건

0.219와 0.621. 같은 사진에서 나온 두 값입니다. 자르기만 했습니다.

얼굴이 화면에서 차지하는 비율입니다. 소스 이미지의 조건은 대부분 이렇게 **어떻게 찍었는가**와 **어떻게 잘랐는가**에서 갈립니다. 하나씩 봅시다.

정면 — 옆얼굴은 돌릴 수 없습니다

옆을 보고 있는 얼굴은 정면으로 돌릴 수 없습니다. 모델은 3D를 복원하는 것이 아니라 2D를 변형합니다. 살짝 돌아간 정도는 괜찮지만, 프로필 사진은 포기하는 편이 빠릅니다.

밝고 고른 조명

한쪽만 어두운 얼굴은 입이 움직일 때 그림자 경계가 어색하게 따라옵니다. 역광은 최악입니다.

입이 살짝 벌어진 상태

의외의 조건입니다. 입을 **꼭 다문 사진은 입을 벌리기 어렵습니다**. 모델이 없는 정보를 지어내야 하기 때문입니다. 치아가 살짝 보이거나 입술이 조금 떨어진 사진에서 결과가 눈에 띄게 깨끗합니다.

입을 다문 사진밖에 없다면 길은 둘입니다. 모션 강도를 올려 강제로 벌리는 것 — 혀가 튀어나오는 부작용이 생길 수 있습니다 — 그리고 이미지 생성 모델로 입이 살짝 벌어진 변형을 만들어 쓰는 것입니다.

해상도는 생각보다 덜 중요합니다

대부분의 파이프라인은 얼굴을 크롭해서 $256 \times 256 \sim 512 \times 512$ 로 처리합니다. 4천만 화소 사진을 넣어도 그 해상도로 줄어듭니다. **얼굴이 화면에서 차지하는 비율이 해상도 자체보다 훨씬 중요합니다**. 전신 사진의 작은 얼굴보다, 화질이 조금 떨어져도 얼굴이 큰 사진이 낫습니다.

배경과 몸 — 스티칭에는 대가가 있습니다

스티칭은 새로 그린 얼굴을 원본 사진에 도로 붙이는 기능입니다. 커면 원본의 몸과 배경이 남고, 끄면 얼굴 크롭만 남습니다. 정장 차림의 전신 사진을 그대로 살리고 싶다면 컵니다. 대신 얼굴이 상대적으로 작아지고, 크롭 경계에서 귀가 잘리기도 합니다. 저자가 실제로 겪은 실패입니다.

손과 몸은 움직이지 않습니다

조건이 아니라 **한계**입니다. 그래도 여기서 못을 박아 둡니다. 이 책의 Track A 도구들은 **얼굴과 머리 포즈만** 움직입니다. 팔짱을 낀 사진은 말하는 내내 팔짱을 끼고 있습니다. चेस 말을 잡은 손은 끝까지 멈춰 있습니다.

토크 헤드에서는 대개 문제가 되지 않습니다. 하지만 "말하면서 손짓도 했으면 좋겠다"는 요구가 들어오면 길은 둘입니다.

원샷 오디오→영상 계열로 갑니다. 이미지 한 장과 음성만으로 상반신과 चेस처까지 한 번에 만드는 모델들이 있습니다(Ch05 §1, 온라인 부록 K §3). 결과는 좋습니다. 대신 원가의 자릿수가 다르고, 단계마다 손떨 곳 없습니다.

Track B로 갑니다. 3D 캐릭터의 팔을 코드로 움직입니다(Ch20). 사실적이지는 않습니다. 대신 즉시 반응하고, 무엇을 언제 할지 전부 통제할 수 있습니다.

즉 "손과 몸은 안 움직인다"는 이 책 Track A가 쓰는 도구의 한계이지, 이 분야의 한계가 아닙니다. 요구사항에 चेस처가 있으면 트랙 선택 자체를 다시 하세요.

스마트폰 스냅을 소스로 바꾸기 — 민트 ★

조건을 읽는 것과 내 사진이 그 조건을 지키는지 아는 것은 다른 일입니다. 저자의 고양이 민트로 끝까지 해 보겠습니다. 시작은 흔한 스마트폰 스냅입니다 — 위에서 내려찍었고, 고양이는 화면 한가운데 작게 있고, 뒤에는 팔레와 의자 다리가 걸려 있습니다.



그림 4.1 왼쪽이 찍은 그대로의 스냅, 오른쪽이 소스로 다듬은 것. 얼굴을 화면에 채우도록 잘랐다(배경 흐림은 보기용)

두 장을 컴패니언 `check_face.py` 로 재면 이렇습니다.

표 4.1 민트 — 스냅과 소스의 차이 (--box로 얼굴 상자를 직접 준 값)

항목	스냅 그대로	다듬은 소스	기준
얼굴 비율	0.219	0.621	0.35 이상(하한 0.20)
밝기	96.2	95.2	60~200
조명 고름	0.547	0.446	0.35 이상
날림(clipping)	0.0	0.0	0.05 이하
정면(대칭)	-0.226	0.424	0.55 이상
판정	실패	여전히 실패	

크롭만으로 얼굴 비율이 0.22에서 0.62로 올라갔습니다. 세 배 가까이입니다. 다시 찍은 것이 아니라 잘라 냈을 뿐입니다. 소스 조건에서 가장 무거운 항목 하나가 그렇게 해결됩니다.

그런데 여전히 실패입니다. 대칭이 0.424로 기준(0.55) 아래입니다. 스냅 그대로일 때는 상자에 배경이 섞여 -0.226이었으니 크롭이 끌어올리기는 했습니다. 원인은 크롭으로 고칠 수 없는 것 — 위에서 내려찍은 각도입니다. 고양이가 카메라를 올려다보고 있어 얼굴 아래쪽이 넓고 위쪽이 좁습니다. 조명 고풐은 잘라 내자 오히려 0.547에서 0.446으로 내려갔습니다 — 배경이 빠지자 천장 조명이 이마만 밝히는 것이 드러난 것입니다. 기준(0.35)을 겨우 넘겼을 뿐이고 원인은 같습니다. 이 둘을 고치려면 **눈높이에서 다시 찍는 수밖에 없습니다.**

여기서 판단이 갑니다. 다시 찍을 수 있으면 찍으세요. 다시 찍을 수 없으면 — 고인이 된 반려동물의 사진이 그렇습니다 — **미달인 채로 가되 무엇이 미달인지 알고 갑니다.** 대칭이 낮은 소스는 리타게팅에서 얼굴이 한쪽으로 쏠립니다. 그러니 Ch12의 `region=lip` 으로 입만 옮겨 쏠림을 줄이는 쪽을 먼저 시도하세요. 체커의 값은 *하지 말라* 가 아니라 *무엇을 조심하라* 입니다.

동물 사진은 이 체커가 스스로 재지 못합니다. 얼굴 검출기가 사람 얼굴용(Haar)이라 민트에 서는 0개를 냅니다. 그래서 위 표는 얼굴 상자를 손으로 준 값입니다 — `check_face.py` 사진.jpg --box x,y,w,h. 동물 소스를 쓸 거라면 이 한 줄을 습관으로 만드세요(Ch13). 파일 이름에 한글이 있으면 영상 라이브러리가 조용히 못 읽는다는 것도 같이 기억하세요 — 체커가 우회해 읽고 그 사실을 알려 줍니다(부록 F 14번).

4.3 안 보이는 얼굴 — 좋은 드라이버 영상의 조건

여기가 이 장의 핵심입니다. 그리고 저자가 가장 많은 시간을 잃은 곳입니다.

클로즈업이어야 합니다 — 오래 믿었던 첫 조건 ★

가장 중요한 조건입니다.

얼굴이 화면의 4분의 1쯤 되는 미디엄샷을 드라이버로 쓰면 입 모션이 낮은 해상도로 뽑힙니다. 그 약한 모션이 소스 얼굴로 옮겨지면 입이 조금밖에 안 움직입니다. 사용자는 그것을 "**싱크가 안 맞는다**" 고 느낍니다.

실제로는 싱크가 어긋난 것이 아니라 **모션의 진폭이 작은** 것입니다. 원인을 오해하면 엉뚱한 곳을 고치게 됩니다. 저자는 이 단계에서 프레임 재배열이니 시간 워핑이니 하는 꼼수를 만들어 며칠을 버렸습니다.

얼굴이 화면을 꽉 채우는 영상 을 쓰면 이 문제가 한 번에 사라집니다.

실측 — 잃는 것은 진폭이 아니라 모양이었습니다

위 설명은 저자가 오래 믿어 온 것인데, 컴패니언 코드로 확인해 보니 **반만 맞았습니다.**

합성한 얼굴을 프레임 안에 여러 크기로 **줄여 넣었습니다.** 카메라가 멀찍이서 담은 것과 같은 상태입니다. 그런 다음 파이프라인이 하는 그대로 얼굴을 크롭해 256×256으로 맞추고, 입 디테일과 총 진폭을 잴습니다.

표 4.2 얼굴이 작아질 때 무엇이 무너지는가 — 얼굴 px · 입 디테일 · 총 진폭

얼굴이 화면의	얼굴 px	입 디테일	확참 대비	총 진폭	확참 대비
0.15	162	2,748	21%	29.9	97%
0.25 (미디엄샷)	270	5,883	46%	26.9	88%
0.40	432	9,050	70%	29.8	97%
0.55	594	9,443	74%	29.9	97%
0.85 (클로즈업)	918	12,838	100%	30.8	100%

입이 열리는 양은 거의 그대로입니다. 미디엄샷과 클로즈업의 총 진폭 차이는 1.14배뿐입니다. 대신 **입 모양의 해상도가 2.2배 무너집니다.**

그러니 정확한 설명은 이쪽입니다. 입은 거칠게 벌어졌다 닫히기는 하는데, **소리에 맞는 모양이 안 나옵니다.** ㄹ과 ㄴ와 ㄷ가 구분되지 않고 몽개진 개폐 운동만 남습니다. 사람은 그것을 "입이 안 맞는다"로 느끼고, 타이밍 탓으로 오해합니다.

그래서 **프레임 재배열이나 시간 워핑으로는 절대 고쳐지지 않습니다.** 타이밍은 처음부터 맞았기 때문입니다. 저자가 며칠을 버린 이유가 이것입니다. 원인을 정확히 짚었다면 5분이면 끝날 일이었습니다 — 드라이버를 바꾸면 됩니다.

증상의 이름을 잘못 붙이면 고칠 수 없는 곳을 고치게 됩니다. 이 표는 `code/ch04_face/closeup.py` 가 매번 다시 계산합니다.

저자의 드라이버가 저자의 기준에 걸렸습니다 — 그런데 결과는 가장 좋았습니다 ★

이 절을 쓰고 나서 `check_face.py --driver` 를 저자가 실제 배포에 쓰는 드라이버 에 돌려 봤습니다. 스톡 사이트에서 "테라피스트"로 찾은, 차분한 정면 화자입니다. 이 책의 첫 기준(얼굴 비율 0.40 이상)으로는 이렇게 나왔습니다.

```
driver_therapist.mp4 - 15.08초 · 29.97fps
[실패] face_ratio      0.235   얼굴이 화면을 꽉 채우는 영상으로 바꾸세요
[OK ] motion          0.0051
[OK ] duration_ratio  2.26
→ 실패
```

얼굴이 화면의 23.5%입니다. 이 절이 "미디엄샷"이라 부른 바로 그 값(4분의 1)입니다. 차분함도 길어도 통과인데 얼굴 비율에서 떨어집니다. 여기까지 쓰고 저자는 "그래서 결과가 나빴다"고 적어 두었습니다.

그런데 이 절 끝의 실험에서 이 드라이버로 고양이를 실제로 돌려 보니, 결과는 셋 중 가장 좋았습니다. 입은 가장 또렷이 열리고 자세는 흔들리지 않았습니다. 기준이 실패를 예측하지 못한 것입니다.

이유는 리타게팅 단계에 있었습니다. LivePortrait는 드라이버 영상에서 **얼굴만 잘라** 쏩니다(--flag-crop-driving-video, Ch12). 원본 화면에서 얼굴이 작아도 잘린 얼굴에 픽셀이 충분하면 입 모션은 그대로 옮겨집니다 — 720p에서 0.235면 세로 약 170픽셀입니다. 얼굴 비율은 **립싱크 (Ch11) 단계의 입 모양 해상도** 에는 여전히 중요합니다. 다만 리타게팅 드라이버의 합격선으로 0.40은 과했습니다.

그래서 기준을 고쳤습니다. 드라이버의 얼굴 비율 하한은 **0.22**(경고선 0.40)로 내리고, 움직임 상한은 그대로 둡니다. 같은 검사를 다시 돌리면 이렇게 나옵니다.

```
driver_therapist.mp4 - 15.08초 · 29.97fps
[경고] face_ratio      0.235  얼굴이 화면을 꽉 채우는 영상으로 바꾸세요 - '싱크가 안 맞는
[OK ] motion         0.0051
[OK ] duration_ratio  2.26
→ 경고
```

기준을 쓰는 것과 기준을 돌리는 것은 다른 일이고, 돌려서 나온 결과와 기준이 어긋나면 **고칠 것은 결과가 아니라 기준**입니다. 이 책에서 가장 오래 쓴 드라이버가 그것을 가르쳤습니다.

차분해야 합니다 — 흔들리는 머리는 그대로 옮겨집니다

머리를 많이 흔드는 화자를 쓰면 그 흔들림이 소스 얼굴로 그대로 옮겨집니다. 사람 얼굴에서 자연스럽던 움직임이 고양이 얼굴에서는 눈알을 굴리고 머리를 흔드는 기괴한 장면이 됩니다.

차분하게 정면을 보며 도박도박 말하는 영상 이 정답입니다. 재미있게도 이런 영상은 스톡 사이트에서 "의사", "상담사", "강사" 같은 키워드로 찾으면 잘 나옵니다. 직업적으로 차분하게 말하는 사람들이기 때문입니다.

가진 드라이버를 전부 재 보세요. 저자가 쓰는 예제 드라이버 열둘에 테라피스트 드라이버를 더한 열셋을 체커에 돌렸습니다. 두 축을 함께 봐야 합니다 — 얼굴이 큰가, 그리고 **차분한가**.

표 4.3 드라이버 열셋 — 얼굴 비율(하한 0.22 · 경고선 0.40) · 움직임(작을수록 차분) · 길이

드라이버	얼굴 비율	움직임	길이	판정
d19	0.642	0.0063	8.3초	둘 다 좋다
d18	0.617	0.0095	7.2초	둘 다 좋다
d0	0.568	0.005	3.1초	차분한데 너무 짧다
d14	0.656	0.0313	17.9초	얼굴은 크고 움직임은 중간
d9	0.61	0.0292	19.6초	〃
d6	0.636	0.0419	33.6초	얼굴은 크나 움직임이 크다

드라이버	얼굴 비율	움직임	길이	판정
d11	0.59	0.0391	9.0초	//
d3	0.671	0.0608	11.8초	얼굴이 가장 큰데 가장 많이 움직인다
d12	0.5	0.0701	7.3초	움직임 최대
d20	0.465	0.0395	7.0초	겨우 통과
d10	0.566	0.0345	15.0초	중간
d13	0.37	0.0128	11.7초	차분하고 하한은 통과 — 경고선 아래
테라피스트	0.235	0.0051	15.1초	가장 차분하고 얼굴은 가장 작다 — 그런데 결과는 가장 좋았다(이 절 끝의 실험)

얼굴 비율만 보면 d3이 최고입니다. 0.671로 열셋 중 1위입니다. 그런데 움직임이 0.0608로 역시 1위입니다 — 테라피스트(0.0051)의 **열두 배** 입니다. 이 절 끝에서 이 드라이버로 실제로 돌린 결과를 보여 드리겠습니다. 한 줄로 먼저 말하면, **더 나빴습니다.**

그래서 조건에는 순서가 있습니다 — 먼저 움직임이 작을 것, 그다음이 얼굴 비율입니다(리타게팅 드라이버는 0.22 이상이면 됩니다 — 이 절 끝의 실험).

이 책의 체커는 움직임 상한을 0.25로 두고 있었습니다. 그런데 실제로 결과를 망친 드라이버가 0.06이었습니다. **상한이 실패를 못 거르는 값이면 그 기준은 없는 것과 같습니다.** 여러분의 드라이버를 몇 개 재 보고 **좋은 것과 나쁜 것 사이 어디**에 선을 그을지 직접 정하세요 — 이 표에서는 0.01 근처가 그 선입니다.

길이도 함께 보세요. d0은 움직임 0.005로 가장 차분한 축입니다. 그런데 3.1초입니다. 6초짜리 음성에 쓰려면 반복해 이어 붙여야 하고, 그 이음매에서 머리가 튕니다.

같은 사진에 드라이버만 셋 — 순위가 뒤집혔습니다 ★

민트 사진 하나와 음성 하나를 고정하고, 드라이버만 셋으로 바꿔 파이프라인(Ch11 → Ch12)을 세 번 돌렸습니다. 나머지 설정은 전부 같습니다 — 배율 3.0, `region=lip`.

세 결과를 **같은 시각(2.5초)**의 프레임으로 나란히 놓았습니다. 그리고 첫 프레임 대비 픽셀이 얼마나 달라졌는지를 입·머리·눈 세 영역에서 했습니다(`code/ch04_face/measure_result.py`). 사람 얼굴 검출에 기대는 Ch09 검증기는 동물에서 무너지므로(아래 주석) 이렇게 영역으로 쪼입니다.



그림 4.2 같은 소스·같은 음성·같은 시각(2.5초), 드라이버만 셋 — 둘째~넷째 칸은 각 드라이버로 만든 결과 프레임. 얼굴이 가장 큰 d3(셋째)이 가장 나쁘고, 얼굴이 가장 작은 테라피스트(둘째)가 입을 가장 또렷이 연다

표 4.4 드라이버 셋의 결과 실측 — 첫 프레임 대비 픽셀 변화(0~255, 클수록 많이 움직임)

드라이버	얼굴 비율 · 움직임	입 영역(평균/최대)	머리 영역	눈 영역	결과
테라피스트	0.235 · 0.005	20.8 / 32.4	10.6	17.7	자세를 지키고 입이 또렷이 열린다
d3	0.671 · 0.061	31.3 / 42.1	19.3	22.7	머리가 돌아가고 눈이 감긴다 — 실패
d19	0.642 · 0.006	14.4 / 23.3	7.3	12.4	가장 안정적이지만 입이 가장 덜 열린다

둘째 칸 — 테라피스트(얼굴 0.235 · 움직임 0.005). 얼굴 비율은 셋 중 가장 작는데 **입 영역 변화가 가장 큼니다(20.8, d19의 약 1.4배)**. 머리 영역 변화 10.6은 입을 열 때 턱과 볼이 따라 움직인 몫입니다. 자세는 그대로입니다. 이 절 앞부분이 "미디어샷이라 입 모션이 저해상도로 추출된다"고 예측한 것과 정반대입니다 — 리타게팅은 드라이버의 얼굴을 잘라 쓰기 때문입니다.

셋째 칸 — d3(얼굴 0.671 · 움직임 0.061). 얼굴 비율만 보면 열셋 중 1위인데, 결과는 **가장 나쁩니다**. 입 영역 변화 31.3은 입이 열려서 나온 값이 아닙니다. **얼굴 전체가 돌아가서 나온 값입니다** — 머리 영역 19.3은 테라피스트의 약 1.8배, 눈 영역 22.7은 눈이 감긴 값입니다. `region=lip`으로 입만 옮기게 했는데도 그렇습니다. 드라이버의 머리가 크게 움직이면 프레임마다 얼굴 크롭이 따라 흔들리고, 그 흔들림이 결과에 남습니다.

넷째 칸 — d19(얼굴 0.642 · 움직임 0.006). 머리 영역 변화 7.3으로 가장 안정적입니다. 대신 입은 가장 덜 열립니다(14.4). 쓸 수는 있습니다. 다만 같은 차분함이라면 테라피스트 쪽이 입이 살아 있습니다.

결론은 순서입니다 — 차분함이 먼저, 얼굴 크기는 그다음. 머리를 움직이는 드라이버는 얼굴이 아무리 커도 결과를 망칩니다. 차분한 드라이버는 얼굴이 작아도 입을 옮깁니다. 이 판정은 눈으로 했습니다 — 이유는 아래 상자에 있습니다.

그래도 셋 중 어느 것도 입이 사람만큼 크게 열리지는 않습니다. 원인은 드라이버가 아니라 소스입니다 — 민트의 대칭이 0.424로 기준 아래이고(§ 2), 위에서 내려찍은 각도라 입 영역이 짧게 잡힙니다. 소스와 드라이버는 각각 통과해야 합니다. 하나가 미달이면 다른 하나를 아무리 잘 골라도 그만큼만 나옵니다.

여기서 이 책의 검증기가 한계를 드러냅니다. Ch09의 싱크 검증기는 입 영역을 *사람 얼굴* 검출로 잡습니다. 고양이에서는 얼굴을 못 찾아 화면 비율로 떨어지고, 그러면 입이 아니라 가슴을 잡니다 — 세 결과에 Ch09 점수를 매기면 **셋 다 믿을 수 없는 숫자**가 나옵니다. Ch04의 소스 체크도 같은 이유로 동물 얼굴을 0개로 셉니다. **사람용 도구는 동물에서 조용히 무너 집니다.** 동물 소스를 다룬다면 검증은 눈으로 하고, 자동 지표는 *사람 드라이버 단계*에서만 믿으세요(Ch13).

충분히 길어야 합니다

드라이버가 음성보다 짧으면 반복해서 이어붙이게 되고, 그 이음매에서 모션이 튕니다. **음성 길이보다 긴 드라이버**가 깔끔합니다.

입꼬리가 과장되지 않아야 합니다

표정이 큰 스포츠 스타일 영상을 쓰면 입꼬리가 좌우로 비틀립니다. 연기가 강한 영상은 드라이버로 부적합합니다.

라이선스를 확인하세요

드라이버 영상은 결과물에 보이지 않습니다. 그래도 **저작물을 쓰는 것은 사실입니다.** 상업적 사용이 가능한 라이선스인지 확인하고, 가능하면 직접 촬영하세요. 카메라 앞에서 30초만 차분하게 말하면 최고의 드라이버가 생깁니다. 부록 B에 조건 체크리스트를 실었습니다.

4.4 얼굴이 없으면 만듭니다

참조 사진 몇 장으로 23분. 그러면 같은 얼굴이 정장에서도 한복에서도 나옵니다. 아래 얼굴 LoRA 이야기입니다.

쓸 만한 소스 이미지가 없으면 만듭니다. 이미지 생성 모델로 정면·고른 조명·입이 살짝 벌어진 인물을 뽑는 것은 이제 어렵지 않습니다.

프롬프트에는 그 모델이 배운 어휘를 쓰세요. 생성 모델은 학습한 캡션의 어휘로 말을 알아듣습니다. 사진 느낌을 원하면 `photo · headshot · portrait`처럼 사진 도메인의 단어를, 그림 느낌을 원하면 `illustration · digital art`를 씁니다. **"실사 같은"**이라고 한국어로 길게 설명하는 것보다 그 한 단어가 잘 듣습니다.

그리고 §2의 조건을 **프롬프트에 그대로 넣으세요.** 정면(`front view`) · 고른 조명(`soft even lighting`) · 단순 배경(`plain background`) · 입이 살짝 벌어진 상태. 나중에 골라내는 것보다 처음부터 그렇게 만드는 편이 훨씬 빠릅니다.

생성 이미지를 쓸 때 조심할 것이 둘 있습니다.

첫째, 손과 귀를 확인하세요. 생성 모델의 고질적 약점입니다. 얼굴만 보고 통과시켰다가, 스티칭을 켜 뒤에야 이상한 손가락이 화면에 남아 있는 것을 발견하기도 합니다.

둘째, 실존 인물과 닮게 만들지 마세요. 특정인의 이름을 프롬프트에 넣어 생성한 얼굴로 말하게 하는 것은, 기술적으로는 쉽지만 Ch29의 선을 넘는 행위입니다.

같은 얼굴을 여러 장 만들어야 할 때 — 얼굴 LoRA. 생성 이미지의 약점은 *다음 장*입니다. 정상 사진은 잘 나왔는데 한복 사진을 뽑으면 다른 사람이 나옵니다.

저자는 이것을 SDXL 위에 **얼굴 LoRA**(그 얼굴 하나만 따로 배우게 하는 작은 추가 학습)를 얹어 풀었습니다. 참조 사진 몇 장으로 1,000스텝을 학습하는 데 RTX 4070 SUPER에서 23분(1,382초)이 걸렸고, 그 뒤로는 정상·니트·한복·흑백·시네마틱 열두 스타일에서 같은 사람이 나왔습니다.

두 가지를 배웠습니다. ① **강도가 정체성입니다.** LoRA 적용 강도를 0.45·0.60·0.75로 훑으니 0.45에서는 *비슷한 다른 사람*이 나오고 0.75에서야 같은 얼굴이 유지됐습니다. 강도는 취향이 아니라 **얼굴이 남는 최소값**으로 정합니다. ② **의상 프롬프트가 얼굴을 끌고 갑니다.** 한복처럼 데이터가 편향된 옷은 얼굴까지 그쪽 평균으로 당깁니다. 그런 스타일은 강도를 더 올리거나 얼굴 참조를 더 넣으세요.

이 실험의 그림은 실지 않습니다. **얼굴의 주인을 확인하지 않은 이미지는 이 책에 실지 않는다**가 Ch29의 규칙이고, 저자의 갤러리 파일에는 그 기록이 없습니다. 숫자만 남깁니다 — 규칙은 저자에게도 적용됩니다.

3D 캐릭터를 렌더해서 소스 이미지로 쓰는 방법도 있습니다. 저자의 경험으로는 **깨끗하게 렌더된 3D 캐릭터 정면 이미지**가 실사 스톡 사진보다 오히려 립싱크 품질이 좋았습니다. 조명이 고르고, 정면이고, 얼굴이 화면을 채우고, 배경이 단순합니다. 소스 이미지의 조건을 전부 만족시키는 것입니다.

4.5 이 장에서 기억할 것

얼굴은 둘입니다. **소스 이미지(보인다)**와 **드라이버 영상(안 보인다)**. 드라이버의 외모는 아무 의미가 없습니다.

좋은 소스 이미지는 **정면 · 밝은 조명 · 입이 살짝 벌어짐 · 얼굴이 큼**. 해상도 자체보다 얼굴 비율이 중요합니다.

좋은 드라이버는 **차분함이 먼저, 그다음 얼굴 비율(리타게팅은 0.22 이상, 경고선 0.40) · 음성보다 김 · 과장 없음**. 미디엄샷은 립싱크 단계의 입 모양 해상도를 떨어뜨리지만, 리타게팅에서는 얼굴을 잘라 쓰므로 차분하기만 하면 통합니다 — 저자의 0.235짜리 드라이버가 셋 중 가장 좋았습니다(§3).

손과 몸은 움직이지 않습니다. 제스처가 필요하면 Track B로 가세요.

다음 장에서는 세 사다리 중 어느 것을 오를지 정합니다. 이 책에서 두 번째로 중요한 결정입니다.

실습 코드: `code/ch04_face/` — 소스 이미지 적합성 체커 + 드라이버 클로즈업 비율 측정

예상 소요: 1시간

관련 부록: B(드라이버 카탈로그), G(윤리)

CHAPTER 5

어느 사다리를 오를 것인가

5.1 사다리는 다섯 — 그리고 이 책 밖의 둘

"AI 휴먼 만들려면 어떤 모델 쓰면 되나요?"

Ch01의 그 질문이 여기서 다시 나옵니다. 여기서도 답은 없습니다. 하늘이와 흙런이와 코치는 서로 다른 방법으로 만들어졌고, 셋을 바꿔 끼울 수 없습니다.

이 장은 그 셋 — 이제는 넷 — 중에서 고르는 법입니다. 이 책에서 여러분이 내리는 두 번째로 중요한 결정입니다.

말하는 얼굴을 만드는 방법은 크게 셋입니다. 위로 갈수록 사실적이고, 아래로 갈수록 빠르고 쌉니다.

1단 — 2D 파츠 리깅. 그림 한 장을 눈·입·몸통 같은 부품으로 쪼개고, 코드가 그 부품을 **통째로** 이동·회전·확대합니다. Ch01의 흙런이가 그 방식입니다. 신경망은 한 줄도 쓰지 않습니다.

1.5단 — 2.5D(메시 변형). 여기서 한 단계가 더 있습니다. 파츠를 사각형 이미지가 아니라 **삼각형 그물(메시)**로 보고, 그 격자점을 개별로 밀고 당깁니다. 그러면 이미지가 **휼니다**.

이 차이가 만드는 것이 **머리 회전 착시**입니다. 정면 그림 한 장으로 고개를 좌우로 돌린 것처럼 보이게 만듭니다 — 얼굴 윤곽을 휘고, 눈·코·입을 원근에 맞춰 밀고, 안쪽 귀를 가립니다. 그러면 사람 눈에는 3D로 보입니다. **그림은 여전히 한 장이고 2D인데 3D처럼 보입니다.** 그래서 2.5D입니다.

상용 도구(Live2D Cubism 계열)가 이 영역이고, 버추얼 스트리머 대부분이 여기 있습니다.

2단 — 3D 캐릭터(VRM). 뼈대와 블렌드셰이프를 가진 3D 모델을 브라우저에서 렌더하고, 코드가 그 파라미터를 조절합니다. 고개를 돌리고 팔을 들고 눈을 감기는 일이 전부 수치 조작입니다. 역시 신경망은 쓰지 않습니다.

3단 — 신경망 립싱크. 실제 사진이나 영상의 픽셀을 신경망이 다시 생성합니다. 소리에 맞춰 입과 턱을 새로 그려 넣습니다. 사실적이고, 무겁고, GPU가 필요합니다.

많은 사람이 3단을 목표로 삼고 1·2단을 "초보용"이라고 생각합니다. 틀렸습니다. **세 사다리는 서로 다른 문제를 풀니다.**

그리고 4단이 생겼습니다 — 손짓까지 한 모델이 그림니다

최근에 한 칸이 더 붙었습니다. **원샷 오디오→영상** — 이미지 한 장과 음성만 주면 입과 표정뿐 아니라 **상반신과 손짓까지** 한 모델이 통째로 만들어 냅니다. 3단이 얼굴만 다루는 것과 달리 이 계열은 몸을 포함합니다.

이 책의 Track A를 3단으로 잡은 이유는 4단이 나빠서가 아닙니다. **원가와 제어 때문**입니다. 4단은 영상 확산 모델을 베이스로 씁니다. 그래서 소비자용 GPU 한 장으로는 감당하기 어렵고, 대부분 API로 부릅니다. 게다가 각 단계를 파라미터로 조절할 수 없습니다 — 결과는 좋지만 **왜 그렇게 됐는지 배울 것이 적습니다**.

동물 얼굴에 약한 것도 3단과 같습니다. 사람 데이터로 학습되었기 때문입니다.

후보 목록과 판단 기준은 **온라인 부록 K**에 있습니다. 그 부록은 6개월마다 갱신되고, 이 장의 결정 트리는 그대로 남습니다.

사다리 위에 칸이 둘 더 있습니다 — 이 책이 오르지 않는 이유

2026년 현재 4단 위로 두 칸이 더 보입니다. 이 책이 다루지 않는 이유는 나쁜 기술이라서가 아니라 **이 책의 전제(소비자 GPU 한 장)를 넘기 때문**입니다. 무엇인지, 언제 올라가는지만 적어 둡니다.

표 5.1 4단 위의 두 칸 — 이 책 밖이지만 알아야 하는 것

칸	하는 일	값	언제 올라가나
5단 · 오디오 구동 확산 모델 (Echo MimicV3 · Omni Human 계열)	음성만 넣으면 얼굴·손·상체가 한 모델에서 나온다 — 리타게팅과 립싱크를 따로 붙일 필요가 없다	VRAM 20GB 이상 (부록 C § 2.5), 영상 1초에 렌더 수십 초, 배치 전용	결과가 콘텐츠 이고 GPU를 빌릴 수 있을 때. 저자는 클라우드 노트북의 A100·L4에 상주 프로세스로 띄워 썼다(Ch28 § 6)
6단 · 3D 가우시안 실사 아바타	다각도 촬영으로 사람을 3D 점구름으로 만들어 실시간 렌더	촬영 세션이 필요, GPU가 상시 켜져 있어야 한다, 얼굴 교체가 곧 재촬영	한 사람의 얼굴로 오래 서비스할 때. 2단(VRM)의 "사실감"이 부족하고 3단의 동시 사용자 정원이 문제일 때

둘 다 사다리를 없애지 않습니다. 5단이 있어도 실시간 대화에는 못 쓰고(배치), 6단이 있어도 얼굴을 매번 바꾸는 서비스에는 못 씁니다(재촬영). § 2의 세 질문은 그대로 유효합니다 — 답이 위쪽을 가리킬 때 이 표로 오면 됩니다.

5.2 세 질문으로 고릅니다

고르는 데 필요한 질문은 셋입니다. 그리고 그중에 **사실감**은 없습니다.

질문 1 — 목표 지연이 얼마인가

첫 번째이자 가장 강력한 필터입니다.

사람이 말을 걸고 **2초 안에** 첫 소리가 나야 한다면, 예전에는 3단이 그냥 후보에서 빠졌습니다. 지금은 사정이 조금 다릅니다. 스트리밍을 전제로 설계된 실사 토크헤드가 나오면서 **"실사 + 실시간"**이 불가능한 칸은 아니게 되었습니다.

다만 판단 기준이 *가능한가*에서 **감당 가능한가**로 옮겨갔을 뿐입니다. 제약 둘은 그대로입니다 — GPU 한 장이 감당하는 **동시 사용자는 사실상 한 명** 이고, **얼굴 전처리 비용은 남습니다**. 사용자가 백 명이면 GPU가 몇 장 필요한지부터 계산하세요. 그 숫자를 감당할 수 없으면 답은 여전히 1·2단입니다.

반대로 결과물이 파일이고 **몇 분을 기다려도 된다면**, 1·2단을 쓸 이유가 별로 없습니다. 사실감이 곧 가치이기 때문입니다.

지연 예산이 아키텍처를 결정합니다. 나중에 최적화해서 실시간으로 만드는 일은 일어나지 않습니다. Part 2 전체가 이 이야기입니다.

질문 2 — 그 얼굴이 사람인가

실사 립싱크 모델은 사람 얼굴에만 작동합니다. 버그가 아니라 학습 데이터의 결과입니다. 모델은 사람 얼굴 영상으로 학습되었고, 사람의 입술과 치아를 그리는 법을 배웠습니다.

고양이 사진에 그 모델을 그대로 적용하면 어떻게 될까요. 모델은 고양이 얼굴에서 "입이 있어야 할 자리"를 찾아내고, 거기에 **사람의 입술과 이를 그립니다**. 저자가 처음 겪은 실패가 정확히 이것이었습니다. 보기 괴로운 결과였습니다.

얼굴이 사람이 아니라면 선택지는 둘입니다. 리타게팅을 경유하거나(Ch13), 애초에 1·2단으로 가는 것입니다.

질문 3 — 얼굴이 고정인가 매번 바뀌는가

3단 모델들은 얼굴마다 **전처리**를 합니다. 얼굴을 검출하고, 크롭 영역을 정하고, 잠재표현을 미리 계산해 캐시합니다. 여기에 수십 초에서 몇 분이 듭니다.

캐릭터가 정해진 한 명이라면 전처리는 한 번뿐입니다. 그 뒤로는 캐시를 재사용합니다. 반대로 사용자가 자기 사진을 올려 매번 다른 얼굴을 쓰는 서비스라면, 전처리 시간이 그대로 사용자 대기 시간이 됩니다.

5.3 결정표 — 다섯 칸을 나란히 놓고

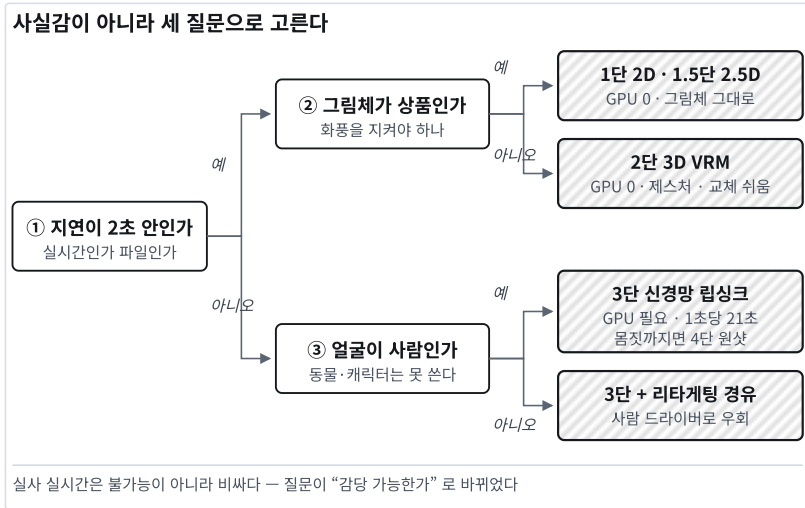


그림 5.1 세 질문으로 사다리가 정해진다

표 5.2 1단 2D 파츠 · 1.5단 2.5D 메시 · 2단 3D VRM · 3단 신경망 립싱크 · 4단 원샷

	1단 2D 파츠	1.5단 2.5D 메시	2단 3D VRM	3단 신경망 립싱크	4단 원샷
GPU	불필요	불필요	불필요	필요	대형(대개 API)
첫 소리까지	약 2초+	약 2초+	약 2초+	분 단위	분~수십 분
사실감	낮음(그림)	중간(그림)	중간(3D)	높음(실사)	매우 높음
얼굴 제약	없음	없음	없음	사람	사람
고개 돌리기	불가	착시 가능	자유	제한적	가능
제스처(손·몸)	제한적	제한적	가능(코드)	불가	가능(자동)
그림체 유지	완전	완전	3D로 재해석	—	—
제작 비용	낮음(파츠 4개)	높음(리깅 작업)	중간(모델 확보)	낮음	낮음
캐릭터 교체	그림만	재리깅 필요	VRM만	전처리	매번 생성
동시 사용자	브라우저	브라우저	브라우저	GPU 대수	API 예산

↑ 첫 문장에서 TTS로 넘기는 겹침 경로 기준. 같은 부품을 순차로 부르는 단순 서버는 실측 3.8 초였습니다(Ch21 § 5).

1.5단의 값은 "그림체를 그대로 유지하면서 입체감을 얻는 것" 입니다. 일러스트의 화풍 자체가 상품인 캐릭터 — 버추얼 스트리머, 게임 캐릭터, IP 마스코트 — 는 3D로 옮기는 순간 원작의 맛이 사라집니다. 2.5D는 그 문제를 피합니다.

대신 **리깅 비용이 큼니다**. 파츠 넷을 잘라 붙이는 1단과 달리, 메시를 깔고 디포머를 걸고 파라미터를 잡는 작업이 캐릭터마다 필요합니다. **캐릭터가 하나면 할 만하고, 열이면 다시 생각해야 합니다**.

GPU 행은 주장이 아니라 측정입니다. 컴패니언 코드로 1단의 파츠 합성(512×512 · 파츠 4개)을 **GPU 없이 CPU만으로** 돌려 채면 프레임당 약 3밀리초입니다. 60fps 예산이 프레임당 16.7밀리초이니 다섯 배 넘게 여유가 있습니다. 브라우저는 GPU 합성을 쓰므로 이보다 빠릅니다 — 이 값은 **가장 느린 쪽의 값**입니다. 3단의 프레임당 600밀리초(Ch06)와 견주면 **200배**입니다.

동시 사용자 행을 특히 보세요. 1·2단은 렌더가 사용자의 브라우저에서 일어납니다. 서버는 텍스트와 오디오만 만듭니다. 사용자가 백 명이면 백 대의 브라우저가 각자 렌더합니다. 3단은 모든 렌더가 여러분의 GPU에서 일어납니다. 이 차이가 서비스의 원가 구조 자체를 바꿉니다.

이 행은 숫자로 환산됩니다. 저자 환경에서 3단은 GPU 한 장이 동시 사용자 한 명 수준을 감당합니다. 즉 **동시 50명 = GPU 50장**입니다. 목표 동시 접속자를 GPU 대수로 바꾼 다음, 그 월 비용이 감당되는지부터 계산하세요. 감당이 안 되면 답은 1·2단입니다 — **사실감을 포기하는 것이 아니라 원가 구조를 고르는 것**입니다. 계산 방법은 부록 N.

표에 넣지 않은 축이 하나 있습니다 — **제어**입니다. 4단은 개발 난이도가 가장 낮는데 제어는 가장 어렵습니다. 부르는 쉽고, 마음에 안 들 때 고칠 손잡이가 없습니다. 이 책이 3단을 본선으로 두는 이유이기도 합니다 — **손잡이가 있어야 배울 것이 있습니다**.

5.4 흔한 시나리오별 답

"우리 회사 대표님 얼굴로 사내 공지 영상을 만들고 싶다" — 3단. 사람 얼굴이고, 실사여야 하고, 미리 만들어 둡니다. Track A.

"학습 사이트에 설명해 주는 캐릭터를 넣고 싶다" — 2단. 즉시 반응해야 하고, 사실적일 필요가 없으며, 동시 접속자가 많습니다. Track B.

"세상을 떠난 반려동물이 말하는 영상" — 3단이되 리타게팅 경유. 사람 얼굴이 아니므로 직접 적용이 불가능합니다. Ch13·Ch15.

"버추얼 스트리머를 만들고 싶다" — 2단. 실시간이고, 제스처가 필요하고, 캐릭터입니다. Track B.

"상담 키오스크" — 2단 또는 1단. 지연이 전부이고, 동시 사용자가 많으며, 하드웨어에 GPU를 넣을 수 없습니다.

"게임 안의 NPC 여럿이 대화한다" — 1·2단 + Track C. 얼굴보다 인격과 상호작용이 본질입니다.

"강연하듯 손짓하며 말하는 홍보 영상" — 4단. 얼굴만으로는 부족하고, 미리 만들어 두며, 원가를 API 예산으로 처리합니다. 온라인 부록 K §3.

"내 사진으로 아바타 만들어주는 앱" — 3단이되 전처리 비용을 설계에 반영 해야 합니다. "만드는 중"을 사용자에게 어떻게 보여줄지가 제품의 성패를 가릅니다. Ch08이 이 문제를 다룹니다.

5.5 사다리를 섞을 수도 있습니다

세 사다리는 배타적이지 않습니다. 실무에서 자주 쓰는 조합이 둘 있습니다.

품질 트랙 안에서 섞기. 신경망 립싱크로 사람 드라이버에 입을 맞추고, 리타게팅으로 그 모션을 대상 얼굴에 옮깁니다. Track A의 핵심 트릭입니다 — Ch01 §5에서 예고한 그 분리입니다. 사람이 아닌 얼굴을 말하게 하는 유일한 실용 경로이기도 합니다.

트랙을 나눠서 섞기. 실시간 대화는 2단(브라우저 VRM)으로 합니다. 사용자가 "이 대화를 영상으로 저장"을 누르면, 그때 3단으로 고품질 렌더를 백그라운드에서 돌립니다. 지연이 중요한 순간과 품질이 중요한 순간을 갈라 놓는 설계입니다.

5.6 이 장에서 기억할 것

사다리는 2D 파츠 · 2.5D 메시 · 3D VRM · 신경망 립싱크 넷이고, 여기에 원샷 오디오→영상 이 다섯째로 붙었고, 여기에 원샷 오디오→영상 이 넷째로 붙었습니다. 위아래가 아니라 서로 다른 문제를 풀니다.

고르는 질문은 셋입니다. 목표 지연 · 얼굴이 사람인가 · 얼굴이 고정인가. 사실감은 맨 마지막입니다.

실사 실시간은 이제 불가능 이 아니라 비싸다 입니다. 질문이 "가능한가"에서 "감당 가능한가"로 바뀌었습니다.

개발이 쉬운 것과 제어가 되는 것은 다릅니다. 4단은 호출하기 가장 쉽고 고치기 가장 어렵습니다.

실사 립싱크는 사람 얼굴에만 작동합니다. 다른 얼굴에 직접 쓰면 사람 입술이 그려집니다.

1.2단은 렌더가 사용자 브라우저 에서, 3단은 여러분의 GPU 에서 일어납니다. 이 차이가 원가 구조를 결정합니다.

Part 1이 끝났습니다. 다음 Part 2는 이 책의 무게중심이고, 모델 이름이 거의 나오지 않습니다. 대신 4분과 2초 사이의 거리를 다룹니다.

실습 코드: `code/ch05_ladder/` — 세 사다리 최소 예제 3종 비교 (같은 문장, 같은 음성)

예상 소요: 2시간

관련 부록: C(벤치마크)

Part 2

지연의 기초

"10.67초 영상에 230초가 걸린다. 같은 날 오후 브라우저 아바타는 2초에 대답한다. 이 21배는 최적화로 메울 수 있는 거리가 아니다."

Ch06	왜 4분이 걸리는가	품질 트랙의 해부. 병목의 85%는 한 단계에 있다
Ch07	지연 예산 2초	TTFA · 문장 청킹 · 스트리밍 · S2S의 대가. 목표 지연이 아키텍처를 정한다
Ch08	지연을 숨기는 법	아이들 루프 · 크로스페이드 · 더블버퍼링. 스피너는 패배다
Ch09	싱크를 어떻게 증명하는가	음량 상관은 틀린 지표다. 발화구간 × 상위 N프레임

CHAPTER 6

왜 4분이 걸리는가

6.1 숫자부터 봅시다

10.67초짜리 영상 하나를 만들었습니다. 512×512 해상도, 30fps, 총 320프레임. 파일 크기는 0.57MB입니다.

만드는 데 **230초**가 걸렸습니다. RTX 4070 SUPER 12GB 한 장을 온전히 쓰고도 그렇습니다.

230초, 거의 4분입니다. 제목의 그 4분이 이 숫자입니다.

영상 길이의 21배입니다. 1분짜리 영상에 21분, 10분짜리 강의 영상에 세 시간 반이라는 뜻입니다. 실제로 79초짜리를 만들었을 때 28분이 걸렸습니다.

이 숫자를 처음 보면 대부분 이렇게 생각합니다. "*최적화하면 되겠지.*" 이 장은 왜 그것이 안 되는지를 설명합니다.

6.2 시간이 어디로 가는가

파이프라인을 단계별로 재면 이렇습니다.

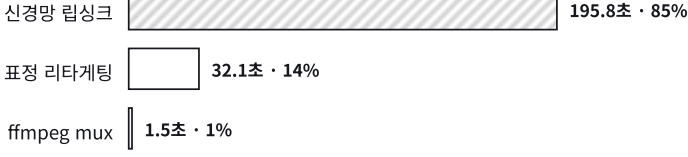
표 6.1 230초의 분해. 병목의 85%가 한 단계에 몰려 있다.

단계	소요	처리율	비중
TTS (대본 → 음성)	수 초	—	별도 — 합계에 넣지 않음
신경망 립싱크	195.8초	~1.6 프레임/초	~85%
표정 리타게팅	32.1초	~10 프레임/초	~14%
ffmpeg mux	1~2초	—	<1%
합계	약 230초		100%

병목의 85%가 한 단계에 있습니다. 이것이 이 장에서 가장 중요한 한 줄입니다.

10.67초 영상 한 편 = 230초

RTX 4070 SUPER 12GB · 512×512 · 30fps · 320프레임

**병목의 85%가 한 단계에 있다**

나머지를 전부 0으로 만들어도 총 소요는 230초에서 196초가 될 뿐입니다. 체감은 그대로입니다.

TTS 는 수 초 — 기계가 아니라 제공자에 달려 있어 위 합계에 넣지 않았습니다.

그림 6.1 230초 중 195.8초가 립싱크 한 단계에 몰려 있다

성능 문제를 만나면 대개 전체를 조금씩 개선하려 듭니다. 여기서는 소용이 없습니다. 나머지 15%를 전부 0으로 만들어도 총 소요는 230초에서 196초가 될 뿐입니다. 체감은 그대로입니다.

최적화는 항상 측정에서 시작합니다. 그리고 측정해 보면 대개 이렇게 한쪽으로 쏠려 있습니다.

21배는 한 번 잦 숫자가 아닙니다

위 표는 10.67초짜리 하나를 잦 값입니다. 점 하나로는 "*우연히 그 파일이 느렸다*"를 배제할 수 없습니다. 그래서 하나 더 잦니다.

표 6.2 음성 길이 · 총 소요 · 배수

음성 길이	총 소요	배수
10.67초	230초	21.6배
79초	약 1,680초 (28분)	약 21배

두 배수가 1% 남짓 안에서 같습니다(둘째는 분 단위로 재 둔 값이라 그 정도가 한계입니다). 이 일치가 말해 주는 것이 있습니다.

파이프라인에 큰 고정 비용 — 모델 로드, 얼굴 전처리 같은 것 — 이 있었다면, 긴 파일일수록 그 비용이 희석되어 배수가 내려가야 합니다. 내려가지 않았습니다. 즉 시간은 거의 전부 **프레임당 비용**에서 나오고, 그 비용은 길이에 정비례합니다.

선형이라는 것은 나쁜 소식이자 좋은 소식입니다. 나쁜 소식 — 긴 영상이 짧은 영상보다 비울로는 하나도 유리하지 않습니다. 좋은 소식 — 길이만 알면 소요 시간을 예측할 수 있습니다. Ch28의 잡 큐가 "*약 4분 남았습니다*"라고 말할 수 있는 근거가 이 표입니다.

측정을 두 번 하는 것은 비용이 거의 안 듭니다. 그런데 한 번과 두 번의 차이는 "숫자"와 "관계"의 차이입니다. 하나는 그 파일이 얼마나 걸렸는지를, 둘은 왜 그만큼 걸리는지를 알려줍니다.

6.3 왜 립싱크가 그렇게 느린가

프레임 하나당 0.6초가 걸립니다. 무슨 일이 일어나고 있는 걸까요.

첫째, 프레임마다 신경망을 통과합니다. 320프레임이면 320번입니다. 게다가 한 번의 통과가 가법지 않습니다. 이미지를 잠재공간(그림을 압축해 담아 두는 좌표계)으로 인코딩하고, 오디오 특징과 함께 U-Net을 통과시키고, 다시 픽셀로 디코딩합니다. 인코더·U-Net·디코더 셋이 프레임마다 도는 셈입니다.

둘째, 오디오 특징 추출이 따로 있습니다. 음성을 프레임 단위 특징으로 바꾸는 별도의 모델이 단에서 돕니다. 이 특징이 "지금 이 순간 입이 어떤 모양이어야 하는가"를 정합니다.

셋째, 얼굴 전처리가 앞에 붙습니다. 첫 실행에서는 얼굴 검출, 크롭 영역 결정, 잠재표현 사전 계산이 더 일어납니다. 이 비용은 캐시로 없앨 수 있고, 없애야 합니다(Ch11).

넷째, 프레임끼리 의존해서 완전 병렬화가 안 됩니다. 배치로 묶으면 나아집니다. 대신 배치를 키우면 VRAM이 먼저 터집니다. 12GB 카드에서 쓸 수 있는 배치 크기는 넉넉하지 않습니다.

6.4 그래서 무엇을 할 수 있나

병목을 알았으니 손댈 곳도 명확합니다. 효과가 큰 순서로 넷입니다.

전처리 캐시. 같은 얼굴을 반복해서 쓴다면 전처리는 한 번이면 됩니다. 두 번째 실행부터 수십 초가 사라집니다. 캐릭터가 고정된 서비스에서는 이것 하나로 체감이 달라집니다.

배치 크기 조정. VRAM이 허용하는 한 키웁니다. 12GB에서 배치 32 정도가 현실적인 상한이고, 8GB라면 절반으로 줄여야 합니다. 배치를 키운 만큼 선형으로 빨라지지는 않습니다. 그래도 20~30%는 나옵니다.

반정밀도. 숫자를 절반 크기로 다루는 모드입니다. 모델을 half로 올려 쓰면 메모리와 시간이 함께 줄어듭니다. 이 분야에서는 사실상 기본값입니다.

해상도와 프레임레이트. 512×512를 유지하고 30fps 대신 25fps로 만들면 프레임 수가 17% 줄어듭니다. 그만큼 시간도 줄어듭니다. 사람 눈에 25fps와 30fps의 차이는 크지 않습니다.

이 넷을 다 해도 실시간의 21배가 3배까지 내려오지는 않습니다. 빨라지는 폭이 2~3배입니다. 그리고 그것이 이 트랙의 현실적인 상한입니다.

6.5 21배는 왜 메울 수 없는가

여기가 이 장의 결론입니다.

실시간이란 1초짜리 오디오를 1초 안에 처리한다는 뜻입니다. 30fps라면 프레임당 33밀리초입니다. 지금 우리는 프레임당 600밀리초를 쓰고 있습니다. **18배**를 줄여야 합니다.

앞의 최적화로 2~3배를 얻습니다. 남은 6~9배는 어디서 오나요.

더 좋은 GPU로 2배 정도. **더 작은 모델**로 2배 정도 — 대신 품질이 떨어집니다. **해상도를 낮춰**서 2배 정도 — 역시 품질이 떨어집니다.

즉 실시간에 닿으려면 **품질을 포기해야 합니다**. 그런데 애초에 이 트랙을 고른 이유가 품질이었습니니다. 제자리로 돌아옵니다.

이것이 Part 1에서 말한 명제의 근거입니다.

품질 트랙과 실시간 트랙은 같은 파이프라인의 두 설정이 아니라, 다른 물건입니다.

실시간이 필요하면 처음부터 다른 아키텍처를 고릅니다. 신경망이 픽셀을 그리는 대신, 이미 있는 그림이나 3D 모델을 코드가 변형합니다.

그러면 프레임당 비용이 600밀리초에서 **약 3밀리초**로 떨어집니다(근거는 Ch05 §3) — 컴패니언 코드의 2D 파츠 합성을 GPU 없이 CPU만으로 켜 값입니다(Ch05 §3). **200배**입니다. 브라우저의 GPU 합성은 그보다 빠르니 실제 격차는 더 큼니다. **최적화가 아니라 아키텍처 교체가 만드는 차이입니다**.

6.6 그렇다면 품질 트랙은 어떻게 쓰나

느리다는 것이 쓸모없다는 뜻은 아닙니다. 느린 것에 맞는 쓰임새가 따로 있습니다.

배치 작업으로 씁니다. 사용자를 세워 두지 않습니다. 작업을 큐에 넣고, 끝나면 알립니다. 영상 편집 도구의 렌더링과 같은 방식입니다.

미리 만들어 둡니다. 자주 나오는 문장을 미리 렌더해 두는 법은 Ch08 §6입니다. 실시간 대화 중에도 자주 나오는 문장은 캐시가 통합니다.

분리해서 씁니다. 실시간 대화는 가벼운 트랙으로 하고, 사용자가 "이 대화를 영상으로 저장"을 누르면 그때 품질 트랙으로 백그라운드 렌더를 겁니다.

전용 하드웨어로 씁니다. 서비스 서버와 렌더 서버를 분리합니다. Ch28에서 원격 GPU 잡으로 이 구조를 만듭니다.

6.7 이 장에서 기억할 것

10.67초 영상에 230초. **실시간의 21배** 이고, 병목의 **85%가 립싱크 한 단계**에 있습니다.

최적화는 측정에서 시작합니다. 병목이 아닌 곳을 0으로 만들어도 체감은 달라지지 않습니다.

전처리 캐시 · 배치 · 반정밀도 · 해상도 조정으로 2~3배 를 연습니다. 그것이 상한입니다.

21배는 최적화로 메울 수 없습니다. 실시간이 필요하다면 아키텍처를 바꿉니다. 그것이 200배의 차이를 만듭니다.

느린 트랙은 **배치 · 사전 렌더 · 분리 · 전용 하드웨어** 로 씁니다.

다음 장은 반대쪽입니다. 2초 안에 첫 소리를 내려면 무엇을 어떤 순서로 놓아야 하는가.

실습 코드: `code/ch06_profile/` — 파이프라인 단계별 계측 스크립트

예상 소요: 1시간 (계측 실행 포함)

관련 부록: C(실측 벤치마크)

CHAPTER 7

지연 예산 2초

7.1 사용자가 재는 시간은 하나뿐입니다

3.8초와 1.3초. 같은 부품으로 만든 두 서버가 같은 질문에 낸 시간입니다. 부품이 아니라 부품을 잇는 방법이 달랐습니다. 곧 그 이야기를 합니다.

실시간 대화 아바타에서 켈 숫자는 하나입니다.

첫 소리가 날 때까지 몇 초인가.

전체 응답 시간이 아닙니다. 사용자는 첫 소리를 들은 순간부터 "반응했다"고 인식합니다. 그다음부터는 소리를 들으며 기다립니다. 문장이 언제 끝나는지는 거의 신경 쓰지 않습니다.

웹에서 이 지표를 TTFB(Time To First Byte)라고 부릅니다. 대화 아바타에서는 TTFA(Time To First Audio) 라고 부르는 편이 정확하겠습니다.

경험적 기준선은 이렇습니다. **1초 이하면 즉각적으로 느껴집니다. 2초까지는 자연스러운 대화입니다. 3초를 넘으면 사용자가 화면을 의심하기 시작합니다. 5초는 이미 대화가 아닙니다.**

목표를 2초로 잡습니다.

7.2 예산을 쪼갭니다

2초를 각 단계에 나눠 줍니다. 실시간 트랙의 전형적인 배분은 이렇습니다.

표 7.1 2초 예산의 단계별 배분. 합이 2초를 넘길 수 있다 — 여유가 없다.

단계	예산	실제로 무엇이 일어나나
입력 확정	0~300ms(음성이면 침묵 판정 700~1000ms가 앞에 더해짐 — Ch23 § 2)	타이핑이면 0. 음성이면 발화 종료 판정 (VAD)
STT	200~500ms	음성 → 텍스트
LLM 첫 토큰	300~800ms	가장 변동이 큰 구간
LLM 첫 문장 완성	+200~400ms	문장 하나만 나오면 다음으로 넘긴다
TTS 첫 청크	300~600ms	문장 하나 → 오디오
전송·재생 시작	50~150ms	네트워크 + 브라우저 디코딩

단계	예산	실제로 무엇이 일어나나
합계	1.1~2.8초	

이 배분은 **설계**입니다. 같은 부품(가벼운 모델 + 클라우드 TTS)을 실제로 불러 켜 값은 Ch21 §5에 있습니다 — 단계를 순차로 기다리는 단순 서버는 **3.8초**, 첫 문장·첫 청크에서 바로 넘기는 겹침 경로는 **1.3~1.7초**였습니다. 합계를 두 배로 가른 것은 표의 각 칸이 아닙니다. **칸과 칸을 어떻게 이었는가**입니다.

합이 2초를 넘길 수 있다는 점이 중요합니다. 여유가 없습니다. **어느 한 단계라도 무겁게 만들면 예산이 무너집니다.**

그런데 이 표를 그냥 더하면 안 됩니다 ★

위 표는 단계를 순차로 돌렸을 때 의 합입니다. 그러면 이렇게 됩니다.

순차 : 총지연 = STT + LLM + TTS

단계를 겹치면 식이 바뀝니다.

겹침 : 총지연 $\approx \max(\text{단계들}) + \text{첫 청크가 통과하는 시간}$

이 한 줄이 이 장 전체의 뼈대입니다.

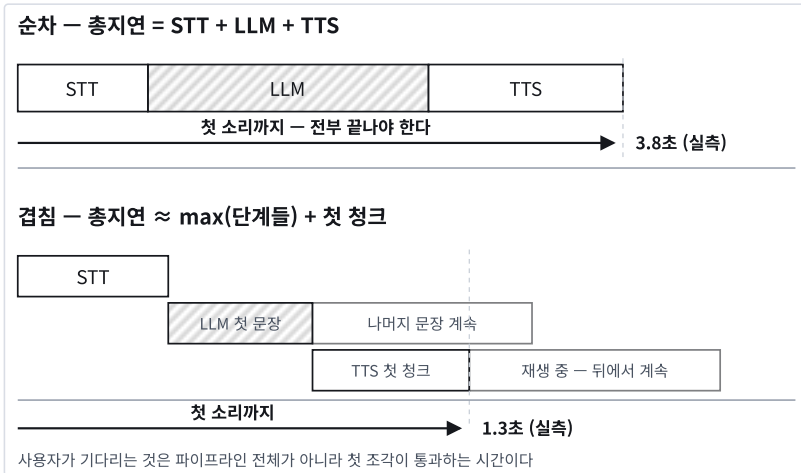


그림 7.1 겹치면 총지연이 $\max(\text{단계들}) + \text{첫 청크}$ 통과 시간으로 줄어든다

LLM이 첫 문장을 뱉는 순간 TTS로 넘깁니다. 그러면 나머지 문장은 **첫 문장이 재생되는 동안 뒤에서 만들어집니다**. 사용자가 기다리는 것은 전체 파이프라인이 아니라 **첫 조각이 통과하는 시간** 뿐입니다.

그래서 최적화의 목표가 달라집니다. **각 단계를 빠르게 만드는 것이 아니라 첫 조각을 빨리 통과시키는 것**입니다. §3의 문장 청킹이 그 구현이고, 뒤의 절들도 전부 이 한 문장의 변주입니다.

접치려면 각 단계가 부분 입력을 받아 부분 출력을 낼 수 있어야 합니다. STT는 청크 단위 처리, LLM은 토큰 스트리밍, TTS는 문장 단위 합성. 셋 중 하나라도 "다 모아야 시작하는" 도구를 쓰면 그 지점에서 파이프라인이 막힙니다. **도구를 고를 때 스트리밍 지원 여부를 먼저 보세요.**

왜 하필 2초인가

경험칙이 아니라 근거가 있습니다.

사람끼리 대화할 때, 상대의 말이 끝나면 보통 **200~500밀리초** 안에 응답이 시작됩니다. 이 주고 받기를 **턴 전환(turn-taking)** 이라고 합니다. 그보다 긴 침묵이 이어지면 듣는 쪽의 뇌는 "**문제가 생겼다**"고 인지합니다.

음성 AI도 이 무의식적 기대치를 그대로 적용받습니다. 사람 수준(200~500ms)은 현재 기술로 어렵습니다. 다만 **2초를 넘기는 순간 사용자는 대화가 아니라 조화를 하고 있다고 느낍니다.**

그리고 이것이 Ch08에서 **지연을 숨기는 기법**이 필요한 이유입니다. 총시간을 줄일 수 없다면 **반응이 시작됐다는 신호**라도 200~500ms 안에 주어야 합니다.

같은 이유로 실시간 트랙에는 얼굴 중에 신경망을 쓸 자리가 없습니다. 프레임당 600밀리초를 먹는 단계는 이 표 어디에도 끼워 넣을 수 없습니다.

7.3 가장 큰 절약 — 문장 단위 청킹

예산 표에서 눈여겨볼 것은 "**LLM 첫 문장 완성**" 행입니다.

LLM 응답 전체를 기다렸다가 TTS를 부르면 생성 시간이 통째로 지연에 들어갑니다. 세 문장짜리 답변이면 2초가 넘습니다.

대신 **첫 문장이 완성되는 순간 그 문장만 TTS로 보냅니다.** 나머지 문장은 LLM이 계속 만들고 있고, 사용자는 이미 첫 문장을 듣고 있습니다. 첫 문장이 재생되는 동안 두 번째 문장의 TTS가 끝납니다. 이어서 재생하면 사용자는 끊김을 느끼지 않습니다.

이 하나로 지연이 절반 이하로 줄어드는 일이 흔합니다.

구현은 스트리밍 응답을 받으면서 문장 경계를 찾는 것입니다. 한국어는 마침표·물음표·느낌표와 종결 어미로 경계를 잡습니다.

왜 하필 문장 단위인가

토큰이 나오는 대로 글자 단위로 TTS에 보내면 안 될까요. 안 됩니다. 이유가 둘입니다.

TTS는 문장 단위로 운율을 계산합니다. 억양은 문장 전체의 구조에서 나옵니다. 어디서 올리고 어디서 내릴지를 문장을 보고 정합니다. 글자나 어절 단위로 쪼개 주면 **운율이 깨져 기계음처럼** 들립니다. 지연을 줄이려다 품질을 잃습니다.

호출마다 고정 오버헤드가 있습니다. 네트워크 왕복, 초기화, 모델 준비. 조각을 잘게 쪼갤수록 이 비용이 내용보다 커집니다.

스트리밍에서 문장을 자르는 것은 생각보다 어렵습니다

완성된 텍스트라면 문장 분리 라이브러리를 쓰면 되지만, 스트리밍에서는 문장이 완성되기 전에 조각이 들어옵니다 — 지금 들어온 마침표가 끝인지 소수점인지를 앞만 보고 정해야 합니다(규칙은 아래 "한국어 문장 분리를 쓸 수 없는 이유"). 실무의 타협은 **종결 부호 + 뒤따르는 공백**을 경계로 보는 것입니다.

여기에 최소 길이 규칙을 하나 둡니다. **너무 짧은 조각을 계속 보내면 재생이 잘게 끊깁니다.** "네." 하나를 TTS에 보내면 오버헤드가 내용보다 큼니다. 그래서 최소 길이(예: 10자)를 두고 그보다 짧으면 다음 문장과 합칩니다.

그런데 첫 청크에는 적용하지 마세요 ★

이 규칙을 그대로 첫 청크에 걸었다가 문제를 만났습니다.

사용자가 "고마워요"라고 하면 캐릭터는 "네!"라고 답합니다. 짧습니다. 최소 길이 규칙에 걸려 다음 문장이 나올 때까지 기다리고, 그동안 아무 소리도 안 납니다. **매끄러우라고 만든 장치**가 첫 소리를 늦추고 있었습니다.

규칙을 나누면 해결됩니다.

첫 청크는 경계가 나오는 즉시 내보냅니다. 길이를 따지지 않습니다. 여기서는 지연이 전부입니다.

두 번째부터 최소 길이를 적용합니다. 사용자는 이미 소리를 듣고 있으므로, 이제부터는 매끄러움이 더 중요합니다.

원칙 — 첫 청크는 지연 우선, 나머지는 매끄러움 우선. 짧은 대답일수록 빨라야 합니다. 대화에서 가장 흔한 응답이 가장 짧은 응답이기 때문입니다.

구현할 때 또 하나 조심할 것이 있습니다. **첫 경계에서 판단을 멈추면 안 됩니다.** 최소 길이에 걸렸을 때 그 자리에서 포기하면, 짧은 문장이 앞에 긴 응답은 영원히 잘리지 않습니다. 기준을 채우는 가장 이른 경계를 찾아야 합니다. 컴패니언 코드의 회귀 테스트가 정확히 이 두 가지를 잡습니다.

아바타가 사용자의 질문을 되읽는 사고 ★

스트리밍을 붙이고 처음 돌리면 이런 일이 일어나기도 합니다. **아바타가 사용자의 질문을 먼저 소리 내어 읽고, 그다음에 답을 합니다.**

사용자 : 볼넷이 뭐야?

아바타 : "볼넷이 뭐야? 볼넷은 투수가 공 네 개를 스트라이크 존 밖으로..."

원인은 파이프라인이 아니라 **스트리머 옵션 한 줄** 입니다. 로컬 모델의 토큰 스트리머는 기본 설정에서 **입력 프롬프트까지 토큰으로 되돌려줍니다**. 화면에 글자로 뿌릴 때는 눈에 띄어도 그러려니 하고 넘어갑니다. 그런데 그 스트림이 그대로 TTS로 들어가면 **소리가 납니다**.

```
streamer = TextIteratorStreamer(
    tok,
    skip_prompt=True,          # ← 입력 프롬프트를 돌려주지 않는다
    skip_special_tokens=True,  # ← <|eot_id|> 같은 제어 토큰을 빼고 준다
)
```

둘째 줄도 같은 종류입니다. 특수 토큰이 문자열로 새어 나오면 TTS가 **그것을 읽으려 시도합니다**. 무슨 소리가 날지는 엔진마다 다르고, 어느 쪽이든 좋지 않습니다.

텍스트 챗에서는 사소한 버그가 음성에서는 사고가 됩니다. Ch03 §4의 태그 정규화와 정확히 같은 구조입니다 — 눈으로 읽는 파이프라인과 귀로 듣는 파이프라인은 허용치가 다릅니다.

화면에 글자를 뿌려 검증하고 넘어가지 마세요. **소리로 한 번 들어 보세요**.

한국어 문장 분리를 쓸 수 없는 이유

'`kss` 같은 한국어 문장 분리 라이브러리를 쓰면 되지 않나요?' 자주 나오는 질문입니다.

쓸 수 없습니다. 그 라이브러리들은 **문장이 다 있는 텍스트**를 전제합니다. 스트리밍에서는 문장이 아직 완성되지 않은 채 조각으로 들어옵니다.

우리가 판단해야 하는 것은 이겁니다 — **"지금 들어온 이 마침표가 문장 끝인가."** 그리고 그 판단을 **뒤에 무엇이 올지 모르는 상태에서** 내려야 합니다.

3.14의 점, Dr.의 점, ...의 점이 전부 같은 문자로 도착합니다. 미래를 보면 쉽고, 못 보면 규칙을 세워야 합니다.

کم페인 코드의 청커가 쓰는 규칙은 **앞만 보고 정할 수 있는 것** 뿐입니다.

표 7.2 스트리밍 청커가 앞만 보고 내리는 판단 — 경우 · 규칙 · 이유

경우	규칙	이유
3.14입니다	경계 아님	점 뒤에 공백이 없다 — 애초에 안 걸린다
Dr. Kim	경계 아님	자주 나오는 라틴 약어 뒤의 점은 예외 목록으로 뺀다
보니... 그렇네요	경계	말줄임표는 생각이 끊긴 자리다. 여기서 끊어도 어색하지 않다
네! 그렇습니까?	경계	느낌표·물음표는 약어 예외가 없다

처음 구현은 **Dr. Kim**을 잘랐습니다. Dr. 이 한 조각으로 TTS에 따로 나갔습니다. 본문에는 **"청커가 처리한다"**고 써 두었는데 코드는 그렇지 않았고, 테스트도 없었습니다. 그 문장을 검증하려 다 발견했습니다. 지금은 위 네 경우가 전부 회귀 테스트에 있습니다.

"코드가 처리한다"고 쓰기 전에 그 케이스를 테스트에 먼저 넣으세요. 테스트가 없으면 그 문장은 바람직하지 않습니다.

7.4 두 번째 절약 — 짧게 대답하게 만들기

가장 저렴한 최적화입니다. 코드가 아니라 프롬프트로 합니다.

시스템 프롬프트에 "한두 문장으로 자연스럽게 짧게 대답하세요"를 넣습니다. 최대 출력 토큰도 낮게 제한합니다. 그리고 생성된 텍스트에서 앞의 두 문장만 잘라 씁니다.

```
s = [x.strip() for x in re.split(r"(?<=[.!?~])\s+", t) if x.strip()][:2]
```

세 겹으로 거는 이유가 있습니다. 프롬프트 지시만으로는 모델이 가끔 길게 답합니다. 토큰 제한만 걸면 문장이 중간에서 잘려 TTS가 이상하게 읽습니다. **프롬프트 · 토큰 제한 · 코드 자르기**를 다 걸어야 안정적입니다.

대화 아바타에서 긴 답변은 그 자체로 나쁩니다. 사람은 대화에서 문단으로 말하지 않습니다.

7.5 세 번째 절약 — 추론 예산 끄기

요즘 모델들은 답하기 전에 속으로 한 번 생각하는 단계를 둡니다. 어려운 문제에서는 정확도를 크게 올립니다. 그런데 **대화 아바타에서는 순수한 지연**입니다.

"안녕하세요, 오늘 기분 어때요?"에 모델이 3초를 생각하면 그건 대화가 아닙니다. API가 추론 예산을 조절하는 옵션을 준다면 0으로 두세요. 잭담에는 추론이 필요 없습니다.

복잡한 판단이 필요한 순간에만 선택적으로 켜는 구조가 이상적입니다. 게임의 추리 판정 같은 곳에서는 켜고, 인사에는 끕니다.

7.6 네 번째 절약 — 모델을 작게, 폴백을 길게

가장 빠른 모델을 첫 번째로 두고, 실패하면 다음으로 넘어가는 리스트를 씁니다.

```
MODELS = ["<가장 가벼운 모델>", "<중간>", "<큰 모델>"]
```

가벼운 모델을 앞에 두는 이유는 대화 아바타의 대부분이 잭담이기 때문입니다. 잭담에 큰 모델은 낭비입니다.

폴백은 모델 안에서 끝나지 않습니다. 저자의 게임 서버는 제공자 자체를 층으로 쌓습니다. 한 곳이 한도에 걸리면 다음 제공자로, 거기도 막히면 그다음으로. 무료 티어를 여러 곳에 걸쳐 두면 하루 종일 시연할 수 있습니다. 무료 한도 초과·일시적 장애·지역 제한은 *가끔* 일어나는 일이 아닙니다. **자주** 일어나는 일입니다.

그리고 전부 실패했을 때를 위해 **오프라인 답변** 을 준비하세요. Ch01의 홈런이가 그 예입니다 — 야구 용어 열두 개의 답이 덕셔너리로 코드에 박혀 있습니다. 우스워 보이지만 강의실 와이파이가 죽은 날 그 열두 줄이 시연을 살렸습니다.

사용자 입력 → 모델 A → 실패 → 모델 B → 실패
→ 제공자 2 → 실패 → 오프라인 사전 → "볼넷은 ..."

화면이 멈추는 것보다 조금 덜 똑똑한 대답이 낫습니다. 도메인이 정해진 서비스라면 자주 나오는 질문 열 개의 답을 박아 두는 것만으로 데모의 생존율이 달라집니다.

7.7 다섯 번째 절약 — 모델을 상시 로드

요청마다 모델을 로드하면 그때마다 수 초가 사라집니다. 당연한 이야기 같지만, 스크립트를 서버로 옮길 때 가장 자주 놓치는 자리입니다.

무거운 것은 프로세스가 뜰 때 한 번 로드해서 전역에 둡니다. 요청 핸들러는 이미 로드된 것을 쓰기만 합니다. 서버가 뜨는 데 1분이 걸려도 상관없습니다. **뜨고 나서 응답이 빠르면 됩니다.**

Track A의 실시간 모드에서는 여기에 하나가 더 붙습니다. 얼굴 전처리 결과도 캐시해 상시 유지합니다. 그러면 두 번째 요청부터는 전처리 없이 바로 추론합니다.

7.8 측정하지 않으면 개선되지 않습니다

지연 최적화에서 가장 흔한 실수는 **감으로 고치는 것** 입니다.

각 단계의 시작과 끝에 타임스탬프를 찍어 로그로 남기세요. 항목은 다섯이면 충분합니다 — 입력 확정, STT 완료, LLM 첫 문장, TTS 첫 청크, 재생 시작.

이 로그를 며칠 쌓으면 **분포** 가 보입니다. 평균만 보면 안 됩니다. 평균 1.2초인데 10%의 요청이 4초라면, 사용자는 열 번에 한 번씩 나쁜 경험을 합니다. 그리고 그 한 번을 기억합니다. **p95(100번 중 95번이 이보다 빠른 값)를 목표로 잡으세요.**

각 단계의 최악값도 따로 기록하세요. 대개 한 단계가 가끔 튀는 것이 원인이고, 그 한 단계는 대개 외부 API입니다.

7.9 파이프라인을 통째로 없애는 선택지 — S2S

지금까지의 예산 표는 STT → LLM → TTS를 이어 붙인 구조를 전제했습니다. 이것을 캐스케이드(cascaded) 파이프라인이라고 부릅니다.

여기에 다른 답이 생겼습니다. 음성을 직접 받아 음성을 내는 단일 모델입니다. 음성-대-음성, 줄여서 S2S라고 부릅니다. 중간에 텍스트가 없습니다.

그런데 S2S도 한 종류가 아닙니다. 세 세대를 구분해야 합니다.

표 7.3 S2S 세 세대 — 구조와 턴의 유무

세대	구조	턴
1세대 캐스케이드	STT → LLM → TTS	있음
2세대 턴 기반 S2S	음성 → 음성 (한 모델)	있음
3세대 전이중	음성 ↔ 음성 (동시)	없음

2세대는 파이프라인만 합쳤을 뿐 여전히 턴 기반입니다. 사용자가 말을 끝내야(엔드포인팅) 모델이 대답을 시작합니다. 빨라지기는 했습니다. 대화의 구조는 무전기 그대로입니다.

3세대가 그 구조를 깎니다. § 10에서 따로 다룹니다 — 이 책의 여러 장이 그 앞에서 다시 쓰여야 하기 때문입니다.

무엇이 좋아지나

지연이 줄어듭니다. 세 번의 왕복이 한 번이 됩니다. 광고되는 수치는 200~300밀리초 대입니다.

끼어들기가 모델 쪽에서 처리됩니다. Ch23에서 직접 만들 끼어들기(barge-in)를 제공자가 해 줍니다.

말의 뉘앙스가 남습니다. 캐스케이드에서는 사용자가 짜증난 목소리로 말해도 STT를 거치면 그냥 글자가 됩니다. S2S는 그 톤을 들었습니다. 그리고 톤으로 답합니다.

무엇을 잃나

텍스트가 없어집니다. 이것이 결정적입니다. 이 책에서 지금까지 만든 것들과 앞으로 만들 것들이 거의 전부 텍스트를 전제 하고 있습니다.

- Ch20의 감정·동작 태그 — 텍스트 앞에 붙이는 방식입니다
- Ch25의 RAG 근거 표시 — 무엇을 참고했는지 보여줄 자리가 없습니다
- Ch27의 말투 채점 — 정규식으로 어미를 검사할 텍스트가 없습니다
- 대화 로그 저장·요약(Ch24) — 별도로 전사해야 합니다

제어가 줄어듭니다. 프롬프트로 형식을 강제하기 어렵고, 출력 검사 단계를 끼워 넣을 자리가 없습니다.

광고 지연과 실제 지연이 다릅니다. 턴 검출, 네트워크 지터, 도구 호출이 없으면 실사용에서는 1.5~2.5초로 올라가는 것이 보통입니다. 캐스케이드와 결국 비슷한 자리에 도착합니다.

비용 구조가 다릅니다. 오디오 토큰 단가가 제공자마다 자릿수 단위로 벌어집니다. 긴 세션을 다루는 서비스에서는 이것이 모델 선택의 1순위가 되기도 합니다.

그래서 어떻게 고르나

얼굴이 없는 순수 음성 서비스라면 S2S가 유리한 경우가 많습니다. 읽는 텍스트가 별로 아쉽지 않기 때문입니다.

얼굴이 있는 아바타라면 캐스케이드가 여전히 기본입니다. 표정과 동작을 텍스트에서 뽑아야 하고(Ch20), 화면에 무언가를 표시해야 하며(Ch25), 인격의 일관성을 자동으로 검사해야 합니다(Ch27).

돌을 섞을 수도 있습니다. S2S가 전사(transcript)를 나란히 내주는 경우가 있습니다. 그 텍스트로 태그와 검증을 돌리면 됩니다. 제공자가 이것을 주는지가 아바타 프로젝트에서는 **모델 선택의 실질적 기준**입니다. 계약이나 가격보다 먼저 확인하세요.

정리 — 이 책은 캐스케이드를 기본으로 잡니다. 학습 가치가 크고, 각 층을 따로 교체할 수 있고, 무엇보다 **텍스트가 있기 때문**입니다. S2S는 그 위에 얹는 최적화로 다루세요. 후보와 현재 지형은 온라인 부록 K § 6.

7.10 3세대 — 턴이 사라집니다 ★

이 절은 뒤의 여러 장(Ch20·Ch23·Ch24·Ch25·Ch27)을 미리 건드립니다. 지금은 "턴이 사라지면 무엇이 바뀌는가"만 보고 넘어가도 됩니다.

2026년 7월, 듣기와 말하기를 **동시에** 하는 음성 모델이 상용화되었습니다. 오디오를 계속 받으면서 오디오를 계속 냅니다. 그리고 **초당 여러 번** 판단합니다 — 지금 말할까, 들을까, 잠깐 멈출까, 맞장구를 칠까, 도구를 부를까.

이름은 전이중(full-duplex)이고, Ch23 § 1에서 무전기와 전화로 비유한 그 구분입니다. **다만 이번에는 우리가 만드는 앱이 아니라 모델 자체가 전화가 되었습니다.**

무엇이 사라지나

엔드포인트가 사라집니다. 사용자가 말을 끝냈는지 판단할 필요가 없습니다. 모델이 계속 듣고 있으니까요. Ch23 § 2의 침묵 임계값 튜닝이 통째로 없어집니다.

"생각하는 시간"이 사라집니다. 어려운 질문을 받으면 더 큰 모델에 백그라운드로 넘기고 대화는 계속 이어갑니다. 사용자에게는 침묵이 안 보입니다. Ch08이 아이들 루프로 숨기려 했던 그 침묵을, 모델이 말로 채웁니다.

TTEA라는 지표가 흔들립니다. 첫 소리까지의 시간을 재려면 *언제부터*를 정해야 하는데, 턴이 없으면 그 시작점이 없습니다.

무엇이 생기나

맞장구를 모델이 냅니다. 사용자가 말하는 중에 "음—", "네"를 끼워 넣습니다. Ch23 §4에서 "맞장구는 끼어들기가 아니다"라고 했는데, 이제 아바타가 **맞장구를 내는 쪽**이 됩니다.

겉쳐 말하기가 정상입니다. 사람의 대화가 원래 그렇습니다. 서로 조금씩 겹칩니다.

이 책의 실시간 트랙과의 관계

이 책의 실시간 트랙(Ch21~Ch23)은 **턴 기반**으로 만듭니다 — 층이 나뉘어 있어 하나씩 배우고, 하나씩 바꿔 끼우고, 텍스트가 남아 검증할 수 있기 때문입니다 (§9). 전이중 모델을 그 위에 얹을 때 달라지는 자리를 미리 적어 둡니다.

표 7.4 전이중이 오면 이 책의 어느 줄이 바뀌는가

이 책의 내용	전이중에서
§2 지연 예산표	단계 합 대신 응답 시작 하나로 다시 켜다
Ch08 지연 숨기기	숨길 침묵이 줄어든다
Ch23 §2 엔드포인트	모델이 맞는다
Ch23 §4 끼어들기	모델이 맞는다
Ch23 §6 상태 기계	Ch23 §7의 형태로 바뀐다
Ch20 감정 태그	그대로 문제로 남는다 — 텍스트가 없다 (§9)

여전히 유효한 것도 분명합니다. 지연을 *예산*으로 사고하는 법, 측정하고 p95를 보는 습관, 그리고 §9의 텍스트를 읽는 대가는 세대와 무관합니다. 바뀌는 것은 표의 숫자이지 표를 만드는 이유가 아닙니다.

지금 무엇을 고를 것인가

얼굴 없는 음성 서비스라면 3세대를 진지하게 보세요. 대화의 질이 다른 차원입니다.

얼굴이 있는 아바타라면 아직 1세대가 안전합니다. 텍스트가 있어야 표정·동작·자막·평가가 돌아 갑니다 (§9). 전사를 함께 주는 제공자라면 3세대도 가능하지만, 겉쳐 말하는 동안 아바타의 입을 어떻게 할지 라는 새 문제가 생깁니다.

판단 — 3세대는 *대화*를 이겼지 *아바타*를 이기지 않았습니니다. 얼굴을 붙이는 순간 텍스트와 상태가 필요해지고, 그 둘을 전이중은 아직 잘 주지 않습니다.

7.11 이 장에서 기억할 것

측정할 숫자는 첫 소리까지의 시간(TTFA) 하나입니다. 목표는 2초, 관리 지표는 p95 입니다.

2초 예산에는 여유가 없습니다. 어느 한 단계도 무겁게 만들 수 없습니다. 실시간 트랙에 신경망 얼굴이 들어갈 자리가 없는 이유입니다.

가장 큰 절약은 문장 단위 청킹 입니다. 첫 문장이 완성되면 즉시 TTS로 보냅니다.

짧게 대답하게 만들기 는 프롬프트 · 토큰 제한 · 코드 자르기 세 겹으로 겹칩니다.

추론 예산을 끄고, 가벼운 모델을 앞에 두고, 풀백과 오프라인 답변을 준비 합니다.

무거운 것은 상시 로드 합니다. 서버가 뜨는 시간은 상관없습니다.

S2S는 지연을 줄이지만 텍스트를 없앱니다. 얼굴이 있는 아바타에서는 캐스케이드가 기본입니다 — 태그·근거 표시·평가가 전부 텍스트를 전제합니다. 전사를 함께 주는 S2S 인지가 실질 선택 기준입니다.

다음 장은 조금 다른 이야기입니다. 지연을 줄이는 것이 아니라 숨기는 법입니다. 그리고 이것이 실제 사용자 경험에서는 더 큰 차이를 만듭니다.

실습 코드: `code/ch07_latency/` — 단계별 계측 미들웨어 + 문장 청킹 스트리머

예상 소요: 2시간

관련 부록: C(벤치마크)

CHAPTER 8

지연을 숨기는 법

8.1 스피너는 패배입니다

바텐더(3D 렌더 얼굴 한 장으로 만든 실시간 대화 캐릭터)가 처음 완성됐을 때, 지연은 충분히 짧았습니다. 응답이 1.5초 만에 왔습니다. 그런데 써 보면 이상했습니다.



그림 8.1 바텐더 — 3D 렌더로 만든 얼굴 한 장(지도 학생 생성). 대기 중에도 이 얼굴이 살아 있어야 한다

메시지를 보낼 때마다 **화면이 한 번 깜빡였습니다**. 그리고 대답을 기다리는 동안 바텐더는 완전히 정지해 있었습니다. 사진처럼. 그래서 그 1.5초가 길게 느껴졌습니다.

로딩 스피너를 올려 봤습니다. 더 나빠졌습니다.

앞 장에서 지연을 2초까지 줄였습니다. 이제 남은 2초를 어떻게 보여줄 것인가의 문제입니다.

가장 흔한 답은 로딩 스피너입니다. 그리고 이것이 대화 아바타에서 할 수 있는 가장 나쁜 선택입니다.

스피너는 **"지금 기다리는 중"**이라고 **대놓고 알리는 장치**이기 때문입니다. 사용자의 주의를 대기 그 자체로 끌어옵니다. 사람과 사람이 대화할 때 상대의 얼굴 위에 회전하는 원이 뜨지는 않습니다. 상대는 그냥 잠깐 생각하는 표정을 짓고, 눈을 깜빡이고, 숨을 쉽니다.

대화 아바타의 대기 상태는 **"로딩"이 아니라 "들고 있음"이어야 합니다.**

8.2 아이들 루프 — 살아 있는 대기 상태

해법은 대기 상태에도 얼굴이 살아 있게 만드는 것입니다. 이것을 **아이들 루프(idle loop)**라고 부릅니다.

브라우저 렌더 트랙(Track B)에서는 저절로 됩니다. 캐릭터는 언제나 렌더되고 있고, 깜빡임과 미세한 호흡 움직임이 계속 돌고 있습니다. 대기 상태라는 것이 따로 없습니다. **입만 안 움직일 뿐**입니다.

신경망 립싱크 트랙에서는 손으로 만들어야 합니다. 방법은 이렇습니다.

무음 오디오를 넣어서 한 번 렌더합니다. 소리가 없으므로 입은 다물린 채로, 소스 영상의 눈 깜빡임과 미세 움직임만 남은 짧은 영상이 나옵니다. 이것을 아이들 영상으로 씁니다.

바텐더의 경우 4초짜리 무음을 만들어 한 번 통과시킵니다. 그러면 4초 동안 조용히 눈을 깜빡이는 바텐더가 나옵니다. 서버가 뜰 때 한 번 만들어 두고 계속 재사용합니다.

여기서 기법이 하나 필요합니다. 짧은 영상을 그냥 반복 재생하면 **이음매에서 튕니다**. 마지막 프레임과 첫 프레임이 다르기 때문입니다.

해결은 단순합니다 — **영상과 그 역재생을 이어붙여** 앞뒤로 왕복하게 만듭니다. 그러면 시작과 끝이 같은 프레임이 되어, 무한히 반복해도 이음매가 보이지 않습니다.

[원본 프레임 1~N] + [역재생 프레임 N~1] → 경계 없는 루프

ffmpeg로는 필터 한 줄입니다. 원본을 역재생한 스트림을 만들고 둘을 이어 붙이면 됩니다.

```
-filter_complex "[0:v]reverse[r];[0:v][r]concat=n=2:v=1[v]"
```

이 한 줄짜리 트릭이 **"살아 있는 대기 상태"**의 대부분을 해결합니다. 4초짜리 소스가 8초짜리 무한 루프가 되고, 이음매가 보이지 않습니다.

8.3 왜 무음을 넣어야 하나

아이들 영상을 만들 때 소스 영상을 그대로 쓰면 안 되는 이유가 있습니다.

실사 립싱크 트랙에서 소스로 쓰는 영상은 대개 **누군가 말하고 있는 영상**입니다. 그것을 그대로 아이들 루프로 돌리면 아바타가 대기 중에 **혼자 계속 떠드는** 것처럼 보입니다. 소리는 안 나는데 입만 움직입니다. 아주 이상합니다.

무음 오디오로 한 번 통과시키면 모델이 입을 다문 상태로 렌더합니다. 눈 깜빡임과 머리의 미세한 움직임은 소스에서 그대로 남습니다. 정확히 원하는 상태입니다.

8.4 크로스페이드 — 전환을 감추기

아이들 루프에서 말하는 영상으로, 그리고 다시 아이들 루프로 돌아오는 순간이 있습니다. 이 두 전환이 어색하면 앞의 노력이 전부 헛것이 됩니다.

하드 컷은 티가 납니다. 아이들 루프의 마지막 머리 각도와 말하기 시작하는 프레임의 머리 각도가 다르면 순간적으로 얼굴이 점프합니다.

짧은 크로스페이드를 겁니다. 100~200밀리초 정도면 충분합니다. 사람 눈은 이 정도의 디졸브를 전환으로 인식하지 못하고, 그냥 자연스러운 움직임으로 받아들입니다.

돌아올 때도 마찬가지로입니다. 말이 끝나는 순간 즉시 아이들로 튀지 말고, 마지막 프레임에서 아이들 루프로 부드럽게 넘어가게 합니다.

8.5 더블 버퍼링 — 깜빡임을 없애기

브라우저에서 영상 요소를 다루면 반드시 만나는 문제가 있습니다. 새 영상 소스를 지정하는 순간 **영상 요소가 잠깐 비면서 화면이 깜빡입니다**. 검은 프레임이나 흰 프레임이 한 번 지나갑니다.

메시지를 주고받을 때마다 화면이 깜빡이면, 아무리 지연이 짧아도 조악해 보입니다.

해법은 그래픽에서 오래 쓰인 기법 그대로입니다. **영상 요소를 두 개 겹쳐 두고 번갈아 씁니다**.

지금 보이는 요소가 A라면, 새 영상은 뒤에 숨어 있는 B에 로드합니다. B의 첫 프레임이 준비되면 그때 B를 앞으로 올리고 A를 뒤로 보냅니다. **A는 그때까지 계속 마지막 프레임을 보여주고 있으므로 화면이 비는 순간이 없습니다**.

이 구조 하나로 "메시지마다 깜빡임" 문제가 완전히 사라집니다.

단, 조건이 하나 붙습니다. 더블 버퍼가 깜빡임을 없애는 것은 **보여줄 이전 프레임이 있을 때**입니다. 아무것도 없는 상태에서 첫 영상을 올리면 요소가 둘이어서 똑같이 빕니다.

컴패니언 코드의 시뮬레이션이 이 차이를 프레임 단위로 셉니다.

표 8.1 버퍼 방식이 만드는 빈 프레임 — 시물레이션

방식	영상 3회 교체 중 화면이 빈 프레임
단일 버퍼	9 / 27
더블 버퍼 (아이들 루프가 이미 돌고 있음)	0 / 27
더블 버퍼 (아무것도 없이 시작)	5 / 27

§ 2의 **아이들 루프**가 § 5의 **전제조건**입니다. 서버가 뜨자마자 아이들을 먼저 올려 두는 것은 "살아 있어 보이려고"만 하는 일이 아닙니다 — 그것이 있어야 이후의 모든 교체가 깜빡이지 않습니다.

"눈으로 보니 안 깜빡이던데요" 대신 **빈 프레임**을 세세요. 0이면 참이고 아니면 거짓입니다.

진짜 브라우저에서도 셧습니다. 컴패니언 `measure_browser.py` 가 헤드리스 크로미움에 `player.html` 을 띄우고, 아이들→응답→아이들 교체 6회 동안 매 애니메이션 프레임마다 *앞 요소에 그릴 프레임이 있는가* 를 기록합니다(`_work/browser.json`). 더블 버퍼는 **323프레임 중 0**, 같은 요소에 `src` 를 바로 갈아끼우는 단일 버퍼는 **326프레임 중 2** 였습니다.

단일 버퍼의 2가 시물레이션의 9보다 작은 이유가 있습니다. 로컬 파일이라 로드가 한두 프레임이면 끝나기 때문입니다. 네트워크에서 내려받는 실제 서비스에서는 그 구멍이 로드 시간만큼 길어 집니다. 반면 더블 버퍼의 0은 로드가 아무리 길어도 0입니다.

8.6 선점 — 기다림을 앞당기기

지연을 숨기는 또 하나의 방법은 **사용자가 기다리기 전에 미리 시작하는 것** 입니다.

미리 렌더된 응답. 인사말, 안내 멘트, 자주 나오는 질문의 답은 미리 만들어 두고 재생만 합니다. 도메인이 좁은 서비스일수록 효과가 큼니다.

추임새 먼저. 사용자가 말을 끝내는 순간 "음—"이나 "네," 같은 짧은 소리를 즉시 재생하고, 그 동안 뒤에서 진짜 응답을 만듭니다. 사람이 실제로 하는 행동입니다. 다만 과하면 어색하니 매번 하지는 마세요.

입력 중 선행 처리. 사용자가 타이핑하는 동안 부분 문장으로 미리 요청을 보내 두는 방법도 있습니다. 버려지는 요청이 생기니 비용과 맞바꾸는 선택입니다.

8.7 실패도 자연스럽게

LLM이 응답하지 않았을 때, TTS가 실패했을 때 화면이 멈추면 안 됩니다.

폴백은 항상 소리가 나야 합니다. Ch07에서 만든 오프라인 답변이 여기서 쓰입니다. 조금 덜 똑똑한 대답이 침묵보다 낫습니다.

에러 메시지를 화면에 띄우지 마세요. 아바타가 "오류가 발생했습니다"라고 말하는 것도 이상하고, 붉은 토스트가 뜨는 것은 더 이상합니다. 로그에 남기고, 사용자에게는 "잠깐만요, 다시 말씀해 주시겠어요?" 정도면 됩니다.

타임아웃을 짧게 잡으세요. 30초 기다렸다가 실패하는 것보다 5초에 포기하고 폴백으로 가는 편이 훨씬 낫습니다.

8.8 이 장에서 기억할 것

스피너는 패배입니다. 대기 상태는 "로딩"이 아니라 "듣고 있음"이어야 합니다.

아이들 루프 를 만들되, 신경망 트랙에서는 **무음 오디오로 한 번 렌더** 해서 입을 다물게 합니다. 루프는 **원본 + 역재생** 으로 이어붙여 이음매를 없앱니다.

전환은 **100~200밀리초 크로스페이드** 로 감춥니다.

더블 버퍼링 으로 영상 교체 시 깜빡임을 제거합니다. 새 소스가 준비될 때까지 이전 프레임을 계속 보여줍니다.

미리 렌더 · 추임새 · 짧은 타임아웃 · 소리 나는 폴백. 실패해도 화면은 살아 있어야 합니다.

다음 장은 Part 2의 마지막이자, 이 책에서 가장 오래 남을 내용입니다. 립싱크가 맞았다는 것을 어떻게 증명하는가.

실습 코드: `code/ch08_hide/` — 아이들 루프 생성기 + 더블버퍼 프론트엔드

예상 소요: 3시간

관련 부록: L(아바타 UI/UX — 첫 3초·기다림)

CHAPTER 9

싱크를 어떻게 증명하는가

9.1 저자가 며칠을 버린 이야기

립싱크가 잘 맞았는지 자동으로 판단하고 싶었습니다. 매번 눈으로 보는 것은 느리고 주관적이니까요.

그런듯한 지표를 하나 만들었습니다. **입 벌림 정도와 음량의 상관계수**입니다. 소리가 클 때 입이 크게 벌어지면 싱크가 맞는 것 아니겠습니까.

재 보니 상관계수가 0에 가까웠습니다. 저는 이것을 "싱크가 안 맞는다"는 뜻으로 읽었고, 그 숫자를 올리려고 며칠을 썼습니다.

프레임을 음량에 맞춰 재배열했습니다. 상관계수는 올라갔고, 영상에서는 머리가 좌우로 90도씩 흔들렸습니다. 시간 축을 워핑해 정렬했습니다. 상관계수는 더 올라갔고, 영상은 끊기고 밀렸습니다. 보간으로 값을 외삽했습니다. 입 모양이 뭉개졌습니다.

지표는 계속 좋아지는데 영상은 계속 나빠졌습니다.

그때 깨달았습니다. 지표가 틀린 것이었습니다.

9.2 왜 음량 상관은 틀린 지표인가

진짜 립싱크는 음소에 맞출니다. 음량이 아닙니다.

"아"를 작게 말하면 입은 크게 벌어지는데 음량은 작습니다. "브"를 크게 말하면 음량은 큰데 입은 닫힙니다. 한국어의 "ㄱ", "ㄴ", "ㅇ"은 입을 다물어야 나는 소리이고, 이 소리들은 종종 큽니다.

즉, 제대로 된 립싱크에서 **입 벌림과 음량의 상관계수는 원래 0에 가깝습니다**. 상관계수가 높다면 그것은 오히려 모델이 음소를 무시하고 음량에만 반응하고 있다는 신호입니다.

저는 잘 작동하는 모델을 망가뜨리려고 며칠을 쓴 것입니다.

이것이 이 장이 Part 2에 있는 이유입니다. **잘못된 지표는 아무것도 측정하지 않는 것보다 나쁩니다**. 아무것도 측정하지 않으면 눈으로 보기도 하는데, 잘못된 지표는 눈을 대체해 버립니다.

9.3 나머지 나쁜 지표들

같이 버려야 할 것들이 있습니다.

단일 프레임 검사. 한 프레임을 캡처해 "입이 벌어져 있으니 잘 되었다"고 판단하는 것. 흔들림, 끊김, 뭉개짐은 전부 프레임 *사이*에서 일어납니다. 정지 화면에서는 절대 보이지 않습니다.

중앙값 편차 같은 통계량. 본포의 모양만 보는 지표는 시간 순서를 무시합니다. 프레임 순서를 무작위로 섞어도 값이 같습니다.

모델이 뱉는 손실값. 학습 손실은 학습이 수렴했는지를 말할 뿐, 이 영상이 잘 맞는지를 말하지 않습니다.

"내가 보기에 괜찮은데요". 만든 사람은 결과를 좋게 봅니다. 특히 며칠 고생한 뒤에는 더 그렇습니다.

9.4 올바른 검증 — 발화구간과 활발한 프레임

그러면 무엇을 재야 하나요. 저자가 정작한 방법은 이렇습니다.

1단계. 음성에서 발화구간을 뽑습니다. ffmpeg의 무음 검출 필터로 "여기는 소리가 나는 구간, 여기는 조용한 구간"을 시간 단위로 얻습니다.

```
ffmpeg -i audio.wav -af silencedetect=n=-30dB:d=0.3 -f null -
```

2단계. 영상에서 입 움직임의 활발도를 프레임마다 계산합니다. 프레임 간 입 영역의 변화량이면 충분합니다. 정교한 랜드마크 추적까지 갈 필요 없습니다.

3단계. 활발도 상위 N개 프레임이 발화구간 안에 들어 있는지 셉니다.

입이 가장 활발하게 움직이는 순간들은 **당연히** 소리가 나는 구간에 있어야 합니다. 조용한 구간에서 입이 활발히 움직이고 있다면 그것은 싱크가 아니라 잠음입니다.

저자의 실측입니다. 차분한 드라이버(d13, 얼굴 0.37 · 움직임 0.013) + 영어 여성 음성 + 입 영역 한정(**region=lip**) 조합으로 만든 영상은 상위 15프레임 중 15개가 전부 발화구간 안에 있었습니다. 15/15입니다.

조건을 이렇게 길게 쓰는 이유가 있습니다. 같은 파이프라인에 출하용 미디엄샷 드라이버(얼굴 비율 0.235, Ch04 §3의 경고선 0.40 아래 — 하한 0.22는 통과)를 넣자 립싱크 원 출력은 8/15(우연 대비 1.11배), 리타게팅을 거친 뒤에도 10/15였습니다. 15/15는 파이프라인의 점수가 아니라 그 *드라이버*의 점수였습니다.

15/15는 그 자체로는 아무 뜻이 없습니다 ★

앞 문단의 "**상위 15프레임 중 15개**"를 다시 보겠습니다. 100%입니다. 좋아 보입니다.

질문 하나면 무너집니다 — 그 영상에서 발화구간은 전체의 몇 퍼센트였습니까?

95%였다면 아무 프레임이나 15개 찍어도 대략 14~15개가 발화구간에 들어갑니다. 지표가 100%를 냈는데 우연도 95%를 냅니다. 이 지표는 그 영상에 대해 아무것도 말하지 않은 것입니다.

§ 2에서 "잘못된 지표는 아무것도 측정하지 않는 것보다 나쁘다"고 썼습니다. 기준선 없는 적중률이 정확히 그 종류입니다. 저자는 음량 상관을 버리고 나서, 새 지표에는 같은 질문을 던지지 않았습니다.

그래서 적중률(①)은 혼자 나오면 안 됩니다.

우연 기준선 = 발화구간의 총 길이 ÷ 영상 길이
우연 대비 = 적중률 ÷ 우연 기준선

우연 대비(②)가 1.0이면 우연과 구분되지 않습니다. 1.25배 정도는 넘어야 무언가를 측정했다고 말할 수 있습니다.

표 9.1 적중률이 같아도 뜻이 같린다 — 상황 · 적중률 · 우연 기준선 · 우연 대비

상황	적중률	우연 기준선	우연 대비	뜻
발화가 3분의 2쯤인 영상	100%	65%	1.54배	측정이 됐다
발화가 거의 전부인 영상	100%	95%	1.05배	아무것도 모른다

두 줄의 적중률이 똑같이 100%입니다. 그리고 아래 줄은 통과시키면 안 됩니다.

발화가 뻑뻑한 소재(뉴스 낭독, 광고 카피)에서는 이 지표가 원래 약합니다.
그럴 때는 지표를 믿지 말고 § 6의 눈 검사로 넘기세요. 지표가 약하다는 것을 아는 것이 지표를 잘못 믿는 것보다 낫습니다.

이 지표가 못 잡는 것 하나

기준선을 붙여도 남는 구멍이 있습니다. 소리에 맞춰 움직이기는 하는데 조용할 때 닫히지 않는 모델입니다.

이런 결과는 상위 프레임이 전부 발화구간에 있고, 우연 대비도 넉넉히 넘깁니다. 두 지표를 다 통과합니다. 그런데 눈으로 보면 어색합니다 — 말이 끝났는데 입이 계속 달싹입니다.

세 번째 숫자 ③이 필요합니다.

조용한 구간 활발도 = 무음 구간의 평균 활발도 ÷ 발화구간의 평균 활발도

제대로 따라가면 이 값이 0에 가깝고, 계속 떨고 있으면 1에 가깝습니다. 0.35를 넘으면 의심하세요.

컴패니언 코드로 세 경우를 채점한 결과입니다.

표 9.2 세 지표로 채점한 세 경우 — 적중률 · 우연 대비 · 조용한 구간 · 판정

	적중률	우연 대비	조용한 구간	판정
잘 맞는 결과	100%	1.54배	3%	통과
조용할 때 안 닫히는 모델	100%	1.54배	59%	실패
발화가 95%인 파일	100%	1.05배	3%	실패

세 줄의 적중률이 전부 100%입니다. 첫 열만 보면 셋 다 합격입니다.

지표 하나로 판정하지 마세요

이 장의 이야기가 두 번 반복된 셈입니다.

한 번은 음량 상관을 믿었다가 며칠을 버렸습니다.

두 번은 그것을 대신한 적중률을, 기준선 없이 그대로 믿을 뻔했습니다.

교훈은 지표를 고르는 법이 아니라 지표를 읽는 법입니다.

새 지표를 만들면 반드시 물으세요 — 아무것도 안 하는 방법은 몇 점을 받습니까?

그 점수가 곧 기준선이고, 기준선을 모르면 어떤 점수도 해석할 수 없습니다. Ch27의 평가 하네스에서 이 질문이 다시 나옵니다.

9.5 다른 목소리로 검증하기

이 방법의 좋은 점이 하나 더 있습니다. 언어와 목소리를 바꿔서 교차 검증 할 수 있다는 것입니다.

같은 얼굴에 한국어 음성, 영어 음성, 다른 성별의 목소리를 각각 넣고 같은 검증을 돌립니다. 모델이 실제로 소리를 듣고 입을 움직이고 있다면 셋 다 통과해야 합니다. 특정 음성에서만 통과한다면 우연이거나 과적합입니다.

저자는 영어 여성 음성으로 이 검증을 통과시킨 뒤에야 "이 파이프라인의 싱크는 언어·목소리와 무관하게 정확하다"고 결론 내렸습니다. 그 전까지는 한국어 하나만 보고 판단하고 있었습니다. 근거가 약했습니다.

③이 공짜로 통과하는 경우 ★

같은 음성으로 Ch10의 모델을 정지 사진에 돌려 검증기에 넣으면 이렇게 나옵니다 — 적중 80% · 우연 대비 1.68배 · 조용한 구간 22%. 통과입니다(`code/ch10_wav2lip/_work/probe.json`).

Ch11의 모델을 미디엄샷 영상 드라이버에 돌린 결과(53% · 1.11배 · 73%, 실패)와 나란히 놓으면, Ch10이 Ch11보다 낫다고 읽힙니다. **그렇게 읽으면 안 됩니다.** 정지 사진은 조용한 구간에 움직일 것이 없습니다 — ③은 검사한 것이 아니라 **검사할 대상이 없었던** 것입니다.

③은 드라이버가 움직이는 파이프라인(Ch11~Ch13)에서만 뜻이 있습니다. 정지 사진 파이프라인에서는 ①·②만 보세요. 검증기가 통과를 낼 때 **무엇을 검사하지 않았는지** 까지 읽어야 합니다.

9.6 눈으로 보는 검증도 규칙이 있습니다

자동 지표가 통과해도 마지막에는 사람이 봅니다. 그때 지켜야 할 규칙이 넷입니다.

연속 프레임으로 보세요. 정지 화면 비교는 금지입니다. 흔들림·끊김·몽개짐은 움직임에서만 보입니다.

소리가 큰 구간과 조용한 구간을 나눠서 보세요. 큰 소리 구간에서 입이 충분히 벌어지는가, 조용한 구간에서 입이 제대로 닫히는가. 조용한 구간에서 입이 계속 달짝이는 것은 흔한 실패입니다.

끝부분을 보세요. 싱크 밀림(드리프트)은 앞에서는 안 보이고 뒤에서 나타납니다. 10초 영상이면 마지막 2초를 봅니다. 원인은 대개 모델이 아니라 프레임레이트 불일치이고, Ch14에서 다룹니다.

비교 화면을 만드세요. 드라이버 · 소스 · 결과를 가로로 붙인 영상이면 한눈에 판단할 수 있습니다. 많은 도구가 이런 비교 영상을 자동으로 만들어 줍니다. 여기에 소리까지 입혀 두면 리뷰가 훨씬 빨라집니다.

9.7 검증을 파이프라인에 넣으세요

한 번 만들어 두면 매번 자동으로 돕니다.

렌더가 끝나면 곧바로 발화구간 검증을 돌립니다. 상위 N프레임 적중률이 기준(예: 80%) 아래면 **경고를 띄우고 결과 파일에 표시** 를 남깁니다. 나중에 파일을 연 사람이 "이건 검증을 통과한 파일"인지 알 수 있어야 합니다.

회귀를 막는 것은 사람의 주의력이 아니라 자동 게이트입니다 (§ 4의 그 질문이 Track C에서 다시 나옵니다). 회귀를 막는 것은 사람의 주의력이 아니라 자동 게이트입니다.

9.8 이 장에서 기억할 것

음량 상관은 틀린 지표입니다. 진짜 립싱크는 음소에 맞추므로 음량과의 상관은 원래 0에 가깝습니다. 이 숫자를 올리려는 시도는 결과를 망칩니다.

잘못된 지표는 아무것도 측정하지 않는 것보다 나쁩니다. 눈을 대체해 버리기 때문입니다.

올바른 검증은 발화구간 × 입 활발도 상위 N프레임 적중률입니다.

적중률은 혼자 나오면 안 됩니다. 발화가 차지하는 시간 비율이 우연 기준선 이고, 그보다 1.25배는 넘어야 무언가를 측정한 것입니다. 발화가 뻑뻑한 영상에서 이 지표는 원래 약합니다.

조용한 구간의 활발도를 같이 보세요. 앞의 두 지표를 다 통과하면서 말이 끝난 뒤 입이 계속 달짝이는 결과가 있습니다.

새 지표를 만들면 물으세요 — 아무것도 안 하는 방법은 몇 점을 받습니까?

다른 언어·다른 목소리로 교차 검증 하세요. 하나만 보고 내린 결론은 근거가 약합니다.

사람이 불 때는 연속 프레임 · 큰소리와 조용한 구간 분리 · 끝부분 확인 · 비교 화면.

검증은 파이프라인 안에 넣어 자동 게이트 로 만듭니다.

Part 2가 끝났습니다. 여기까지가 이 책에서 모델이 바뀌어도 남는 내용입니다. 이제 트랙으로 들어갑니다.

실습 코드: `code/ch09_verify/` — 발화구간 추출 + 입 활발도 계산 + 적중률 리포트

예상 소요: 2시간

관련 부록: F(실패 카탈로그 18·19번)

Part 3 · Track A

사진 한 장이 말한다

시그니처 데모 : 세상을 떠난 반려동물 사진 1장 + 대본 → 한국어 추모 영상

Ch10	Wav2Lip — 가장 빠른 첫 성공	30분 안에 첫 말하는 얼굴. 그리고 그 한계
Ch11	MuseTalk v1.5 — 실사 립싱크의 기준선	잠재공간 인페인팅, bbox_shift, 전처리 캐시
Ch12	LivePortrait — 모션만 옮긴다	표정 리타게팅, region · multiplier · stitching
Ch13	사람이 아닌 얼굴	고양이에 사람 입술이 그려지는 이유와 우회
Ch14	fps 드리프트와 mux	29 vs 29.97. 싱크가 뒤로 밀리는 진짜 원인
Ch15	트랙 A 완성 — 추모 영상 파이프라인	대본 → TTS → MuseTalk → LivePortrait → mp4

CHAPTER 10

Wav2Lip — 가장 빠른 첫 성공

10.1 왜 여기서 시작하나

30분. 이 장이 요구하는 시간은 그게 전부입니다. 그 안에 화면 속의 얼굴이 말을 합니다.

품질은 나중에 올립니다. 지금 필요한 것은 "된다"는 확인 하나입니다. 학생이 첫 시간에 말하는 얼굴을 손에 넣으면 남은 열 시간을 스스로 씁니다. 첫 시간을 오류와 싸우다 끝내면 그 남은 회복 되지 않습니다.

그래서 이 장은 최신 모델로 시작하지 않습니다.

립싱크 분야에서 오래 쓰인 이 모델의 장점은 셋입니다. 구조가 **단순하고**, 요구 사양이 **낮고**, 실패 **해도 원인이 명확합니다**. 그리고 무엇보다 30분 안에 첫 결과물이 나옵니다.

교육 현장에서 이 순서는 결정적입니다. 첫 결과물까지의 거리가 짧을수록, 남은 시간을 학생이 스스로 씁니다.

이 장의 목표는 **작동하는 기준선 하나**입니다. 품질은 다음 장에서 올립니다.

10.2 구조 — 입만 지우고 다시 그린다

입력은 둘입니다. 얼굴이 있는 이미지 또는 영상, 그리고 음성.

동작은 네 걸음입니다. 얼굴을 검출해 입 주변을 잘라냅니다. 그 자리를 **지우다시피 마스킹** 합니다. 음성 특징을 조건으로 주고 신경망이 그 자리를 다시 그립니다. 그려진 입을 원래 프레임에 되 붙입니다.

입 영역만 다시 그립니다. 눈, 코, 머리카락, 배경은 원본 그대로입니다. 그래서 빠르고, 그래서 한 계도 명확합니다.

품질을 담당하는 장치가 하나 더 있습니다. 생성된 입이 소리와 맞는지 판정하는 별도의 판별기가 학습 과정에 붙어 있습니다. 이 판별기 덕분에 이 모델은 **싱크 정확도**에서 오랫동안 기준선 역할을 해 왔습니다.

10.3 실행은 명령 한 줄입니다

준비물은 셋입니다. 가중치 파일, 얼굴 검출기 가중치, 그리고 입력 두 개.

실행은 명령 한 줄입니다. 얼굴 이미지(또는 영상)와 음성 파일을 지정하면 결과 영상이 나옵니다. 이미지를 넣으면 그 이미지를 음성 길이만큼 이어 붙여 영상으로 만들고 입만 움직입니다.

주의할 점 셋입니다.

음성은 16kHz 모노로 정규화해서 넣으세요. Ch03에서 말한 그대로입니다. 다른 포맷을 넣으면 에러 없이 이상한 결과가 나옵니다.

얼굴이 검출되지 않으면 즉시 실패합니다. 좋은 성질입니다. 조용히 나쁜 결과를 내는 것보다 낫습니다. 실패하면 Ch04의 소스 이미지 조건으로 돌아가세요.

얼굴 검출이 프레임마다 돌아 느립니다. 영상 입력에서 특히 그렇습니다. 검출 결과를 캐시하거나 배치 크기를 조절하는 옵션이 있으니 확인하세요.

10.4 실행보다 설치가 어렵습니다 ★

앞 절의 명령은 한 줄입니다. 그런데 여기서 막히는 사람이 실행에서 막히는 사람보다 훨씬 많습니다.

이유는 모델이 아니라 **코드의 나이**입니다.

연구 코드는 그 시점에 굳어 있습니다

이 모델은 논문과 함께 공개된 코드이고, 논문이 나온 **그때의 라이브러리 버전**에 맞춰져 있습니다. 그 뒤로 세상이 움직였습니다.

- 수치 라이브러리가 주요 버전을 올리면서 **일부 별칭을 없앴습니다**
- 오디오 라이브러리가 함수 시그니처를 **키워드 전용으로 바꿨습니다**
- 영상 라이브러리와 딥러닝 프레임워크도 각자 움직였습니다

그래서 최신 환경에 그냥 넣으면 **모델과 아무 상관 없는 곳에서** 터집니다. `AttributeError`, `TypeError`, 인자 이름이 다르다는 오류. 초심자는 이것을 "**모델이 안 되는구나**"로 읽고 포기합니다.

이것은 결함이 아니라 상태입니다. 유지보수되는 제품이 아니라 발표 시점에 멈춘 스냅샷입니다. 그것을 전제로 다뤄야 합니다.

requirements.txt가 있어도 안심할 수 없습니다

오래된 저장소의 의존성 파일은 **일부만 버전이 박혀 있고 나머지는 열려 있는** 경우가 많습니다. 열려 있는 것들이 그 사이에 올라가면서 조합이 깨집니다.

"**어제는 됐는데 오늘 안 된다**"의 대부분이 이것입니다. 여러분이 바꾼 것이 없어도 **설치할 때마다 다른 조합이 내려옵니다.**

세 단계로 대응합니다

① 버전을 직접 못 박습니다. 저장소가 안 박아 뒀으면 여러분이 박습니다. 저자의 실습 노트북은 수치·오디오 라이브러리를 옛 버전으로 고정해 두고 시작합니다.

```
numpy==1.23.5
librosa==0.9.2
```

숫자 자체는 시간이 지나면 바뀝니다. 남는 것은 "돌아간 조합을 적어 두고 그것으로 재현한다"는 습관입니다. 컴패니언 코드의 wav2lip_run.py 가 그 조합을 PINS 로 들고 있습니다. 돌리기 전에 저장소·가중치·얼굴 검출 모델이 있는지 전부 한 번에 알려줍니다 — 없는 것을 하나씩 세 번 발견하는 것보다 낫습니다.

② 환경을 격리합니다. Ch02 §3의 두 별 환경이 여기서 필요합니다. 이 낡은 조합을 다른 작업과 같은 환경에 넣으면, 그 다른 작업이 깨집니다.

③ 굳혀서 옮깁니다. 한 번 돌아간 조합을 컨테이너로 구워 두면 다시는 이 싸움을 하지 않습니다. Ch28 §4가 그 이야기이고, 거기서 "실행 시점에 아무것도 내려받지 않는다"를 강조하는 이유 중 하나가 이것입니다.

이 장을 쓴 뒤 다시 돌려 보니 — 같은 저장소, 세 번째 환경 ★

저자의 실습 저장소(컴패니언 paths.where("wav2lip"))에는 6월의 결과물이 두 개 남아 있습니다. result.mp4 와 result_np126.mp4 . 뒤의 것은 이름 그대로 numpy 1.26으로 돌린 것이고, 그 때 저장소에 patch.py 를 만들어 두었습니다. 없어진 별칭(np.float → float)을 되돌리고 오디오 라이브러리의 mel() 호출을 키워드 인자로 바꾸는 열댓 줄입니다.

즉 위 ①(버전을 낮춰 못 박기)이 아니라 ①'(코드를 지금 버전에 맞게 고치기)로 뚫었던 것입니다. 두 길 다 있습니다. 핀은 저장소를 안 건드리는 대신 낡은 환경을 떠안고, 패치는 환경을 최신으로 두는 대신 저장소에 손을 댑니다.

그리고 9월에 같은 명령을 기본 파이썬으로 다시 쳤습니다. 4초 만에 죽었습니다. Numba needs NumPy 2.4 or less. Got NumPy 2.5. — 6월에 돌던 조합이 그 사이 numpy 2.5로 올라가며 오디오 라이브러리의 의존성이 거부한 것입니다.

저자가 바꾼 것은 없습니다. 이것이 위 "어제는 됐는데 오늘 안 된다"의 실물이고, 두 결과물 사이 석 달이 그 간격입니다(_work/attempts.json). 그래서 ② 격리 가 필요합니다. 이 장의 컴패니언은 환경의 python을 절대 경로로 받고, 없으면 실행 대신 "무엇이 없는지"를 냅니다.

격리 환경에서 실제로 돌린 결과 — Ch11과 같은 잣대로 ★

패치된 저장소를 파이썬 3.10 격리 환경(numpy 1.26.4 · librosa 0.10.2 · torch 2.6+cu124)에 넣었습니다. 그리고 Ch11과 똑같은 드라이버와 음성(driver_therapist.mp4 · _script.wav 6.07 초)으로 돌린 뒤, Ch09의 검증기로 셋을 같은 표에 놓았습니다(ch10_wav2lip/_work/probe.json · ch11_musetalk/_work/probe.json).

표 10.1 같은 음성 — 세 실행의 Ch09 판정 (상위 15프레임 · 얼굴 기준 입 영역)

실행	벽시계	출력	적중률	우연 대비	조용한 구간	판정
이 모델 · 정지 사진(1024 ²)	—	1024 ² · 25fps · 149f	80%	1.68배	22%	통과
이 모델 · 영상 드라이버, 절반 해상도	37초	640×360 · 29.97fps · 178f	73%	1.54배	52%	실패
Ch11 모델 · 같은 영상 드라이버	108초	256 ² 얼굴 · 29.97fps · 176f	53%	1.11배	73%	실패

세 줄을 따로 읽으세요. **첫 줄의 통과는 공짜입니다.** 정지 사진은 조용한 구간에 움직일 것이 없으니 ③을 검사한 것이 아닙니다(Ch09 §5).

둘째 줄과 셋째 줄이 진짜 비교이고, 같은 드라이버에서 이 모델이 Ch11 모델보다 셋 다 낮습니다. 그러나 둘 다 실패입니다. 조용한 구간 52%와 73%는 모델이 아니라 드라이버가 만든 값입니다. 쉬지 않고 말하는 드라이버(Ch11 §6)가 침묵 구간에서도 계속 움직이기 때문이고, 어느 모델도 상류에서 들어온 움직임을 지우지 못합니다. 이 장의 제목이 이 모델을 "가장 빠른 첫 성공"이라고 부른 것은 정지 사진 줄의 이야기이고, 영상 드라이버로 넘어가는 순간 Ch04가 먼저입니다.

두 가지가 더 있습니다. **원본 720p로 돌리자 얼굴 검출에서 6분 넘게 멈춰 타임아웃**이었습니다. 배치 16의 720p 프레임이 GPU 메모리를 넘겨, 배치를 절반씩 줄이며 다시 시작하는 루프에 빠진 것입니다. `--resize_factor 2`로 절반 해상도를 주니 37초(얼굴 검출 22초)에 끝났습니다. 이 모델은 어차피 96×96 입 영역만 다시 그리므로 입력 해상도를 낮춰도 잃는 것이 거의 없습니다.

그리고 **출력이 178프레임, 5.94초**입니다. 음성 6.07초에 4프레임(0.13초) 모자랍니다. Ch11의 176/182와 같은 종류의 부족이고, Ch14 §4의 검산이 여기서도 필요합니다.

첫 성공은 굳어 있는 환경에서 얻으세요

이 장의 제목은 이 모델을 **가장 빠른 첫 성공**이라고 불렀습니다. 그 약속을 지키려면 순서가 중요합니다.

설치를 먼저 하지 마세요. 이미 조합이 맞춰진 환경 — 준비된 노트북이나 컨테이너 — 에서 **먼저 결과물을 한 번 보세요.** 화면 속의 얼굴이 말하는 것을 본 다음에 로컬 설치를 시작하면, 그때부터는 **무엇을 향해 싸우는지** 알고 하는 것입니다.

Ch02 §1이 **"먼저 무GPU 트랙으로 끝까지 한 번 가 보라"**고 한 것과 같은 사상입니다. **전체 그림을 모르는 채로 버전 충돌과 싸우는 것은 학습이 아니라 소모입니다.**

언제 버티고 언제 갈아تنا

낡은 코드를 살려 쓰는 데에도 한계가 있습니다. 판단 기준 셋입니다.

표 10.2 신호 · 뜻

신호	뜻
고정 버전이 여러분의 GPU 드라이버와 안 맞는다	갈아탈 때다. 아래로 못 내려가는 벽이다
커스텀 연산자를 빌드해야 하는데 컴파일러가 거부한다	우회 가능(부록 A). 한 번만 하면 된다
라이브러리 API만 바뀌었다	버티세요. 버전 고정으로 해결된다

드라이버·CUDA는 아래로 내려가기 어렵고 파이썬 패키지는 쉽습니다. 이 비대칭이 판단의 축입니다.

10.5 첫 성공 뒤에 보이는 네 가지 한계

첫 결과물을 본 사람은 반드시 이 넷을 만납니다. 미리 알고 있으면 다음 장으로 넘어가는 판단이 빨라집니다.

입 영역의 해상도가 낮습니다. 다시 그리는 영역이 96×96 급으로 작습니다. 전체 화면에서는 그럴듯하지만, 얼굴을 확대하면 입 주변만 흐릿한 것이 보입니다. 세로 영상이나 클로즈업에서 특히 티가 납니다.

입 주변에 경계가 보입니다. 다시 그린 사각형과 원본의 경계에서 색과 질감이 미세하게 어긋납니다. 조명이 강한 사진일수록 두드러집니다.

치아 표현이 약합니다. 저해상도의 직접적인 결과입니다.

표정이 변하지 않습니다. 입만 움직입니다. 눈은 원본 그대로이고, 이미지 한 장을 넣었다면 눈이 한 번도 깜빡이지 않습니다. 10초만 지나도 사람은 이것을 알아챱니다.

마지막 항목이 특히 중요합니다. 이미지 한 장으로 만든 결과가 어딘가 불편한데 이유를 모르겠다면, 대개 눈이 깜빡이지 않아서입니다.

10.6 눈 문제를 어떻게 풀 것인가

세 가지 길이 있고, 이 책은 세 번째를 씁니다.

첫째, 영상을 소스로 쓰기. 이미지 대신 사람이 가만히 있는 짧은 영상을 소스로 넣으면 눈 깜빡임이 원본에서 옵니다. 가장 쉬운 해법이고, 소스 영상이 있다면 이것으로 충분합니다.

둘째, 눈 깜빡임을 후처리로 합성하기. 가능은 하지만 티가 납니다.

셋째, 리타게팅 모델을 엮기. 립싱크는 립싱크대로 하고, 표정과 눈 깜빡임은 별도의 모델이 드라이브 영상에서 옮겨옵니다. Ch12에서 다룰 방식이고, Track A의 최종 파이프라인이 이 구조입니다.

세 번째가 정답인 이유는 문제를 **층으로 쪼개기** 때문입니다. 입은 소리를 따라가고, 표정은 드라이브를 따라갑니다. 각각을 따로 제어하게 됩니다.

10.7 그림에도 이 모델을 쓰는 경우

구식이라고 버릴 이유는 없습니다. 이것이 최선인 상황이 실제로 있습니다.

빠른 프로토타입. 아이디어를 검증하는 단계에서는 4분이 아니라 30초 안에 결과가 나오는 편이 훨씬 유용합니다.

저사양 환경. 요구 VRAM이 작습니다. 노트북 GPU나 무료 클라우드 노트북에서 돌립니다.

긴 영상의 대략적인 처리. 강의 영상의 더빙 교체처럼 화면에서 얼굴이 작게 나오는 경우, 입 영역 해상도의 한계가 보이지 않습니다.

교육. 구조가 단순해서 "무엇이 어떻게 일어나는가"를 설명하기에 가장 좋습니다.

10.8 이 장에서 기억할 것

이 모델은 **입 영역만 마스킹하고 다시 그립니다**. 그래서 빠르고, 그래서 한계도 그 영역에 몰립니다.

30분 안에 첫 결과가 나오는 것이 이 장의 목적입니다. **작동하는 기준선**을 먼저 갖습니다.

그런데 **실행보다 설치에서 막히는 사람이 훨씬 많습니다**. 연구 코드는 발표 시점에 굳어 있고 그 뒤로 라이브러리들이 움직였습니다. 모델과 무관한 곳에서 나는 오류를 "**모델이 안 된다**"로 읽지 마세요.

대응은 **버전 못 박기 → 환경 격리 → 컨테이너로 굳히기** 순입니다. 그리고 첫 성공은 이미 조합이 맞춰진 환경에서 얻으세요. 결과물을 한 번 본 다음에 로컬 설치를 시작해야 무엇을 향해 싸우는지 압니다.

드라이버·CUDA는 아래로 내리기 어렵고 파이썬 패키지는 쉽습니다. 버틸지 갈아탈지는 이 비대칭으로 판단합니다.

한계는 넷입니다. **저해상도 입 · 경계선 · 약한 치아 · 표정 불변**. 특히 이미지 입력에서는 **눈이 깜빡이지 않습니다**.

눈 문제의 정답은 **충을 쪼개는 것**입니다. 입은 소리를, 표정은 드라이버를 따라가게 만듭니다. Ch12에서 그 구조를 만듭니다.

다음 장에서는 실사 품질의 현재 기준선으로 올라갑니다.

실습 코드: `code/ch10_wav2lip/` — 최소 실행 스크립트 + 16kHz 모노 사전 점검 + 얼굴 검출 실패 진단
예상 소요: 30분 (첫 결과) / 1시간 (한계 관찰까지)
관련 부록: F(실패 카탈로그), H(Windows 트러블슈팅)

CHAPTER 11

MuseTalk v1.5 — 실사 립싱크의 기준선

11.1 무엇이 달라졌나

프레임 하나에 0.6초.

Ch06에서 본 그 숫자가 이 장의 모델 것입니다. 320프레임이면 195.8초, 전체 파이프라인의 85%. 느립니다. 그런데도 이 책은 이것을 Track A의 기준선으로 삼습니다.

무엇을 얻고 그 시간을 쓰는지 보겠습니다.

앞 장의 모델이 픽셀 공간에서 작은 입 영역을 다시 그렸다면, 이 모델은 잠재 공간에서 입과 턱을 인페인팅 합니다. 잠재 공간은 이미지를 압축해 담아 둔 좌표계이고, 인페인팅은 지운 자리를 주변과 어울리게 채우는 일입니다.

순서는 이렇습니다. 이미지를 VAE로 잠재표현으로 압축합니다. 그 잠재표현에서 입·턱에 해당하는 부분을 마스킹합니다. 오디오 특징을 조건으로 U-Net이 그 자리를 채웁니다. 채워진 잠재표현을 다시 이미지로 디코딩합니다.

이 구조가 주는 이점은 셋입니다.

해상도가 실용적입니다. 얼굴 크롭 기준 256×256 급으로 다시 그려지므로 확대해도 뭉개짐이 적습니다.

경계가 자연스럽습니다. 잠재 공간에서 섞이므로 픽셀 경계선이 남지 않습니다. 얼굴 파싱을 이용해 턱선을 따라 부드럽게 합성하는 처리도 들어 있습니다.

치아와 혀가 표현됩니다. 사실적이지만, 이것이 나중에 문제가 되기도 합니다(Ch13).

대신 느립니다. 저자 환경(RTX 4070 SUPER)에서 **벽시계 108초 ÷ 176프레임 ≈ 프레임당 0.6초**, 실시간의 18배입니다. Ch06의 21배는 리타게팅·mux까지 더한 값입니다. Ch06의 그 숫자가 바로 이 모델의 것입니다.

한 가지를 미리 적어 둡니다. 저장소는 *V100에서 30fps 이상* 이라고 말합니다. 거짓이 아닙니다. 그쪽은 모델 적재·얼굴 검출·VAE 인코딩을 뺀 **추론 루프만** 잔 숫자이고, 이쪽은 명령을 치고 파일이 나올 때까지 전부입니다. 둘을 같은 저울에 올리면 스무 배가 어디로 갔는지 찾느라 하루를 씁니다.

왜 이것을 기준선으로 삼나

최신 모델이어서가 아닙니다. **속도와 품질의 균형점**이고, **실시간 모드가 있기 때문**입니다.

같은 계열에 **확산 기반 립싱크**가 따로 있습니다. 입 안쪽·치아·혀 묘사가 한 단계 더 뚜렷하지만 그만큼 더 느립니다. 싱크 정확도보다 시각 충실도가 전부인 최종 렌더라면 그쪽이 맞습니다. 옮겨타도 손해는 없습니다 — **이 장에서 익히는 파라미터 감각이 그대로 따라갑니다.** bbox 계열의 입 벌림 조절, 배치 크기, 전처리 캐시는 계열 공통입니다.

30초를 넘는 긴 영상에서 얼굴이 서서히 무너진다면 **장문 정체성 유지에 특화된 계열**을 보세요. 후보 목록은 온라인 부록 K §1에 있고 6개월마다 갱신됩니다.

어느 모델도 전부를 잘하지 않습니다. **싱크 정확도 · 시각 충실도 · 속도 · 길이 · 감정** 중 하나에 강하고 나머지에 약합니다. 고품질 작업에서 컷마다 다른 모델로 라우팅하는 것이 실제 관행인 이유입니다.



그림 11.1 같은 프레임 번호의 입력과 출력. 오른쪽은 입·턱만 잠재공간에서 다시 그린 것이라 그 영역이 나머지보다 부드럽다 — 이 모델이 손대는 범위가 정확히 그 부분이다

이 그림에서 볼 것은 입 모양보다 **경계**입니다. 코 아래부터 턱까지가 원본보다 흐립니다. 얼굴 크롭 256²로 다시 그려진 영역이 그 자리이고, §3의 bbox_shift 값이 움직이는 것도 그 사각형입니다. 결과가 이상하면 먼저 이 경계가 어디에 그어졌는지 보세요 (§3).

11.2 이 모델은 이미지가 아니라 영상을 받습니다

실행은 설정 파일에 **참조 영상**과 **음성**을 적고 추론 스크립트를 부르는 방식입니다. 버전에 따라 가중치 경로와 설정 파일이 달라지므로, 실행 명령에 버전을 명시하는 습관을 들이세요.

여기서 이름에 주의할 점이 있습니다. 설정에 넣는 것은 소스 *이미지* 가 아니라 **참조 영상** 입니다. 이 모델은 영상을 받아 그 영상의 입만 바꿉니다. 이미지 한 장으로 시작하고 싶다면 그 이미지를 먼저 영상으로 만들어야 합니다.

이 사실이 Track A 전체의 구조를 결정합니다. **이 모델은 얼굴을 만들어내는 도구가 아니라, 이미 있는 얼굴 영상의 입을 소리에 맞춰 바꾸는 도구입니다.**

11.3 손델 파라미터는 셋뿐입니다

bbox_shift — 입 벌림 폭

입 영역 경계 상자를 위아래로 옮기는 값입니다. 양수를 주면 입이 더 벌어지는 방향으로 결과가 바뀝니다.

무표정한 클로즈업이나 입을 꼭 다문 소스에서는 양수를 조금만 줘도 결과가 눈에 띄게 살아납니다. 저자는 클로즈업 소스에서 10 근처를 씁니다. 너무 키우면 턱이 부자연스럽게 내려갑니다.

배치 크기

VRAM과 직결됩니다. 12GB에서 32 정도가 현실적이고, 8GB라면 절반으로 줄입니다. 배치를 키우면 20~30% 빨라집니다.

얼굴 파싱 모드와 볼 너비

합성 경계를 어디에 둘지 정합니다. 턱선 기준 모드가 기본이고, 볼 너비 값으로 좌우 합성 범위를 조절합니다. 얼굴이 화면을 꽉 채우는 클로즈업에서는 기본값이 잘 맞습니다.

11.4 전처리 캐시 — 두 번째부터 빨라집니다

이 모델은 얼굴마다 전처리를 합니다. 얼굴 검출, 크롭 좌표 계산, 프레임별 잠재표현 사전 계산. 수십 초에서 몇 분이 걸립니다.

전처리 결과는 아바타 단위로 캐시됩니다. 같은 얼굴을 다시 쓸 때는 캐시를 재사용하고 전처리를 건너뜁니다.

서비스를 만든다면 이 성질을 설계에 반영하세요.

캐릭터가 고정이라면 서버가 뜰 때 전처리를 끝내 두고 캐시를 상시 유지합니다. 그 뒤로는 추론만 돕니다.

사용자가 매번 얼굴을 올린다면 전처리 시간이 그대로 대기 시간입니다. Ch08의 기법으로 그 시간을 어떻게 보여줄지 설계해야 합니다.

11.5 실시간 모드 — GPU 한 장에 사용자 한 명

이 모델에는 실시간 모드가 따로 있습니다. 전처리를 미리 끝낸 아바타에 대해, 오디오가 들어오는 대로 프레임을 만들어 스트리밍합니다.

전제 조건이 있습니다. **모델이 상시 메모리에 올라가 있고, 아바타 전처리가 캐시되어 있어야** 합니다. 그리고 25fps 정도로 낮춰 씁니다.

저자의 실측에서 이 구성으로 실제 대화 아바타가 만들어졌습니다. 다만 두 가지를 인정해야 합니다.

GPU 한 장이 동시 사용자 한 명 수준을 감당합니다. 서비스로 키우려면 GPU 대수가 곧 동시 접속자 수입입니다. 브라우저 렌더 트랙과 원가 구조가 완전히 다릅니다.

얼굴을 바꿀 수 없습니다. 미리 전처리한 아바타만 쓸 수 있습니다.

그래서 이 책은 실시간 대화의 기본 트랙을 Track B로 잡고, 이 모드는 "실사 얼굴이 반드시 필요한 실시간"이라는 좁은 경우의 선택지로 둡니다.

스트리밍 우선 설계

최근에는 처음부터 **스트리밍을 전제로 설계된** 실사 토크헤드 계열이 나왔습니다. 정체성과 무관한 모션 공간에서 확산을 돌려 **첫 프레임 지연을 크게 낮춘** 것들입니다. 이 장의 실시간 모드가 "원래 배치용 모델에 실시간 경로를 붙인 것" 이라면, 그쪽은 반대 방향에서 출발합니다.

두 제약은 그대로입니다. GPU 한 장이 동시 사용자 한 명 수준을 감당하고, 얼굴 전처리 비용이 납습니다. 달라진 것은 *가능한가*가 아니라 *얼마에 감당하는가*입니다(Ch05 §2, 온라인 부록 K §4).

11.6 흔한 실패 셋

한글 파일명으로 실패합니다. Windows에서 OpenCV가 한글 경로의 파일을 읽지 못합니다. 조용히 빈 이미지를 돌려주거나 예외를 던집니다. **ASCII 경로로 복사한 뒤 처리** 하는 것이 확실한 해법입니다. 파이프라인 안에서 자동으로 하도록 만들어 두세요.

fps 값이 정수라서 실패합니다(자세한 것은 Ch14 §5). 저장된 모션 데이터의 fps 필드가 정수형이면 "fps must be float" 류의 오류가 납니다. 실수형으로 저장하세요.

참조 영상이 계속 말하고 있습니다. 참조 영상으로 쓴 것이 말하는 영상이면, 무음 구간에서도 입이 달싹입니다. 아이들 루프를 무음으로 따로 렌더해야 하는 이유입니다(Ch08).

실제로 돌리면서 두 번 죽었습니다

저자 환경에서 실제로 돌렸습니다. 모델이 도는 데 108초, 돌기까지 두 번 죽는 데 6분 이 걸렸습니다. 셋 다 Ch10 §4가 말한 것이었습니다.

표 11.1 회차 · 결과 · 원인

회차	죽은 자리	원인
1	<code>diffusers</code> 안에서 <code>name 'nn' is not defined</code>	<code>transformers 5.13</code> — 요구는 4.39
2	<code>accelerate.utils.memory</code> 에 함수 없음	<code>peft 0.19</code> — <code>requirements.txt</code> 에 아예 없는 패키지 가 밀려 올라옴
3	통과	<code>peft==0.12.0</code> 까지 못박은 뒤

둘째 줄이 요점입니다. 저장소의 요구 파일에 **적혀 있지도 않은** 패키지가 다른 패키지에 딸려 올라와 조합을 짰습니다. 요구 파일을 그대로 설치해도 이런 것은 못 막습니다. `musetalk_run.py` 가 돌리기 전에 네 패키지의 버전을 먼저 대조하는 이유입니다 — 6분짜리 실패를 첫 1초 안에 잡습니다.

그리고 산출물이 6프레임 모자랐습니다

6.072초 음성에 29.97fps면 182프레임이어야 합니다. **176프레임이 나왔습니다**. 컨테이너의 영상 스트림은 5.87초, 음성 스트림은 6.07초입니다.

그 176프레임을 Ch09의 검증기에 넣었습니다. 같은 음성, 얼굴 기준 입 영역. 상위 15프레임 중 **8개** 만 발화구간이고 우연 대비 **1.11배**, 조용한 구간 활발도 **73%** 입니다. 판정은 실패입니다.

모델이 나빠서가 아닙니다. Ch04 §3이 '경고'로 넘긴 미디어샷 드라이버가 침묵 구간에서도 계속 말하고 있었던 결과이고, Ch12의 리타게팅이 이것을 10/15로 조금 올릴 뿐 살리지는 못합니다(`_work/probe.json`).

이 부족분은 이 장에서는 안 보입니다. 다음 단계(Ch12)가 이 176프레임을 **29fps로 다시 써서** 길이를 6.07초에 "맞춰" 버리기 때문입니다. 길이는 맞고, 재생은 3.3% 느려집니다 — Ch14 §2의 드리프트가 **여기서 태어납니다**. 산출물을 받으면 길이가 아니라 **프레임 수** 를 세세요. `check_lengths` 가 그것을 냅니다.

11.7 이 장에서 기억할 것

이 모델은 **잠재 공간에서 입·턱을 인페인팅** 합니다. 해상도·경계·치아 표현이 모두 나아지지만 **프레임당 0.6초** 를 씹니다.

입력은 **이미지가 아니라 영상**입니다. 얼굴을 만드는 도구가 아니라 이미 있는 얼굴의 입을 바꾸는 도구입니다.

조절할 것은 **bbox_shift · 배치 크기 · 파싱 모드** 셋입니다.

전처리 캐시가 체감 성능의 핵심입니다. 고정 캐릭터면 상시 유지, 가변 얼굴이면 대기 시간 설계.

실시간 모드는 있습니다. 그러나 **GPU 한 장 $\approx$ 동시 사용자 한 명** 이고 얼굴을 바꿀 수 없습니다.

첫 6프레임만 뽑아 확인하세요. 6분을 기다린 뒤 실패를 아는 것과 첫 1초에 아는 것의 차이입니다 (§3).

다음 장에서 표정과 눈 깜빡임을 담당할 다른 층을 엿봅니다. Track A의 구조가 거기서 완성됩니다.

실습 코드: `code/ch11_musetaik/` — 설정 생성 + 네 패키지 버전 대조 + 산출물 프레임 수 검사

예상 소요: 3시간

관련 부록: A(환경), C(벤치마크), H(Windows)

CHAPTER 12

LivePortrait — 모션만 옮긴다

12.1 이 모델이 푸는 문제

하늘이(세상을 떠난 고양이 — 사진 한 장으로 말하게 하는 Track A의 얼굴) 영상에서 가장 자주 듣는 질문은 이것입니다. *"이거 어떻게 눈을 깜빡여요?"*

립싱크 모델은 입만 만듭니다. 사진 한 장으로 시작하면 눈은 처음 상태 그대로 굳어 있습니다. 10초만 지나도 사람은 그걸 알아챱니다. 이유는 몰라도 불편해집니다.

깜빡임은 다른 층에서 옵니다. 이 장의 모델입니다.

앞의 두 모델은 **소리에서 입을 만듭니다**. 이 모델은 다릅니다.

영상에서 표정과 머리 포즈를 뽑아 다른 얼굴에 그대로 옮깁니다. 소리는 아예 보지 않습니다. 이것을 리타게팅(retargeting) 또는 리인액트먼트(reenactment)라고 부릅니다. 한 얼굴의 움직임을 다른 얼굴에 입히는 일입니다.

입력은 소스 이미지 한 장과 드라이버 영상 하나입니다. 출력은 소스의 얼굴이 드라이버의 표정과 움직임을 그대로 따라 하는 영상입니다.

여기서 Ch04의 이야기가 다시 나옵니다. **드라이버는 결과에 보이지 않습니다**. 옮겨지는 것은 움직임의 궤적뿐입니다.

12.2 왜 이것이 필요한가

두 가지 이유입니다.

첫째, 눈 깜빡임과 표정. 립싱크 모델은 입만 다룹니다. 이미지 한 장으로 시작하면 눈이 한 번도 깜빡이지 않습니다. 리타게팅 모델은 드라이버의 깜빡임과 미세 표정을 그대로 옮겨 옵니다.

둘째, 사람이 아닌 얼굴. 이 모델에는 동물용 모드가 따로 있습니다. 립싱크 모델은 사람 얼굴에만 작동하지만, 이 모델은 동물 얼굴에도 표정을 옮깁니다. 이 차이가 Ch13 전체의 근거입니다.

그래서 Track A의 최종 구조는 이렇게 됩니다.

음성 → [립싱크] → 입이 정확한 사람 영상(모션 소스)

↓

소스 이미지 → [리타게팅] → 그 입·눈·표정을 대상 얼굴로 전사

층을 나눈 것입니다. 입은 소리를 따라가고, 표정은 드라이버를 따라갑니다.

12.3 조절해야 할 파라미터 넷 — 그리고 다섯째

ani-mation-region — 무엇을 옮길 것인가

가장 중요한 선택입니다.

입만 옮기기. 싱크가 가장 선명합니다. 드라이버의 잡스러운 움직임이 섞이지 않기 때문입니다. 대신 **눈이 소스 상태로 고정** 되어 한 번도 깜빡이지 않습니다.

표정 전체 옮기기. 입 + 눈 깜빡임 + 미세 표정이 전부 옮겨집니다. 자연스럽지만, 드라이버의 잡 모션이 섞여 **싱크 선명도가 조금 희석** 됩니다.

이 트랙의 대표적인 트레이드오프입니다. 정답은 드라이버가 정합니다. **차분한 드라이버일수록 전체 옮기기를 써도 흔들림이 적습니다.** Ch04에서 드라이버 조건을 그렇게 강조한 이유입니다.

driving-multi-plier — 모션 강도

옮기는 움직임의 크기를 배율로 조절합니다.

1.0 근처 는 원본 그대로입니다. **1.5 정도** 는 입이 시원하게 벌어지고 표정이 살아납니다. **2.5 이상** 은 위험합니다 — 혀가 나오고 입이 왜곡됩니다.

입을 꼭 다문 소스 이미지에서는 배율을 올려야 입이 제대로 열립니다. 그러나 올릴수록 왜곡이 따라옵니다. 소스 이미지를 잘 고르는 것이 배율을 올리는 것보다 항상 낫습니다.

stitching — 몸을 되붙일 것인가

커면 얼굴 결과를 원본 사진의 몸과 배경에 다시 합성합니다. 정장 차림의 전신 포트레이트를 살리고 싶을 때 필요합니다.

끄면 얼굴 크롭만 남습니다. 얼굴이 크게 나오고 경계 문제가 없습니다.

저자는 스티칭을 켜다가 **얼굴이 작아지고 한쪽 귀가 잘리는** 실패를 겪었습니다. 결과의 프레이밍이 중요하다면 끄고, 몸과 배경이 중요하다면 켜되 경계를 반드시 확인하세요.

crop-driving-video — 드라이버를 크롭할 것인가

드라이버 영상에서 얼굴 영역을 자동으로 잘라 줍니다. 드라이버가 미디엄샷일 때 도움이 됩니다. 이 플러그 덕분에 리타게팅에서는 미디엄샷 드라이버도 쓸 만합니다(Ch04 §3). 얼굴 비율보다 **차분함** 을 먼저 보세요.

ani-mation-region은 값이 다섯입니다

위에서 둘만 소개했지만 실제로는 다섯 중에 고릅니다.

표 12.1 값 · 옮기는 것

값	옮기는 것
lip	입만
eyes	눈만
exp	표정 전체 (입 + 눈 + 미세 표정)
pose	머리 자세만
all	표정 + 자세 (기본값)

동물에는 **lip** 이 사실상 정답입니다. Ch13 §5의 "눈알을 굴리는 기괴한 결과"는 눈이 함께 옮겨져서 생기는데, **lip** 이면 애초에 옮기지 않습니다. 드라이버를 차분한 것으로 고르는 것보다 **확실하고 공짜인 해법** 입니다.

pose 를 따로 쓸 일도 있습니다. 입은 다른 방법으로 만들고 **머리 움직임만** 가져오고 싶을 때입니다.

12.4 다섯째 파라미터 — driving-option ★

파라미터가 넷이라고 했는데, 하나가 더 있습니다. 그리고 이것이 앞의 넷보다 먼저 정해져야 합니다.

```
--driving-option expression-friendly | pose-friendly
```

expression-friendly 는 드라이버의 첫 프레임을 기준으로 삼습니다. 첫 프레임과 소스의 차이를 재 놓고, 이후 모든 프레임을 첫 프레임으로부터의 차분 으로 바꿔 적용합니다.

왜 이것이 필요한가요. 드라이버 영상의 사람은 **가만히 있을 때도 고개가 조금 기울어져 있습니다**. 그대로 옮기면 소스의 고개도 같이 기울어집니다. 첫 프레임을 기준으로 빼 두면 **드라이버의 평상시 자세가 소스의 평상시 자세에 맞춰집니다**.

pose-friendly 는 그 보정을 하지 않고 드라이버의 자세를 그대로 따릅니다. 드라이버의 머리 움직임을 **의도적으로** 가져오고 싶을 때 씁니다.

12.5 옵션이 조용히 무시되는 세 경우 ★

이 도구를 쓰다 보면 "옵션을 켜는데 아무 변화가 없다"는 순간이 옵니다. 대개 셋 중 하나입니다. 셋 다 **에러가 나지 않습니다**.

① 동물 모드에서는 driving-option이 안 들립니다

사람용과 동물용 파이프라인이 별도 파일로 갈라져 있고, `driving_option` 을 보는 분기는 사람 쪽에만 있습니다. 동물용 스크립트도 인자는 받아 주지만 아무것도 하지 않습니다.

저자의 배포용 잡 스크립트가 실제로 이 옵션을 동물 모드에 넘기고 있었습니다. 결과에는 영향이 없었으니 무해했지만, 넘겼기 때문에 효과가 있다고 믿고 있었습니다.

② 소스가 영상이면 안 들립니다

`expression-friendly` 의 보정은 소스가 이미지일 때만 적용됩니다. 소스로 영상을 넣으면 그 분기를 통과하지 못합니다. 이 책은 사진 한 장을 소스로 쓰므로 해당되지 않지만, 영상-투-영상으로 넘어가는 순간 조용히 달라집니다.

③ 문서와 코드가 다릅니다

인자 정의에는 `driving_multiplier` 가 "*expression-friendly*일 때만 쓰인다"고 적혀 있습니다. 코드는 그렇지 않습니다. 배울은 마지막 단계에서 옵션과 무관하게 곱해집니다. 옛 위치에 있던 줄은 주석 처리되어 남아 있습니다.

Ch03A §2에서 "코드 주석과 실물이 다를 때는 실물을 믿으세요"라고 썼습니다. 그 원칙은 남의 라이브러리에도 그대로 적용됩니다. 파라미터가 안 듣는 것 같으면 문서 말고 호출되는 자리를 직접 여세요.

12.6 실제로 출하한 조합 — 배울 3.0의 사연

파라미터를 하나씩 설명했지만 이것들은 서로 독립이 아닙니다. 저자가 배포용 컨테이너에서 실제로 쓰는 조합입니다 — 팻 사진 한 장 + 한국어 음성으로 추모 영상을 만드는 잡입니다.

```
inference_animals.py
-s 팻사진.jpg
-d 드라이버.mp4
--animation-region lip
--driving-multiplier 3.0
--flag-crop-driving-video
--no-flag-stitching
```

← MuseTalk이 만든 '말하는 사람'
 ← 입만. 눈알 굴리기를 원한 차단
 ← 얼굴 크롭만. 털 경계 아티팩트 회피

배울 3.0이 눈에 걸릴 것입니다. §3에서 "2.5 이상은 위험"이라고 썼는데 그 위입니다. 세 가지가 겹쳐서 가능해졌습니다.

첫째, `lip` 이라서 왜곡이 입에 갇힙니다. 배울이 커져도 눈과 머리는 애초에 옮기지 않으므로 흔들릴 것이 없습니다. §3의 "2.5 이상은 위험"은 `a11` 을 전제한 값입니다.

둘째, 동물 얼굴은 허용치가 다릅니다. Ch13 §5가 말한 그대로 — 고양이·개에서 혀가 살짝 보이는 것은 사람 얼굴에서만큼 이상하지 않습니다.

셋째, 입을 꼭 다문 펫 사진은 2.5 아래 배율로는 입이 열리지 않습니다. 소스를 고를 수 없는 상황(고인이 된 반려동물의 사진은 한 장뿐입니다)에서는 배율로 메울 수밖에 없습니다.

사람 얼굴에는 3.0을 쓰지 마세요. 위 세 이유가 하나도 성립하지 않습니다.

이 조합을 오늘 돌린 결과 ★

위 명령을 그대로 저자 환경(RTX 4070 SUPER)에서 돌렸습니다. 소스는 고양이 `s39.jpg`, 드라이버는 Ch11에서 만든 176프레임(영상 5.87초 · 음성 6.07초)짜리 `driver_therapist_script.mp4` 입니다. 로그와 출력은 컴패니언 `ch12_liveportrait/_work/` 에 있습니다.

표 12.2 출하 조합 1회 실행 (`_work/timing.txt` · `run.log` · `ffprobe`)

항목	값	읽는 법
벽시계	28초	6.07초 입력 → 실시간의 4.6배. Ch06의 32.1초와 같은 자릿수
그중 위밍업	3.9초	얼굴 분석 1.6 + 랜드마크 1.0 + X-Pose 1.3 — 두 번째 실행부터는 없다
출력 해상도	512×512	크롭 결과. 원본 사진 크기가 아니다
출력 fps	29	정수 29. 드라이버는 29.97이었다 — Ch14 §4의 재해석 대상
프레임 수	176	드라이버와 같다. 길이는 $176/29 = 6.07$ 초로 "맞춰진다" (Ch11 §6)
Ch09 판정	실패	적중 67% · 우연 대비 1.38배 · 조용한 구간 활발도 77% (상한 35%)

(Ch09 점수는 사람 얼굴 검출 기준이라 동물 산출물에서는 절대값을 믿을 수 없습니다(Ch04 §3). 여기서 읽을 것은 배율 간 *상대* 변화뿐입니다.)

마지막 줄이 이 절의 진짜 결과입니다. **배율 3.0이 문제가 아니었습니다.** Ch13 §5의 스윙대로 1.5에서도 84%였습니다.

침묵 구간에 움직임은 넣는 것은 배율이 아니라 **드라이버** 입니다. 그 드라이버는 Ch04 §3의 경고선(0.40) 아래인 0.235짜리 미디엄샷이었고, 무엇보다 쉬지 않고 말하는 영상이었습니다. 파라미터 표를 아무리 잘 맞춰도 상류가 틀리면 하류에서 못 고칩니다. 이 장의 파라미터 다섯은 *드라이버가 맞다는 전제* 위에서만 의미가 있습니다.

파라미터 표를 외우지 말고 조합의 이유를 보세요. 같은 3.0이 한 조합에서는 정상이고 다른 조합에서는 실패입니다.

12.7 사람 모드와 동물 모드

추론 스크립트가 두 개 따로 있습니다. 사람은 사람용을, 동물은 동물용을 씁니다.

동물 모드는 별도의 키포인트 검출기를 쓰고, 그 검출기를 쓰려면 **커스텀 CUDA 연산자를 직접 빌드해야 합니다**. Windows에서 이 빌드가 이 책 전체에서 가장 까다로운 환경 작업이고, 부록 A에 우회 방법을 정리했습니다.

한 번 빌드하면 다시 볼 일이 없습니다.

12.8 비교 영상을 활용하세요

이 모델은 실행할 때마다 **[드라이버 | 소스 | 결과]**를 가로로 붙인 비교 영상을 함께 만듭니다.

이 파일이 리뷰에 결정적입니다. 결과만 보면 "잘 된 건가" 판단이 어렵지만, 드라이버와 나란히 보면 어떤 모션이 어떻게 옮겨졌는지 즉시 보입니다. 드라이버가 입을 크게 벌리는데 결과가 조금만 벌어진다면 배울 문제입니다. 드라이버가 차분한데 결과가 흔들린다면 소스 문제입니다.

여기에 음성을 입혀 두면(fps 보정은 Ch14) 그대로 팀 리뷰용 자료가 됩니다.



그림 12.1 저자의 비교 영상 한 프레임 — 왼쪽 소스, 오른쪽 결과(드라이버 칸은 지면상 잘라 냈다). 이 모델은 결과만이 아니라 이런 나란히 영상을 함께 내놓는다

나란히 놓고 보면 눈에 들어오는 것이 둘입니다. 입은 사람 드라이버의 모션대로 크게 열렸고, **눈도 조금 커졌습니다** — 이 프레임은 `animation-region`을 입으로 제한하지 않은 초기 실험이라 눈·이마의 표정까지 따라왔습니다. §3의 값을 `lip`으로 좁히면 오른쪽의 눈이 왼쪽과 같아집니다. 비교 영상은 그 차이를 한 프레임에서 보여 주기 때문에 파라미터를 바꿀 때마다 이것부터 여세요.

12.9 이 모델의 한계

얼굴과 머리만 움직입니다. 손, 팔, 몸은 정지합니다. 스티칭을 켜면 원본의 몸이 그대로 붙어 있습니다. 제스처가 필요하면 Ch04 §2의 두 갈래 — 원샷 오디오→영상 계열 또는 Track B — 로 가세요.

3D를 복원하지 않습니다. 정면 얼굴을 옆모습으로 돌릴 수 없습니다. 드라이버가 크게 고개를 돌리면 결과가 무너집니다.

드라이버의 품질이 상한입니다. 드라이버가 흐릿하면 결과의 모션도 약합니다. 이것이 Ch04에서 클로즈업을 강조한 이유입니다.

12.10 이 장에서 기억할 것

이 모델은 소리를 보지 않습니다. 영상에서 표정·포즈를 뽑아 다른 얼굴로 옮깁니다.

Track A의 구조는 **층 나누기**입니다. 립싱크가 입을, 리타게팅이 표정과 눈 깜빡임을 맡습니다.

파라미터는 **region · multiplier · stitching · crop**, 그리고 그보다 먼저 정할 **driving-option**입니다. region은 **싱크 선명도 ↔ 눈 깜빡임**의 트레이드오프이고, 동물이면 **lip**이 사실상 정답입니다.

넷은 서로 독립이 아닙니다. 같은 배율 3.0이 **lip** 조합에서는 왜곡을 부르지 않고 **all** 조합에서는 얼굴을 무너뜨립니다. 표를 외우지 말고 조합의 이유를 보세요.

옵션이 조용히 무시되는 경우가 있습니다 — 동물 모드의 **driving-option**, 소스가 영상일 때의 **expression-friendly**, 그리고 문서와 어긋나는 배율 적용 조건. **에러가 나지 않으므로 직접 확인해야 합니다.**

배율은 동물·**lip** 조합이면 **3.0**(§6), 사람·**all** 조합이면 **1.5** 근처. **all**에서 2.5 이상은 혀와 왜곡을 부릅니다.

비교 영상을 리뷰의 기본 자료로 쓰세요.

다음 장에서 이 두 모델의 조합이 왜 사람이 아닌 얼굴을 말하게 하는 유일한 실용 경로인지 봅니다.

실습 코드: `code/ch12_liveportrait/` — 사람/동물 모드 실행 + 파라미터 스왑 + 비교 영상 생성

예상 소요: 3시간 (동물 모드 빌드 별도)

관련 부록: A(X-Pose 빌드), B(드라이버 카탈로그), F(실패 4번 스티칭)

CHAPTER 13

사람이 아닌 얼굴

13.1 하늘이에게 사람 입술이 그려졌습니다

하늘이 사진에 한국어 음성을 주고 실사 립싱크 모델을 돌렸습니다. 모델은 그 얼굴에서 "입이 있어야 할 자리"를 찾아냈습니다. 그리고 거기에 사람의 입술과 치아를 그렸습니다.

이 책의 출발점이 된 실패입니다.



그림 13.1 소스 이미지(왼쪽)과 §3의 해법을 거쳐 말하는 순간(오른쪽). 이것은 §1이 말한 실패 프레임이 아니라 그 뒤의 결과다 — 사람 입술이 그려진 것이 아니라 고양이 자신의 입이 열렸다

보기 괴로웠습니다. 그리고 저는 그 결과를 만들려고 며칠을 쓴 참이었습니다.

버그가 아닙니다. 배운 대로 정확히 동작한 것입니다.

이 모델은 사람 얼굴 영상으로 학습되었습니다. 오디오 특징이 주어졌을 때 그 자리에 무엇을 그려야 하는지를 배웠고, 그 "무엇"은 언제나 사람의 입이었습니다. 고양이 입이 어떻게 생겼는지는 한 번도 본 적이 없습니다.

학습 데이터의 분포 밖에서는 모델이 조용히 이상해집니다. 에러를 내지 않기 때문에 더 위험합니다.

13.2 시도했다가 버린 것 넷

정답에 닿기 전에 저자가 시도한 것들입니다. 전부 실패했고, 실패의 이유가 각각 다릅니다. 부록 F의 앞부분이 이 목록입니다.

픽셀 프레임 재배열. 음량에 따라 입이 벌어진 프레임과 다문 프레임을 골라 재배열했습니다. 입은 움직이는 것처럼 보였습니다. 대신 **머리 포즈가 프레임마다 튀어 좌우로 90도씩 흔들렸습니다.** 각 프레임의 머리 각도가 다르다는 것을 계산에 넣지 않은 것입니다.

시간 워핑과 정렬. 프레임 시퀀스를 시간 축에서 늘이고 줄여 소리에 맞췄습니다. 같은 프레임이 여러 번 반복되면서 **영상이 멈췄다 튀는** 결과가 되었습니다.

모션 데이터 재타이밍. 저장된 모션 시퀀스를 음량에 맞춰 재배열했습니다. 발음과 전혀 맞지 않는 **가짜 싱크**가 나왔습니다. 입은 부지런히 움직이는데 아무 말도 하지 않는 것처럼 보입니다.

보간과 외삽. 중간 값을 만들어 부드럽게 이으려 했습니다. **입 모양이 뭉개졌습니다.**

이 실패들에는 공통 원인 하나가 있어 보였습니다 — **전부 소리를 음량으로만 이해했다는 것.** § 4에서 이 진단이 절반만 맞았다는 것을 보입니다. Ch09의 잘못된 지표와 같은 뿌리입니다.

다만 이 목록을 통째로 버리지는 마세요. 넷을 한꺼번에 시도했다가 다 같이 실패했기 때문에, 저자는 오랫동안 **재타이밍 자체가 틀렸다**고 생각했습니다. 나중에 하나씩 떼어 보니 그렇지 않았습니다 — § 4에서 다시 다룹니다.

13.3 정답 — 층을 나눈다

해법은 문제를 다시 정의하는 데서 나왔습니다.

필요한 것은 "고양이 얼굴에 소리에 맞는 입 움직임"입니다. 이것을 둘로 쪼갭니다.

1. **소리에 맞는 입 움직임을 만든다** — 사람 얼굴에서. 모델이 잘하는 일입니다.
2. **그 움직임을 고양이 얼굴로 옮긴다** — 리타게팅으로. 다른 모델이 잘하는 일입니다.

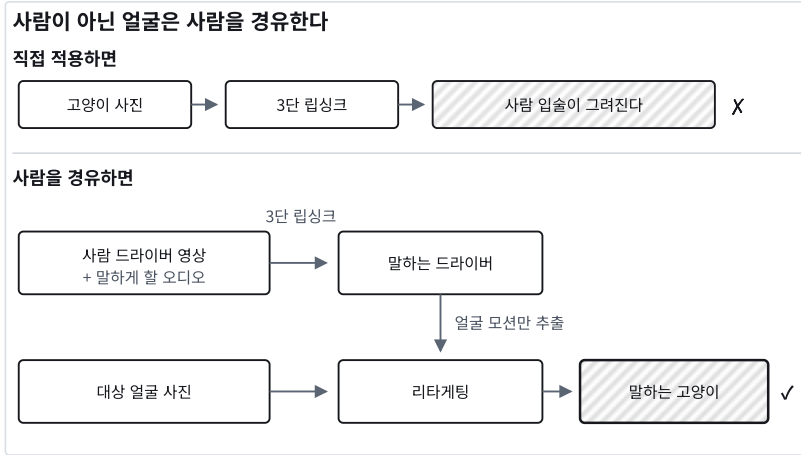


그림 13.2 직접 적용하면 사람 입술이 그려진다 — 사람 드라이버의 모션만 옮겨야 한다

드라이버로 쓴 사람은 결과에 전혀 보이지 않습니다. 가져오는 것은 입이 벌어지고 닫히는 꺾적뿐입니다.

이것이 Track A의 핵심 트릭이고, 사람이 아닌 얼굴을 말하게 하는 유일한 실용 경로입니다.

13.4 실패했다고 한 것을 다시 봅니다 ★

§ 2에서 넷을 전부 실패로 묶었습니다. 며칠을 버린 뒤에 내린 결론이었습니다.

그 결론은 너무 넓었습니다.

실패한 것은 **재타이밍**이라는 발상이 아니라, 그 안에 섞여 있던 **세 개의 선택**이었습니다. 셋은 서로 독립이고, 각각이 혼자서도 결과를 망칩니다.

세 번 고쳐 썼습니다.

표 13.1 재타이밍 세 판 — 무엇을 신호로 썼나

판	신호	결과
1판 — 음량 판	음량(RMS)을 단조 증가로 맞춤	입이 열린긴 하는데 음소와 어긋난다 — 'ㅂ'에서 벌어지고 'ㅌ'에서 닫힌다
2판 — 음소 판	모음·자음을 나눠 매핑	나아졌지만 경계에서 뒤틀다 — 자음 구간이 짧아 프레임이 급하게 건너뛴다
3판 — 모션 판	원본 모션의 입 개방도를 프레임별로 재고 그 곡선에 맞춤	채택 — 튀지 않고 어긋남도 줄었다

세 판의 차이는 신호였습니다. 같은 재배열 코드에 무엇을 기준으로 줄을 세우느냐만 바꿨습니다. 1판이 실패한 이유가 §2의 결론("음량을 신호로 쓰면 음소를 놓친다")입니다. 3판이 통과한 이유는 신호를 음성이 아니라 **이미 잘 만들어진 모션 자체**에서 뽑았기 때문입니다.

세 판의 스크립트와 결과 영상이 저장소에 나란히 남아 있습니다. 같은 소스·같은 음성이라 세 개를 이어서 재생하면 이 표가 눈으로 확인됩니다.

표 13.2 선택 · 왜 망하는가

선택	왜 망하는가
픽셀을 재배열한다	프레임을 골라 이어붙이면 좌우가 된다
음량을 신호로 쓴다	Ch09 §2 그대로 — 립싱크는 음소에 맞는다
외삽 한다	클립에 없는 입 모양을 만들어내면 뭉개진다

셋을 하나씩 걷어내면 남는 것이 있습니다.

① 픽셀이 아니라 모션을 재배열합니다

프레임 이미지를 골라 이어붙이면 이음매마다 화면이 튕니다. 그런데 리타게팅 도구는 **모션 파라미터**(표정·회전·이동·배율)를 입력으로 받고, 그것으로 **매 프레임을 새로 렌더** 합니다.

그러면 재배열의 대상을 픽셀이 아니라 모션으로 바꿀 수 있습니다. 순서를 아무리 바뀌도 렌더러가 새로 그리므로 **픽셀 흔들림이 원리적으로 생기지 않습니다.**

세 가지를 같이 지킵니다.

단조(monotonic) 진행. 소스 프레임 인덱스가 뒤로 가지 않게 합니다. 되돌아가면 머리가 흔들립니다.

한 걸음의 상한. 출력 한 프레임에 소스를 최대 세 프레임까지만 전진시킵니다. 이 값이 클수록 소리에는 잘 맞고 머리 움직임은 거칠어집니다. **3 정도가 균형점**이었습니다.

부메랑으로 이어 붙입니다. 소스 클립이 짧으면 정방향과 역방향을 번갈아 이어 원하는 길이를 만듭니다 — Ch08 §2의 아이들 루프와 같은 트릭입니다. 시작과 끝이 같은 프레임이라 이음매가 없습니다.

② 음량이 아니라 발음 타이밍을 신호로 씁니다

여기가 이 절의 핵심입니다.

음량으로 맞추면 Ch09 §2의 함정에 그대로 빠집니다. 그러면 무엇을 신호로 쓰나요.

사람 드라이버에 립싱크를 걸어 만든 영상의 입 벌림을 씁니다. 그 영상은 소리에 맞춰 입이 열리고 닫히므로, 거기서 뽑은 곡선이 곧 **발음 타이밍**입니다.

정리하면 이렇게 됩니다.

표 13.3 가진 것 · 방법

가진 것	방법
사진 한 장뿐	§ 3 — 사람 드라이버의 모션을 통째로 리타게팅
짧은 영상도 있음	이 절 — 타이밍만 사람에게서, 입 모양은 자기 것으로

후자가 낫습니다. 그 동물의 실제 입 모양을 쓰기 때문입니다. 사진뿐이라면 §3으로 가되, **혹시 영상이 없는지**를 먼저 물어보세요. 대개 있습니다.

결론을 줍니다

§ 2의 문장을 이렇게 고칩니다.

~~프레임 재배열 · 시간 워핑 · 모션 재타이밍 · 보간 외삽은 전부 실패한다~~

픽셀을 재배열하면 튀고, 음량을 신호로 쓰면 음소를 놓치고, 외삽하면 뭉개진다.

셋을 다 건어낸 재타이밍은 동작한다.

처음의 결론이 틀렸던 것이 아니라 **덜 나뉘어 있었습니다**. 실패를 정리할 때 흔한 일입니다. 여러 선택을 한꺼번에 시도했다가 다 같이 실패하면, 무엇이 원인이었는지 모르는 채로 전부 버리게 됩니다.

한 번에 하나씩 바꿨어야 했습니다.

13.5 동물 얼굴에 붙는 조건 넷

구조가 돌아가더라도, 동물에는 사람보다 까다로운 조건이 붙습니다.

정면성이 더 중요합니다. 동물 얼굴은 구조가 사람과 달라서, 조금만 돌아가도 키포인트 검출이 흔들립니다.

모션 강도가 딜레마입니다. 입을 다문 동물 사진에서 입을 벌리려면 배율을 올려야 하는데, 올리면 허가 나옵니다. 고양이나 개에서는 어느 정도 자연스러워 보이는 수준이라 참을 만하지만, 배율이 3.0을 넘으면 확실히 이상해집니다. §4의 방법을 쓸 수 있으면 이 딜레마 자체가 사라집니다 — 없는 입 모양을 만들어낼 일이 없기 때문입니다.

드라이버의 차분함이 사람보다 훨씬 중요합니다. 사람 얼굴에서 자연스럽던 눈동자 움직임이 고양이 얼굴에서는 **눈알을 굴리는** 기괴한 결과가 됩니다.

털과 수염 경계에서 아티팩트가 생깁니다. 스티칭을 끄고 얼굴 크롭으로 가는 편이 대개 낫습니다.

입 벌린 소스는 입을 다물지 못합니다 ★

리타게팅은 소스의 입 모양에서 출발 합니다. 드라이버가 주는 것은 "얼마나 더 벌려라"이지 "이 모양으로 만들어라"가 아닙니다. 그래서 소스가 이미 입을 벌리고 있으면 드라이버가 다물어도 결과는 다물지 못합니다.

같은 드라이버로 두 고양이를 돌려 확인했습니다. 하늘이 자리에 쓴 공개 예제 사진(등장인물 페이지 참조)은 입을 다문 정면이고, 줄무늬 고양이는 입을 벌린 채 찍힌 사진입니다.

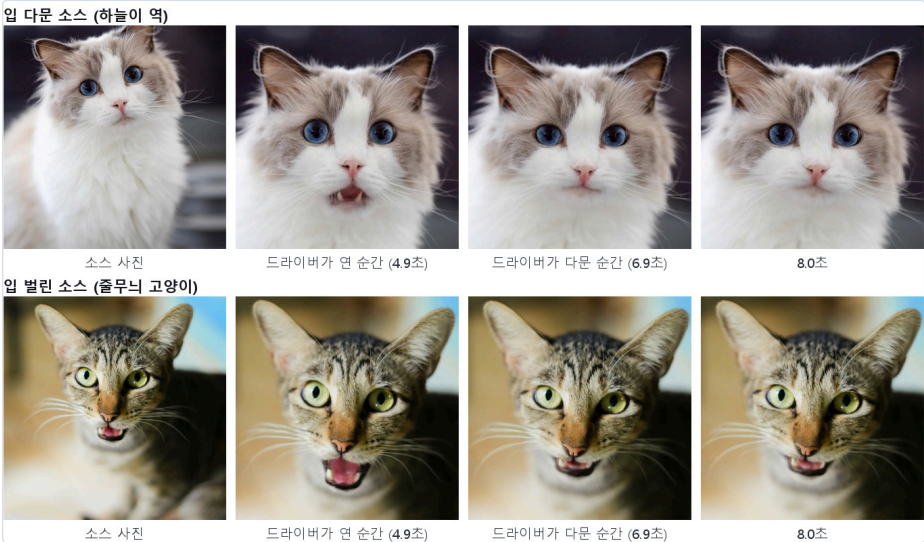


그림 13.3 같은 드라이버, 소스만 다르다. 윗줄(입 다문 소스)은 드라이버가 다물 때 다물고 열 때 연다. 아랫줄(입 벌린 소스)은 어느 순간에도 입을 다물지 못한다

숫자로도 그렇습니다. 결과 영상의 입 영역에서 **입 공동(어두운 픽셀)의 비율**을 프레임마다 재면 (code/ch13_nonhuman/measure_open.py), 입 다문 소스는 0.005에서 출발해 최대 0.075까지 열렸다가 320프레임 중 299프레임이 다시 다물려 있습니다. 입 벌린 소스는 0.204에서 출발해 **가장 다문 순간에도 0.165** 이고, 다문 프레임이 0 입니다. 헛까지 계속 보입니다.

그래서 동물 소스의 첫 조건은 사람과 반대입니다. 사람 립싱크(Ch11)는 입이 살짝 벌어진 소스를 좋아하지만(Ch04 §2), 동물 리타게팅의 소스는 **입을 다문 정면** 이어야 합니다. 입을 벌린 예쁜 사진은 표지에는 좋아도 소스로는 못 씁니다.

배율을 바꿔 봤습니다 — 조용한 구간 수치는 꿈쩍도 안 합니다

혀 딜레마를 숫자로 보려고 저자 환경에서 배율만 바꿔 넷을 돌렸습니다(고양이 s39 · 같은 드라이버 · lip).

표 13.4 배율 · 적중률 · 우연 대비 · 조용한 구간 · 판정

배율	적중률	우연 대비	조용한 구간	판정
1.5	53%	1.11배	84%	실패
2.0	60%	1.24배	82%	실패
3.0	67%	1.38배	77%	실패
4.0	67%	1.38배	73%	실패

(이 값들은 사람 얼굴 검출 기준이라 동물 산출물에서는 절대값을 믿을 수 없습니다(Ch04 § 3). 여기서 읽을 것은 배율 간 **상대** 변화뿐입니다.)

배율을 올리면 적중률은 오릅니다. 말할 때 입이 더 크게 벌어지니 상위 프레임이 발화구간으로 몰립니다. 그런데 **조용한 구간은 넷 다 70%를 넘습니다**. 배율은 말할 때와 조용할 때를 **같이** 키우므로 그 비율을 건드리지 못합니다.

즉 **말이 끝나도 입이 안 닫히는 것은 배율의 문제가 아닙니다**. 드라이버(사람 립싱크 영상)가 이미 조용할 때 73%로 떨어지고 있었고(Ch11 § 6), 리타게팅이 그 떨림을 줄이기는커녕 낮은 배율에서 더 키웠습니다, 고양이네 그것을 물려받았을 뿐입니다. 배율은 혀를 만들지만, 조용한 입을 만들지는 못합니다.

파라미터를 스왑하기 전에 **상류를 먼저 채점하세요**. 상류가 떨어지면 하류의 어떤 값도 통과하지 못합니다.

13.6 캐릭터와 3D 렌더 얼굴

동물만이 아닙니다. 애니메이션 캐릭터, 3D 렌더 이미지, 일러스트도 "사람이 아닌 얼굴"에 속합니다.

애니메이션 스타일 캐릭터는 리타게팅에서도 입이 거의 움직이지 않는 경우가 많습니다. 얼굴 구조가 학습 분포와 너무 다르기 때문입니다. 이런 얼굴은 Track B로 가는 편이 빠릅니다 — 2D 파츠로 쪼개서 코드로 움직입니다.

사실적으로 렌더된 3D 캐릭터는 의외로 잘 됩니다. 저자의 경험에서는 오히려 실사 스톡 사진보다 결과가 좋았습니다. 이유는 Ch04에서 말한 그대로입니다 — 정면이고, 조명이 고르고, 얼굴이 크고, 배경이 단순합니다. **소스 이미지의 조건을 전부 만족하기 때문**입니다.

실무에서 바로 쓰이는 발견입니다. 좋은 실사 소스를 찾기 어렵다면, 3D 캐릭터를 렌더해서 쓰는 쪽이 더 나은 결과를 낼 수 있습니다.

13.7 이 장에서 기억할 것

실사 립싱크 모델은 사람 얼굴에만 작동합니다. 다른 얼굴에 쓰면 그 자리에 사람 입술과 치아를 그립니다. 에러가 나지 않아서 더 위험합니다.

픽셀을 재배열하면 튀고, 음량을 신호로 쓰면 음소를 놓치고, 외삽하면 뭉개집니다. 셋은 서로 독립이고, 셋을 다 건어낸 재타이밍은 동작합니다.

여러 선택을 한꺼번에 시도했다가 다 같이 실패하면 무엇이 원인이었는지 알 수 없습니다. 한 번에 하나씩 바꾸세요.

정답은 층 분리입니다. 사람 드라이버에 립싱크를 걸고, 그 모션만 대상 얼굴로 리타게팅합니다.

그 동물의 짧은 영상이 있다면 한 단계 더 갑니다 — 타이밍만 사람에게서 가져오고, 입 모양은 그 동물 자신의 프레임을 씁니다. 실제로 벌린 입을 쓰므로 혀가 나오지 않습니다.

동물에는 조건이 더 붙습니다. 정면성 · 배울 딜레마(혀) · 드라이버의 차분함 · 스티칭 끄기.

잘 렌더된 3D 캐릭터는 실사보다 결과가 좋을 수 있습니다. 소스 조건을 전부 만족하기 때문입니다.

다음 장은 마지막 남은 문제입니다. 입이 잘 움직이는데도 뒤로 갈수록 소리보다 늦어지는 현상.

실습 코드: `code/ch13_nonhuman/` — 사람 드라이버 경유 파이프라인 + 배울 스왑 비교 + 모션 재타이밍 (단조 DTW · 외삽 없는 보간)

예상 소요: 3시간

관련 부록: F(실패 카탈로그 1~13번)

CHAPTER 14

fps 드리프트와 mux

14.1 앞은 맞는데 뒤가 밀립니다

처음 2초는 완벽합니다. 5초쯤에서 미묘하게 어긋납니다. 10초에서는 확실히 늦습니다.

파이프라인은 완성됐고, 입도 잘 움직이고, 검증도 통과했습니다. 그런데 영상을 끝까지 보면 **뒤로 갈수록 입이 소리보다 늦습니다.**

이 증상을 만나면 대부분 모델을 의심합니다. 배울을 바꾸고, 드라이버를 바꾸고, region을 바꿉니다. 전부 소용없습니다. **원인이 모델이 아니기 때문입니다.**

14.2 원인은 29와 29.97입니다

파이프라인의 각 단계가 서로 다른 프레임레이트로 돌고 있었습니다.

립싱크 단계에서 쓴 드라이버 영상은 **29.97fps**였습니다. 정확히는 30000/1001입니다. NTSC 시대에서 내려온 이 어중간한 숫자가 지금도 대부분의 영상 파일에 남아 있습니다.

리타이밍 단계는 결과를 **29fps**로 내보냈습니다.

두 값의 차이는 3.3%입니다. 프레임 수는 같은데 재생 속도가 3.3% 느립니다. 10초짜리 영상이라면 끝에서 **0.33초**가 밀립니다.

330밀리초는 확실히 보입니다. 방송 규격(ITU-R BT.1359)이 잡는 검출 한계가 소리가 앞설 때 약 45ms, 뒤질 때 약 125ms이고, 허용 한계도 90ms/185ms입니다. 330ms는 그 어느 쪽보다도 멀리 있습니다.

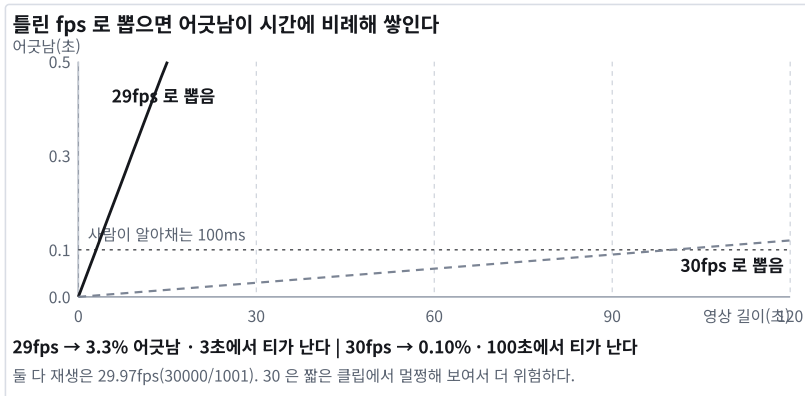


그림 14.1 어긋남은 영상 길이에 비례해 쌓인다 — 30fps는 100초에서야 티가 나서 더 위험하다

30은 29보다 더 위험합니다

29로 뽑았다면 3초 만에 티가 납니다. 시연 영상 한 번 돌려 보면 바로 압니다.

30으로 뽑으면 0.1%입니다. 10초짜리 테스트 클립에서 어긋남은 0.01초 — 아무도 못 느낍니다.

그래서 통과합니다. 그리고 100초짜리 실제 콘텐츠에서 처음 터집니다.

표 14.1 뽑은 fps · 어긋남 비율 · 100ms를 넘는 시점

뽑은 fps	어긋남 비율	100ms를 넘는 시점
29	3.3%	3초 — 즉시 걸린다
30	0.1%	100초 — 테스트를 통과하고 실전에서 터진다

"짧은 클립에서 잘 되던데요"는 검증이 아닙니다. 길이를 늘려 보는 것이 검증입니다.

전체가 균일하게 밀리는 것이 아니라 뒤로 갈수록 누적 되는 것이 이 문제의 지문입니다. 이 패턴을 보면 프레임레이트를 의심하세요.

14.3 잘못된 해결책 — setpts로 늘리기

가장 먼저 떠오르는 해법이 있습니다. 영상 길이를 음성 길이에 맞춰 늘리거나 줄이는 것입니다. ffmpeg의 setpts 필터로 재생 속도를 조절하면 됩니다.

하지 마세요.

전체 길이는 맞습니다. 대신 프레임 타이밍이 다시 왜곡 됩니다. 프레임을 복제하거나 버리게 되고, 그 과정에서 미세한 끊김이 생깁니다. 그리고 원인은 그대로 남아 있으므로 다른 조건에서 다시 나타납니다.

저는 이 방법으로 한동안 버텼습니다. 조건이 바뀔 때마다 값을 다시 맞춰야 한다는 것을 깨닫고 포기했습니다. 증상을 덮는 해법은 반드시 다시 돌아옵니다.

14.4 올바른 해결책 — fps를 재해석한다

프레임 데이터 자체는 맞습니다. 잘못된 것은 파일에 적합한 재생 속도 뿐입니다.

그러면 재생 속도만 고쳐 주면 됩니다. 프레임은 건드리지 않고, 컨테이너에 "이 영상은 사실 29.97fps로 재생해야 한다"고 알려주는 것입니다.

ffmpeg에서는 입력 앞에 프레임레이트를 지정 하면 됩니다. 입력 옵션이므로 반드시 -i 앞에 와야 합니다.

```
ffmpeg -r 30000/1001 -i lp_output.mp4 -i voice.wav \
-map 0:v -map 1:a -shortest \
```

```
-metadata comment="AI-generated · 생성 로그 ID" final.mp4
```

마지막 줄은 fps와 무관하지만 **같은 명령에 늘 붙어 다녀야** 합니다. 2026년부터 합성 영상은 표시가 의무이고(Ch29 §3), 파일 단계에서 가장 싼 표시가 이 메타데이터 한 줄입니다. 컴패니언 `mux_lint.py` 는 mp4 출력에 이 줄이 없으면 지적합니다 — 잇는 것은 사람이고, 세는 것은 코드입니다.

`-r` 이 `-i` 앞에 있으면 "이 입력을 이 프레임레이트로 해석하라"는 뜻입니다. 뒤에 있으면 "출력을 이 프레임레이트로 만들어라"는 뜻입니다. **완전히 다른 동작입니다.** 이 위치 하나가 문제를 고치기도 하고 더 망가뜨리기도 합니다.

`-shortest` 는 둘 중 짧은 쪽에서 끊습니다. 음성 끝의 무음이 조금 잘려도 무방합니다.

재해석이 못 고치는 경우 — 프레임이 모자랄 때 ★

실물에 적용해 보니 결과가 이 장의 서사와 **반쯤 어긋났습니다.**

립싱크 단계가 6.072초 음성에 **176프레임** 을 냈습니다(29.97fps면 182이어야 합니다). 리타게팅이 그것을 **29fps** 로 써서 길이가 6.069초 — 음성과 3ms 차이로 "맞았습니다". 길이 대조는 통과합니다.

그래서 §4 대로 `-r 30000/1001` 로 재해석했습니다.

표 14.2 fps · 길이 · 음성과의 차 · 재생 속도

	fps	길이	음성과의 차	재생 속도
재해석 전	29	6.069s	0.003s	3.3% 느림 — 끝에서 0.2s 밀림
재해석 후	29.97	5.873s	0.199s	정확 — 그런데 0.2s 짧다

둘 다 틀렸습니다. 하나는 속도가 틀리고 하나는 길이가 틀립니다. 재해석은 속도 를 고치는 도구이지 *프레임* 수를 만드는 도구가 아닙니다. 원인은 상류가 6프레임을 덜 낸 것이고(Ch11), 그것은 컨테이너의 어떤 값으로도 고쳐지지 않습니다.

공용 `media.check_lengths` 가 처음에는 이것을 **못 잡았습니다.** 길이만 비교했기 때문에 29fps 판을 통과시켰습니다. 지금은 **음성 길이 × fps** 로 기대 프레임 수를 계산해 부족분을 같이 냅니다.

길이가 맞다고 타이밍이 맞는 것은 아닙니다. 프레임 수 ÷ fps가 음성 길이와 같아야 하고, 그 등식의 세 항을 다 봐야 합니다.

14.5 원칙 — 파이프라인 전체에서 fps를 하나로

근본 해법은 처음부터 값을 통일하는 것입니다.

파이프라인의 fps를 한 곳에서 정의 하고, 모든 단계가 그 값을 참조하게 합니다. 상수 하나로 두세요.

드라이버 영상을 미리 정규화 하세요. 29.97fps 소스를 쓸 거라면 파이프라인 진입 시점에 목표 fps로 한 번 변환해 두는 편이 안전합니다.

각 단계의 출력 fps를 로그로 찍으세요. ffmpeg으로 확인해서, 예상과 다르면 즉시 경고합니다. 이 한 줄이 나중에 며칠을 아깁니다.

정수와 실수를 구분하세요. 모션 데이터를 저장할 때 fps를 정수형으로 넣으면 라이브러리가 "fps must be float" 류의 오류를 냅니다. 저자가 실제로 겪은 실패입니다.

그리고 29.97은 30000/1001이 아닙니다.

```
30000 / 1001 = 29.97002997...
29.97       = 29.97000000
```

백만분의 1 차이입니다. 실무에서 문제를 일으키는 크기는 아닙니다. 문제는 되돌릴 수 없다 는 것입니다. 어딘가에서 29.97 이라는 소수로 한 번 반올림되고 나면, 그 값에서 30000/1001 을 복원할 방법이 없습니다 — 2997/100 이 훨씬 가깝기 때문입니다.

그래서 컴패니언 코드는 되돌리려 하지 않고 표로 압니다.

```
NTSC_FAMILY = {
    # 표기값 → 진짜 값
    Fraction(2997, 100): Fraction(30000, 1001), # 29.97
    Fraction(5994, 100): Fraction(60000, 1001), # 59.94
    Fraction(23976, 1000): Fraction(24000, 1001), # 23.976
}
```

반올림된 표기값은 원본을 복원하지 못합니다. 이 원칙은 fps 밖에서도 그대로 쓰입니다.

14.6 mux 단계에서 명시할 넷

영상과 음성을 합치는 마지막 단계에서 확인할 것들입니다.

스트림 매핑을 명시하세요. 여러 입력이 있을 때 어느 스트림을 쓸지 자동으로 고르게 두면, 소스 영상에 딸린 원본 오디오가 섞이는 사고가 납니다.

픽셀 포맷을 지정하세요. yuv420p가 아니면 일부 플레이어와 브라우저에서 재생되지 않습니다. 자동 생성된 영상이 "왜 이 브라우저에서만 안 나오지" 하는 경우의 대부분이 이것입니다.

길이를 최종 확인하세요. 음성 길이와 결과 영상 길이를 초 단위로 비교해서, 0.1초 이상 차이 나면 경고합니다.

코덱을 명시하세요. H.264 + AAC가 가장 넓게 호환됩니다.

14.7 이 장에서 기억할 것

뒤로 갈수록 밀리는 싱크의 원인은 대개 프레임레이트 불일치입니다. 29 vs 29.97은 3.3% 차이이고, 10초 영상에서 0.33초를 만듭니다.

setpts로 늘리지 마세요. 증상을 덮고 프레임 타이밍을 왜곡합니다.

해법은 **입력 앞의 -r 로 fps를 재해석** 하는 것입니다. -i 앞과 뒤는 완전히 다른 동작입니다.

파이프라인 전체에서 fps를 상수 하나로 통일 하고, 각 단계의 출력 fps를 로그로 검증하세요.

mux 단계에서는 스트림 매핑 · 픽셀 포맷 · 길이 확인 · 코덱 을 명시합니다.

그리고 이 체크리스트는 눈으로 지키는 것이 아닙니다. 컴패니언 코드의 `mux_lint.py` 가 ffmpeg 명령을 받아 위 항목을 기계적으로 확인합니다.

```
$ python mux_lint.py -- ffmpeg -i v.mp4 -i a.wav -r 30000/1001 ... out.mp4
[ERROR] r-after-i      -r 이 -i 뒤에 있다 - 입력을 재해석하는 것이 아니라
                        출력을 그 fps 로 다시 만든다. 프레임이 버려지거나 복제된다.
```

원하는 명령과 이 명령은 **토큰이 완전히 같고 순서 하나만 다릅니다**. 그래서 눈으로 비교해서는 찾지 못합니다. 렌더가 4분 걸린 뒤에 알게 되는 것보다, 돌리기 전에 0.1초 만에 아는 편이 낫습니다.

다음 장에서 지금까지의 다섯 장을 하나의 파이프라인으로 묶습니다. Track A의 완성입니다.

실습 코드: `code/ch14_mux/` — fps 검증기 + 안전한 mux 래퍼 + 길이 대조 리포트

예상 소요: 1시간

관련 부록: F(실패 8·15번), H(Windows)

CHAPTER 15

트랙 A 완성 — 추모 영상 파이프라인

15.1 만들 것

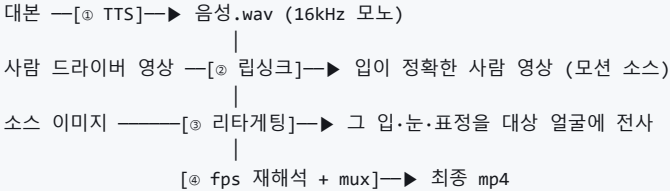
사진 한 장, 대본 세 줄, 그리고 4분.

사진 한 장과 대본을 넣으면, 그 얼굴이 한국어로 말하는 영상이 나옵니다. 사람 얼굴도, 동물 얼굴도, 3D 캐릭터도 됩니다. 실행 한 번에 끝나고, 실패하면 어느 단계에서 왜 실패했는지 말해 줍니다.

이 장의 예제가 하늘이인 이유가 있습니다. 이 얼굴이 가장 어려운 조건을 전부 갖고 있기 때문입니다. 이 얼굴이 왜 가장 어려운 조건인지는 등장인물 페이지에 적어 두었습니다 — 여기서 통하면 다른 곳에서는 다 통합니다.

반대로 말하면, 사람 얼굴에 이 파이프라인을 쓰는 것은 훨씬 쉽습니다. 리타게팅을 사람 모드로 바꾸고 배율을 조금 낮추면 끝입니다. 어려운 쪽을 먼저 하는 이유입니다.

15.2 전체 구조



네 단계이고, 각각이 Ch03과 Ch11~Ch14에 대응합니다.

① **TTS** (Ch03) — 감정의 80%가 여기서 결정됩니다. 최종 출력이므로 프리미엄 엔진을 씁니다. 16kHz 모노로 정규화하고 길이를 기록합니다.

② **립싱크** (Ch11) — 드라이버는 차분한 정면 클로즈업. 결과에 보이지 않으므로 외모는 무관합니다. 전체 소요의 85%가 여기서 나옵니다.

③ **리타게팅** (Ch12~Ch13) — 대상이 동물이면 동물 모드. region과 배율이 이 단계의 두 손잡이입니다.

④ **mux** (Ch14) — 입력 앞의 **-r**로 fps를 재해석하고 음성을 붙입니다. setpts로 늘리지 않습니다.

15.3 파라미터 기본값

수십 번의 시행 끝에 정착한 조합입니다. 출발점으로 쓰고, 소스에 맞춰 조정하세요.

드라이버 — 먼저 차분할 것(움직임 0.01 아래), 그다음 얼굴 비율 0.22 이상, 음성보다 긴 길이 (Ch04 §3), 차분하게 정면을 보며 말하는 영상, 음성보다 긴 길이.

립싱크 — v1.5, 배치는 VRAM이 허용하는 최대, 클로즈업 소스라면 bbox_shift를 조금 양수로.

리타게팅(동물) — lip (입만). 동물에서 표정 전체는 눈알 굴림을 부릅니다(Ch12 §3). 드라이버가 충분히 차분하면 전체를, 싱크 선명도가 우선이면 입만. 배율은 3.0에서 시작합니다(동물 · lip 기준, Ch12 §6). 혀가 보이면 낮추세요. 스티칭은 끝니다. 드라이버 크롭은 컵니다.

mux — 입력 앞 -r , -map 명시, -shortest , yuv420p, H.264+AAC.

각 파라미터의 근거는 Ch11~Ch14에 있습니다. 여기서는 조합만 제시합니다.

15.4 파이프라인을 스크립트로 묶기

네 단계를 손으로 실행하면 반드시 실수합니다. 스크립트 하나로 묶으세요. 설계에서 지킬 원칙이 넷, 그리고 그 넷이 못 막는 구멍이 하나입니다.

환경을 절대 경로로 호출합니다. 립싱크와 리타게팅은 conda 환경이 다릅니다. 활성화 대신 각 환경의 python 실행 파일을 절대 경로로 부르면, 한 스크립트에서 두 환경을 순서대로 쓸 수 있습니다.

출력을 실시간으로 흘려보냅니다. 4분 동안 아무것도 찍지 않는 스크립트는 멈춘 것과 구분되지 않습니다. 각 단계의 출력을 그대로 내보내고, 단계마다 소요 시간을 찍습니다.

단계마다 결과를 검증하고 즉시 멈춥니다. 립싱크 결과 파일이 없는데 리타게팅으로 넘어가면, 나중에 나오는 에러 메시지는 원인과 아무 상관이 없습니다. 각 단계가 끝나면 결과 파일의 존재와 크기를 확인하고, 없으면 그 자리에서 중단합니다.

한글 경로를 자동으로 회피합니다. 입력을 작업 디렉터리에 ASCII 이름으로 복사한 뒤 처리하고, 마지막에 원하는 이름으로 되돌립니다. 사용자가 한글 파일명을 쓰지 않게 하는 것보다, 코드가 알아서 처리하는 편이 낫습니다.

끝난 단계는 다시 하지 않습니다. 원칙 넷을 다 지켜도 남는 구멍입니다. mux가 실패하면 — 코덱 하나 잘못 적어서 — 스크립트를 다시 돌리게 되고, 그러면 195초짜리 립싱크가 처음부터 다시 돌립니다(Ch06). 저자의 배포용 잡 스크립트가 실제로 그랬습니다.

단계마다 어떤 입력으로 만들었는지 를 파일 하나에 기록해 두세요. 다음에 돌릴 때 입력이 같으면 그 단계는 건너뛰고, 실패한 단계부터 다시 시작합니다.

표 15.1 상황 · 다시 도는 것

상황	다시 도는 것
아무것도 안 바뀔	없음 — 넷 다 건너뛸
사진만 바뀔	리타게팅 · mux (립싱크는 안 돈다)
대본이 바뀔	넷 다 — 하류가 전부 달라진다
mux가 실패해서 다시	mux 만

건너뛰어야 할 것을 돌리면 시간을 잃고, 돌려야 할 것을 건너뛰면 결과를 잃습니다. 컴패니언 코드의 `pipeline.py` 가 이 표를 회귀 테스트로 갖고 있고, `plan()` 은 각 단계를 왜 돌리는지 이유까지 냅니다 — "입력이 바뀌어서", "지난번에 여기서 실패해서".

한 줄로 돌리기 — 실제 배포 스크립트

절마다 명령을 따로 치면 순서를 틀립니다. 저자는 이 파이프라인 전체를 인자 다섯 개짜리 스크립트 하나로 묶어 두고 씁니다.

```
pet_talk -PetImage 펫사진.jpg          ← 소스: 정면 · 밝은 조명일수록 좋다
          -Audio   대본음성.wav
          -HumanRef data/video/doctor.mp4 ← 드라이버: 차분한 정면 화자(스톡)
          -Multiplier 3.0                ← 입 크기. 너무 키우면 혀가 나온다
          -Region   lip                    ← lip=입만 · exp=표정 전체(입·눈·미세 표정)
```

기본값이 곧 이 책의 결론입니다. 드라이버는 입을 크게 벌리는 차분한 정면 화자, 배율은 3.0, 영역은 lip .

스크립트가 하는 일은 셋뿐입니다. 음성을 립싱크 환경의 입력 폴더로 옮기고, 두 환경의 python 을 절대 경로로 차례로 부르고, 결과에 음성을 다시 붙입니다. 환경을 활성화하지 않고 절대 경로로 부르는 이유는 Ch02 §3과 같습니다. 셸 상태에 의존하면 *어제는 됐는데 오늘 안 되는* 스크립트가 됩니다.

인자로 뺄 것과 박아 둘 것을 나누세요. 사진·음성처럼 매번 바뀌는 것은 인자로, 파이썬 경로·저장소 위치·ffmpeg 위치처럼 기계에 매인 것은 스크립트 위쪽 한 곳에 모읍니다. 다른 기계로 옮길 때 고칠 줄이 다섯 줄 안에 있어야 합니다.

15.5 원격 GPU로 보내기

GPU가 없거나, 있어도 오래 걸리는 작업을 다른 곳에서 돌리고 싶을 때가 있습니다.

이 파이프라인은 그런 형태에 잘 맞습니다. **입력이 파일 세 개(사진·음성·드라이버)이고 출력이 파일 하나(영상)이며, 중간에 사용자 상호작용이 없기 때문**입니다. 전형적인 배치 잡입니다.

컨테이너 이미지에 두 모델의 저장소와 가중치를 미리 넣어 두면, 네트워크가 차단된 환경에서도 완주합니다. 이 부분은 Ch28에서 다룹니다.

완성된 결과물 — 이 파이프라인이 실제로 내놓는 것

여기까지가 이 장의 목적지입니다. 사진 한 장과 대본 하나로 나온 실제 결과물의 프레임 넷입니다.

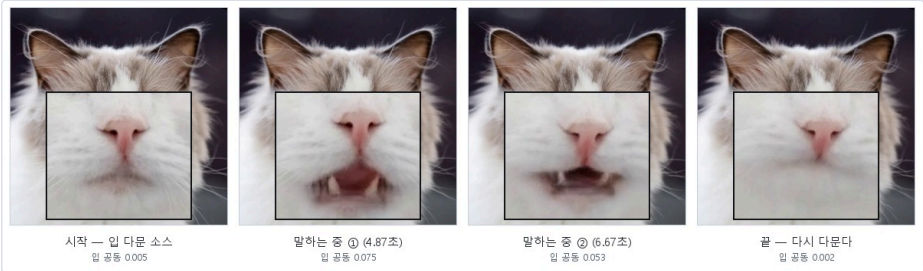


그림 15.1 소스는 입을 다문 정면이고, 말하는 동안 입이 열렸다 닫힌다. 아래 칸은 입 부분을 세 배로 확대한 것 — 입 공동 비율 0.005 → 0.075 → 0.053 → 0.002. 입을 다문 소스는 이만큼만 열린다(Ch13 § 5). 512×512 · 30fps · 320프레임

입이 얼마나 열리는지도 재 두었습니다. `ch13_nonhuman/measure_open.py` 로 프레임마다 입 공동 비율을 재면 시작 0.005 → 최대 **0.075**(4.87초) → 0.053(6.67초) → 끝 0.002입니다. 320프레임 중 299프레임이 다문 상태입니다. 입을 다문 소스에서 출발했으니 이것이 이 파이프라인이 낼 수 있는 최대치입니다 — 입 벌린 소스를 넣으면 반대로 다물지 못합니다(Ch13 § 5). 도판의 확대 칸은 그 차이를 인쇄 크기에서 보이게 하려고 넣었습니다.

대본은 세 줄이었습니다.

지수야, 나야. 하늘이.
이름 불러줘서 고마워. 어디서든 달려갔잖아, 나.
보고 싶어, 지수야. 잘 지내.

세 줄에 4분이 걸립니다. 부록 C의 230초가 이 세 줄의 값입니다. 그리고 그 4분의 85%가 립싱크한 단계에 몰려 있습니다(Ch06). 이 비율을 알고 나면 최적화할 자리가 하나로 좁혀집니다.

결과물을 보는 순서가 있습니다. 먼저 소리를 끄고 입만 보세요 — 말하지 않는 구간에서 입이 달싹이면 드라이버가 잘못된 것입니다(Ch04). 그 다음 소리를 켜고 끝부분을 보세요 — 마지막 문장에서 입이 밀리면 fps 드리프트입니다(Ch14). 마지막으로 처음부터 끝까지 한 번 봅니다. 이 순서를 지키면 어느 장으로 돌아가야 하는지가 정해집니다.

15.6 검증과 리뷰

자동 검증 — Ch09의 발화구간 적중률을 파이프라인 끝에 붙입니다. 단, 대상이 동물이면 이 게이트는 ② 립싱크 단계(사람 드라이버)에만 겁니다 — 최종 산출물의 Ch09 점수는 믿을 수 없습니다(Ch04 §3). 기준 미달이면 경고를 남깁니다. 그리고 Ch14의 길이 대조를 함께 돌립니다.

사람 리뷰 — 비교 영상(드라이버·소스·결과)에 음성을 입혀 봅니다. 연속으로 보고, 큰 소리 구간과 조용한 구간을 나눠 보고, 끝부분을 확인합니다.

한 번 더 볼 것 — 추모 영상처럼 감정적으로 중요한 결과물은 **혀가 나오는 구간**과 **눈이 부자연스러운 구간**을 따로 확인하세요. 결과를 불편하게 만드는 것은 대개 이 둘입니다.

15.7 실측 성능 — 영상 1초당 21초

RTX 4070 SUPER 12GB에서 10.67초 영상 한 편 기준입니다.

단계별 소요(립싱크 195.8초 · 리타게팅 32.1초 · mux 1~2초 → 합계 약 230초, TTS는 별도로 수초)는 부록 C §2에 표로 있습니다 — 립싱크 한 단계가 85%입니다.

79초짜리 대본으로는 약 28분이 걸렸습니다. 대략 **영상 1초당 21초**로 어렵하면 됩니다.

Ch06의 최적화를 적용하면 2~3배까지 줄어듭니다. 그 이상은 이 트랙의 성격상 어렵습니다.

15.8 사람 얼굴은 순서가 뒤집힙니다 — 여섯 번의 실험 ★

이 장의 순서(TTS → 립싱크 → 리타게팅 → mux)는 하늘이, 즉 **사람이 아닌 얼굴**을 위한 것입니다. 사람 사진으로 같은 파이프라인을 만들 때 저자는 순서를 두고 여섯 가지를 한 오후에 돌려 비교했습니다(2026-07-12, 같은 음성 파일, 같은 사진).

표 15.2 립싱크와 리타게팅의 순서 실험 — 같은 입력, 여섯 구성

구성	순서·설정	소요	결과
동물 모드	립싱크 → 리타게팅(동물, lip , ×3.0, 스티칭 끄)	196 + 43초	사람 얼굴에 동물 모드 — 입만 옮겨 어색
립싱크만	사람 드라이버 영상에 립싱크만	175초	깔끔하지만 사진이 아니라 영상이 필요
정지 사진	립싱크 모델에 사진 한 장	95초	오류 — MuseTalk은 정지 사진을 받지 못한다(Ch11 § 2). Ch10의 모델은 받는다
사람 모드	립싱크 → 리타게팅(사람, 전체 표정)	189 + 48초	표정은 자연스러운데 리타게팅이 입을 다시 건드린다
역순	리타게팅(전체) → 립싱크	55 + 196초	눈·머리는 살아 있고 입은 립싱크가 소유
역순 + 눈만	리타게팅(eyes) → 립싱크	48 + 169초	채택 — 깜빡임만 얹고 입은 립싱크가 그린다

결론은 두 줄입니다.

사람 사진은 리타게팅을 먼저 돌려 눈 깜빡임·머리 움직임이 있는 영상을 만들고, 그 영상에 립싱크를 겁니다. 립싱크 모델이 사진을 못 받기 때문이고, 순서를 거꾸로 하면 리타게팅이 립싱크가 그린 입을 몽개 웅얼거리기 때문입니다. 그리고 리타게팅은 **눈만**(`--animation-region eyes`) 옮깁니다. 입까지 옮기면 두 모델이 같은 입을 두고 다룹니다.

동물 얼굴이 반대 순서인 이유는 Ch13에 있습니다. 립싱크 모델이 동물 입을 못 그리니, 사람 드라이버에 입을 먼저 만들고 그 **움직임**만 옮겨야 합니다. 같은 부품, 다른 순서. 얼굴이 사람인지가 순서를 정합니다.

15.9 이 결과물을 만들 때의 태도

기술 이야기를 마치고 한 가지만 덧붙입니다.

이 파이프라인은 **세상을 떠난 존재를 말하게 만듭니다**. 그것이 이 장의 예제인 이유이고, 동시에 이 기술이 갖는 무게이기도 합니다.

만들기 전에 확인할 것이 있습니다. 이 영상을 보게 될 사람이 그것을 원하는가. 대본의 내용이 그 존재가 실제로 했을 법한 말인가, 아니면 남은 사람이 듣고 싶은 말인가. 이 둘은 다르고, 후자는 위로가 되기도 하고 상처가 되기도 합니다.

그리고 사람의 얼굴이라면 — 동의가 있는가. 이 질문에는 예외가 없습니다. Ch29에서 다시 다룹니다.

15.10 이 장에서 기억할 것

파이프라인은 TTS → 립싱크 → 리타게팅 → mux 네 단계입니다.

스크립트 설계의 원칙은 절대 경로 호출 · 실시간 출력 · 단계별 검증 후 즉시 중단 · 한글 경로 자동 회피 입니다.

영상 1초당 약 21초 로 어렵습니다. 배치 잡으로 설계하고, 원격 GPU로 보낼 수 있습니다.

검증은 자동 게이트 + 비교 영상 리뷰 두 겹으로 합니다.

대상이 사람 얼굴이면 순서가 뒤집힙니다 — 리타게팅을 먼저, 립싱크를 나중에 (§ 8). 동물은 그 반대입니다.

Track A가 끝났습니다. 다음 트랙은 정반대의 세계입니다. GPU를 쓰지 않고, 4분 대신 2초 안에 대답하는 아바타를 만듭니다.

실습 코드: `code/ch15_pipeline/` — 4단계 통합 스크립트(`pipeline.py` · 매니페스트 재개 · 단계별 게이트) + 원격 잡 판단(`run_remote.py`)

예상 소요: 4시간 (렌더 대기 포함)

관련 부록: A·B·C·F·G

Part 3 · Track B

GPU 없이 움직이는 아바타

시그니처 데모 : 브라우저 VRM 3D 휴먼, 말 걸면 첫 소리까지 2초(겹침 경로), 감정에 맞춰 손을 흔든다

Ch16	그림 한 장을 파츠로 쪼갬다	파츠 아핀 리깅·2.5D(메시 변형)와의 경계
Ch17	음량이 입을 연다	WebAudio 실시간 립싱크, 모음 근사, 무음 처리
Ch18	VRM — 브라우저에서 도는 3D 휴먼	three-vrm 로컬 서빙, 본·블렌드셰이프
Ch19	코드로 만드는 생명감	깜빡임·시선·호흡·아이들 모션의 난수 설계
Ch20	감정 태그와 제스처	LLM이 [happy][wave] 를 붙이게 하는 법
Ch21	트랙 B 완성 — 2초 응답 대화 아바타	STT → LLM → TTS → 렌더까지 한 서버

CHAPTER 16

그림 한 장을 파츠로 쪼갬다

16.1 신경망을 한 줄도 쓰지 않는 얼굴

홈런이는 그림 파일 네 개입니다. 야구 용어를 설명하는 2D 마스크트이고, Track B의 첫 얼굴입니다.



그림 16.1 원본 그림(지도 학생 생성) → 눈·입을 잘라내고 주변 색으로 메운 몸통 → 왼눈 파츠와 입 파츠. 몸통에서 표정이 사라진 것이 정상이다 — 그 위에 파츠를 얹는다

```
parts/
├─ body.png    몸통 (눈과 입 자리는 비워 둠)
├─ eyeL.png    왼눈
├─ eyeR.png    오른눈
└─ mouth.png   입
```

이게 전부입니다. 모델 가중치도, CUDA도, 전처리 캐시도 없습니다. 그런데 홈런이는 눈을 깜빡이고, 고개를 갸웃하고, 입을 벌려 볼넷을 설명합니다.

Track A에서는 프레임 하나에 0.6초를 썼습니다. 그 숫자를 떠올리면 조금 허탈합니다.

이 트랙은 정반대입니다. 이미 그려진 그림을 코드가 변형합니다. 프레임당 4.15밀리초입니다 (512×512 · 파츠 넷 · CPU만으로 241fps — `ch05_ladder/render_2d.py`). 립싱크 신경망의 프레임당 0.6초와 견주면 약 145배 차이이고, 이 차이는 최적화가 아니라 접근이 달라서 생깁니다.

원리는 오래된 2D 애니메이션 기법 그대로입니다. 그림을 부품으로 쪼개고, 부품을 옮기고 돌리고 늘려서 움직이는 것처럼 보이게 만듭니다. 눈을 감기는 것은 눈 이미지를 세로로 눌러 찌그러뜨리는 일이고, 고개를 기울이는 것은 몸통 이미지를 조금 회전시키는 일입니다.

단순합니다. 그런데 잘 만들면 놀랍도록 살아 있어 보입니다.

16.2 최소 파츠 넷

처음부터 잘게 쪼갤 필요는 없습니다. 휴먼이가 증명합니다 — 넷이면 시작됩니다.

몸통 — 얼굴과 몸 전체이고 배경도 겸합니다. 눈과 입이 있던 자리는 지워 두거나, 어차피 가려지므로 그대로 뒤도 됩니다.

왼쪽 눈 · 오른쪽 눈 — 따로 쪼깁니다. 한쪽만 감는 윈크가 되고, 좌우 깜빡임 타이밍을 아주 미세하게 어긋나게 하면 훨씬 자연스러워집니다.

입 — 닫힌 상태 하나면 시작할 수 있습니다. 여는 것은 세로로 늘려서 표현합니다.

각 파츠는 투명 배경 PNG로 저장하고, 원본 그림에서의 좌표를 함께 적어 둡니다. 이 좌표가 나중에 기준점이 됩니다.

16.3 직접 그리거나, 오려 내거나

직접 그리는 경우 가 가장 깔끔합니다. 처음부터 레이어를 나눠 그리고, 레이어마다 PNG로 내보냅니다.

기존 그림을 쪼개는 경우 는 두 단계입니다. 눈과 입 영역을 잘라내고, 원본에서 그 자리를 주변 색으로 메웁니다. 이미지 편집 도구의 내용 인식 채우기나 간단한 인페인팅 — 빈자리를 주변 픽셀로 자동으로 메우는 기능 — 이면 충분합니다. 눈과 입이 항상 그 위에 올라가므로 완벽할 필요는 없습니다.

여백을 남기세요. 딱 맞게 자르면 변형할 때 경계가 잘립니다. 눈을 세로로 늘리고 입을 크게 벌릴 여유를 두세요.

컴패니언 `split_parts.py --demo` 가 이 두 단계를 그대로 합니다. 샘플 그림(260×320)에서 눈 52×44 돌, 입 102×57, 구멍을 메운 몸통 260×320이 나옵니다. **0.20초** 걸립니다(`_work/measure.json`).

눈 상자가 눈동자보다 눈에 띄게 큰 것이 방금 말한 여백입니다 — 사방에 눈 지름의 절반쯤을 남겼습니다.

`--auto` 로 좌표를 자동 추정하게 하면 어떻게 될까요. 사진용 얼굴 검출기(Haar)는 **그림에서는 얼굴을 못 찾습니다**. 도구는 화면 비율로 찍는 예비 규칙으로 물려납니다. 그 추정은 수동 좌표에서는 눈 9px · 입 15px 빔나갔습니다(`_work/auto_points.json`).

눈 상자 높이가 44px이니 5분의 1, 입은 57px의 4분의 1입니다. 변형 기준점이 그만큼 어긋나면 입이 위쪽이 아니라 가운데에서 벌어집니다. **그림에는 좌표를 직접 주세요.** 자동은 사진용입니다.

16.4 변형의 문법

파츠를 캔버스에 그릴 때 거는 변환은 다섯입니다. 전부 단순한 2D 변환이고, 조합하면 표현이 확 넓어집니다.

이동 — 기준 위치에서 파츠를 옮깁니다. 고개 끄덕임은 얼굴 파츠 전체를 위아래로 조금 움직이는 것입니다.

세로 축소(스퀴시) — 눈 감기의 핵심입니다. 눈 이미지의 세로 크기를 줄이면 감긴 것처럼 보입니다.

기준점은 파츠마다 다릅니다 ★

여기가 이 장에서 가장 자주 틀리는 곳입니다. 모든 파츠를 같은 기준으로 변형하면 안 됩니다.

저자의 홈런이는 이렇게 나뉘어 있습니다.

표 16.1 파츠 · 변형 기준 · 왜

파츠	변형 기준	왜
눈	중앙	눈은 위아래 눈꺼풀이 함께 좁혀진다
입	위쪽(top)	윗입술은 고정, 아래턱이 내려간다
몸통	바닥	발이 뜨면 안 된다

같은 축소, 다른 기준점 — 결과가 완전히 달라진다

점선 = 원래 크기 · 채운 칸 = 세로 0.3배로 누른 뒤 · ● = 고정되는 기준점

파츠	세로 0.3배	기준점	그래서
눈		중앙	감을 때 위아래가 같이 오므라든다
입		위	윗입술은 고정, 아래턱만 내려온다
몸통		바닥	호흡해도 발이 바닥에서 뜨지 않는다

그림 16.2 같은 세로 축소라도 기준점이 중앙이나 위쪽이나 바닥이나에 따라 결과가 완전히 달라진다

입을 중앙 기준으로 벌리면 윗입술이 코 쪽으로 올라갑니다. 몸통을 중앙 기준으로 기울이면 발이 바닥에서 떨어집니다.

CSS에서는 `transform-origin` 한 줄입니다.

```
.eye { transform-origin: center center; }
.mouth { transform-origin: center top; }
```

해부학을 한 번만 생각하고 기준점을 정하세요. 그 다음엔 값만 바꾸면 됩니다.

눈은 0까지 감지 마세요

```
const eyeOpen = Math.max(0.05, eyeScale * (1 - blink * 0.92));
```

안전장치가 돌입니다. 깜빡임을 0.92까지만 적용하고, 결과에 0.05 하한 을 겁니다.

완전히 0으로 만들면 눈이 사라집니다. 두께가 없는 선이 되어버려서, 감은 눈이 아니라 지원진 눈처럼 보입니다. 아주 얇게라도 남겨 두면 감은 눈 이 됩니다.

세로 확대 — 입 벌리기입니다. 다만 세로로만 늘리면 안 됩니다.

```
mouth.style.transform = `scaleY(${1 + m*0.6}) scaleX(${1 - m*0.18})`;
```

세로로 60% 늘리는 동안 가로로 18% 좁힙니다. 실제로 입을 벌리면 입꼬리가 안쪽으로 모입니다. 세로만 늘리면 입이 네모나게 커져서 어색합니다.

부피를 지키려고 반대 축을 줄이는 것 — 이것을 스쿼시·스트레치라고 부릅니다. 2D 애니메이션의 가장 오래된 원리 중 하나이고, 파츠 리깅에서 값 하나로 얻는 가장 큰 개선입니다.

회전과 기울임 — 몸통을 살짝 기울이면 자세가 생깁니다. 말할 때 몸이 아주 조금씩 기울면 생명감이 크게 올라갑니다.

투명도 — 표정 파츠를 겹쳐 쓸 때 씁니다. 불 홍조 같은 것을 감정에 따라 서서히 켵니다.

16.5 파츠 대신 손잡이 넷을 잡으세요

파츠를 직접 조작하지 마세요. 뜻이 담긴 숫자 로 한 겹 감쌉니다. 손잡이는 넷입니다.

eyeOpen 이 0이면 눈을 감고 1이면 뜹니다. mouthOpen 이 0이면 입을 다물고 1이면 최대한 벌립니다. bodyLean 은 -1에서 1 사이로 몸의 기울기를 나타냅니다. headY 는 고개의 위아래 위치입니다.

컴패니언 코드의 params.py 가 이 넷을 받아 파츠별 변형을 냅니다. 거기서 두 가지를 처리합니다 — 범위 밖 값은 잘라냅니다(Ch17의 음량 구동이 1을 넘겨도 입이 찢어지지 않습니다). 그리고 기준점은 §4의 표에서 가져옵니다. 눈은 중앙, 입은 위, 몸통은 바닥이 코드에 한 번만 적혀 있습니다.

"한 겹 더 감싸면 느려지지 않나" — 이 질문이 꼭 나오므로 비용부터 재 두었습니다. `clamp()` + `apply()` 한 번에 **2.5μs** 입니다(2만 번 반복 평균). 60fps 한 프레임 예산 16,667μs의 **0.015%** 입니다. 프레임 예산은 렌더링(캔버스 그리기)이 전부 쓰고, 이 층은 없는 것과 같습니다. 비용이 아니라 구조 때문에 두는 층입니다.

이 파라미터 층이 있으면 좋은 점이 셋입니다.

렌더링과 제어가 분리됩니다. 파츠를 다시 그려도 제어 코드는 그대로입니다.

LLM이 제어할 수 있습니다. "이 말에 어울리는 표정"을 파라미터 값으로 뺄게 만들 수 있습니다(Ch20).

3D로 옮기기 쉽습니다. 같은 파라미터 이름을 VRM의 블렌드셰이프에 매핑하면 제어 로직을 그대로 재사용합니다(Ch18).

16.6 팔이 없어도 몸은 말합니다

2D 파츠에는 대개 팔 파츠가 없습니다. 그리는 비용이 크고, 관절이 없어 자연스럽게 움직이지 않기 때문입니다.

그래도 제스처를 포기할 필요는 없습니다. 홈런이는 팔 없이 **몸통 세 채널**로 동작을 만듭니다.

```
bob    위아래 이동
lean   좌우 회전(도)
squash 세로 눌림
```

이 셋을 몸통 전체에 겁니다.

```
avatar.style.transform =
  `translateY(${ -bob*100}%) rotate(${lean}deg) scaleY(${1+squash})`;
```

대기와 발화의 리듬이 다릅니다

```
// 대기 - 느리고 작게
bob  = sin(st*1.6) * 0.012
lean = sin(st*0.7) * 1.3

// 말하는 동안 - 위에 더한다
bob += |sin(st*5.0)| * 0.03    // 절대값 → 위로만 튼다
lean += sin(st*3.3) * 3.4
squash += sin(st*5.0) * 0.02
```

bob에 절대값을 씌운 것을 보세요. 사인파를 그대로 쓰면 위아래로 같이 흔들립니다. 절대값을 씌우면 아래로는 안 내려가고 위로만 통통 튀니다. 말할 때의 들뜬 리듬이 이 한 글자에서 나옵니다.

대기 주기(1.6:0.7)와 발화 주기(5.0:3.3)는 세 배 이상 차이 납니다. Ch19의 3D도 같은 원리입니다 — 상태가 바뀌면 리듬이 바뀝니다.

동작도 같은 함수 꼴입니다

Ch20에서 볼 3D 제스처와 형태가 정확히 같습니다. 진행도 p 와 시간 t 를 받아 채널 값을 돌려 주고, 양 끝에 엔벨로프 — 동작의 시작과 끝을 부드럽게 깎는 곡선 — 를 곱합니다.

```
cheer: (p,t) => ({ bob: -0.16*Math.abs(Math.sin(p*Math.PI*2)),
                    squash: -0.07*Math.sin(p*Math.PI) }), // 만세 점프
shrug: (p,t) => ({ lean: 9*Math.sin(t*5),
                    squash: 0.07*Math.sin(p*Math.PI) }), // 어깨 으쓱
```

채널 이름만 다르고 구조는 같습니다. 그래서 2D로 만든 동작 목록을 3D로 옮길 때 매핑 표 하나면 됩니다. § 5의 파라미터 층이 3D 이식의 통로라고 한 것의 실물입니다.

캐릭터 전용 동작을 두세요

홈런이에게는 야구 마스크트만의 동작이 있습니다 — 홈런 폴스윙.

```
homerun: (p,t) => ({ lean: swingArc(p, 48), ... })
```

p 에 따라 48도까지 몸을 회전시키는 스윙 궤적 함수입니다. 범용 동작 목록에는 없고, 있을 이유도 없습니다.

도메인이 정해진 아바타라면 그 도메인의 동작을 한둘 만드세요. 사용자가 기억하는 것은 표준 열세 개가 아니라 그 하나입니다.

16.7 이 장이 만든 것은 Live2D가 아닙니다 ★

용어부터 못 박고 갑니다. 이 장의 방식을 흔히 "*Live2D 방식*"이라고 부르지만, 엄밀히 말하면 아닙니다.

이 장에서 한 것은 **아핀 변환**입니다. 이미지를 통째로 옮기고 돌리고 키우고 줄였다는 뜻입니다. 파츠는 사각형 이미지 그대로 움직였고, 이미지 자체는 절대 휘지 않았습니다.

진짜 2.5D는 메시를 변형합니다. 파츠 위에 삼각형 그물을 깔고, 그 격자점을 하나씩 밀고 당깁니다. 그러면 이미지가 휩니다.

그 차이가 만드는 것 — 고개 돌리기

이 장의 방식으로는 **고개를 좌우로 돌릴 수 없습니다**. 얼굴 파츠를 회전시켜 봐야 그림이 기울 뿐입니다. 옆을 보는 것처럼 보이지 않습니다.

메시 변형은 다릅니다. 얼굴 윤곽을 한쪽으로 휘고, 눈·코·입을 원근에 맞춰 밀고, 반대쪽 귀를 서서히 가립니다. 그러면 **정면 그림 한 장으로 고개를 돌린 것처럼 보입니다**.

그림은 여전히 한 장이고 2D인데 3D처럼 보입니다. 그래서 **2.5D입니다**.

언제 1.5단으로 올라가나

올라가야 할 때 — 캐릭터의 그림체가 상품일 때입니다. 버추얼 스트리머, IP 마스코트, 게임 캐릭터. 3D로 옮기면 원작의 화풍이 사라집니다. 그리고 시청자가 캐릭터의 얼굴을 오래 보는 콘텐츠일수록 고개 돌리기가 있고 없고의 차이가 큼니다.

안 올라가도 될 때 — 캐릭터가 정면으로 설명만 하면 되는 경우입니다. 안내·학습·키오스크가 여기 속합니다. **홈런이는 야구 용어를 정면으로 설명하므로 이 장의 방식으로 충분합니다**.

대신 3D로 갈 때 — 캐릭터가 여러티거나, 손동작이 필요하거나, 시점을 바꿔야 할 때입니다. 리깅 비용이 캐릭터마다 드는 2.5D와 달리 VRM은 모델만 바꾸면 됩니다(Ch18).

비용의 모양이 다릅니다

표 16.2 1단 파츠 · 1.5단 메시 — 비용의 모양

	1단 파츠	1.5단 메시
준비	그림을 4조각으로 자르기	메시 깔기 · 디포머 · 파라미터 잡기
걸리는 시간	한두 시간	캐릭터당 며칠
도구	이미지 편집기 + 코드	전용 저작 도구
캐릭터 추가	자르면 끝	처음부터 다시

1단은 코드 문제이고, 1.5단은 작업 문제입니다. 이 책이 1단을 본선으로 둔 이유이기도 합니다 — 프로그래머가 혼자 끝낼 수 있는 범위이기 때문입니다. 2.5D는 대개 **일러스트레이터와 리거가 함께** 하는 일입니다.

정리 — 이 장의 결과물을 **2.5D** 라고 부르지 마세요. **파츠 리깅** 입니다. 그리고 그것으로 충분한 경우가 생각보다 많습니다.

16.8 이 방식의 장단점

장점 — GPU가 필요 없습니다. 브라우저에서 60fps로 돌립니다. 사용자 수만큼 브라우저가 렌더하므로 서버 부담이 없습니다. 그림만 있으면 어떤 캐릭터든 됩니다 — 사람, 동물, 로봇, 추상적인 형태.

단점 — 사실적일 수 없습니다. 그림이니까요. 고개를 크게 돌리는 3D적 표현이 불가능합니다. 파츠를 쪼개는 수작업도 필요합니다.

어디에 쓰나 — 캐릭터 마스코트, 학습 도우미, 키오스크, 게임 UI, 그리고 **저사양 환경 전반**. 태블릿과 구형 브라우저에서도 돌립니다.

16.9 이 장에서 기억할 것

이 트랙은 **신경망 대신 코드가 그림을 변형** 합니다. 프레임당 비용이 약 145배 낮습니다(4.15ms 대 0.6초).

최소 파츠는 넷 — **몸통 · 좌우 눈 · 입**. 여백을 남기고 자릅니다.

변형의 문법은 **이동 · 스퀘시 · 확대 · 회전 · 투명도** 입니다. 눈 감기는 반드시 **중앙 기준 스퀘시** 로 합니다 (§ 4의 표).

파츠를 직접 다루지 말고 **의미 있는 파라미터 층** 으로 감싸세요. 이 층이 LLM 제어와 3D 이식의 통로가 됩니다.

다음 장에서 이 아바타에게 소리를 줍니다. 음량 하나로 입이 움직이기 시작합니다.

실습 코드: `code/ch16_parts/` — 이미지 파츠 분할 스크립트 + 네 손잡이(`params.py` · `eye` · `Open/mouthOpen/bodyLean/headY`) + 기준점 검사

예상 소요: 2시간

관련 부록: F § E(살아 있어 보이지 않는 실패) · L § 2(상태를 얼굴로)

CHAPTER 17

음량이 입을 연다

17.1 Ch09가 버린 신호를 다시 씁니다

Part 2에서 저는 음량으로 입을 움직이는 꺾음을 며칠 동안 만들었다가 전부 버렸습니다. 그런데 이 장에서는 바로 그 음량으로 홈런이의 입을 엽니다. 홈런이는 야구 용어를 설명하는 2D 그림 마스코트, Track B의 첫 얼굴입니다.

모순처럼 보이지만 아닙니다. 오히려 이 대비가 이 책에서 가장 중요한 교훈 하나를 보여 줍니다.

Ch09는 실사 립싱크의 품질을 판정하는 지표 이야기였습니다. 실사 모델은 음소를 보고 입을 그립니다. 그 결과를 음량으로 채점하면 틀립니다.

이 장은 캐릭터 아바타를 움직이는 신호 이야기입니다. 사실적인 음소 표현은 목표가 아닙니다. 목표는 "말하고 있다는 것이 자연스럽게 보이는 것" 이고, 그 목적에는 음량이 놀랍도록 잘 통합니다.

목표가 다르면 옳은 지표도 다릅니다. 이 구분이 이 장의 전부입니다.

17.2 왜 음량으로 충분한가

만화와 애니메이션에서 캐릭터의 입은 원래 정확하지 않습니다. 입 모양 몇 개를 소리 리듬에 맞춰 갈아 끼울 뿐입니다. 사람들은 수십 년 동안 그것을 자연스럽게 받아들여 왔습니다.

양식화된 캐릭터에서는 음소 정확도보다 리듬의 일치가 훨씬 중요합니다. 소리가 날 때 입이 움직이고, 소리가 멈추면 입이 닫히면 됩니다. 리듬이 맞으면 뇌가 나머지를 채웁니다.

실무적으로 결정적인 이점도 있습니다. 오디오를 미리 분석할 필요가 없습니다. 재생하면서 음량을 실시간으로 읽어 입을 움직이므로, TTS가 오디오를 만들자마자 바로 재생을 시작합니다. Ch07의 지연 예산에 아무것도 더하지 않습니다.

17.3 브라우저에서 음량 읽기

웹 오디오 API로 재생 중인 오디오의 순간 음량을 읽습니다. 구조는 간단합니다.

오디오 요소를 오디오 컨텍스트에 연결하고, 그 사이에 분석 노드를 끼웁니다. 매 렌더 프레임마다 분석 노드에서 지금 파형을 꺼내 크기를 잽니다. 그 값을 정규화해서 입 벌림 파라미터로 씁니다.

크기는 파형 값의 제곱 평균 제곱근(RMS)이 무난합니다. 절대값 평균도 충분합니다. 정밀할 필요가 없습니다.

주의할 점 하나. **오디오 컨텍스트는 사용자 상호작용 이후에만 시작됩니다.** 브라우저 정책입니다. 페이지 로드 시점에 만들면 정지 상태로 생성되고, 사용자가 버튼을 한 번 누르면 재개됩니다. 첫 클릭에서 컨텍스트를 깨우는 코드를 넣어 두세요. 이것을 놓치면 "왜 첫 번째 메시지만 입이 안 움직이지" 하는 증상이 나옵니다.

17.4 날것의 음량은 쓸 수 없습니다

측정한 값을 그대로 입 벌림에 넣으면 결과가 나쁩니다. 네 가지 처리가 필요합니다.

부드럽게 만들기. 음량은 프레임마다 심하게 요동칩니다. 그대로 쓰면 입이 덜덜 떨립니다. 이전 값과 현재 값을 섞는 간단한 평활화로 잡힙니다. 다만 너무 부드럽게 하면 입이 소리보다 늦게 반응합니다. 여는 쪽은 빠르게, 닫는 쪽은 느리게 하는 비대칭 평활화가 좋습니다. 실제 입의 물리적 특성과도 맞습니다.

바닥 자르기. 무음 구간에도 미세한 잡음이 있습니다. 임계값 아래는 0으로 눌러 입을 확실히 닫습니다. 이것이 없으면 아바타가 조용할 때도 입을 달짝거립니다.

천장 정하기. 음량의 최대값은 오디오마다 다릅니다. 고정된 값으로 나누면 어떤 오디오에서는 입이 조금만 벌어지고, 어떤 오디오에서는 늘 최대로 벌어집니다. 최근 구간의 최대값을 추적해 그것으로 정규화하면 안정적입니다.

⚠️ 그런데 이 둘을 같이 쓰면 무음이 안 막힙니다

바닥 자르기와 동적 천장은 각각 옳은데, **함께 쓰면 서로를 무력화합니다.**

조용한 구간이 이어지면 **천장도 함께 내려잡니다.** 그러면 0.004짜리 미세 잡음이 **천장 대비**로는 큰 소리가 되어 바닥을 통과합니다. 아무 소리도 안 나는데 아바타가 입을 달짝입니다 — 바닥 자르기를 넣었는데도 그렇습니다.

컴패니언 코드의 회귀 테스트가 이것을 잡았습니다. 무음 30프레임 뒤 입 벌림이 **0.15**였습니다.

이 결함이 **실제 파일에서** 얼마나 나는지도 확인했습니다(`measure.py`). 저자의 TTS 출력과 마이크 녹음 넷(`_script · _chat · _stt · _live`)은 무음 구간이 **디지털 0**이었습니다 — 하위 10% 프레임의 RMS가 0.0000. 게이트가 있든 없든 열림 비율이 같았습니다.

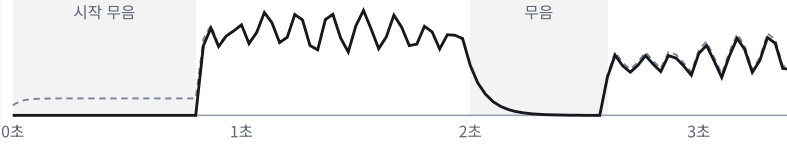
게이트는 이런 파일이 아니라 **잡음 바닥이 있는 오디오**를 위한 것입니다. 압축 음원, 팬 소리가 섞인 마이크 녹음이 그렇습니다. 회귀 테스트는 0.004를 합성해 그 경우를 지킵니다. 실측은 **결함의 조건**을 좀혀 줍니다.

날것의 음량은 못 쓴다 — 다섯 처리를 거친다

① 날것 RMS



② MouthDriver 출력 — 입 벌림 0~1



점선 = 노이즈 게이트를 끈 것 — 시작 무음에서 0.15 까지 열린다 (게이트 있으면 0.00).

새 드라이버의 동적 천장은 최저값이라 0.004 짜리 잡음이 천장 대비 큰 소리가 된다.

두 번째 무음에서는 천장이 아직 높아 같은 잡음이 묻힌다 — 상대 기준만으로는 못 막는다.

그림 17.1 게이트를 끄면 시작 무음에서 입이 0.15까지 열린다 — 상대 기준만으로는 못 막는다

해법은 절대 기준을 하나 더 두는 것입니다.

정규화 전에 → 원시 음량이 절대 임계보다 작으면 그냥 0
정규화 후에 → 천장 대비 바닥 자르기

상대 기준과 절대 기준은 서로를 대신하지 못합니다. 상대 기준은 지금 이 오디오 안에서 큰가를 묻고, 절대 기준은 애초에 소리가 나긴 하는가를 묻습니다. 무음을 막는 것은 후자입니다.

그래서 처리는 넷이 아니라 다섯 입니다 — 절대 게이트 · 비대칭 평활화 · 상대 바닥 · 동적 천장 · 곡선.

닫는 속도는 Ch09의 잣대로 정했습니다 ★

"여는 쪽 빠르게, 닫는 쪽 느리게"는 맞는 방향입니다. 그런데 얼마나 느리게인지는 제가 감으로 정해 두었습니다(release 0.15).

실제 파일 넷에 돌려 봤습니다. 조용한 프레임(RMS < 0.06)의 50%에서 입이 열려 있었습니다. Ch09 §5가 조용한 구간 활발도 상한을 35%로 두었으니, 이 장의 기본값이 이 책의 자체 기준에 걸린 것입니다.

표 17.1 release 값 스윙 — 실제 파일 넷 평균 (sweep_release.py → _work/release_sweep.json)

release	조용한 프레임 중 열림	발화 끝→닫힘	발화 중 떨림(\)	$\Delta\backslash$	평균)
0.15 (이전 기본)	50%	17.2프레임 (573ms)	0.049		
0.25	38%	10.2프레임	0.063		
0.35 (새 기본)	33%	7.5프레임 (250ms)	0.075		
0.50	29%	5.5프레임	0.088		
1.00 (평활 없음)	19%	3.0프레임	0.114		

닫힘을 빠르게 할수록 조용한 구간은 깨끗해지고 떨림은 커집니다. 두 지표가 반대로 움직이니 하나만 보고 고르면 안 됩니다. **35% 아래로 내려가는 첫 값** 인 0.35를 골랐습니다. 떨림 0.075는 3단 이미지 전환(§5)에서 한 단 폭(0.33)의 4분의 1이라 화면에 거의 드러나지 않습니다.

곡선 적용. 음량과 입 벌림을 선형으로 이으면 맞깁니다. 제곱근을 취하면 작은 소리에서도 입이 어느 정도 열려 생동감이 생깁니다.

17.5 열린 입 이미지를 따로 두세요

닫힌 입을 세로로 늘려 벌리는 방식은 한계가 명확합니다. 입술이 부자연스럽게 늘어나 보입니다.

훨씬 나은 방법은 **입 모양 이미지를 두세 장 준비해 갈아 끼우는 것**입니다. 닫힌 입, 조금 벌린 입, 크게 벌린 입. 입 벌림 값에 따라 어느 이미지를 그릴지 고릅니다.

세 장이면 대부분의 상황에 충분합니다. 한국어의 주요 모음 입 모양을 몇 개 더 넣을 수도 있습니다. 다만 그러려면 음소 정보가 필요해지고, 이 트랙의 장점인 단순함이 사라집니다.

전환에도 약간의 보간을 넣으세요. 이미지가 툭툭 바뀌면 눈에 띕니다. 전환 순간에 두 이미지를 짧게 겹쳐 그리면 훨씬 부드럽습니다.

17.6 분석이 실패해도 입은 움직여야 합니다 ★

여기가 실전과 예제가 갈리는 자리입니다.

오디오 분석은 조용히 실패합니다. 컨텍스트가 안 깨어났거나 (§3), 브라우저 정책이 바뀌었거나, 오디오 요소 연결이 어긋났거나. 그러면 음량이 계속 0으로 읽히고 — **아바타가 소리는 내면서 입을 다물고 있습니다.**

이것은 립싱크가 조금 어긋난 것보다 **훨씬 나쁩니다.** 사용자는 즉시 고장으로 인식합니다.

저자의 구현은 안전망을 한 줄 둥니다.

```
if (speaking) {
  target = Math.max(target, 0.42 + 0.4 * Math.sin(performance.now() * 0.013));
}
```

말하는 중이라는 것을 아는 동안에는, 음량과 무관하게 최소한의 입 움직임을 보장합니다. 분석이 정상이면 실제 음량이 이 값을 넘어 그대로 쓰이고, 분석이 죽었으면 이 사인파가 입을 펴려이게 합니다.

값을 보세요. 중심 0.42에 진폭 0.4이므로 0.02~0.82 사이 를 오갑니다. 완전히 닫히지도, 최대로 벌어지지도 않습니다. *말하고 있는 입* 처럼 보이는 범위입니다.

원칙 — 실시간 시스템에서 감지에 의존하는 표현에는 폴백을 두세요. 감지가 죽었을 때 표현이 함께 죽으면 사용자는 서비스 전체가 고장 났다고 판단합니다. Ch07 §6의 오프라인 답변과 같은 사상이고, 여기서는 한 줄이면 됩니다.

단, 폴백은 **speaking** 이 참일 때만 동작해야 합니다. 조건을 빼면 조용할 때도 입이 계속 펴려입니다 — §4의 바닥 자르기를 무효로 만드는 실수입니다.

17.7 말이 끝나면 확실히 닫으세요

흔한 버그가 있습니다. 오디오 재생이 끝났는데 입이 마지막 상태로 멈춰 있는 것입니다. 반쯤 벌어진 채 굳은 얼굴은 대단히 이상합니다.

재생 종료 이벤트에서 입 벌림 값을 0으로 되돌리고, 분석 루프를 멈추세요. 평활화에 맡겨도 스스로 닫히긴 합니다 — 실측으로 release 0.35에서 마지막 소리 뒤 7.5프레임(250ms), 0.15였다면 17.2프레임(573ms)입니다.

그 4분의 1초 동안 반쯤 열린 입은 정지 화면처럼 보입니다. 기다리지 말고 닫으세요. 그리고 안전 장치로 일정 시간 이상 음량 갱신이 없으면 자동으로 닫히게 해 두면 예외 상황에서도 안전합니다.

17.8 서버가 할 일은 거의 없습니다

이 방식에서 서버가 하는 일은 놀랍도록 적습니다. 텍스트를 받아 오디오 파일을 만들고 URL을 돌려주는 것 이 전부입니다. 입을 움직이는 일은 브라우저가 합니다.

그래서 서버가 가볍고, 그래서 동시 사용자를 많이 받습니다. Ch05의 결정표에서 "동시 사용자" 행의 차이가 바로 여기서 나옵니다.

오디오 파일에는 순번을 붙여 저장하고 정적 파일로 서빙합니다. 오래된 파일을 주기적으로 지우는 정리 작업만 넣어 두면 됩니다.

17.9 이 장에서 기억할 것

목표가 다르면 옳은 지표도 다릅니다. 양식화된 캐릭터에서는 음소 정확도보다 리듬의 일치가 중요하고, 음량이 그것을 잘 표현합니다.

음량 구동의 결정적 이점은 사전 분석이 필요 없다 는 것입니다. 지연 예산에 아무것도 추가하지 않습니다.

오디오 컨텍스트는 사용자 상호작용 이후에 시작 됩니다. 첫 클릭에서 깨우세요.

날것의 음량은 못 씁니다. 절대 게이트 · 비대칭 평활화 · 상대 바닥 · 동적 천장 · 곡선 다섯 처리를 겁니다. 그중 절대 게이트가 없으면 나머지가 무음을 못 막습니다.

입 이미지를 두세 장 두고 전환 하는 편이 늘리기보다 훨씬 낫습니다.

재생이 끝나면 확실히 닫으세요. 반쯤 벌어진 채 굳은 입은 눈에 찡니다.

다음 장에서 같은 원리를 3D로 옮깁니다. 파라미터 층을 만들어 둔 것이 여기서 값어치를 합니다.

실습 코드: `code/ch17_volume/` — 웹오디오 분석기 + 평활화 파이프라인 + 입 이미지 전환

예상 소요: 2시간

관련 부록: F §E(살아 있어 보이지 않는 실패) · L §2(상태를 얼굴로)

CHAPTER 18

VRM — 브라우저에서 도는 3D 휴먼

18.1 왜 3D로 올라가나

코치를 남성 캐릭터에서 여성 캐릭터로 바꿨습니다. 코드는 한 줄도 고치지 않았습니니다. 모델 파일 경로만 바꿨습니다. 코치는 브라우저에서 도는 3D VRM 홈트레이닝 코치, Track B 2단의 얼굴입니다.

깜빡임도, 제스처도, 립싱크도 그대로 동작했습니다. 스퀴트를 시키면 새 캐릭터가 스퀴트를 했습니다.

이게 가능한 이유가 이 장의 절반입니다.

2D 파츠로도 살아 있는 아바타를 만들 수 있습니다. 그런데도 3D로 올라가는 이유가 셋 있습니다.

제스처가 가능해집니다. 2D에서 팔을 흔들려면 팔 파츠를 따로 만들어 회전시켜야 하는데, 각도가 조금만 커져도 어색해집니다. 3D는 뼈대가 있으므로 팔이 자유롭게 움직입니다. Ch20에서 만들 감정 기반 제스처는 3D에서 훨씬 쉽습니다.

시점을 바꿀 수 있습니다. 카메라를 옮겨 얼굴을 클로즈업하거나 전신을 보여 줍니다. 2D는 그런 각도가 전부입니다.

모델을 갈아끼울 수 있습니다. 표준 포맷을 쓰면 캐릭터를 교체할 때 코드를 고칠 필요가 없습니다. 아마 이것이 가장 실무적인 이점입니다.

그리고 여전히 GPU는 필요 없습니다. 브라우저의 3D 렌더링은 노트북 내장 그래픽에서도 60fps로 돕니다.

18.2 VRM — 뼈 이름이 정해져 있는 포맷

3D 아바타를 위한 표준 포맷입니다. 일반적인 3D 모델 포맷 위에 아바타에 필요한 규약을 얹었습니다.

표준화된 뼈대 이름과 표정 이름. 이 둘이 전부입니다.

어떤 VRM 모델이든 머리 뼈는 `head`, 왼쪽 위팔은 `leftUpperArm` 이라고 부릅니다. 표정도 `blink`, `happy`, `aa` 같은 표준 이름을 갖습니다. 그래서 **한 번 만든 제어 코드가 모든 VRM 모델에서 동작합니다.**

이 성질의 가치는 모델을 직접 바꿔 보면 압니다. 캐릭터를 교체해도 깜빡임, 제스처, 립싱크가 전부 그대로 둡니다. 코드 한 줄 고치지 않고 남성 캐릭터를 여성 캐릭터로, 사람을 판타지 캐릭터로 바꿉니다.

무료로 배포되는 VRM 모델이 많고, 캐릭터 제작 도구로 직접 만들 수도 있습니다.

VRM이 아닌 3D 모델도 받을 수 있습니다. 일반 glTF/GLB 파일은 VRM 확장 플러그인을 등록하지 않고 로더에 그대로 넘기면 됩니다. 다만 그때는 표준 뼈 이름을 못 쓰므로, 모델의 실제 뼈 이름을 찾아 매핑 표를 손으로 만들어야 합니다.

그 수고가 곧 VRM의 값어치 입니다. 캐릭터를 여럿 다룰 계획이면 VRM으로 통일하세요.

18.3 라이브러리를 로컬에 두세요

3D 렌더링 라이브러리와 VRM 확장 라이브러리, 둘이 필요합니다. 예제는 대개 CDN에서 불러오지만 **로컬에 받아 직접 서빙하는 편을 권합니다.**

이유가 셋입니다. 네트워크가 없는 환경(강의실, 폐쇄망)에서 둡니다. CDN이 죽거나 버전이 바뀌어도 어제 되던 것이 오늘도 됩니다. 그리고 로딩이 빠릅니다.

파일 몇 개를 정적 디렉터리에 두고 서버에서 마운트하면 끝입니다.

18.4 세 가지 제어 통로

VRM 모델을 움직이는 방법은 셋입니다. 각각 담당하는 영역이 다릅니다.

표정 — 블렌드셰이프

얼굴을 통째로 다시 그리지 않고 미리 만든 표정을 섞는 방식입니다. 이름을 주고 0에서 1 사이의 값을 넣으면 표정이 변합니다. `blink` 에 1을 넣으면 눈을 감고, `happy` 에 0.7을 넣으면 웃습니다.

입 모양도 여기 있습니다. 표준 이름으로 모음에 해당하는 표정이 정의되어 있고, 그중 하나를 음량에 비례해 열면 립싱크가 됩니다. **Ch16의 파라미터 층에 Ch17의 음량을 흘려 넣으면 됩니다.** `mouthOpen` 값을 그 표정에 넣기만 하면 3D 캐릭터가 말하기 시작합니다.

여러 표정을 동시에 켤 수 있지만 서로 충돌하는 것들이 있습니다. 웃는 표정과 슬픈 표정을 같이 켜면 이상해집니다. 감정은 하나씩 켜고, 깜빡임과 입만 그 위에 겹치세요.

자세 — 뼈대 회전

뼈를 이름으로 찾아 회전값을 주면 자세가 바뀝니다. 팔을 들거나, 고개를 기울이거나, 허리를 돌립니다.

정규화된 뼈 노드로 접근하세요.

```
const bone = vrm.humanoid.getNormalizedBoneNode('rightUpperArm');
```

모델의 실제 뼈 계층을 직접 뒤지지 않습니다. VRM 규격이 표준 이름에서 실제 노드로 가는 다리를 놔 줍니다. 이 한 줄이 §2에서 말한 "표준화"의 실체입니다.

그리고 로드 직후에 기본 회전값을 전부 복사해 두세요.

```
BONELIST.forEach(n => {
  const b = vrm.humanoid.getNormalizedBoneNode(n);
  if (b) { bones[n] = b; restRot[n] = b.rotation.clone(); }
});
```

이후 모든 조작은 이 값에 더하기만 합니다. 왜 그래야 하는지는 Ch19 §2에서 다룹니다 — 캐릭터 교체가 가능한 시스템의 전제 조건입니다.

주의할 것은 **회전의 기준**입니다. VRM의 기본 자세는 팔을 옆으로 벌린 자세이고, 각 뼈의 회전은 그 기준에서의 상대값입니다. 처음에는 감이 안 오니 값을 조금씩 바꿔 가며 눈으로 확인하는 편이 빠릅니다.

시선 — 룩앳

캐릭터가 특정 지점을 바라보게 합니다. 카메라를 바라보게 하면 화면 앞의 사용자와 눈이 마주칩니다. 이것 하나만 켜도 존재감이 크게 달라집니다.

18.5 렌더 루프의 구조

브라우저는 매 프레임 그리기를 반복합니다. 그 루프 안에서 할 일이 넷입니다.

시간 갱신 — VRM 라이브러리에 경과 시간을 알려 줍니다. 스프링 본(머리카락, 옷자락의 흔들림) 같은 물리 시뮬레이션이 이 값으로 돕니다.

파라미터 적용 — 지금의 깜빡임·입·감정·자세 값을 모델에 씁니다.

애니메이션 진행 — 진행 중인 제스처가 있으면 한 프레임 진행시킵니다.

렌더 — 장면을 그립니다.

모든 제어가 이 루프 안에서 값을 쓰는 방식입니다. 애니메이션 파일을 재생하는 것이 아닙니다. 매 프레임 코드가 값을 계산해서 넣습니다. 그래서 어떤 조합이든 즉석에서 만들고, 중간에 끊고 다른 것으로 갈아탈 수 있습니다.

18.6 카메라와 조명

기본 설정만 잡아도 결과가 크게 달라집니다.

카메라 는 얼굴 높이에 두고 **수평으로** 바라보게 하는 것이 저자의 기본값입니다 — `viewer.html` 은 카메라 높이를 모델 중심 + 키의 15%에 두고 시선도 같은 높이로 잡습니다. 거리는 키의 1.4배, 상반신이 들어오는 정도.

대화 아바타에서는 전신보다 상반신이 낮습니다. 얼굴이 커서 표정이 보이고, 손 제스처도 화면 안에 들어옵니다.

그런데 이 값을 고정하면 안 됩니다. 캐릭터마다 키가 다릅니다. 같은 카메라 위치에서 어떤 모델은 얼굴이 잘리고, 어떤 모델은 화면 아래에 작게 놓입니다.

모델을 로드한 뒤 바운딩 박스로 중심과 높이를 재고, 그 값으로 카메라를 다시 겨누세요.

```
function aimCam() {
  controls.target.set(center.x, center.y + camYoff, center.z);
  camera.position.set(center.x, center.y + camYoff, center.z + camDist);
}
```

이 장 첫머리에서 "코드는 한 줄도 고치지 않았다"고 했는데, 카메라만은 그냥 되지 않습니다. 표준 뼈 이름은 자세를 지켜 주지만 신장은 지켜 주지 않습니다.

조명 은 정면 방향광 하나와 환경광 하나면 충분합니다. VRM 모델은 대개 툰 셰이딩을 쓰므로 복잡한 조명이 필요 없습니다. 저자는 방향광을 앞쪽 위(0.5, 1.5, 1.2)에 두고 환경광으로 그림자를 띄웁니다. 환경광을 충분히 주는 것 이 요령입니다. 어두우면 캐릭터가 침울해 보입니다.

디버그 창구를 하나 열어 두세요

저자의 뷰어에는 콘솔에서 부를 수 있는 함수가 하나 있습니다. 모델의 **바운딩 크기·중심·키·카메라 거리·메시 수·스킨드 메시 수** 를 한 번에 돌려줍니다.

새 캐릭터가 이상하게 보일 때 원인은 대개 셋 중 하나입니다 — **스케일이 다르거나, 중심이 어긋났거나, 스킨드 메시가 아니거나**. 이 함수 하나면 셋을 3초 만에 구분합니다. Ch19의 고정 포즈 장치와 함께, **눈으로 판단하지 않기 위한 최소 장비** 입니다.

배경 은 단색이나 흐린 그라데이션으로 두세요. 3D 배경을 넣으면 렌더 비용이 올라가고, 정작 캐릭터에 대한 주목도는 떨어집니다.

18.7 모델을 바꿔도 코드가 그대로인 이유 ★

"모델 파일 경로만 바꿨는데 전부 그대로 동작했다"는 이 트랙의 가장 큰 약속입니다. 그것이 성립하는 이유가 셋이고, 하나라도 빠지면 깨집니다.

저자의 실제 뷰어는 VRM만이 아니라 Blender로 리깅한 glb와 Mixamo 리그까지 세 계열을 한 코드로 돌립니다. 그 셋을 관통하는 규칙이 이것입니다.

① 뼈는 표준 이름 하나로만 부릅니다

VRM은 규격 자체가 `leftUpperArm` 같은 표준 이름을 강제합니다. 그런데 Blender는 `upper_arm.L`, Mixamo는 `mixamorig:LeftArm` 입니다.

코드가 그 차이를 알면 안 됩니다. 계열마다 표준 이름 → 실제 이름 표를 한 장 두고, 코드는 표준 이름만 씁니다.

표 18.1 표준 · VRM · Blender · Mixamo

표준	VRM	Blender	Mixamo
<code>leftUpperArm</code>	<code>leftUpperArm</code>	<code>upper_arm.L</code>	<code>mixamorig:LeftArm</code>
<code>chest</code>	<code>chest</code>	<code>chest</code>	<code>mixamorig:Spine2</code>

표를 바꾸는 것이 모델을 바꾸는 것이고, 코드는 한 줄도 안 바꿉니다.

없는 뼈는 그냥 건너뛸니다. 상반신만 있는 모델도 있고 어깨 뼈가 없는 모델도 있습니다. 하나 없다고 전체가 죽으면 교체가 안 됩니다. `getNormalizedBoneNode` 가 `null` 을 주면 그 뼈는 없는 것으로 두고 나머지를 진행하세요.

이름에서 점과 콜론이 사라집니다 — 저자가 해맨 곳

three.js는 glTF를 읽으면서 **노드 이름의 . 과 :** 을 지웁니다. 그래서 `upper_arm.L` 로 찾으면 없고, `upper_armL` 로 들어 있습니다. 어떤 익스포터는 `_` 로 바꿉니다.

뼈가 분명히 있는데 `getObjectByName` 이 `undefined` 를 돌려주는 상황입니다. 세 가지 표기를 차례로 시도하세요 — 원래 이름, 기호를 지운 것, 기호를 `_` 로 바꾼 것.

② 회전은 휴식 포즈로부터의 오프셋입니다

Ch19 §2의 원칙인데, 여기서 왜 그래야 하는지가 드러납니다.

같은 `rotg('neck', 0.04)` 를 세 모델에 넣으면 절대각은 셋 다 다르게 나옵니다. 목이 곧은 모델, 살짝 숙인 모델, 젖힌 모델이 각자의 자세 위에서 0.04만큼 끄덕입니다. 절대값 0.04를 썼다면 셋이 같은 자세로 굽었을 것입니다. 그 순간 교체가 깨집니다.

T 포즈 모델에는 부호가 하나 더 필요합니다

어떤 모델은 팔을 벌린 T 포즈로 로드됩니다. 휴식 포즈를 잡기 전에 팔을 내려야 하는데, **같은 각도인데 어떤 모델은 팔이 내려가고 어떤 모델은 올라갑니다.** 리그를 만든 도구에 따라 회전축 방향이 뒤집혀 있기 때문입니다.

모델마다 부호 하나(+1 / -1)를 정해 두면 됩니다. 그것 말고는 전부 같습니다.

③ 크기는 뼈로 재고, 단위가 이상하면 고칩니다

모델은 단위가 제각각입니다. 미터로 만든 것은 키가 1.6으로 들어오지만, **센티미터로 만든 것은 160, 어떤 것은 0.016** 으로 들어옵니다. 둘 다 화면에 안 보입니다 — 하나는 너무 커서, 하나는 너무 작아서.

메시의 바운딩 박스로 재지 마세요. 스킨드 메시는 애니메이션 상태와 모디파이어에 따라 값이 흔들립니다. **뼈의 월드 좌표**가 실제 몸의 범위를 정확히 줍니다. 뼈는 표면보다 조금 안쪽이므로 6% 정도 여유를 둡니다.

그리고 **정상 범위(0.5~4m)면 건드리지 않습니다.** 단위가 다른 것만 사람 키로 맞추니다.

셋을 한 함수에

로드 직후 한 번 도는 절차가 이것입니다.

표준 이름으로 뼈를 찾는다 (없으면 건너뛰다)

- 휴식 포즈를 복사한다
- T 포즈면 팔을 내린다 (부호 반영)
- 뼈로 키를 재고 필요하면 정규화한다

컴패니언 코드의 `rig.py`가 이 절차를 순수 함수로 갖고 있고, 세 계열 모델을 흉내낸 입력으로 33개 항목을 검사합니다. **점이 지워진 이름을 찾는가, 없는 뼈에서 터지지 않는가, 부호를 뒤집으면 팔이 반대로 가는가, cm 모델을 줄이는가** — 전부 저자가 실제로 걸렸던 자리입니다.

세 규칙을 실제 파일 아홉에 대 보니 ★

말로 된 규칙은 파일을 열어 보기 전까지 가설입니다. 컴패니언 `survey.py`가 저자 폴더의 VRM 여섯.glb 셋을 외부 라이브러리 없이(glTF의 JSON 청크만 읽어서) 열고, 세 규칙을 각각 셉니다.

표 18.2 실제 모델 아홉 — 표준 뼈 15개 중 잡힌 수 · 키 · 배율 (_work/survey.json)

파일	노드 이름으로 (VRM 표)	휴머노이드 지도로	Blender 표	Mixamo 표	메시 키(m)	배율
VRM 1.0 셋 (Avatar Sample_T · avatar · avatar_t)	0	15	0	0	1.64~1.78	1.0
VRM 0.x 둘 (avatar_f · avatar_m)	0	15	0	0	1.61~1.77	1.0
hanhwa.vrm (VRM 0.x, Blender 제작)	5	15	15	0	0.95	1.0
avatar_hface.glb (Blender)	5	—	15	0	0.95	1.0
gm_model.glb · avatar_g.glb (Mixamo)	0	—	0	15	0.012	×133

첫 열이 이 표의 발견입니다. 저자가 **젠 VRM 파일들에서는 노드 이름으로 표준 뼈가 하나도 안 잡혔습니다**. 노드는 `J_Bip_C_Hips` 같은 제작 도구의 이름이고, `hips` 라는 표준 이름은 확장 블록의 휴머노이드 지도(`humanBones`)에만 있습니다.

브라우저에서 "VRM은 이름이 그대로"로 보였던 것은 three-vrm이 그 지도를 읽어 정규화된 뼈를 내주기 때문이었습니다. 그러니 규칙 ①의 진짜 문장은 *"뼈 이름은 표준 이름 하나로 부른다"*가 아닙니다. "표준 이름으로 가는 지도를 하나 둔다 — VRM은 파일 안의 지도, glb는 코드 안의 표"입니다. 컴패니언은 이 조사 뒤 `humanoid_map()`을 얻었습니다(VRM 0.x는 목록, 1.0은 사전 — 모양까지 다릅니다).

Mixamo 둘의 메시 키 **0.012m**이 규칙 ③입니다. 제작 도구가 다른 단위로 내보낸 값이 미터로 읽힌 것이고, `scale_for()`가 ×133으로 1.6m에 맞춥니다. 이 한 줄이 없으면 캐릭터가 1.2cm로 서 있습니다 — 안 보이는 것이 아니라 *너무 작아서* 안 보입니다.

18.8 더 사실적인 3D는 없나 — 있습니다, 그런데

3D 아바타의 사실감 최전선은 리깅된 캐릭터가 아니라 **가우시안 스피래팅** 계열입니다. 실제 사람을 촬영해 만든 3D 표현이고, 실시간 렌더가 되며, 표정 블렌드셰이프를 붙이는 방법도 성숙했습니다. 사진 한 장에서 3D 얼굴을 만드는 연구도 나와 있습니다.

그런데 이 책의 Track B는 VRM입니다. 이유가 셋입니다.

제어 가능성. Ch19에서 만들 깜빡임·호흡·시선·제스처는 전부 *이름 붙은 파라미터에 값을 쓰는* 방식입니다. VRM은 그 이름이 표준화되어 있습니다. 스플래팅 아바타는 제어 인터페이스가 구현마다 다릅니다.

교체 가능성. 캐릭터를 바꿔도 코드가 그대로입니다. 스플래팅 아바타는 개체마다 촬영과 학습이 필요합니다.

배포. VRM은 파일 하나를 브라우저가 읽으면 끝입니다.

정리 — 가우시안 스플래팅은 사실감을 사고, VRM은 제어와 배포를 삽니다. 실시간 대화 아바타에서는 아직 후자가 더 자주 이깁니다. 이 균형은 바뀔 수 있고, 바뀌는 시점은 온라인 부록 K §8에서 추적합니다.

18.9 이 장에서 기억할 것

3D로 올라가는 이유는 제스처·시점·모델 교체 이고, 여전히 GPU는 필요 없습니다.

VRM의 핵심 가치는 표준화된 **빠·표정 이름**입니다. 한 번 만든 제어 코드가 모든 모델에서 동작합니다.

라이브러리는 **로컬에 두고 직접 서빙** 하세요. 폐쇄망·버전 고정·속도 모두에 이득입니다.

제어 통로는 셋 — **블렌드셰이프(표정·입) · 뼈 회전(자세) · 룩앳(시선)**. Ch16의 파라미터 층에 Ch17의 음량을 흘려 넣으면 즉시 말합니다.

시선을 카메라로 향하게 하는 것만으로 존재감이 크게 달라집니다.

모델을 바꿔도 코드가 그대로인 이유는 **규칙 셋**입니다 — 표준 이름으로만 부르기 · 휴식 포즈로 부터의 오프셋 · 뼈로 크기 재기 (§7).

다음 장에서 이 캐릭터에게 생명감을 넣습니다. 깜빡임과 호흡을 어떻게 설계하느냐가 "인형"과 "존재"를 가릅니다.

실습 코드: `code/ch18_vrm/` — 로컬 서빙 VRM 뷰어 + 파라미터 층 연결 + 모델 교체 데모

예상 소요: 3시간

관련 부록: A(환경) · F §E(살아 있어 보이지 않는 실패)

CHAPTER 19

코드로 만드는 생명감

19.1 완벽하게 규칙적인 것은 죽어 보입니다

코치를 3초마다 정확히 한 번 깜빡이게 만들었습니다. 30초쯤 보고 있으면 뭔가 불편해집니다. 코치는 Ch18에서 만든 3D VRM 홈트레이닝 코치입니다.

깜빡이기는 합니다. 그런데 메트로놈 같습니다. 사람은 그 규칙성을 의식적으로 세지 않지만 느낍니다.

기계이기 때문입니다. **살아 있는 것은 규칙적이지 않습니다.**

생명감의 대부분은 **적절한 양의 불규칙성**에서 나옵니다. 이 장은 그 불규칙성을 설계하는 이야기입니다. 코드 몇십 줄이 만드는 차이가 3D 모델의 품질보다 크다는 것을 보게 됩니다.

19.2 먼저 — 절대 각도를 쓰지 마세요 ★

이 장의 모든 기법에 앞서는 원칙이 하나 있습니다. 이걸 어기면 **캐릭터를 바꾸는 순간 자세가 무너집니다.**

VRM 모델은 저마다 기본 자세가 조금씩 다릅니다. 어떤 모델은 팔이 더 벌어져 있고, 어떤 모델은 고개가 살짝 숙여져 있습니다. 여기에 "목뼈를 0.04 라디안으로 설정"이라고 절대값을 쓰면 그 모델의 원래 자세를 지워 버립니다.

로드 시점의 기본 회전값을 저장해 두고, 거기에 더하세요.

```
// 로드 직후: restRot[본이름] = 본의 기본 회전값을 복사
function rotg(name, dx, dy, dz) {
  const b = bones[name], r = restRot[name];
  if (b && r) b.rotation.set(r.x + dx, r.y + dy, r.z + dz);
}
```

이 함수를 통해서만 뼈를 건드립니다. 이후 나오는 모든 수치는 **기본 자세로부터의 오프셋**입니다.

Ch18에서 "모델 파일 경로만 바꿨는데 전부 그대로 동작했다"고 했습니다. 표준 뼈 이름이 절반이고, **나머지 절반이 이 오프셋 방식**입니다. 절대 각도를 썼다면 캐릭터를 바꿀 때마다 값을 다시 잡아야 했을 것입니다.

19.3 깜빡임 — 가장 중요한 하나

하나만 넣을 수 있다면 깜빡임입니다.

주기는 난수로. 사람은 평균 3~5초에 한 번 깜빡이지만 편차가 큼니다. 다음 깜빡임까지의 간격을 매번 새로 뽑으세요. 2초에서 5초 사이의 난수면 충분합니다.

속도는 비대칭으로. 눈을 감는 것은 빠르고, 뜨는 것은 조금 느립니다. 감는 데 60밀리초, 뜨는 데 120밀리초 정도. 이 비대칭이 자연스러움의 상당 부분을 만듭니다.

가끔 두 번 연속으로. 사람은 종종 두 번 빠르게 깜빡입니다. 10~15% 확률로 이중 깜빡임을 넣으면 눈에 띄게 살아납니다.

말할 때는 조금 더 자주. 대화 중에는 깜빡임 빈도가 올라갑니다. 말하는 동안 간격을 35% 줄입니다(0.65배) — 아래 문헌 대조에서 정한 값입니다.

좌우를 미세하게 어긋나게. 두 눈은 완전히 동시에 감기지 않습니다. 20~30밀리초의 차이를 두면 훨씬 자연스럽습니다. 이 차이는 눈에 보이지는 않지만 느껴집니다.

깜빡임을 문헌에 대 보니 ★

"3~5초에 한 번"은 어디서 온 숫자일까요. 문헌은 **실 때 약 17회/분, 대화 중 약 26회/분** (Bentivoglio 등 1997; 일반적으로 15~20회/분)입니다. 컴패니언 `alive.py` 의 `Blink` 를 한 시간 돌려 셧습니다(`_work/measure.json`).

표 19.1 깜빡임 모델 1시간 시뮬레이션 — 분당 횟수

	실 때	말할 때	이중 깜빡임
문헌	17	26	있음
처음 코드 (2~5초 균등 · 말할 때 0.8배)	16.2	20.1	0%
고친 코드 (말할 때 0.65배 · 이중 12%)	18.1	27.1	11.5~12.3%

두 가지가 틀려 있었습니다. 말할 때 0.8배는 20회로 문헌의 26회에 한참 못 미쳤습니다. 그리고 위에서 "10~15% 확률로 이중 깜빡임을 넣으면 눈에 띄게 살아난다"고 써 놓고 **코드에는 이중 깜빡임이 없었습니다.** 본문이 약속한 것을 코드가 안 하고 있었던 것입니다.

둘 다 고쳐 0.65배·12%로 맞췄습니다. 숫자보다 이 절차를 기억하세요 — *자연스럽다* 는 느낌의 근거는 문헌의 빈도이고, 코드가 그 빈도를 내는지는 세어 봐야 합니다.

실제 구현은 이렇습니다

저자의 코치가 쓰는 값입니다. 위 권장안보다 **단순합니다.**

```
// 다음 깜빡임까지 2~5초 난수
if (blinkT > nextBlink + 0.15) nextBlink = blinkT + 2 + Math.random() * 3;
```

```
// 0.3초 창에서 사인 반파장 - 0 → 1 → 0
if (blinkT >= nextBlink - 0.15 && blinkT <= nextBlink + 0.15) {
  const p = (blinkT - (nextBlink - 0.15)) / 0.3;
  v = Math.sin(p * Math.PI);
}
expressionManager.setValue('blink', v);
```

주기는 2~5초 난수, 폭은 0.3초, 곡선은 사인 반파장. 감는 속도와 뜨는 속도가 같습니다 — 앞서 말한 비대칭이 아닙니다.

이 단순한 버전으로도 충분히 자연스럽습니다. 비대칭·이중 깜빡임·좌우 시차는 그 위에 얹는 개선이고, 없다고 어색하지는 않습니다. **먼저 이걸 넣고, 부족하다고 느낄 때 다듬으세요.**

`Math.sin(p *  $\pi$ )` 를 쓰는 이유는 따로 있습니다. **양 끝이 정확히 0이라 이음매가 없습니다.** 선형으로 오르내리면 시작과 끝에서 각이 지고, 그것이 눈에 띍니다. 이 트릭은 Ch20의 제스처에서도 그대로 다시 나옵니다.

19.4 호흡 — 눈에 보이지 않지만 느껴집니다

가슴과 어깨가 아주 조금 오르내립니다. 진폭은 눈으로 겨우 알아챌 정도, 주기는 4초 안팎.

느린 사인파 하나면 됩니다. 여기에도 약간의 불규칙을 섞으세요. 완전한 사인파는 다시 기계적입니다.

감정과 연동하면 좋습니다. 흥분한 상태에서는 호흡이 빠르고 깊어지고, 차분한 상태에서는 느리고 얕아집니다. 파라미터 하나로 이 둘을 오갑니다.

19.5 미세 움직임 — 정지하지 않기

사람은 가만히 있을 때도 완전히 정지하지 않습니다. 머리가 아주 조금씩 움직이고, 무게중심이 미세하게 이동합니다.

여러 주기의 느린 파동을 겹치세요. 서로 다른 주기의 사인파를 작은 진폭으로 더하면 반복되지 않는 것처럼 보이는 흔들림이 생깁니다. 주기끼리 나누어떨어지지 않게 고르는 것이 요령입니다.

사인 여섯 개가 전부입니다

저자의 코치가 대기 상태에서 돌리는 것 전부입니다. `st` 는 누적 시간(초)입니다.

```
roto('spine', sin(st*1.2)*0.012, 0, 0); // 호흡
roto('hips', 0, sin(st*0.4)*0.05, 0); // 무게중심 좌우
```



```
rotg('neck', sin(st*0.45)*0.04, sin(st*0.7)*0.05, 0);    // 머리
rotg('head', sin(st*0.6)*0.02, sin(st*0.9)*0.03, 0);    // 머리 미세
```

두 가지만 보세요.

첫째, 진동수가 전부 다릅니다 — $0.4 \cdot 0.45 \cdot 0.6 \cdot 0.7 \cdot 0.9 \cdot 1.2$ (초당 라디안). 둘씩 정수비인 짝은 있지만(0.45와 0.9, 0.6과 1.2) 여섯 전체의 공통 주기는 약 126초라, 겹친 파형은 **두 분 넘게 같은 모양으로 돌아오지 않습니다**. 사람이 "아, 반복되네" 하고 느끼지 못하는 이유입니다.

둘째, 진폭이 믿기 어려울 만큼 작습니다. 호흡은 0.012 라디안 — 0.7도입니다. 가장 큰 것도 0.05 라디안(약 3도)입니다.

숫자만 보면 "이게 보이거나 할까" 싶습니다. 보이지 않습니다. 그런데 끄면 즉시 알아챱니다. § 10의 판단 기준이 여기서 나왔습니다.

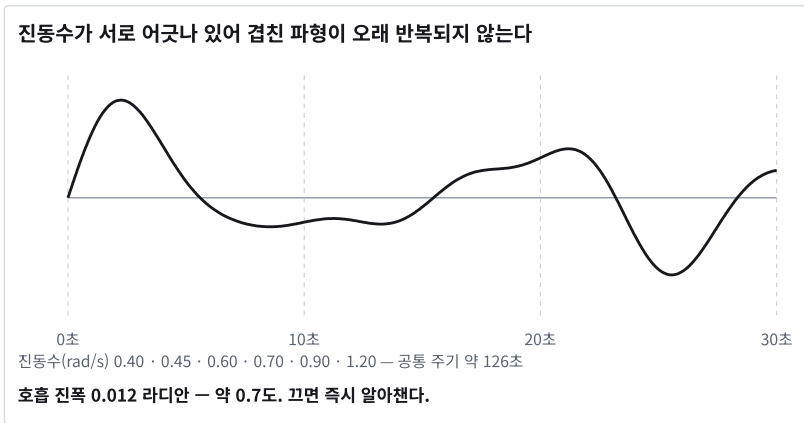


그림 19.1 서로 배수가 아닌 여섯 주기를 겹치면 파형이 반복되지 않는다

머리를 neck 과 head 둘로 나눈 것도 의도적입니다. 한 뼈에 큰 진폭을 주는 것보다, 두 뼈에 작게 나눠 주면 목이 꺾이지 않으면서 움직임이 살아납니다.

19.6 시선 — 존재감의 핵심

기본은 카메라를 봅니다. 사용자와 눈이 마주칩니다.

가끔 시선을 돌립니다. 계속 응시하면 부담스럽습니다. 몇 초에 한 번 짧게 옆이나 위를 봤다가 돌아옵니다. 생각하는 것처럼 보입니다.

말할 때와 들을 때가 다릅니다. 사람은 들을 때 상대를 더 오래 보고, 말할 때 시선을 더 자주 돌립니다. 이 패턴을 반영하면 대화의 리듬이 생깁니다.

시선 이동에 머리를 조금 따라 움직이세요. 눈만 굴리면 기괴합니다. 눈이 먼저 가고 머리가 조금 따라가는 순서가 자연스럽습니다.

19.7 아이들 모션 — 심심하지 않게

오래 아무 일도 없으면 캐릭터가 뭔가 하도록 만듭니다. 가벼운 스트레칭, 고개 갸웃, 자세 고쳐 잡기.

빈도는 낮게. 20~30초에 한 번 이하. 자주 하면 산만합니다.

대화가 시작되면 즉시 중단 하고 응답 상태로 전환합니다.

중단이 부드러워야 합니다. 진행 중인 동작을 뚝 끊지 말고, 지금 자세에서 목표 자세로 짧게 보간해 넘어갑니다.

19.8 층으로 쌓기

지금까지의 요소는 서로 독립적으로 돌면서 같은 파라미터에 값을 더합니다. 충돌을 막는 방법은 층으로 쌓는 것 입니다. 층은 여덟입니다.

기본 자세 층 — 캐릭터의 기준 포즈.

미세 움직임 층 — 위의 느린 파동들. 항상 둥니다.

호흡 층 — 항상 둥니다.

감정 층 — 지금의 감정 표정. 하나만 활성화. 강도는 상태에 따라 다릅니다 — 저자의 구현은 말할 때 0.7, 대기 중에는 0.12로 두고 그 사이를 보간합니다. 대기 중에도 표정을 완전히 0으로 내리지 않는 것이 요령입니다. 감정이 남아 있는 얼굴이 됩니다.

깜빡임 층 — 감정 위에 겹칩니다. 감정 표정을 덮어쓰지 않고 눈만 건드립니다.

입 층 — 음량 구동. 얼굴 층 가운데 가장 위.

동작 층 — 제스처가 실행 중일 때만. 다른 층 위에 덮어씹니다.

물리 층 — 머리카락·옷자락·장신구의 흔들림. 위의 모든 층이 끝난 뒤 자동으로 따라옵니다.

아래에서 위로 순서대로 적용하면 서로 간섭하지 않습니다. 그리고 어느 층 하나를 꺼도 나머지는 계속 둥니다.

19.9 공짜로 얻는 생명감 — 스프링본

지금까지는 전부 여러분이 계산해서 넣는 움직임이었습니다. **하나는 공짜입니다.**

VRM 모델에는 대개 **스프링본** 이 설정되어 있습니다. 머리카락, 옷자락, 리본, 꼬리, 귀. 이 뼈들은 여러분이 건드리지 않아도 **몸의 움직임을 따라 흔들립니다.** 관성과 감쇠를 물리가 계산합니다.

여러분이 할 일은 한 줄뿐입니다.

```
vrn.update(deltaTime); // 렌더 루프 안에서 경과 시간만 넘긴다
```

이 한 줄을 빠뜨리면 **머리카락이 조각처럼 굳습니다.** 그리고 이건 의외로 자주 놓칩니다 — 뼈 회전은 잘 되는데 뭔가 죽어 보일 때, 대개 이 호출이 없습니다.

왜 이게 큰가

§5에서 미세 움직임의 진폭이 0.012 라디안이라고 했습니다. 눈에 안 보이는 값입니다.

그런데 스프링본이 그것을 증폭합니다. 고개가 0.7도 움직이면 머리카락 끝은 그보다 훨씬 크게 흔들립니다. 시간차를 두고 따라오고, 몇 번 출렁이다 멈춥니다. **보이지 않는 움직임이 보이는 움직임으로 번역됩니다.**

긴 머리 캐릭터와 짧은 머리 캐릭터의 생명감이 다르게 느껴지는 이유가 이것입니다. 같은 코드인 데도요.

함정 둘

제스처가 클수록 물리가 과해집니다. 팔벌려뛰기처럼 큰 동작에서 머리카락이 심하게 날립니다. 캐릭터에 따라 뚫고 나가기도 합니다. 큰 동작을 쓸 계획이면 **그 동작으로 한 번 돌려 보고 스프링 강성을 조절하세요.**

모델마다 설정이 천차만별입니다. 어떤 모델은 스프링본이 아예 없고, 어떤 모델은 과하게 걸려 있습니다. Ch18의 캐릭터 교체가 **코드 수정 없이** 되는 것과 달리, **흔들림의 인상은 모델마다 다릅니다.** 새 캐릭터를 넣으면 이것부터 확인하세요.

19.10 얼마나 넣어야 하나

과하면 산만합니다. 미세 움직임의 진폭이 크면 취한 것처럼 보이고, 시선이 자주 돌면 불안해 보이고, 아이들 모션이 잦으면 대화에 집중하지 못하는 것처럼 보입니다.

판단 기준은 하나입니다. **의식적으로 알아채면 과한 것입니다.** 좋은 생명감은 보이지 않습니다. 꺾을 때 비로소 "뭔가 죽었네" 하고 느껴지는 정도가 적절합니다.

만든 뒤에 각 요소를 하나씩 꺼 보세요. 끄면 어색해지고 켜면 의식되지 않는 값이 정답입니다.

19.11 이 장에서 기억할 것

규칙적인 것은 **죽어 보입니다**. 생명감은 적절한 양의 불규칙성에서 나옵니다.

하나만 넣는다면 **깜빡임**입니다. 난수 주기 · 말할 때 0.65배 · 이중 깜빡임 · 좌우 미세 차이.

호흡 은 느린 사인파, **미세 움직임** 은 서로 나누어떨어지지 않는 여러 주기의 겹침.

시선 이 존재감의 핵심입니다. 기본은 카메라, 가끔 돌리기, 말할 때와 들을 때 다르게, 머리가 조금 따라가기.

요소들은 **층으로 쌓아** 관리합니다. 아래에서 위로 적용하면 간섭하지 않습니다.

의식적으로 알아채면 과한 것입니다. 컸을 때 비로소 허전한 정도가 적절합니다.

다음 장에서 LLM이 이 캐릭터의 표정과 동작을 고르게 만듭니다.

실습 코드: `code/ch19_alive/` — 깜빡임·호흡·미세움직임·시선 층 구현 + 온오프 비교 데모

예상 소요: 3시간

관련 부록: F §E(살아 있어 보이지 않는 실패) · L §1(첫 3초)

CHAPTER 20

감정 태그와 제스처

20.1 말과 몸이 따로 놀면

코치가 "자, 같이 스트레칭할까요!"라고 말하면서 팔짱을 낀 채 서 있으면, 아무도 따라 하지 않습니다. 코치는 Ch18·Ch19를 거친 3D VRM 홈트레이닝 코치입니다.

비유가 아니라 실제로 일어난 일입니다. 코치의 첫 버전은 말은 잘했지만 몸이 가만히 있었습니다. 시연을 본 사람들의 반응은 한결같았습니다. "말은 하는데 좀 이상하네요." 무엇이 이상한지는 아무도 정확히 짚지 못했습니다.

캐릭터가 "정말 기뻐요!"라고 말하는데 얼굴이 무표정이면 말의 내용이 통째로 무효가 됩니다. 사람은 말보다 몸을 먼저 믿습니다.

그러면 무엇이 말에 맞는 표정과 동작인지는 누가 정할까요. **LLM이 정하게 하면 됩니다.** 이미 문장을 만들고 있으니, 그 문장에 어울리는 감정과 동작도 함께 고르게 하면 됩니다.

이 장의 방법은 단순한데 효과가 큼니다. 그리고 추가 지언이 거의 없습니다.

20.2 태그를 앞에 붙이게 합니다

LLM이 응답 맨 앞에 대괄호 태그 두 개를 붙이게 합니다.

[excited][jumpingjack] 자 같이 운동해요!

첫 번째가 감정, 두 번째가 동작입니다. 코드가 태그를 떼어내고, 감정은 감정 층에, 동작은 동작 층에 넣고, 남은 텍스트만 TTS로 보냅니다.

왜 뒤가 아니라 앞인가. 스트리밍 때문입니다. 첫 토큰에 태그가 나오면 텍스트가 생성되는 동안 이미 표정과 동작을 시작합니다. 뒤에 붙이면 전체 응답을 기다려야 하고, Ch07의 문장 청킹과 충돌합니다.

왜 JSON이 아닌가. JSON으로 감싸면 파싱은 깔끔합니다. 대신 모델이 형식을 어기는 순간 전체가 깨집니다. 대괄호 태그는 없으면 없는 대로 기본값으로 넘어갑니다. **실패해도 우아하게 퇴화**하는 형식이 대화 시스템에는 더 맞습니다.

이것이 텍스트투모션입니다 ★

코치가 하는 일을 한 줄로 부르면 **텍스트투모션(text-to-motion)** 입니다. 사용자가 "같이 팔벌려 뛰기 하자"라고 말하면, 그 말이 코치의 동작이 됩니다. 경로는 셋입니다 — 말이 LLM에 들어가고, LLM이 응답 맨 앞에 [jumpingjack] 을 고르고, 브라우저가 그 이름의 동작 함수 (§ 5)를 대기 동작 위에 얹어 실제로 팔을 벌리고 뛸니다. 정장을 입은 남성 프리셋(아저씨)이든 여성 프리셋이든 같은 태그가 같은 뼈를 움직입니다 — Ch18의 표준 뼈대 이름 덕분입니다.

이 책의 텍스트투모션은 '생성'이 아니라 '선택'입니다. 연구 쪽의 텍스트투모션은 문장에서 새 동작 궤적을 **합성** 하는 모델을 뜻합니다(사람 동작 데이터로 학습한 **확산·트랜스포머** 계열). 대화 아바타에는 그 길을 쓰지 않았습니다. 이유는 셋입니다. ① **지연** — 태그는 첫 토큰과 함께 도착해 말이 시작되기 전에 동작이 시작되지만(위 "왜 앞인가"), 동작을 합성하는 모델은 문장을 다 받은 뒤 몇 초를 더 씁니다. ② **통제** — 열세 개짜리 닫힌 목록은 이상한 자세가 나올 수 없고, 강도·길이를 § 6에서 곱하기 한 번으로 바꿉니다. ③ **끼어들기** — 사용자가 말을 끊으면 동작도 엔벨로프 한 줄로 접힙니다 (§ 5). 합성된 클립은 중간에 끊으면 굳습니다 (§ 5의 클립·코드 비교).

그래서 코치의 텍스트투모션은 *어떤 동작을 할지* 만 언어모델에게 맡기고, *어떻게 움직일지* 는 코드가 씁니다. 공연·연출처럼 새 동작이 계속 필요한 자리라면 합성 모델이 맞고, 그때도 이 장의 태그 층은 그 모델 앞에 두는 라우터로 그대로 남습니다.

20.3 목록을 좁게 유지하세요

감정과 동작의 후보 목록을 시스템 프롬프트에 적습니다. 그리고 **짧게 유지** 하세요.

감정은 여섯 개면 충분합니다. 인사 · 기쁨 · 신남 · 생각 · 슬픔 · 평상.

대화용 동작은 여덟에 '없음'을 더한 아홉으로 시작합니다. 손 흔들기 · 인사 · 끄덕임 · 가리키기 · 박수 · 만세 · 고민 · 어깨 으쓱 · 없음. 운동 동작 다섯(팔벌려뛰기 · 팔 돌리기 · 스트레칭 · 비틀기 · 스쿼트)은 § 6에서 따로 다룹니다 — 합쳐 열세 개입니다.

목록이 길어지면 두 가지가 나빠집니다. 모델이 엉뚱한 것을 고르는 빈도가 올라가고, 여러분이 그 동작들을 전부 구현해야 합니다.

각 태그의 의미를 프롬프트에 한 줄로 설명 하세요. 이름만으로는 모델이 오해합니다. point=가리키기, shrug=어깨 으쓱 같은 짧은 주석이 정확도를 눈에 띄게 올립니다.

코치의 동작 목록은 이렇게 생겼습니다. 앞의 여덟 개가 일반 대화용, 뒤의 다섯 개가 운동용입니다.

일반 : wave · bow · nod · point · clap · cheer · think · shrug
운동 : jumpingjack · armcircle · stretch · twist · squat

그리고 한 줄이 더 필요했습니다.

"운동·체조·PT·준비운동·스트레칭 요청이면 *jumpingjack/armcircle/stretch/twist/squat* 중에서 고르세요."

이 한 줄을 넣기 전에는 코치가 "같이 운동해요"라고 말하면서 `nod` 를 골랐습니다. 끄덕이기만 한 것입니다. 동작 목록에 스퀴트가 있는데도 고르지 않았습니다. 모델은 목록을 보지만, 그 목록이 언제 쓰이는지는 모릅니다.

한 줄을 넣고 나서는 이렇게 나왔습니다.

[excited][jumpingjack] 자 같이 운동해요!

도메인이 정해진 아바타라면 이런 조건 지시 한 줄이 태그 품질의 절반입니다. 목록을 늘리는 것보다 훨씬 효과가 큼니다.

20.4 파싱은 관대하게

모델은 형식을 어깁니다. 반드시 어깁니다. 파서는 그것을 전제로 만듭니다.

태그가 없으면 기본값(평상·동작 없음)으로 갑니다.

태그가 하나만 있으면 그것이 감정인지 동작인지 목록과 대조해 판단합니다.

모르는 이름이면 무시하고 기본값을 씁니다.

태그가 세 개 이상이면 앞의 두 개만 씁니다.

태그가 문장 중간에 있으면 — 이것도 일어납니다. 텍스트 전체에서 대괄호 패턴을 지우는 안전망을 넣으세요. 태그가 TTS로 넘어가 "대괄호 익사이트드"라고 읽는 사고를 막습니다.

파싱 실패가 대화를 끊어서는 안 됩니다. 최악의 경우에도 무표정으로 말은 하게 하세요.

20.5 동작을 어떻게 만드나

각 동작은 시간에 따른 뼈 회전값의 시퀀스입니다. 만드는 방법은 셋입니다.

코드로 직접 작성. 간단한 동작은 대부분 이것으로 충분하고, 수정이 쉽습니다.

동작 = 시간을 받는 순수 함수

저자의 구현은 각 동작을 함수 하나 로 정의합니다. 진행도 p (0~1)와 경과 시간 t (초)를 받아 관절 채널별 각도를 돌려줍니다.

```

wave: (p,t) => ({ ruz:0.95, rux:0.1, shz:0.35,
                  rly: 0.45 + Math.sin(t*15)*0.45, tilt:0.05 } ),
bow:  (p,t) => ({ lean: 0.55*Math.sin(p*Math.PI),
                  nod: 0.25*Math.sin(p*Math.PI) } ),
nod:  (p,t) => ({ nod: 0.20*Math.sin(t*9) } ),

```

p와 **t**를 나눠 받는 것이 설계의 핵심입니다.

반복되는 움직임은 **t**를 씁니다. 손목을 흔드는 `sin(t*15)`, 끄덕이는 `sin(t*9)`는 지속 시간과 무관하게 같은 속도로 씁니다.

한 번 갔다 오는 움직임은 **p**를 씁니다. 인사(bow)의 `sin(p*π)`는 진행도가 0→1로 가는 동안 0→1→0을 그립니다. 숙였다가 저절로 돌아옵니다. §6의 "기본 자세로 돌아옵니다"가 별도 코드 없이 해결됩니다.

Ch19 §3의 사인 반파장 트릭이 여기서 다시 씁니다. 양 끝이 정확히 0인 함수는 이음매를 만들지 않습니다.

시작과 끝을 감싸는 한 줄

동작을 그냥 켜고 끄면 첫 프레임과 마지막 프레임에서 팔이 튕니다. 저자는 **엔벨로프** 한 줄로 막습니다 — Ch16 §6에서 본 그 곡선입니다.

```
const env = Math.min(1, Math.min(actT, dur - actT) / 0.3);
```

시작 후 0.3초 동안 0→1로 올라오고, 끝나기 0.3초 전부터 1→0으로 내려갑니다. 모든 채널에 이 값을 곱합니다.

이 한 줄이 동작 열세 개의 시작·끝 처리를 전부 해결합니다. 동작마다 페이드인·아웃을 따로 짤 필요가 없습니다.

엔벨로프 한 줄이 동작 13종의 시작·끝을 처리한다



양 끝 0.3초에서 0으로 접한다 — 동작마다 페이드를 따로 짤 필요가 없다.

그림 20.1 엔벨로프가 동작의 시작과 끝 0.3초를 0으로 접는다

절차적 생성. 파라미터 몇 개로 동작군을 만듭니다. "끄덕임"을 진폭·횟수·속도의 함수로 정의하면 상황에 따라 다른 느낌의 끄덕임이 나옵니다.

애니메이션 클립 재생. 미리 만들어진 모션 데이터를 불러와 재생합니다. 아래에서 따로 봅니다.

처음에는 코드로 직접 쓰는 편을 권합니다. 열 개 정도의 동작은 하루면 만듭니다.

클립을 쓸 것인가, 코드로 쓸 것인가 ★

이 선택이 제스처 시스템의 성격을 결정합니다.

클립은 모션 캡처나 애니메이터가 만든 데이터입니다. 사람이 실제로 움직인 궤적이라 **디테일**이 **다롭니다** — 무게중심 이동, 관성, 손가락의 미세한 움직임. 코드로는 흉내내기 어렵습니다.

무료·유료 모션 라이브러리가 많고, VRM 진영에는 아바타 애니메이션 전용 포맷(VRMA)도 있습니다. 범용 모션은 대개 사람형 뼈대 기준이라 **리타게팅**을 거쳐 VRM 표준 뼈에 매핑합니다.

그런데 대화 아바타에서는 클립이 생각보다 불편합니다.

표 20.1 클립 · 코드

	클립	코드
디테일	높음	낮음
길이	고정	자유
강도 조절	어려움	곱하기 한 번
중간에 끊기	어색함	자연스러움
조합	블렌딩 필요	더하면 됨
캐릭터 교체	리타게팅	그대로
추가 비용	파일·용량	함수 한 줄

길이와 강도가 핵심입니다. 대화 아바타의 동작은 *말의 길이에 맞춰야* 하고(§ 6), *감정에 따라 크기가 달라져야* 합니다(§ 6의 '감정은 크기와 빈도를 동시에 바꿉니다'). 클립은 둘 다 어렵습니다 — 2초짜리 클립을 1.3초 발화에 맞추려면 재생 속도를 바꿔야 하고, 그러면 어색해집니다.

끼어들기도 문제입니다. 사용자가 말을 끊으면 동작도 즉시 멈춰야 하는데(Ch23), 클립을 중간에 끊으면 어정쩡한 자세로 굳습니다. 코드 동작은 엔벨로프가 알아서 접어 줍니다.

판단 — *대화하는* 아바타는 **코드**, *공연하는* 아바타는 **클립**. 춤·체조 시연·정해진 인사 동작처럼 **길이와 내용이 고정된** 것만 클립으로 두고, 대화 반응은 코드로 만드세요. 저자의 코치가 운동 동작까지 코드로 짰 이유입니다 — 운동은 반복 횟수를 상황에 맞춰 늘려야 하니까요.

쉬울 수도 있습니다. 클립을 하위 층으로 재생하고 그 위에 코드 제스처를 더하는 구조가 가능합니다. Ch19 §8의 층 구조에서 동작 층을 둘로 나누면 됩니다. 다만 두 층이 같은 뼈를 건드리면 층 돌하므로 **담당 뼈를 나눠 두는 것**이 안전합니다 — 예를 들어 하체는 클립, 상체는 코드.

20.6 동작 실행의 규칙

말과 동시에 시작합니다. 오디오 재생과 동작 시작을 같은 순간에 트리거합니다. 태그가 먼저 도착하므로 가능합니다.

말보다 짧게 끝냅니다. 동작이 말보다 길면 말이 끝난 뒤에도 팔이 허공에서 움직입니다.

저자가 쓰는 지속 시간은 **두 무리로 갈립니다**.

표 20.2 갈래 · 동작 · 지속

갈래	동작	지속
대화 제스처	인사·끄덕임·어깨 으쓱	1.5~2.3초
운동 동작	팔벌려뛰기·스쿼트·팔 돌리기	4.0~4.5초

대화 제스처는 **한 번 하고 끝나는 것**이라 짧고, 운동은 **반복을 보여줘야** 하므로 깁니다. 4.5초 동안 $\sin(t*6)$ 이면 대략 네 번 반복됩니다. **용도가 지속 시간을 정하지, 미학이 정하지 않습니다.**

기본 자세로 돌아옵니다. 동작이 끝나면 부드럽게 원래 자세로 복귀합니다. 이 복귀가 없으면 팔이 든 채로 굳습니다.

중첩을 허용하지 않습니다. 새 동작이 들어오면 진행 중인 것을 지금 자세에서 보간해 전환합니다. 두 동작을 동시에 재생하면 팔이 이상한 곳으로 갑니다.

표정은 더 오래 유지합니다. 대화 제스처는 2초 안에 끝나지만, 감정 표정은 그 발화가 끝날 때까지 두는 편이 자연스럽습니다.

감정은 크기와 빈도를 동시에 바꿉니다 ★

태그로 받은 감정을 표정에만 쓰면 절반만 쓰는 것입니다. **감정은 몸의 리듬도 바꿔야 합니다.**

말하는 동안 코치는 손동작 자세 몇 가지를 랜덤으로 오갑니다. 그 **진폭**과 **전환 주기**를 감정이 결정합니다.

표 20.3 감정 · 진폭 · 전환 주기

감정	진폭	전환 주기
신남	1.35	0.35~0.6초
인사·기쁨	1.0~1.1	0.5~0.9초
평상	0.8	0.6~1.1초
생각	0.5	1.0~1.7초
슬픔	0.4	1.2~2.0초

신남은 **크고 빠르게**, 슬픔은 **작고 느리게**. 두 축이 같은 방향으로 움직입니다.

이것이 Ch03 §5에서 "감정은 목소리와 표정 두 경로로 들어간다"고 한 것의 3D 판입니다. 3D 캐릭터에는 세 번째 경로 — 몸의 리듬 이 있습니다. 그리고 이 경로가 실사 트랙에는 아예 없습니다(Ch12 §6). Track B가 감정 표현에서 Track A를 이기는 지점입니다(Ch05 §3 결정표).

20.7 몸을 만지다 배운 것 셋

교과서에 없고, 직접 관절을 돌려 보면 반나절 안에 만나는 것들입니다.

팔을 올리면 어깨도 올려야 합니다

위팔만 회전시키면 어깨에서 팔이 끊겨 보입니다. 사람은 팔을 들 때 어깨가 따라 올라갑니다.

저자의 모든 동작 정의에 shz (어깨 z회전) 채널이 함께 들어 있는 이유입니다. 만세(cheer)는 팔 1.05에 어깨 0.4, 스트레칭은 팔 1.5에 어깨 0.45. 팔 각도의 3분의 1 정도 를 어깨에 주면 자연스럽습니다.

팔꿈치는 돌리지 마세요

아래팔(forearm)의 회전축은 모델마다 불안정합니다. 돌리면 팔이 비틀립니다. 특히 여러 캐릭터를 바꿔 쓰는 구조에서는 모델마다 결과가 다릅니다.

저자의 동작 적용부는 위팔·어깨·목·가슴·다리만 건드리고 아래팔은 굳게 유지 합니다. 손 흔들기 처럼 아래팔이 꼭 필요한 동작만 예외로 두고, 그것도 한 축만 씁니다.

표현력을 조금 잃고 안정성을 얻는 거래 이고, 캐릭터 교체가 가능한 시스템에서는 이쪽이 맞습니다.

스쿼트는 관절만으로 안 됩니다

무릎과 엉덩이를 굽히면 다리는 접히는데 몸이 제자리에 떠 있습니다. 실제로 앉으려면 캐릭터 전체가 내려가야 합니다.

```
modelRoot.position.y = modelBaseY - (o.drop || 0) * env;
```

동작 정의에 `drop` 채널을 두고 **루트 오브젝트의 높이를 직접 내립니다**. 끝나면 원래 높이로 되돌립니다.

일반화하면 — 뼈 회전으로 표현되지 않는 움직임이 있습니다. 점프, 앉기, 뒷걸음. 이런 것은 **루트 변환** 이 필요하고, 그때는 원위치 복구를 반드시 짝지어 두세요.

그리고 하나 더. 좌우 대칭 동작에서 **다리의 부호가 서로 반대** 입니다. 원다리는 +z가 바깥, 오른 다리는 -z가 바깥. 이런 것은 문서를 봐서 알 수 없고 **한 번 돌려 보고 적어 두는 수밖에 없습니다**. 저자의 코드에도 그 자리에 "실축"이라는 주석이 붙어 있습니다.

20.8 이모지와 특수문자 제거

이모지가 TTS로 넘어가면 엔진에 따라 이름을 잃어버리거나 침묵으로 처리합니다. 어느 쪽인지 예측할 수 없다는 것이 문제입니다. 그래서 이 장의 구조에서는 반드시 걸려야 합니다.

시스템 프롬프트에 **"이모지나 특수문자는 쓰지 마세요"** 를 명시하세요. 그리고 그래도 나올 것을 전제로 코드에서 한 번 더 거릅니다. Ch03에서 만든 정규화가 여기서 다시 쓰입니다.

20.9 S2S를 쓰면 이 장이 통째로 무너집니다

Ch07 §9에서 예고한 대가가 여기서 청구됩니다.

음성-대-음성 모델은 텍스트를 거치지 않습니다. 그런데 이 장의 전체 구조가 **"LLM이 만든 텍스트 앞에 태그를 붙인다"** 는 전제 위에 서 있습니다. 텍스트가 없으면 붙일 자리가 없습니다.

선택지는 셋입니다.

전사를 함께 받습니다. 많은 S2S 제공자가 오디오와 별도로 전사 텍스트를 스트리밍해 줍니다. 그것이 있으면 이 장의 방식이 그대로 동작합니다. **아바타 프로젝트에서 S2S를 고를 때 가장 먼저 확인할 항목** 입니다.

표정을 오디오에서 뽑습니다. 텍스트 대신 음성의 톤·속도·음높이에서 감정을 추정해 표정에 매핑합니다. 정확도는 떨어지지만 지연이 0에 가깝고, Ch17의 음량 구동과 같은 계통의 발상입니다. 동작까지는 어렵고 표정 정도가 한계입니다.

감정만 별도 경로로 받습니다. S2S와 병렬로 가벼운 모델에게 "지금 대화의 분위기"를 묻습니다. 지연이 붙지만 태그 품질은 유지됩니다.

판단 — 얼굴이 있는 아바타에서 S2S를 쓰려면 **전사를 주는 제공자**를 고르세요. 그것이 없으면 표정만 오디오에서 뽑고 동작은 포기하는 것이 현실적입니다. 그리고 그 순간 Ch27의 말투 채점도 함께 포기하게 된다는 것을 알고 결정하세요.

20.10 이 장에서 기억할 것

LLM 응답 맨 앞에 **대괄호 태그 두 개** — 감정과 동작. 앞에 붙이는 이유는 스트리밍입니다.

JSON보다 대괄호 태그가 낫습니다. 실패해도 우아하게 퇴화합니다.

목록은 **짧게** 유지하고, 각 **태그의 의미를 한 줄로 설명**하세요.

파서는 **관대하게** 만듭니다. 태그 없음 · 하나만 · 모르는 이름 · 중간 삽입을 전부 처리하고, 최악의 경우에도 말은 하게 합니다.

동작은 **말과 동시에 시작, 말보다 짧게, 기본 자세로 복귀, 중첩 금지**. 표정은 발화가 끝날 때까지 유지.

이모지는 프롬프트와 코드 두 겹으로 제거합니다.

말이 곧 동작이 되는 이 구조가 **텍스트투모션**입니다. 새 궤적을 만드는 것이 아니라 목록에서 고르는 방식이라 실패해도 말은 나옵니다(§ 2).

감정은 동작의 크기와 빈도를 동시에 바꿉니다 — 신남은 크고 잣게, 슬픔은 작고 뜸하게(§ 6).

S2S를 쓰면 **태그를 붙일 텍스트가 없습니다**. 전사를 주는 제공자를 고르거나, 표정만 오디오에서 추정하고 동작은 포기하세요.

다음 장에서 지금까지의 다섯 장을 하나의 서버로 묶습니다. Track B의 완성입니다.

실습 코드: `code/ch20_gesture/` — 태그 파서 + 동작 라이브러리 13종 + 표정 매핑

예상 소요: 4시간

관련 부록: F § E(살아 있어 보이지 않는 실패) · L § 5(여럿이 나올 때)

CHAPTER 21

트랙 B 완성 — 2초 응답 대화 아바타

21.1 만들 것

여기서 코치가 완성됩니다. 브라우저에서 도는 3D VRM 홈트레이닝 코치, Track B의 완성형입니다.

앞의 네 장에서 만든 것들 — 브라우저 3D 렌더, 음량 립싱크, 깜빡임과 호흡, 감정 태그와 제스처 — 이 한 서버 뒤에서 하나로 붙습니다. 그리고 GPU는 여전히 한 장도 쓰지 않습니다.

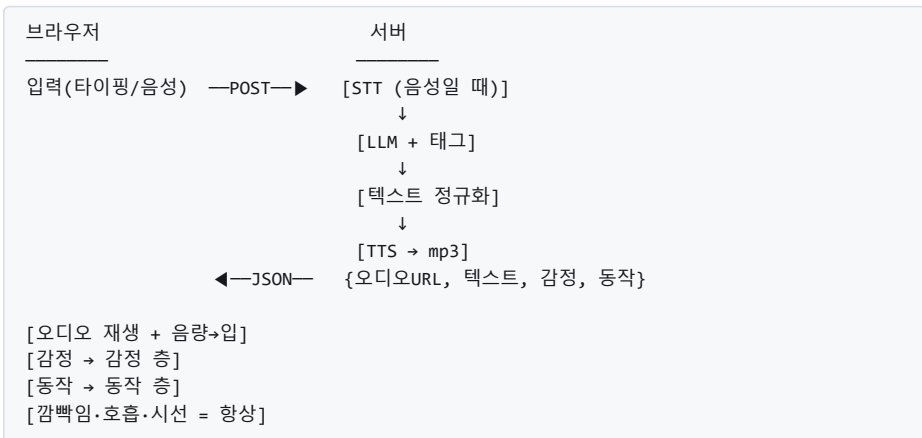
브라우저를 열면 3D 캐릭터가 서 있습니다. 눈을 깜빡이고, 숨을 쉬고, 이쪽을 보고 있습니다.

말을 걸면 **2초 안에** 대답이 시작됩니다. 첫 문장에서 TTS로 넘기고 첫 청크에서 재생하는 **겹침** 경로로 만들었을 때의 이야기입니다 — §3의 읽기 쉬운 단순 서버는 §5의 실측대로 그 두 배가 걸립니다.

말하는 동안 입이 움직이고, 내용에 어울리는 표정을 짓고, 손을 흔듭니다. 말이 끝나면 다시 조용히 이쪽을 봅니다.

서버는 노트북 한 대면 됩니다.

21.2 전체 구조



서버는 텍스트와 오디오만 만듭니다. 얼굴에 관한 모든 일은 브라우저에서 일어납니다. 이 분담이 이 트랙의 성격을 결정합니다.

21.3 서버 — 한 파일로 충분합니다

Track B의 서버는 놀랍도록 단순합니다. 웹 프레임워크 하나에 엔드포인트 셋이면 됩니다.

정적 파일 서빙 — 3D 라이브러리, VRM 모델, 생성된 오디오.

대화 엔드포인트 — 텍스트를 받아 LLM을 부르고, 태그를 파싱하고, TTS를 돌리고, 결과를 JSON으로 반환.

메인 페이지 — 프론트엔드.

Ch07에서 만든 지연 최적화를 전부 여기에 넣습니다. 모델 폴백 리스트, 추론 예산 끄기, 짧은 응답 강제, 오프라인 폴백 답변.

LLM 타임아웃은 5초로 잡습니다. 첫 모델이 5초를 넘기면 끊고 사다리의 다음 모델로 갑니다 (Ch07 §6) — §5에서 이 5초가 어떻게 되돌아오는지 봅니다.

TTS 호출은 락으로 보호하세요. 파일 이름에 순번을 쓴다면 동시 요청에서 충돌합니다. 카운터를 락으로 감싸는 몇 줄이면 됩니다.

나머지 타임아웃도 짧게 잡습니다. TTS 20초. 넘으면 폴백 답변으로 갑니다.

무거운 것을 이벤트 루프에 올리지 마세요

여기서 초보자가 가장 자주 서버를 멈춥니다.

비동기 웹 서버의 이벤트 루프는 **한 번에 하나만** 돕니다. 그 안에서 오래 걸리는 작업을 직접 실행하면 **루프 전체가 멈춥니다**. 그동안 다른 사용자의 요청도, 진행 중인 스트리밍도 전부 정지합니다.

TTS 실행, 모델 추론, 파일 변환이 전부 여기 해당합니다.

무거운 것은 스레드로 떼어 내고, 가벼운 I/O만 비동기로 두세요. 프레임워크가 주는 스레드 풀 실행 헬퍼를 쓰면 한 줄입니다. 저자의 서버는 TTS 호출을 그렇게 감쌉니다.

증상이 특이해서 알아보기 쉽습니다 — **혼자 쓸 때는 멀쩡한데 두 명이 동시에 쓰면 둘 다 느려집니다**. 이 패턴이 보이면 이벤트 루프를 막고 있는 것입니다.

21.4 프론트엔드 — 세 개의 루프

브라우저 쪽에는 서로 다른 주기로 도는 루프가 셋 있습니다.

렌더 루프 (60fps) — 매 프레임 모든 층의 값을 계산해 모델에 적용하고 그립니다. Ch18·Ch19의 내용이 여기 있습니다.

오디오 분석 루프 (렌더와 동기) — 재생 중인 오디오의 음량을 읽어 입 파라미터에 넣습니다. Ch17입니다.

타이머 루프 (비정기) — 깜빡임 예약, 아이들 모션 예약. 각자 난수 주기로 다음 실행을 예약합니다.

이 셋이 독립적으로 돌면서 같은 파라미터 집합에 값을 씁니다. Ch19의 층 구조가 여기서 충돌을 막습니다.

21.5 실측 지연 ★

먼저 **설계 예산**입니다 — Ch07 §3의 겹침 경로(첫 문장에서 TTS로, 첫 청크에서 재생)를 전제로 한 배분입니다.

표 21.1 단계 · 소요

단계	소요
요청 전송	~50ms
LLM 응답 (가벼운 모델, 추론 끄기)	400~900ms
텍스트 정규화	~1ms
TTS	400~800ms
오디오 전송	50~150ms
재생 시작 + 입 반응	~50ms
합계 (첫 소리까지)	약 1~2초

이 표만 보면 Ch07의 목표를 만족하고 여유도 있습니다. 그런데 이 장의 서버(§3)는 이 표대로 동작하지 않습니다. 캠페인인 `measure_ttfa.py` 로 서버의 함수(`gemini()` · `tts()`)를 그대로 불러 재 봤습니다 — 같은 모델(`gemini-2.5-flash-lite`), 같은 TTS(`edge-tts`), 8발화.

표 21.2 서버 경로 실측 — 비스트리밍 LLM → 문장 전체를 TTS 파일로 → 재생 (`_work/ttfa.json`)

	LLM	TTS	첫 소리까지	2초 안
중앙값	2.18초	1.32초	3.77초	0 / 8
범위	1.80~5.97초	1.02~1.82초	3.18~7.08초	

예산 상단(2초)의 두 배 가까운 3.8초입니다. 원인 셋이 겹쳤습니다.

첫째, LLM을 끝까지 기다립니다. 스트리밍을 안 쓰니 첫 토큰이 아니라 *마지막* 토큰까지의 시간입니다. 게다가 §3의 5초 타임아웃과 모델 사다리가 여기서 역효과를 냅니다 — 무료 등급의 분당 한도에 걸리면 첫 모델에서 5초를 다 쓰고 다음 모델로 넘어갑니다. 그래서 5.97초짜리 두 건은 **빈 답**으로 끝나 폴백 문장이 나왔습니다. 빠르게 포기하라는 규칙(Ch08 §7)이 *한 번에 5초씩* 포기하는 규칙이 된 것입니다.

둘째, TTS를 파일로 받습니다. 서브프로세스로 edge-tts를 띄우고(인터프리터 기동만 0.6초) 문장 전체의 mp3가 다 써질 때까지 기다립니다.

셋째, 첫 문장을 떼지 않습니다. 답이 두 문장이면 둘 다 기다립니다.

§3의 서버는 *읽기 쉬운* 서버이지 *빠른* 서버가 아닙니다. 위 예산표는 컴퍼니언 `stream.py`의 접속 경로 — Gemini 스트리밍에서 **첫 문장이 끝나는 순간** 멈추고(`first_sentence()`), edge-tts 스트림의 **첫 오디오 청크**에서 재생을 시작하는 — 를 전제로 합니다. 그 경로의 실측은 아래에 있습니다.

표: 접속 경로 실측 — Gemini SSE 첫 문장 + edge-tts 첫 청크 (`_work/ttfa_stream.json`)

이 표의 TTS는 측정용입니다. 출하물에는 공식 API나 셀프호스트로 바꿉니다 — Ch03 §3의 '상용으로 내보낼 때'.

표 21.3 접속 경로 실측 — 첫 문장만 TTS로 보내 재생 (표본 넷)

	첫 토큰	첫 문장 완성	TTS 첫 청크	첫 소리까지	2초 안
flash-lite · 표본 1	1.01초	1.01초	0.24초	1.26초	○
flash-lite · 표본 2	0.98초	1.0초	0.69초	1.7초	○
flash-latest · 2발화 중앙값	1.42초	1.81초	0.53초	2.35초 (2.34~2.36)	0 / 2

같은 TTS, 같은 사다리의 모델들입니다. 바뀐 것은 **기다리는 지점** 뿐입니다 — 마지막 토큰 대신 첫 문장, 파일 완성 대신 첫 청크. 그런데 표가 둘로 갈립니다.

flash-lite는 첫 문장이 1.0초에 끝나 **1.3~1.7초로 예산 안**입니다. 사다리의 다음 모델(flash-latest)은 첫 문장에만 1.8초가 걸려 **2.3초로 예산 밖**입니다. 접속은 TTS 쪽 대기를 없애 주지만 *모델이 첫 문장을 내는 속도*는 못 줄입니다 — §3의 서버가 429를 만나 다음 모델로 넘어가는 순간 예산을 잃는 이유가 이 줄입니다.

표본은 넷(무료 등급의 호출 한도 안에서 켜 값)이므로 분포가 아니라 *자릿수와 갈림*으로 읽으세요. 이 장의 서버를 그대로 쓸 거라면 예산표가 아니라 3.77초가 여러분의 숫자입니다. 예산표의 숫자를 얻으려면 `stream.py`의 두 함수를 서버에 넣고 **첫 문장이 1초 안에 나오는 모델**을 사다리 맨 위에 두어야 합니다.

변동이 큰 구간은 LLM과 TTS 둘입니다. 둘 다 외부 API라면 네트워크 상태에 따라 튕니다. p95를 관리하려면 이 둘의 최악값을 따로 로깅하세요.

여섯 턴을 연달아 돌린 기록 ★

지연을 한 번 재는 것과 *대화가 이어지는가*는 다른 질문입니다. 저자의 실사 실시간 아바타 바텐더(Ch08 — Track B가 아닌 실사 트랙이지만, 턴이 이어지는가는 트랙과 무관합니다)가 여섯 턴을 연달아 응답한 산출물이 남아 있습니다 — 512×512 · 25fps, 턴마다 파일 하나입니다.

표 21.4 실시간 6턴 — 턴별 음성 길이와 실제 렌더된 프레임 수 (ch21_realtime/_work/realtime_turns.json)

턴	음성	있어야 할 프레임	실제 프레임	차이
1	6.22초	156	65	-91
2	6.22초	156	154	-2
3	3.07초	77	75	-2
4	3.07초	77	75	-2
5	4.80초	120	119	-1
6	3.60초	90	89	-1

둘째 턴부터는 한두 프레임 차이입니다 — 반올림 수준이고 눈에 보이지 않습니다. 그런데 첫 턴만 91프레임, 3.6초여치가 없습니다. 음성은 6.22초를 다 재생하는데 영상은 2.6초에서 멈춘 채 마지막 프레임이 남습니다. 사용자에게는 "말은 계속 나오는데 얼굴이 굳었다"로 보입니다.

원인은 바로 다음 § 6의 것과 같습니다. 첫 호출에서만 일어나는 준비(연산 커널·메모리 할당)가 렌더 루프와 같은 시간을 다투고, 그 사이 프레임이 버려집니다. 그래서 § 6의 위밍업은 *지연을 줄* 이려고 하는 것이 아니라 **첫 턴의 얼굴을 살리려고** 하는 것입니다.

저자의 산출물에도 4초짜리 위밍업 클립과 7.9초짜리 아이들 루프가 함께 남아 있습니다 — 위밍업으로 커널을 데우고, 그동안 아이들 루프를 돌려 화면을 채웁니다(Ch08 § 2).

턴마다 프레임 수를 세세요. 한 줄이면 됩니다 — 음성 초 × fps 와 실제 프레임 수를 비교하고 차이가 다섯을 넘으면 로그에 남깁니다. 눈으로는 "첫 턴이 좀 어색하네" 로만 느껴지는 것이 숫자로는 91프레임입니다.

재 봤습니다 — 모델이 직접 소리를 내면 첫 소리까지 0.67초 ★

위 표들은 이 책 경로 안의 숫자였고, 이것은 바깥과 견준 숫자입니다. 2026년 9월, 같은 질문 셋(코치 페르소나)을 이 책의 겹침 경로와 **네이티브 오디오 모델**(텍스트를 넣으면 모델이 바로 소리를 내는 것) 두 종에 넣고 같은 자료 썼습니다 — 질문 전송 완료부터 첫 오디오 청크까지.

표 21.5 첫 소리까지 — 이 책의 경로 vs 네이티브 오디오 (ch21_realtime/_work/live_ttfa.json)

경로	첫 소리 (중앙값)	비고
이 책 — 서버 경로 (§3 구조·위 실측)	3.77초	LLM을 끝까지 기다림
이 책 — 겹침 경로 (위 표)	1.26초	첫 문장 + TTS 첫 청크
네이티브 오디오 · 2025년형	3.29초	말하기 전에 생각을 글로 먼저 씀 — 그 글이 첫 소리를 3초 늦춘다
네이티브 오디오 · 2026년형	0.67초	세션 준비 0.43초는 별도(한 번)

겹침 경로보다 두 배 빠릅니다. 그리고 세 가지가 함께 옵니다.

- ① **얼굴은 남습니다** — 소리가 어디서 오는 입을 움직이는 것은 Ch17의 음량 구동이고, 그 코드는 한 줄도 안 바뀝니다.
- ② **목소리 선택을 잃습니다** — 클로닝(Ch03 §3)도, 사투리(Ch03A)도 그 모델 안에는 없습니다. 목소리가 곧 제품인 서비스는 여기서 멈춥니다.
- ③ **초당 과금과 종속** — 세션이 열려 있는 동안 계속 과금되고(Ch28 §6), 그 모델이 사라지면 이 장의 구조로 돌아와야 합니다.

그래서 이 책은 겹침 경로를 기본으로 두고, 네이티브 오디오를 **어댑터 하나로 같이 끼우는 선택지**로 둡니다.

2025년형이 2026년형보다 느린 이유가 교훈입니다. 로그를 보면 2025년형은 소리를 내기 전에 ****Prioritizing Knee Safety**** 같은 생각을 글로 먼저 씁니다. 그 글이 첫 소리를 3초 늦춥니다. 모델이 좋아진 것이 아니라 **말하기 전에 생각을 밖으로 내지 않게 된 것**입니다 — Ch07 §3이 청킹에서 한 얘기가 모델 안에서 그대로 반복됩니다.

21.6 첫 호출이 유독 느린 이유

서버를 띄우고 첫 요청을 보내면 **이상하게 오래 걸립니다.** 두 번째부터는 정상입니다.

모델을 상시 로드했는데도 그렇습니다(Ch07 §7). 로딩과 별개로 **첫 실행에서만 일어나는 준비**가 있기 때문입니다 — 연산 커널 컴파일, 메모리 할당, 캐시 워밍.

저자의 음성 서버는 이 비용이 **1.5초 이상**이었습니다. 첫 사용자가 그것을 전부 뒤집어씁니다.

해결은 간단합니다. 서버가 뜰 때 짧은 터미 요청을 한 번 돌리세요.

로드 완료 → 짧은 문장으로 1회 합성(버림) → 준비 완료 표시

준비 비용을 로딩 단계로 옮기는 것입니다. 서버가 뜨는 데 1.5초 더 걸리고, 첫 사용자가 1.5초를 아깁니다. 어차피 서버 시작 시간은 아무도 안 봅니다(Ch07 §7).

준비가 끝나기 전에는 요청을 받지 마세요. 준비 완료 신호를 낸 뒤에 트래픽을 열어야 의미가 있습니다.

21.7 여러 캐릭터 지원

VRM의 표준화 덕분에 캐릭터 교체가 쉽습니다. 모델 파일 목록을 두고 사용자가 고르게 하면 됩니다.

캐릭터마다 목소리를 다르게 매핑하세요. 남성 캐릭터에서 여성 목소리가 나오면 몰입이 깨집니다. 캐릭터 정의에 VRM 경로와 TTS 음성 이름을 함께 둡니다.

페르소나도 함께 바꿀 수 있습니다. 캐릭터마다 시스템 프롬프트를 다르게 두면 성격까지 달라집니다. Track C의 내용이 여기서 미리 한 발을 걸칩니다.

전환은 부드럽게. 모델 로딩에 시간이 걸리므로, 새 모델이 준비될 때까지 이전 모델을 유지하다가 교체합니다. Ch08의 더블 버퍼링과 같은 발상입니다.

21.8 확인해야 할 것들

완성 후 점검 목록입니다.

첫 클릭에서 오디오 컨텍스트가 깨어나는가. 안 그러면 첫 메시지만 입이 안 움직입니다.

말이 끝나면 입이 확실히 닫히는가. 반쯤 벌어진 채 굳지 않아야 합니다.

LLM이 실패해도 소리가 나는가. 폴백 답변이 재생되어야 합니다.

연속으로 빠르게 말을 걸면 어떻게 되는가. 이전 오디오를 멈추고 새 것으로 넘어가야 합니다. 두 개가 겹쳐 재생되면 안 됩니다.

30초간 아무것도 안 하면 살아 있어 보이는가. 깜빡임·호흡·시선·아이들 모션이 돌아야 합니다.

태그가 화면이나 소리에 노출되지 않는가. 대괄호가 읽히면 즉시 몰입이 깨집니다.

21.9 여기서 멈춰도 되는 이유

Track B만으로 완성된 제품이 됩니다.

키오스크, 학습 도우미, 상담 프론트, 전시 안내, 버추얼 스트리머. 이 목록의 대부분은 여기서 더 필요한 것이 없습니다. 실사 얼굴이 필요 없고, 기억이 필요 없고, 도메인 지식이 필요 없다면 완성입니다.

추가할 것이 있다면 그것은 얼굴이 아니라 인격입니다. Track C로 갑니다.

이 구조가 서비스가 된 모습 — 같은 브라우저·서버·GPU 구조를 *남의 GPU* 위에 올려 사람 휴면을 파는 저자의 실서비스(실시간 휴먼 스튜디오)는 Ch28 §6에 화면과 함께 있습니다. 바뀌는 것은 GPU가 누구 것인가뿐입니다.

21.10 이 장에서 기억할 것

구조는 서버는 텍스트·오디오, 브라우저는 얼굴입니다. 이 분담이 GPU 0과 높은 동시 접속을 가능하게 합니다.

프론트엔드에는 렌더·오디오 분석·타이머 세 루프가 독립적으로 돌니다. 총 구조가 충돌을 막습니다.

실측 첫 소리까지 — 단순 서버 경로 3.8초(8발화 중 2초 안 0), 겹침 경로 1.3~1.7초(flash-lite)·2.3초(flash-latest). 같은 부품이고 기다리는 지점만 다릅니다 (§5). 변동이 큰 구간은 LLM과 TTS입니다.

캐릭터 교체 시 VRM·목소리·페르소나를 한 묶음으로 관리하세요.

바깥과 견주면 첫 소리 0.67초 짜리 경로가 이미 있습니다 (§5 끝). 얼굴은 그대로 남고, 목소리 선택과 종속을 내줍니다.

점점 목록 여섯 개를 반드시 확인하세요. 특히 첫 클릭 오디오 컨텍스트와 연속 발화 처리.

Track B가 끝났습니다. 다음 트랙은 얼굴을 떠나 인격으로 갑니다.

실습 코드: `code/ch21_realtime/` — 완성 서버 + 프론트엔드 + 캐릭터 3종 프리셋
예상 소요: 5시간
관련 부록: C(지연 실측)

Part 3 · Track C

인격을 넣는다

시그니처 데모 : 서로 다른 인격의 아바타 여럿이 한 화면에서 대화하고 추론한다

Ch22	페르소나 — 인격의 90%는 프롬프트다	시스템 프롬프트 설계, 말투 고정, 붕괴 방지
Ch23	귀를 붙인다 — STT와 끼어들기	VAD · barge-in · 상태 기계 · 오인식 복구
Ch23A	실시간 통역 — 듣고, 옮기고, 다른 목소리로 말한다	문장 단위 순차 통역 · 용어 잠금 · 겹쳐 말하지 않기 · 수어 브리지
Ch24	기억 3층 — 그리고 원장	단기 컨텍스트 · 요약 · 장기 벡터 기억
Ch25	이 캐릭터만 아는 것 — RAG	한국어 청킹 · 하이브리드 검색 · 근거 표시
Ch26	여러 아바타가 함께	멀티 에이전트 진행자 구조, 비밀 정보 격리
Ch27	트랙 C 완성 — 평가 하네스	골든셋 · 페르소나 회귀 게이트 · 자동 채점

CHAPTER 22

페르소나 — 인격의 90%는 프롬프트다

22.1 파인튜닝 전에, 한 줄 고치고 새로고침

질문 12개를 던졌더니 답 12개가 전부 말투를 어겼습니다. 시스템 프롬프트에 규칙 몇 줄을 엮자 위반이 2개로 떨어졌습니다. 학습은 한 번도 돌리지 않았습니다(§ 3의 실험).

가장 자주 받는 질문이 이것입니다. "이 캐릭터의 말투를 학습시키려면 어떻게 하나요."

대부분 답은 "학습시키지 마세요"입니다. 시스템 프롬프트로 충분합니다.

파인튜닝은 데이터를 모으고, 학습을 돌리고, 평가하고, 배포하는 일입니다. 마음에 안 들면 처음부터 다시 합니다. 시스템 프롬프트는 한 줄 고치고 새로고침하면 끝입니다.

프롬프트로 되는 것을 먼저 만드세요. 그것으로 안 되는 지점이 또렷해졌을 때 파인튜닝을 꺼내세요. 실무에서 그 시점은 생각보다 늦게 옵니다.

22.2 좋은 시스템 프롬프트의 뼈대 — 딱 다섯 부분

정체성 · 말투 · 분량 · 형식 · 금지. 이 다섯이면 됩니다.

정체성 — 누구인가. 한두 문장. "당신은 친근한 한국어 대화 상대입니다"처럼 단순해도 됩니다.

말투 규칙 — 어떻게 말하는가. **여기가 가장 중요합니다.** 존댓말인가 반말인가, 문장은 몇 개인가, 자주 쓰는 표현과 쓰지 않는 표현은 무엇인가.

분량 제약 — Ch07에서 만든 것입니다. "한두 문장으로 짧게"는 페르소나이면서 동시에 지연 최적화입니다.

출력 형식 — Ch20의 태그 규칙, 이모지 금지.

금지 사항 — 이 캐릭터가 하지 않는 것. 역할 이탈 방지와 안전 경계입니다.

여섯 번째를 붙이고 싶어진다면 참으세요. 프롬프트가 길어지면 준수율이 오히려 떨어집니다. **짧고 명확한 프롬프트가 길고 상세한 프롬프트를 대개 이깁니다.**

규칙 층과 상태 층을 갈라 놓으세요 ★

저자의 추리극에서 마을 사람들은 규칙과 상태를 아예 다른 인자로 받습니다. 규칙은 `system` 으로, 상태는 사용자 메시지로.

```
system : "너는 심문에 응하는 인물이다. 한두 문장. 메타발언·이모지·따옴표 금지."
user   : [상황] + [너의 비밀] + [너의 입장] + [공개된 단서] + [최근 대화] + "네 차례다"
```

위의 다섯 자리를 한 덩어리로 쓰면 곧 관리가 안 됩니다. **바뀌는 것과 안 바뀌는 것을 가르세요.**

규칙 층(시스템 프롬프트) — 이 캐릭터가 **어떻게** 말하는가. 말투·분량·금지 사항·출력 형식. **대화** **내내** 고정입니다.

상태 층(매 호출 조립) — 이 캐릭터가 **지금 무엇을** 아는가. 상황, 방금 오간 말, 현재 목표. **매번** 달라집니다.

이 분리가 셋을 줍니다. 규칙을 고칠 때 상태 조립 코드를 안 건드립니다. 캐릭터가 여럿이어도 규칙 층은 그대로 공유합니다(§ 7). 그리고 **무엇이** **새는지** **쫓을 때** **상태 층만 보면 됩니다**(Ch26 § 3).

22.3 말투를 고정하는 기법 — 예시 · 금지 · 호칭 · 어미

"친근하게 말하세요." 이 한 줄만 주면 모델은 이모지를 답니다. 아래 실험에서 실제로 그랬습니다. 추상적인 지시는 잘 안 듣습니다.

예시를 주세요. 짧은 대화 예시 두세 개가 설명 열 줄보다 강합니다. 원하는 말투의 실제 문장을 보여 주는 것이 가장 확실합니다.

금지 목록을 주세요. "~인 것 같아요"를 쓰지 않는다, 사과로 문장을 시작하지 않는다, 상대를 "고객님"이라고 부르지 않는다. 하지 말 것이 하라는 것보다 잘 지켜집니다.

호칭을 고정하세요. 상대를 뭐라고 부르는지가 말투의 인상을 크게 좌우합니다.

어미를 지정하세요. 한국어에서는 어미가 인격입니다. "~해요"인지 "~합니다"인지 "~해"인지를 못 박으세요.

12 대 2 — 규칙 층 하나가 만든 차이 ★

숫자로 확인했습니다. 컴패니언 `ch22_persona/experiment.py` 는 같은 모델(gemini-2.5-flash-lite, 온도 0.9)에 같은 질문 12개를 던지되 **시스템 프롬프트만 세 단계로** 바꿉니다. 채점은 이 장의 `validate()` 가 합니다 — 이모지·메타발언·금지어·어미·문장 수를 코드로 봅니다(태그 규칙은 제외).

표 22.1 시스템 프롬프트 층별 위반 응답 수 (12문항, _work/experiment.json)

프롬프트	위반 응답	무엇이 걸렸나
A. 정체성 한 줄 — "너는 홈트레이닝 코치다. 친근하고 활기차다."	12 / 12	이모지 11 · 문장 수 초과 11(4~11문장) · 어미 6
B. A + 어미·금지어·메타발언·분량 규칙	2 / 12	어미 2
C. B + 예시 두 줄	2 / 12	어미 1 · 빈 응답 1

A의 첫 답이 이렇습니다 — "안녕하세요! 반갑습니다! 🍌 집에서 즐겁게 운동할 준비 되셨나요? 저는 여러분의 홈트레이닝 여정을 함께할 코치입니다! 😊 ..." 여덟 문장, 이모지 둘, 합쇼체와 요체가 섞였습니다. 같은 질문에 B는 이렇게 답합니다 — "안녕하세요. 오늘도 저와 함께 신나고 건강하게 몸을 움직여 봐요."

세 가지를 읽으세요. 첫째, 정체성 한 줄은 말투를 전혀 고정하지 못합니다. 12 전부 위반이고, 모델이 스스로 고른 "친근함"은 이모지와 장광설이었습니다. 둘째, 규칙 총 하나로 12건이 2건이 됩니다. 이 절이 하는 말이 바로 이것입니다. 셋째, 예시는 이 실험에선 더 줄이지 못했습니다. 남은 2건은 어미가 섞이는 것이고, 규칙으로는 못 없앴니다. Ch27의 검증으로 잡아야 하는 몫입니다. 예시의 값은 말투의 결을 정하는 데 있지 위반 수를 줄이는 데 있지 않았습니다.

이 실험은 채점기의 결함도 하나 잡았습니다. 처음 돌렸을 때 A의 어미 위반이 11이었습니다. 이모지가 문장 끝에 붙는 바람에 어미 분류가 전부 "기타"로 떨어진 것이었습니다 — 이모지 한 건이 어미 위반 한 건을 덩으로 만들고 있었습니다. 이모지를 떼고 분류하도록 고치니 6이 됐습니다. 채점기를 채점하라 는 Ch27 § 4의 말이 여기서 먼저 필요했습니다.

어미는 매칭이 아니라 분류로 판정하세요 ★

자동 검증(Ch27)을 붙이면 반드시 밟는 함정입니다.

요체를 검사한다고 "문장이 요:조:어:요로 끝나는가"를 보면 이 문장이 통과합니다.

"함께 진행하시죠."

조로 끝나니 규칙에는 맞습니다. 그런데 이것은 요체가 아니라 합쇼체 권유형입니다. 캐릭터가 말투를 바꿨는데 검증기가 못 잡습니다.

그렇죠 는 요체이고 하시죠 는 합쇼체입니다. 끝 글자가 같은데 말투가 다릅니다.

해법은 순서를 두는 것입니다. 더 구체적인 합쇼체를 먼저 판정하고, 남은 것을 요체로 봅니다.

- ① 합쇼체 : 습니다 · 뵙니다 · 십시오 · 시조 · 하조
- ② 요체 : 예요 · 예요 · 어요 · 해요 · 세요 · 네요 · 조 · 요
- ③ 반말 : 아 · 어 · 야 · 지 · 해 · 다 · 자

"이 말투인가"를 묻지 말고 "어느 말투인가"를 물어주세요. 그래야 기대와 실재를 나란히 놓고 비교합니다. 오류 메시지도 "요체 기대 · 합쇼체 나옴"처럼 쓸모 있어집니다.

이것은 한국어라서 생기는 문제입니다. 영어 페르소나 검증에는 이런 층이 없습니다. **한국어 AI 휴먼을 만든다면 어미 관정기 하나는 직접 짜야 합니다.**

메타발언을 금지하세요. 이게 빠지면 캐릭터가 "제가 맡은 역할상..."이나 "주어진 정보에 따르면..." 같은 말을 합니다. 한 번 나오면 물입이 통째로 깨집니다. 저자의 프롬프트에는 예외 없이 **"메타발언·이모지·따옴표 금지"** 한 줄 가 붙어 있습니다.

비밀은 주되, 말하지 말라고 하세요 ★

비밀을 전 캐릭터가 첫 마디에 자백합니다. 페르소나 설계에서 가장 섬세한 대목입니다.

캐릭터에게 비밀을 주면(Ch26 § 3) 그 캐릭터는 비밀을 **압니다**. 그런데 **말하면 안 됩니다**. 정보를 주는 것과 발화를 허락하는 것은 다른 문제입니다. 프롬프트에서 이 둘을 구분하지 않으면 마을 사람들이 자기 정체를 스스로 붙어 버립니다.

[너의 비밀] (여기에 비밀을 준다 - 알아야 연기가 되니까)

+

"비밀이나 범인 여부는 절대 직접 밝히지 마라" ← 발화 금지를 따로 명시

주되, 말하지 말라고 하세요. 이 두 줄이 같이 있어야 합니다.

같은 원리가 훨씬 넓게 적용됩니다. 상담 아바타가 이전 상담 기록을 알지만 먼저 꺼내면 안 될 때, 게임 NPC가 지도를 알지만 알려주면 안 될 때. **컨텍스트에 넣는 순간 발화 가능성이 생기고, 그것을 막는 것은 프롬프트의 몫입니다.**

그리고 이것은 완벽하지 않습니다. 확실히 막아야 하는 정보라면 **애초에 컨텍스트에 넣지 마세요**. 프롬프트는 방어선이자 자물쇠가 아닙니다.

22.4 페르소나가 무너지는 네 자리

"저는 AI 언어모델로서..." 이 한 문장이 나오는 순간 캐릭터는 죽습니다. 그 순간은 정해진 네 자리에서 옵니다.

긴 대화에서 서서히. 대화가 길어지면 초반의 지시가 희석됩니다. 시스템 프롬프트를 주기적으로 다시 넣으세요. 요약할 때 페르소나 규칙을 함께 남기는 방법도 있습니다.

어려운 질문에서 갑자기. 모델이 곤란한 질문을 받으면 기본 어시스턴트 모드로 돌아갑니다. 이런 상황의 대응을 프롬프트에 미리 써 두세요 — 모르는 것을 어떻게 말할지를 캐릭터의 말투로 정해둡니다.

감정적인 주제에서. 사용자가 슬픈 이야기를 하면 모델이 갑자기 정중해집니다. 반말을 쓰던 캐릭터의 입에서 존댓말이 나옵니다.

출력 형식과 충돌할 때. 태그를 붙이라는 지시와 말투 지시가 부딪히면 하나를 놓칩니다. 형식 지시는 프롬프트 끝에 두는 편이 준수율이 높습니다.

이 넷이 Ch27의 평가 하네스에서 검사할 항목이 됩니다.

22.5 온도 0.7~0.9, 그리고 반복 방지

이 장의 실험은 온도 0.9로 돌렸습니다. 대화에서는 그 근처가 좋습니다.

너무 낮추지 마세요. 0에 가까우면 같은 질문에 항상 같은 답이 나옵니다. 정확성이 중요한 시스템에서는 좋지만, 대화 상대로서는 지루합니다. 0.7~0.9 정도가 적당합니다.

너무 높이지도 마세요. 1.0을 넘기면 말투가 흔들리기 시작합니다.

최근 응답을 기억해 반복을 피하세요. 사용자가 비슷한 질문을 반복할 때 같은 답이 그대로 나오면 기계 티가 납니다. 최근 몇 개의 응답을 컨텍스트에 넣고 "다르게 말하세요"를 붙이는 것만으로 좋아집니다.

22.6 폴백도 캐릭터여야 합니다

여섯 명이 동시에 폴백에 걸렸는데 여섯이 똑같은 문장을 말합니다. 그 순간 사용자는 시스템이 고장 난 것을 압니다.

Ch07 §6에서 LLM이 실패하면 오프라인 답변으로 간다고 했습니다. **그 답변이 페르소나를 깨면 폴백의 값은 절반입니다.**

저자의 추리극은 폴백을 두 겹으로 만듭니다.

자기소개 실패 시 — 캐릭터의 이름과 역할을 끼워 문장을 조립합니다.

```
f"{역할}, {이름}입니다. 그날 밤 저는 제 자리에 있었을 뿐, 이 일과는 무관합니다."
```

LLM이 죽어도 **그 캐릭터의 이름과 직업이 나옵니다.** 밋밋하지만 딱 사람은 아닙니다.

대사 실패 시 — 일반 대사 세 개를 준비하고, 고를 때 캐릭터 이름을 섞습니다.

```
generic[(idx + 이름_숫자값) % len(generic)]
```

이 한 줄이 영리합니다. 이름을 숫자로 바꾸는 함수는 실행마다 같은 값을 내는 것으로 쓰세요(파이썬 `hash()` 는 실행마다 달라집니다). 그러면 같은 캐릭터는 언제나 같은 순서로 폴백을 쓰고, 캐릭터마다 시작점이 다릅니다. 여섯이 동시에 걸려도 **여섯이 서로 다른 문장** 을 말합니다.

원칙 — 폴백은 **덜 똑똑해도 되지만** **만 사람이면 안 됩니다.** 이름·역할·말투 중 하나라도 살려 두세요. 문자열 하나 조립하는 비용입니다.

그리고 API 키 없이 도는 모드를 만드세요

저자의 엔진에는 **더미 모드** 가 있습니다. 캐릭터별 대사가 미리 적혀 있어서 API 키 없이도 전체 흐름이 돕니다.

이게 개발 속도를 크게 바꿉니다. UI를 고칠 때, 진행 순서를 바꿀 때, 화면 배치를 볼 때 — LLM 을 부를 이유가 없습니다. 무료 한도도 안 쓰고, 응답을 기다리지도 않고, **결과가 매번 같아서 비교** 가 됩니다.

Ch27의 평가와도 이어집니다. 더미 모드로 돌리면 *LLM*을 **뺀 나머지** 가 정상인지 갈라서 확인합니다.

22.7 캐릭터는 코드가 아니라 데이터입니다

캐릭터가 여섯이면 프롬프트도 여섯입니다. 코드에 박아 두면 한 명 늘릴 때마다 코드를 고칩니다.

각 캐릭터를 **이름 · 시스템 프롬프트 · TTS 음성 · VRM 모델 · 감정 매핑** 의 묶음으로 정의하세요. 설정 파일에 두면 캐릭터 추가가 파일 편집만으로 끝납니다.

공통 부분을 분리 하세요. 출력 형식 규칙, 안전 규칙, 분량 제약은 모든 캐릭터가 공유합니다. 캐릭터별 프롬프트는 정체성과 말투만 담습니다. 공통 규칙이 바뀔 때 캐릭터를 하나씩 고치지 않아도 됩니다.

22.8 언제 파인튜닝으로 넘어가나

사투리는 파인튜닝을 해도 절반만 해결됩니다. 프롬프트의 한계가 또렷해지는 자리가 셋 있습니다.

말투가 아주 특수할 때. 사투리, 고어체, 특정 인물의 독특한 화법. 예시 몇 개로는 안정적으로 재현되지 않습니다.

사투리에는 함정이 하나 더 있습니다. LLM을 **파인튜닝해도 어미와 어휘만 바꿉니다.** 그 텍스트를 표준어 TTS에 넣으면 **표준어 억양으로 사투리 단어를** 읽습니다.

억양은 텍스트에 없습니다. 음성 모델 안에 있습니다. 즉 이것은 *머리* 층이 아니라 *목소리* 층의 문제입니다(Ch01). 층을 잘못 짚으면 프롬프트를 아무리 고쳐도 해결되지 않습니다.

실제 해법과 그 원가는 **Ch03A** 에 사례로 실었습니다.

프롬프트가 너무 길어질 때. 지시가 스무 줄을 넘어가면 준수율이 떨어지고 토큰 비용도 올라갑니다.

일관성이 사업적으로 중요할 때. 브랜드 캐릭터처럼 말투가 흔들리면 안 되는 경우입니다.

이때 선택지는 둘입니다. **선호 데이터로 정렬** 하거나, **소량 데이터로 어댑터 학습** 을 하는 것입니다. 둘 다 전체 모델을 다시 학습하지 않습니다. 다만 이 책의 범위를 넘어가고, 대부분의 프로젝트에서는 필요하지 않습니다.

22.9 이 장에서 기억할 것

파인튜닝하기 전에 시스템 프롬프트로 해 보세요. 대부분 충분합니다.

프롬프트 빠대는 다섯 — **정체성 · 말투 · 분량 · 형식 · 금지.** 짧고 명확한 것이 길고 상세한 것을 이깁니다.

말투는 **예시 · 금지 목록 · 호칭 · 어미** 로 고정합니다. 한국어에서는 어미가 인격입니다.

붕괴는 네 곳에서 일어납니다 — **긴 대화 · 어려운 질문 · 감정적 주제 · 형식 충돌.** 이 넷이 평가 항목이 됩니다.

온도 **0.7~0.9**, 최근 응답 반복 방지.

캐릭터는 **데이터로 관리** 하고 **공통 규칙을 분리** 하세요.

다음 장에서 이 인격에게 귀를 붙입니다. 그리고 끼어들 수 있게 만듭니다.

실습 코드: `code/ch22_persona/` — 캐릭터 정의 스키마 + 프롬프트 템플릿 + 말투 붕괴 재현 테스트
예상 소요: 2시간
관련 부록: F § F(인격이 무너지는 실패) · 온라인 K(모델 지형도)

CHAPTER 23

귀를 붙인다 — STT와 끼어들기

23.1 타이핑에서 음성으로

마이크를 붙인 첫날, 코치(Ch18~Ch21)가 자기 자신과 대화를 시작했습니다.

스피커에서 나온 코치의 목소리를 마이크가 다시 잡았습니다. 시스템은 그것을 사용자 발화로 인식했습니다. 코치는 자기 말에 대답했고, 그 대답을 또 들었습니다. 끄기 전까지 멈추지 않았습다.

헤드셋을 끼면 사라지는 문제입니다. 그리고 전시장에 스피커로 놓는 순간 다시 나타납니다.

지금까지 입력은 타이핑이었습니다. 음성으로 바꾸는 순간 두 가지가 새로 생깁니다.

언제 말이 끝났는지 판단해야 합니다. 타이핑에는 엔터라는 명확한 신호가 있습니다. 음성에는 없습니다.

말하는 도중에 끼어들 수 있어야 합니다. 아바타가 말하는 중에 사용자가 말을 시작하면 아바타는 멈춰야 합니다. 사람이 그렇게 하기 때문입니다.

두 번째가 특히 중요합니다. **끼어들 수 없는 대화는 대화가 아니라 안내방송입니다.**

무전기에서 전화로

통신에는 이 전환을 부르는 이름이 이미 있습니다.

반이중(half-duplex)은 한 번에 한 방향만 통하는 것입니다. 무전기가 그렇습니다. 내가 말할 때는 못 듣고, 들을 때는 못 말합니다.

전이중(full-duplex)은 양방향에 동시에 통하는 것입니다. 전화가 그렇습니다.

지금까지 만든 아바타는 **반이중**이었습니다. 사용자가 입력하고, 아바타가 답하고, 다시 사용자가 입력합니다. 이 장에서 만드는 것은 **전이중**입니다.

이 한 단어가 §4의 난이도를 미리 설명해 줍니다. **아바타가 말하면서 동시에 들어야 합니다.** 그러면 스피커로 나가는 자기 목소리를 마이크가 다시 잡습니다. §1의 무한루프가 바로 그것이고, 전이중 시스템이면 반드시 만나는 문제입니다.

23.2 언제 말이 끝났는가 — 침묵을 재는 일

"음... 그러니까..." 여기서 아바타가 대답을 시작하면 대화가 깨집니다. 사용자는 아직 말하는 중이었습니다.

정식 명칭은 **엔드포인팅(endpointing)**, 우리말로 발화 종료 검출입니다. 이름이 붙어 있다는 것은 그만큼 어려운 문제라는 뜻입니다.

침묵 기반 이 가장 단순하고 실용적입니다. 소리가 일정 시간 이상 나지 않으면 발화가 끝난 것으로 봅니다.

다만 침묵만으로는 부족한 순간이 있습니다. 위의 "음... 그러니까..."처럼 **생각하는 침묵**과 말이 정말 끝난 침묵이 소리로는 똑같기 때문입니다. 사람은 억양의 하강, 문장의 완결, 시선을 함께 봅니다. 우리가 쓸 수 있는 것은 그중 가장 다루기 쉬운 침묵뿐입니다. 그래서 사람보다 어색합니다.

VAD는 두 갈래입니다

VAD(voice activity detection — 지금 마이크에 들어온 소리가 사람 말인지 아닌지 판정하는 층)를 고르는 문제입니다. 갈래는 둘입니다.

에너지 기반 — 음량이 임계값을 넘으면 말로 봅니다. 단순하고 빠르지만 **잡음에 약합니다**. 에어컨 소리, 키보드 소리, 옆 사람의 기침이 전부 발화가 됩니다.

신경망 기반 — 사람 목소리의 특징을 학습해서, 음량이 비슷해도 잡음과 말소리를 구분합니다. 조금 무겁지만 실전에서는 이쪽이 답입니다.

조용한 사무실이면 에너지 기반으로 충분합니다. **전시장·매장·카페처럼 배경 소음이 있는 곳이면 신경망 VAD로 가세요.** §2의 임계값 튜닝으로 버티려다 실패하는 전형적인 경우입니다.

임계 시간이 관건입니다. **짧으면** 사용자가 문장 중간에 잠깐 숨을 쉬었을 때 끊깁니다. **길면** 말이 끝났는데 아바타가 반응하지 않아 답답합니다.

경험적으로 **700밀리초에서 1초** 사이가 무난합니다. 이보다 짧으면 오작동이 늘고, 길면 지연 예산을 잡아먹습니다. 이 시간이 Ch07 표의 "입력 확정" 항목이고, **2초 예산은 이 판정이 끝난 다음부터 셉니다** — 사용자가 느끼는 시간에는 이 0.7~1초가 앞에 붙습니다.

음량 임계값도 조정이 필요합니다. 조용한 방과 시끄러운 전시장의 차이가 큼니다. 시작할 때 주변 소음을 몇 초간 재서 기준을 정하는 방식이 실전에서 잘 통합니다.

최소 발화 길이를 두세요. 기침이나 문 닫는 소리로 STT가 돌면 낭비입니다. 200밀리초 이하의 소리는 무시합니다.

녹음 넷에 실제로 돌려 보니 ★

컴패니언 `vad.py`를 저자의 녹음 넷(TTS 출력 둘 · 마이크 둘)에 20ms 프레임으로 돌렸습니다. 침묵 길이를 바꿔 가며 발화 조각 수와 **말이 끝난 뒤 END 까지의 지연**을 잰습니다(`ch23_stt/work/measure.json`).

표 23.1 침묵 길이별 — 조각 수 · 끝 지연

파일	임계(보정)	300ms	500ms	800ms	1000ms
_script (TTS 6.1초)	0.010	2 · 300ms	2 · 500ms	2 · 800ms	2 · 1000ms
_chat (마이크 4.2초)	0.010	1 · 320ms	1 · 520ms	1 · 820ms	1 · 1020ms
_stt (마이크 4.8초)	0.029	2 · 280ms	2 · 480ms	2 · 780ms	2 · 920ms †
_live (TTS 3.1초)	0.010	1 · 340ms	1 · 540ms	1 · 840ms	1 · 1040ms

† 파일이 끝나 스트림 종료로 닫힘 — 아래.

읽을 것이 셋입니다.

첫째, 끝 지연은 곧 침묵 설정값입니다. 정의상 그렇습니다. 800ms를 고르면 사용자가 말을 마친 뒤 0.8초를 반드시 기다린 다음에야 Ch07의 2초 예산이 시작됩니다. "700ms~1초"는 그 값을 통째로 응답 지연에 없다는 뜻이고, 이 표가 그것을 파일마다 보여 줍니다.

둘째, 조각 수는 300ms 에서도 안 늘었습니다. 이 녹음들은 문장 사이 쉬이 300ms보다 짧아서입니다. 쉬이 긴 화자라면 갈라집니다. 여러분의 사용자 녹음으로 이 표를 다시 만드세요.

셋째가 이 실측의 진짜 수확입니다. 처음 돌렸을 때 네 파일 모두 **발화 0개**였습니다. 보정 (§ 2 "시작할 때 주변 소음을 재라")을 첫 1초로 했는데, 이 녹음들은 첫 1초부터 말이었습니다. 말소리를 주변 소음으로 재니 임계가 0.23~0.54까지 올라갔습니다. 말소리보다 높은 기준이니 그 뒤로는 아무것도 말이 아니었습니다. 보정 창을 **가장 조용한 30%**로 바꾸니 0.010~0.029가 됐습니다.

그리고 침묵 1000ms에서 **_stt**의 마지막 발화는 파일이 끝나도록 닫히지 않았습니다. 마이크가 꺼지거나 연결이 끊겨도 같은 일이 납니다. 그래서 스트림 끝에서 침묵을 기다리지 않고 닫는 **finish()**를 두었습니다. VAD의 두 가장자리 — **시작의 보정과 끝의 종료** — 는 둘 다 **"사용자가 조용할 것"**을 가정하고 있었고, 둘 다 실제 파일에서 깨졌습니다.

23.3 STT를 어디서 돌릴 것인가

STT — 소리를 글자로 바꾸는 인식기 — 를 돌릴 자리는 셋입니다. 클라우드 API 왕복은 200~500밀리초이고, 이 숫자가 Ch07의 예산 안에 들어오는지가 이 절의 전부입니다.

브라우저 내장 인식은 설치가 필요 없고 빠릅니다. 다만 브라우저마다 지원이 다르고, 한국어 정확도가 상황을 탐니다.

클라우드 API는 정확도가 좋고 관리가 필요 없습니다. 오디오를 서버로 보내야 하므로 지연이 높고, 프라이버시 문제가 따라옵니다.

로컬 모델은 외부 전송이 없고 무제한입니다. 대신 모델을 로드해야 하고, 작은 모델은 한국어 정확도가 떨어집니다.

대화 아바타에서는 **클라우드 API로 시작** 하세요. 위의 200~500밀리초가 예산 안에 들어가고, 정확도가 안정적입니다. 프라이버시가 중요한 환경이면 그때 로컬로 옮깁니다.

오디오는 짧게 잘라 보내세요. 긴 녹음을 통째로 보내면 전송과 처리가 둘 다 느려집니다. 발화 종료 판정 직후에 그 구간만 보냅니다.

23.4 끼어들기 — barge-in

아바타가 세 번째 문장을 말하는 중에 사용자가 "아니, 그게 아니라"라고 합니다.

즉시 멈춰야 합니다. 오디오 재생을 중단하고, 입을 닫고, 듣는 자세로 바꿉니다.

구현에서 주의할 점이 다섯입니다.

마이크는 계속 열어 둡니다. 아바타가 말하는 동안 마이크를 끄면 끼어들 수 없습니다.

자기 목소리를 걸러야 합니다. 스피커에서 나오는 아바타의 목소리를 마이크가 다시 잡습니다. 이것을 사용자 발화로 오인하면 아바타가 자기 말에 반응해 무한 루프에 빠집니다. §1의 첫날이 그것입니다.

원리를 알면 대응이 정해집니다. **반향 제거(AEC)는 뺄셈입니다.** 스피커로 내보낸 신호를 참조 신호로 들고 있다가, 마이크로 들어온 소리에서 **그 신호의 지연·변형된 버전을 추정해 뺍니다.** 남은 것이 사람 목소리입니다. 방마다 울림이 다르므로 그 변형은 **적응 필터**가 실시간으로 학습합니다.

여기서 세 가지가 따라 나옵니다.

헤드셋이면 공짜입니다. 스피커 소리가 마이크에 애초에 안 들어오므로 뺄 것이 없습니다. 키오스크·전시장처럼 스피커를 쓸 수밖에 없는 환경에서만 진짜 문제입니다.

참조 신호를 잃으면 못 뺍니다. 오디오 출력이 브라우저·OS·외부 믹서를 거치면서 참조 경로가 끊기면 AEC는 동작하지 않습니다. "*라이브러리를 켜는데 안 되네*"의 대부분이 이것입니다.

직접 만들지 마세요. WebRTC 스택에 **에코 제거·잡음 억제·자동 이득 조절(AGC)**이 이미 들어있고, 이 셋은 십수 년 다듬어진 것들입니다. WebRTC를 쓰는 이유가 *빠르다*만이 아닌 *까닭*입니다. UDP 기반이라 약간의 패킷 손실을 감수하고 지연을 줄이는 데다(음성은 조금 끊겨도 늦는 것보다 낫습니다), 이 전처리들이 달려 옵니다.

무엇을 쓰든 **아바타가 말하는 동안 임계값을 올려 두는 것**은 같이 하세요. AEC가 완벽하지 않은 순간을 위한 이중 안전장치입니다. 컴패니언 코드의 `is_self_echo()`가 그 층입니다.

진행 중인 작업을 취소합니다. 오디오만 멈추면 안 됩니다. 뒤에서 들고 있던 LLM 요청과 TTS 요청도 취소해야 합니다. 그러지 않으면 잠시 후 이전 응답이 뒤늦게 재생됩니다.

멈춤이 자연스러워야 합니다. 소리를 툭 끊지 말고 100밀리초 정도 페이드아웃하세요. 입도 그 시간 동안 닫습니다. Ch08의 전환 처리와 같은 사상입니다.

입이 소리와 함께 멈춰야 합니다. 이것이 아바타 특유의 요구입니다. 끼어들기로 오디오를 멈췄는데 입이 계속 움직이면, 아무 소리도 안 나면서 혼자 떠드는 얼굴이 됩니다. **소리만 멈추는 것보다 더 어색합니다.** 정지 플래그는 TTS 재생뿐 아니라 립싱크 층에도 같이 걸어야 합니다.

취소할 것을 목록으로 두세요

이 실수는 눈으로 보면 즉시 알지만 코드에서는 조용히 빠집니다. 끼어들기 처리를 여기저기 흠여 놓으면, 나중에 층을 하나 더 붙였을 때 그 층만 안 멈춥니다.

취소 대상을 한 곳에 목록으로 두세요.

```
CANCEL_ON_BARGEIN = ("tts_playback", "lipsync", "llm_stream", "pending_render")
```

그러면 검증이 한 줄이 됩니다.

```
assert "lipsync" in CANCEL_ON_BARGEIN
assert set(cancelled) == set(CANCEL_ON_BARGEIN)  # 하나도 빠지지 않았는가
```

층을 새로 추가할 때 이 목록에 넣는 것을 잊으면 테스트가 먼저 알려줍니다. 목록이 없으면 아무도 알려주지 않고, 사용자가 발견합니다.

맞장구는 끼어들기가 아닙니다

흔한 오작동이 하나 있습니다. 사용자가 "응", "네", "아하" 같은 맞장구를 하면 아바타가 말을 멈춥니다.

이런 짧은 추임새는 말차레를 뺏으려는 것이 아닙니다. 언어학에서 백채널(backchannel)이라고 부르는, "듣고 있어요"라는 신호입니다. 사람은 상대가 맞장구를 치면 오히려 편하게 말을 이어갑니다.

정교하게 하려면 이런 표현을 목록으로 두고 끼어들기에서 제외하세요. 간단하게 하려면 아주 짧은 발화(예: 0.5초 미만)를 끼어들기로 보지 않는 것 만으로도 상당히 좋아집니다.

23.5 STT는 틀립니다 — 오인식 다루기

한국어에서는 고유명사와 짧은 단어가 특히 자주 틀립니다. 완벽하게 만들려 하지 말고 네 겹으로 막으세요.

너무 짧은 인식 결과는 버리세요. 한 글자나 두 글자 결과는 대개 잡음입니다.

신뢰도가 제공되면 쓰세요. 낮은 신뢰도의 인식은 되물습니다. "죄송해요, 다시 말씀해 주시겠어요?"는 틀린 답변보다 낫습니다.

도메인 어휘를 힌트로 주세요. API가 지원한다면 자주 나올 단어 목록을 전달합니다. 캐릭터 이름, 제품명, 지명의 인식률이 크게 올라갑니다.

LLM이 복구하게 하세요. 인식 결과가 조금 이상해도 LLM은 문맥으로 의도를 잡아냅니다. STT 결과를 완벽하게 만들려고 애쓰기보다, 시스템 프롬프트에 "음성 인식 결과라 오타가 있을 수 있으니 문맥으로 이해하세요"를 넣는 편이 낫습니다.

23.6 상태 기계로 관리하세요

상태는 넷입니다. 이 넷을 명시적으로 적어 두지 않으면 전환이 곧 엉킵니다.

대기 — 아무 일도 없음. 아이들 모션이 돕니다.

듣는 중 — 사용자가 말하고 있음. 아바타는 듣는 표정.

처리 중 — STT·LLM·TTS 진행. Ch08의 자연 숨기기가 여기서 동작합니다.

말하는 중 — 오디오 재생. 입·표정·동작 활성화.

전환 규칙을 명시하세요. 특히 **말하는 중** → **듣는 중** 이 끼어들기이고, 이 전환에서 취소해야 할 것들이 있습니다.

각 상태에서 아바타가 어떻게 보이는지 도 정하세요. 듣는 중에는 살짝 고개를 기울이고 시선을 고정, 처리 중에는 생각하는 표정. 이 시각적 피드백이 Ch08에서 말한 "스피너 대신"의 실체입니다.

23.7 이 장을 건너뛰게 되는 경우

Ch07 §9의 S2S(음성을 넣으면 곧바로 음성이 나오는 단일 모델)를 쓰면 이 장의 상당 부분을 제공자가 대신 해 줍니다. 발화 종료 판정, 끼어들기, 자기 목소리 차단이 모델 쪽에 들어 있습니다.

그래도 **이 장을 읽고 넘어가세요**. 이유가 셋입니다.

여전히 여러분이 처리해야 하는 것이 있습니다. 아바타의 입과 표정을 멈추는 것, 진행 중인 렌더를 취소하는 것, 상태를 전환하는 것은 제공자가 모릅니다. §4의 다섯 중 뒤의 셋(진행 중 작업 취소 · 부드러운 멈춤 · 입도 함께 멈추기)은 그대로 남습니다.

튜닝이 필요합니다. 침묵 임계값과 소음 기준은 여전히 환경마다 다릅니다. 조용한 사무실에서 맞춘 값이 전시장에서는 오작동합니다.

폴백이 필요합니다. S2S가 실패하거나 한도에 걸렸을 때 캐스케이드로 내려갈 수 있어야 합니다. 그러려면 이 장의 구조를 갖고 있어야 합니다.

§6의 **상태 기계는 어느 쪽이든 필요합니다**. 그것이 이 장에서 가장 오래 남는 부분입니다.

그런데 전이중은 그 상태 기계를 깎니다 ★

Ch07 §10의 3세대 모델을 쓰면 이야기가 달라집니다.

§6의 네 상태 — 대기 · 듣는 중 · 처리 중 · 말하는 중 — 는 **서로 배타적**입니다. 한 번에 하나만 참입니다. 그것이 상태 기계라는 도구의 전제입니다.

전이중은 듣기와 말하기가 동시에 참입니다. 아바타가 말하는 중에도 계속 듣고 있고, 사용자가 말하는 중에도 맞장구를 냅니다. 배타적인 네 칸에 들어가지 않습니다.

상태가 아니라 채널로 다시 보세요.

입력 채널 : 조용함 / 사용자 발화 중
출력 채널 : 조용함 / 맞장구 / 발화 중

두 채널이 독립적으로 켜지고 꺼집니다. 조합이 여섯 가지이고, 그중 둘 다 켜진 상태(겹쳐 말하기)가 정상이라는 점이 이전과 다릅니다.

얼굴이 있으면 새 문제가 생깁니다

여기부터는 음성 서비스에 없고 **아바타에만 있는 문제**입니다.

맞장구칠 때 입을 어떻게 할 것인가. "음—"은 0.3초짜리입니다. 정식 발화처럼 입을 크게 열면 과합니다. Ch17의 음량 구동을 그대로 쓰되 **최대 벌림을 절반으로 눌러** 놓으면 자연스럽습니다.

겹쳐 말할 때 표정을 어떻게 할 것인가. 사용자가 말하는 중에 아바타는 듣는 표정 이어야 합니다. 그런데 동시에 맞장구를 내면서 입은 움직입니다. 듣는 **얼굴 + 움직이는 입**이라는 조합이 필요하고, 이것은 § 6의 네 상태에 없습니다.

어디까지가 한 발화인지 모호합니다. 자막을 언제 끊을지, 대화 로그에 어떻게 남길지가 애매해집니다. 부록 L § 3의 자막 규칙이 겹쳐 말하기를 전제하지 않고 쓰였습니다.

정리 — 전이중은 음성 대화를 크게 개선했지만, **얼굴을 붙이는 순간 풀리지 않은 문제가 남습니다.** 이 책이 아직 답을 갖고 있지 않은 영역이고, 그렇게 적어 둡니다.

23.8 이 장에서 기억할 것

음성 입력이 추가하는 것은 둘 — **발화 종료 판정** 과 **끼어들기**. 끼어들 수 없으면 대화가 아닙니다.

종료 판정은 **침묵 700ms~1초**, 주변 소음으로 임계값 보정, 최소 발화 길이 필터.

STT는 **클라우드 API로 시작**, 프라이버시가 필요하면 로컬로. 오디오는 발화 구간만 잘라 보냅니다.

끼어들기 구현의 핵심 다섯 — **마이크 상시 개방 · 자기 목소리 차단 · 진행 중 작업 취소 · 부드러운 멈춤 · 입도 함께 멈추기**.

오인식은 **짧은 결과 버리기 · 신뢰도 활용 · 도메인 어휘 힌트 · LLM 문맥 복구**로 다룹니다.

상태 기계로 관리하고, 각 상태의 시각적 표현을 정하세요.

다음 장은 기억입니다. 어제 한 얘기를 기억하는 것이 인격의 그 나머지를 채웁니다.

실습 코드: `code/ch23_stt/` — VAD + STT 연동 + 끼어들기 상태 기계

예상 소요: 4시간

관련 부록: L § 4(기다림을 정직하게) · F § E

CHAPTER 23A

실시간 통역 — 듣고, 옮기고, 다른 목소리로 말한다

23A.1 통역은 대화가 아닙니다

"수수료는 12,000원입니다." 창구에서 이 한 문장을 잘못 옮기면 그것은 오역이 아니라 사고입니다.

Ch21의 코치는 **대답** 합니다. 사용자가 묻고 코치가 답하니 턴이 분명합니다. 통역 아바타는 아닙니다. 화자는 아바타에게 말하는 것이 아니라 **다른 사람에게** 말하고, 아바타는 그 말을 다른 언어로 **대신** 합니다. 이 차이가 구조의 절반을 바꿉니다.

세 곳에서 이 요구가 나옵니다. 관광 안내 창구와 민원 창구(Ch01 §3의 도메인 표), 병원 접수, 그리고 청각장애인을 위한 수어 통역 보조입니다. 셋의 공통점이 있습니다 — **틀리면 안 되는 낱말**이 있고(수수료·호수·시간), 화자가 말하는 동안 아바타가 끼어들면 안 되며, 상대가 답을 기다리는 시간이 대화보다 길게 느껴집니다.

이 장은 새 모델을 쓰지 않습니다. Ch23의 귀, Ch07의 문장 분할, Ch22의 규칙, Ch21의 목소리, Ch18의 얼굴을 **다른 순서로 잇는** 장입니다.

23A.2 동시통역이 아니라 순차 통역입니다

사람 통역사도 둘로 나뉩니다. 동시통역은 화자와 겹쳐 말하고, 순차 통역은 화자가 문장이나 단락을 끝낸 뒤 옮깁니다. 아바타는 **순차**로 갑니다. 이유가 셋입니다.

겹쳐 말하면 아무것도 안 들립니다. Ch26 §5가 말한 그대로입니다. 한 스피커에서 두 목소리가 나오는 순간 둘 다 죽습니다.

문장이 끝나야 번역이 됩니다. 한국어는 동사가 끝에 옵니다. "수수료는 12,000원이..."까지 듣고는 긍정인지 부정인지 모릅니다. 문장 단위가 번역 품질의 하한입니다.

턴 기반 부품을 그대로 씁니다. Ch07 §10의 전이중 모델은 통역에 오히려 맞지 않습니다. 모델이 스스로 끼어드는 것이 통역에서는 사고입니다.

구조는 한 줄입니다.

STT(Ch23) → 문장 분할(Ch07) → 번역(LLM · 용어 잠금 · 숫자 보존) → 정규화(Ch03) → TTS(대상)

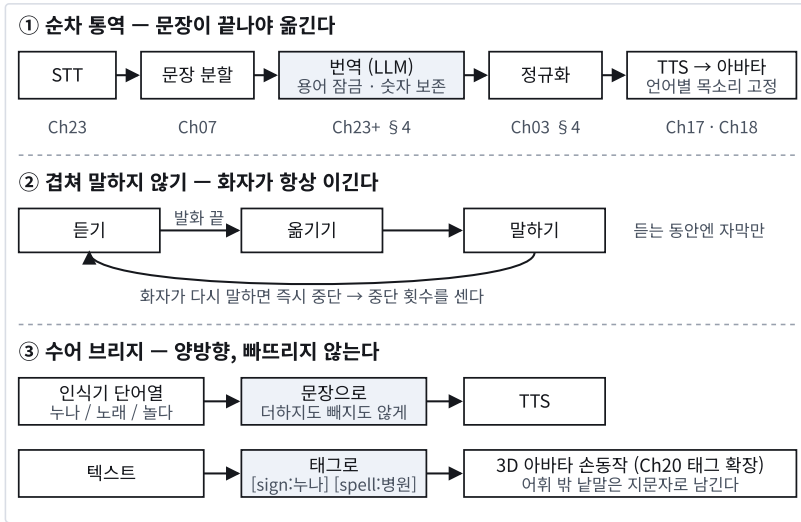


그림 23A.1 위 — 순차 통역 파이프라인과 각 칸의 출처 장. 가운데 — 화자가 항상 이기는 상태 기계. 아래 — 손이 입력이자 출력인 수어 브리지 양방향

컴패니언 `interpreter.py` 가 이 줄에서 `판단` 에 해당하는 부분을 전부 순수 함수로 갖고 있습니다. 네트워크가 필요한 것은 번역 호출 하나뿐이고, 그것도 함수를 넣어 주는 방식이라 어떤 모델 이든 됩니다.

23A.3 지연 예산 — 어디서부터 재는가

Ch07은 *사용자가 말을 끝낸 순간* 부터 켜습니다. 통역도 같습니다. 화자가 문장을 끝낸 순간부터 통역 음성이 시작되기까지가 지연이고, 순차 통역의 최소 지연은 세 항의 합입니다.

표 23A.1 순차 통역의 최소 지연 — 문장 하나

단계	예산	근거
발화 종료 판정	0.8초	Ch23 § 2의 침묵 길이 — 이보다 짧으면 문장이 갈라진다
번역 (한 문장)	1.0초	경량 모델, 비스트리밍 — 문장이 짧아 첫 토큰과 마지막 토큰이 가깝다
TTS 첫 청크	0.5초	Ch21 § 5의 실측 0.24~0.69초
합	2.3초	<code>latency_budget()</code>

2.3초는 대화 아바타의 2초 예산보다 깁니다. 그런데 통역에서는 이것이 **덜 길게 느껴집니다**. 화자가 말하는 동안 상대는 이미 원문 자막을 보고 있고(§ 5), 사람 통역사도 문장 뒤에 한 박자를 두기 때문입니다.

순서를 바꿔 이 박자를 없애려 하지 마세요. 발화 종료 판정을 줄이면 문장이 갈라지고, 갈라진 문장은 틀리게 옮겨집니다.

23A.4 번역기에 거는 네 가지 규칙

LLM에 "번역해" 라고만 하면 통역이 아니라 *의역* 이 나옵니다. 창구에서 의역은 사고입니다. 컴패니언은 프롬프트에 네 가지를 박고, 돌아온 답을 코드로 검사합니다.

문장 단위로만 보냅니다. `segment()` 가 스트리밍 STT 텍스트에서 *완성된 문장* 만 떼어 넘기고 조각은 남깁니다. 마침표 뒤의 공백, 소수점, 약어(Dr.)를 구분하는 규칙은 Ch07 §3의 청커와 같습니다. 일본어·중국어의 종결 부호(。！？)는 뒤에 공백이 없어도 문장 끝입니다.

용어를 잠급니다. 기관명·상품명·직함은 Glossary 에 원어→대상어 쌍으로 넣습니다. 프롬프트에 "정확히 이 용어를 쓰라" 고 넣는 것으로 끝이 아닙니다. 돌아온 번역에 그 용어가 **있는지 코드**가 **확인** 하고, 없으면 빠진 것을 지목해 한 번 더 부릅니다.

숫자를 보존합니다. 원문의 숫자가 번역문에 전부 있어야 합니다. 모델은 "12분"을 "twelve minutes"로 풀어 쓰기를 좋아하는데, 그러면 검사가 잡습니다. 숫자를 글자로 푸는 것은 번역기의 몫이 아니라 **TTS 정규화**의 몫 이고(Ch03 §4), 대상 언어가 한국어면 그 정규화가 자동으로 걸립니다.

목소리는 언어마다 하나로 고정합니다. 한 통역사가 두 언어를 오가는 인상을 지키려면 성별과 톤이 비슷한 목소리를 언어별로 하나씩 정해 둡니다. 문장마다 목소리가 바뀌면 상대는 누가 말하는지 놓칩니다.

23A.5 겹쳐 말하지 않기 — 화자가 항상 이깁니다

Ch23 §6의 상태 기계에서 가운데 상태 하나가 바뀝니다. 대화 아바타는 *듣기* → *생각* → *말하기* 였고, 통역 아바타는 *듣기* → *옮기기* → *말하기* 입니다. 네 상태는 그대로 두고, 규칙 셋을 새로 겁니다.

표 23A.2 통역 세션의 세 규칙 (Session)

규칙	동작	이유
듣는 동안 소리를 내지 않는다	완성된 문장은 큐에 쌓고 원문 자막만 먼저 띄운다	겹쳐 말하지 않기 · 상대가 기다리는 동안 볼 것이 있다
발화가 끝나면 큐의 첫 문장을 옮겨 말한다	자막의 원문 옆에 번역이 붙는다	순차 통역
말하는 중에 화자가 다시 말하면 멈춘다	중단 횟수를 센다	Ch23 §4의 끼어들기와 같은 우선순위 — 화자가 항상 이긴다

세 번째 규칙의 중단 횟수가 운영 지표입니다. 이 값이 크면 둘 중 하나입니다. 통역이 너무 느려 화자가 기다리지 못했거나, 화자에게 "한 문장씩 끊어 말해 달라"는 안내가 화면에 없거나. 부록 L의 기다림 표시가 통역 화면에서는 "통역 중"이 아니라 "말씀 계속하세요" 여야 하는 이유입니다.

23A.6 수어 브리지 — 손이 입력이고 손이 출력입니다

저자는 한국수어(KSL) 단어 인식 MVP를 따로 만들었습니다. 카메라 영상에서 MediaPipe Holistic으로 상체와 양손 75개 점을 뽑고, BiLSTM으로 단어 20개(+ 무동작)를 분류합니다. AI-Hub 수어 영상 100편으로 학습해 처음 보는 각도에서 85%(17/20), 창 하나에 4.8ms 었습니다.

그 인식기는 이 책 밖입니다. 이 장의 몫은 그 인식기와 아바타 사이입니다. 컴패니언 `signbridge.py`가 양방향을 맡습니다.

입력 쪽 — 단어열을 문장으로. 인식기는 "누나 / 노래 / 놀다" 같은 단어열을 냅니다. 문장이 아닙니다. LLM에 *이 단어들만으로, 더하거나 빼지 말고, 조사와 어순만 채워* 문장을 만들게 하고 그것을 TTS로 읽습니다. "더하지 말라"가 핵심입니다 — 모델은 단어 셋으로 이야기를 지어내려 합니다. LLM이 없어도 단어를 이어 붙인 최소 문장으로 동작합니다.

출력 쪽 — 문장을 손동작 태그로. Ch20의 [검정][동작] 태그를 `[sign:단어]`로 확장합니다. 문장의 내용어를 인식기 어휘에 대면, 어휘에 있는 것은 `[sign:누나]`, 없는 것은 `[spell:병원]` — 지문자 표식으로 남깁니다. **빠뜨리지 않는 것**이 규칙입니다. 어휘에 없다고 낱말을 버리면 상대는 문장의 절반을 잃습니다. 활용형과 조사는 벗겨서 맞추되, 한 글자 어간은 어미까지 붙입니다 — "놀라운"은 "놀다"가 아니고 "힘들었어요"는 "힘"이 아닙니다.

인식기 쪽 사실을 적어 둡니다. 이 브리지가 기대는 인식기가 무엇인지 모르면 브리지의 한계도 모릅니다.

표 23A.3 저자의 KSL 단어 인식 MVP — 브리지가 전제하는 것

항목	값
입력	카메라 영상 → MediaPipe Holistic 상체·양손 75개 점
모델	Conv1d → BiLSTM(192×2) → 어텐션 풀 → 21클래스(단어 20 + 무동작)
데이터	AI-Hub 수어 영상 100편 (20단어 × 5각도)
정확도	처음 보는 각도에서 85% (17/20)
지연	창 하나에 4.8ms
출력	단어열 — 문장이 아니다 (그래서 이 절의 입력 쪽이 필요하다)

어휘 20개로는 어디까지 되나. `coverage()`가 문장 묶음에서 어휘가 덮는 내용어 비율을 냅니다. 창구 문장 넷에서 54%였습니다 — 나머지 46%는 지문자입니다. 이 숫자가 이 브리지의 정직한 상태이고, 인식기 어휘가 늘어나는 만큼만 올라갑니다. 브리지의 값은 어휘가 작을 때 무엇을 놓쳤

는지 보이게 하는 것입니다.

23A.7 실측 — 문장 하나가 첫 소리가 되기까지

컴패니언 `measure.py` 가 창구 문장 3개를 영어로 옮기며 세 시각을 켜었습니다 — 번역 호출, 대상 언어 TTS의 첫 청크, 그리고 발화 종료 판정 0.8초를 더한 **발화 끝 → 통역 첫 소리**. 번역은 경량 모델(gemini-flash-latest) 비스트리밍, 용어 잠금(올댓에이아이 → AllThatAI)과 숫자 보존 검사를 켜 채입니다.

표 23A.4 문장 하나의 통역 지연 — 중앙값 (ch23plus_interpreter/_work/measure.json)

방향	문장	번역	TTS 첫 청크	발화 끝 → 첫 소리	용어·숫자 누락
한→영	3	4.28초	0.37초	5.46초	0

§ 3의 예산 2.3초와 견주어 보세요. 발화 끝에서 첫 소리까지 중앙값 **5.46초**, 최대 5.48초입니다. TTS 첫 청크는 Ch21의 실측과 같은 자릿수이고, 남은 시간은 전부 번역 호출입니다. **통역 지연을 줄이는 자리는 TTS가 아니라 번역기의 첫 문장입니다.**

용어·숫자 누락은 0건이었습니다. "처리에는 영업일 기준 3일이 걸리고 수수료는 12,000원입니다."는 "It will take 3 business days to process, and the fee is 12,000 won."로 나왔고 검사를 통과했습니다. 무료 등급의 호출 한도 안에서 켜 값이라 표본이 적고, 한도에 걸린 문장(빈 응답)은 표에서 뺐습니다 — 본포가 아니라 자릿수로 읽으세요.

그리고 2026년의 라이브 모델에 같은 문장을 말로 넣어 봤습니다 ★

처음엔 텍스트로 넣었습니다. 범용 라이브 모델은 0.59초 만에 영어로 답했습니다. 그런데 **번역 전용 라이브 모델은 20초 동안 아무 메시지도 보내지 않았습다** — 오류조차 없이.

원문 메시지를 그대로 찍어 보니 접속은 받고(`setupComplete`) 텍스트 턴을 *무시* 하고 있었습니다. 라이브 번역기는 말소리를 듣는 물건이라서입니다. 그래서 같은 창구 문장 셋을 TTS로 말소리(16kHz)로 만들어 100밀리초 조각으로 실시간처럼 흘려 넣었습니다. 그리고 **말이 시작된 시각과 끝난 시각 둘 다에서 첫 영어 소리까지**를 켜었습니다.

표 23A.5 통역 첫 소리까지 — 말소리 입력, 세 경로 (ch23plus_interpreter/_work/live_translate.json)

경로	말 끝 → 첫 영어 소리	말 시작 → 첫 영어 소리	성격
이 장 — 순차 (§ 7)	5.46초 (종료 판정 0.8 포함)	—	순차
범용 라이브 모델	0.29초	4.2~6.2초 (= 문장 길이 + 0.3)	순차 — 말이 끝나야 시작
번역 전용 라이브 모델	-1.07초 (끝나기 전)	3.45초	동시 — 문장 중간에 시작

범용 라이브 모델은 순차입니다. 화자가 끝내면 0.29초 뒤에 시작합니다. 이 장의 5.46초와 같은 구조에서 열아홉 배 빠른 것입니다. 그래도 §4의 규칙 중 용어 잠금·숫자 보존은 모델 밖에서 코드가 여전히 검사해야 합니다. 전사를 보세요 — "올댓에이아이"를 한 모델은 *"the Old Say in - formation desk"*, 다른 모델은 *"the Oldest AI"* 로 알아들었고, "3일"은 *"three business days"*로 풀어 읽었습니다. 사람 이름과 기관명은 말소리만으로는 못 맞춥니다. **Glossary**가 필요한 이유가 여기서 실물로 나옵니다. **빨라진 것은 옮기는 시간이지 통역의 규칙이 아닙니다.**

번역 전용 모델은 동시통역입니다. 한국어가 시작되고 3.45초 — 문장이 끝나기 0.4~2.4초 전 — 에 영어가 나옵니다. §2에서 "이 구조로는 안 된다"고 한 그것이 2026년에는 API 호출 하나입니다.

그런데 §5의 규칙이 뒤집힙니다. 순차 통역의 규칙은 *겹쳐 말하지 않기* 였는데, 동시통역은 *겹쳐 말하는 것이 가능*입니다. 그러면 문제가 옮겨 갑니다 — 한 스피커에서 두 목소리가 나오면 둘 다 죽는다는 Ch26 §5의 그 문제로. 동시통역 아바타는 **소리를 분리**해야 합니다. 듣는 쪽에는 이어폰, 화면에는 자막, 아바타의 입은 영어 소리에만 맞춥니다(Ch17). 이 모델을 쓸지 말지는 지언이 아니라 **소리를 분리할 수 있는 자리인가**로 정합니다.

한 가지 더. 번역 전용 모델은 문장이 끝난 뒤에도 출력 스트림을 열어 둡니다. 턴 종료 신호가 오지 않습니다. 출력 길이를 표에 넣지 않은 이유입니다 — **재지 못한 것은 비워 둡니다.** 텍스트 무응답도 처음엔 "음성 전용으로 보인다"고 추정으로 적었다가, 원문 로그로 확인한 뒤에야 이렇게 고쳐 썼습니다.

23A.8 이 장의 경계

이 장의 구조는 순차입니다. 화자와 겹쳐 말하는 동시통역은 이 구조로 안 됩니다. 그리고 §7에서 봤듯 2026년에는 그것을 하는 모델이 따로 있습니다. 그 모델을 쓰는 순간 §5의 규칙은 "겹쳐 말하지 않기"에서 **"겹쳐 말하되 소리를 분리하기"**로 바뀝니다. 규칙이 사라지는 것이 아니라 자리를 옮깁니다.

번역 품질은 이 장이 재지 않습니다. 용어와 숫자가 빠졌는지는 코드가 잡습니다. 옮긴 문장이 맞는지 는 Ch27의 골든셋과 같은 방식 — 창구 문장과 그 정답 번역을 짝지어 둔 목록 — 으로 따로 채점해야 합니다.

수어는 단어 인식까지입니다. 문장 단위 수어 인식과 자연스러운 수어 생성은 각각 별도의 연구 주제입니다. 이 장은 그 둘 사이에 **잃어버리지 않는 다리**를 놓는 데서 멈춥니다.

23A.9 이 장에서 기억할 것

통역 아바타는 **순차**로 말합니다. 문장이 끝나야 옮기고, 옮기는 동안 상대는 원문 자막을 봅니다.

번역기에는 **용어 잠금·숫자 보존** 을 걸고, 돌아온 답을 코드로 검사합니다. 숫자를 말로 푸는 것은 TTS 정규화의 몫입니다.

상태 기계의 규칙은 하나입니다 — **화자가 항상 이깁니다**. 중단 횟수가 지표입니다.

수어 브리지는 어휘 밖 낱말을 **버리지 않고 지문자로 남깁니다**. 뎀 비율이 그 브리지의 상태입니다.

실습 코드: `code/ch23plus_interpreter/` — 문장 분할·용어 잠금·숫자 보존·상태 기계(`interpreter.py`) · 수어 브리지(`signbridge.py`) · 실측(`measure.py`)

예상 소요: 2시간 (번역 모델 키 필요)

관련 부록: L(기다림 표시) · G(음성·얼굴 동의)

CHAPTER 24

기억 3층 — 그리고 원장

24.1 기억이 없으면 인격도 없습니다

코치(Ch21)에게 이름을 알려준 지 5분 만에 다시 물어봤습니다. 코치는 몰랐습니다.

그때까지 코치는 잘 만들어진 아바타였습니다. 2초 안에 대답하고, 눈을 깜빡이고, 운동을 시켰습니다. 그런데 그 한 번의 되물음에 **관계가 없다는 것**이 드러났습니다.

매번 처음 만나는 사람과 대화하는 것을 상상해 보세요. 아무리 성격이 좋아도 관계가 생기지 않습니다.

대화 아바타에서 사용자가 가장 빨리 실망하는 지점이 여기입니다. 어제 말한 것을 오늘 모릅니다. 5분 전에 이름을 말했는데 다시 묻습니다.

기억은 세 층으로 나눠 생각하면 깔끔합니다. **단기 · 요약 · 장기.**

그 전에 — 기억과 지식은 다른 것입니다

이 장은 **기억** 을, 다음 장(Ch25)은 **지식** 을 다룹니다. 둘을 섞으면 설계가 무너지므로 먼저 갈라놓겠습니다.

인지과학에는 이 구분에 이름이 있습니다.

표 24.1 일화 기억과 의미 기억 — 예 · 성격

	예	성격
일화 기억(episodic)	"어제 이 사람이 무릎이 아프다고 했다"	누구의 것인가 가 중요하다
의미 기억(semantic)	"무릎 통증에는 대퇴사두근 강화가 도움이 된다"	누구든 같다

사람의 뇌도 이 둘을 다르게 다룹니다. 우리 시스템도 그래야 합니다.

일화 기억은 사용자마다 따로 쌓이고, 틀리면 관계가 깨집니다. **의미 기억은 모두가 공유** 하고, 틀리면 정보가 잘못 나갑니다. 저장하는 곳도, 지우는 규칙도, 검증하는 방법도 달라야 합니다.

실무에서 이 구분이 갖는 뜻은 분명합니다 — 사용자가 "**내 데이터를 지워 달라**" 고 하면 지워지는 것은 **일화 기억뿐입니다**. 의미 기억은 그 사람의 것이 아니었습니다. Ch29의 철회 요청을 이행할 때 이 경계가 없으면 무엇을 지워야 할지 알 수 없습니다.

이 장은 일화, 다음 장은 의미입니다.

24.2 1층 — 단기 기억 (대화 컨텍스트)

최근 6~10턴을 그대로 다시 보냅니다. 1층은 그게 전부입니다.

몇 턴을 넣을 것인가가 유일한 결정입니다. 많이 넣을수록 문맥이 좋아지지만 토큰 비용과 지연이 올라갑니다.

대화 아바타에서는 **최근 6~10턴** 정도가 적당합니다. 응답이 짧으므로(Ch07) 이 정도는 토큰 부담이 크지 않습니다.

시스템 프롬프트는 항상 맨 앞에. 대화가 길어져도 페르소나가 유지되게 합니다. Ch22에서 말한 붕괴의 예방입니다.

턴이 아니라 토큰으로 자르세요. 사용자가 긴 문장을 하나 넣으면 턴 수는 적는데 토큰이 폭발합니다. 토큰 예산을 정하고 그것을 넘으면 오래된 것부터 버립니다.

숫자로 보면 이렇습니다. 컴패니언 `ch24_memory/experiment.py` 가 300턴짜리 대화(짧은 발화 사이에 25턴마다 672자짜리 긴 발화)를 두 방식에 넣습니다.

표 24.2 컨텍스트 크기(글자) — 최근 8턴 창 vs 토큰 예산 1,200 (`_work/experiment.json`). 한국어는 1토큰이 대략 한 글자여서 예산과 글자 수를 나란히 읽어도 됩니다.

방식	최대	평균	긴 발화를 2,016자로 늘리면
최근 8턴 창	832	250	2,176
토큰 예산 1,200	1,200	1,013	2,049 → 수정 후 1,200

턴 창은 **상한이 없습니다**. 긴 발화가 오면 평균의 8.7배까지 튕니다. 토큰 예산은 상한이 있고, 평균이 예산에 붙어 있어 예산을 씁니다. 턴 창은 평균 250으로 예산의 5분의 1만 씁니다.

그런데 마지막 칸이 이 실험의 발견입니다. **토큰 예산도 처음엔 뚫렸습니다**. 예산 구현이 "최소 2턴은 남긴다"고 보장하므로, 턴 하나가 예산보다 크면 막을 수 없습니다. 그래서 컴패니언은 **턴 자체를 먼저 자르는 상한**(`max_turn`, 기본 예산의 절반)을 두고 잘린 앞부분을 요약 층으로 넘깁니다. 예산은 두 겹이어야 합니다 — 전체에 하나, 턴 하나에 하나.

24.3 2층 — 요약 기억

버리는 대화를 그냥 버리지 말고 요약해서 남깁니다.

언제 요약하나. 단기 기억이 예산을 넘어 오래된 턴을 버릴 때가 자연스러운 시점입니다. 버려지는 턴들을 요약해 요약 슬롯에 누적합니다.

무엇을 요약하나. 전부가 아닙니다. **다시 필요할 정보**만 남깁니다. 사용자에 대한 사실(이름, 선호, 상황), 진행 중인 맥락(무엇을 하려던 중이었나), 그리고 관계의 톤(친해졌는가).

요약도 압축됩니다. 대화가 아주 길어지면 요약 자체가 커집니다. 요약을 다시 요약합니다. 같은 실험에서 300턴 동안 버려진 17,672자가 요약 예산 400자 안에 머물렀습니다. 다만 컴패니언의 요약기는 자리표시자(앞 120자 자르기)라 **압축비**는 의미가 없고, 예산 안에 머무는 구조만 확인

한 것입니다. 실제 요약 모델을 붙이면 무엇이 남는지는 다시 재야 합니다.

요약에 페르소나 규칙을 함께 남기세요. 요약이 시스템 프롬프트 자리를 밀어내는 상황을 막습니다.

요약은 비용이 듭니다. 매 턴 하지 말고, 배치로 몇 턴에 한 번씩 하거나 백그라운드로 돌리세요. 사용자를 기다리게 하면 안 됩니다.

24.4 3층 — 장기 기억 (벡터 저장소)

"사용자는 고양이를 키운다." 이런 한 줄이 세션이 끝나도 남는 층입니다. 저장소는 **벡터 저장소** — 문장을 숫자 배열로 바꿔 두고 뜻이 가까운 것을 찾아 주는 창고입니다(Ch25).

무엇을 저장하나. 대화 전체를 저장하면 검색 품질이 나빠집니다. **의미 있는 사실 단위** 로 저장하세요. "사용자의 이름은 ○○", "사용자는 고양이를 키운다", "사용자는 아침형 인간이 아니다".

이 추출은 LLM에게 시킬 수 있습니다. 대화 조각을 주고 "기억해 둘 만한 사실을 뽑아라" 고 하면 됩니다. 백그라운드로 돌립니다.

언제 꺼내나. 매 턴 검색하면 지연이 늘어납니다. 전략은 둘입니다.

세션 시작 시 한 번 최근·중요 기억 몇 개를 불러 컨텍스트에 넣습니다. 저렴하고, 대부분 충분합니다.

필요할 때만 검색합니다. 사용자 발화에 "저번에", "내가 말했던" 같은 신호가 있을 때, 또는 LLM 이 도구 호출로 직접 검색을 요청하게 합니다.

중복을 관리하세요. 같은 사실이 여러 번 저장되면 검색 결과가 그것으로 채워집니다. 저장 전에 비슷한 기억이 있는지 확인하고, 있으면 갱신합니다.

시간을 함께 저장하세요. 오래된 기억과 최근 기억의 가중치가 달라야 합니다. 그리고 "언제 그 얘기를 했는지" 자체가 대화에서 쓸모 있습니다.

24.5 기억은 대화 로그가 아니라 이벤트 원장입니다 ★

앞 세 층은 전부 **말** 을 다뤘습니다. 실제 서비스에서 더 오래 쓰이는 것은 **무슨 일이 있었는가** 입니다.

저자의 플랫폼은 대화와 별개로 **행동 이벤트 테이블** 을 하나 둡니다.

```
play_events(user_id, 대상, 위치, 종류, 부가정보, 시각)
```

들어온 것, 시작한 것, 힌트를 쓴 것, 끝낸 것. **구조화된 한 줄씩** 쌓입니다.

왜 이게 요약보다 나은가

§ 3의 요약은 **한 번 압축하면 되돌릴 수 없습니다**. 요약할 때 중요하지 않다고 판단한 것은 영원히 사라집니다.

이벤트 원장은 반대입니다. **원본이 남아 있으므로 요약을 언제든 다시 만들 수 있습니다**. 나중에 "사용자가 어떤 주제에서 자주 멈추는가"를 알고 싶어진다면, 그때 원장을 다시 집계하면 됩니다. 요약만 갖고 있었다면 불가능합니다.

규칙 — 일어난 일은 구조화해서 원본으로, 말은 요약해서 압축으로. 둘은 다른 저장소입니다.

프로필은 저장하지 말고 조립하세요

사용자에 대해 아는 것을 하나의 레코드에 저장해 두고 싶어집니다. **하지 마세요**.

저자의 프로필 함수는 매 호출마다 **원장에서 조립** 합니다 — 가입일, 최근 활동, 최근 기록 30건, 누적 집계, 중도포기 수, 등급.

파생값을 저장하면 반드시 원본과 어긋납니다. 이벤트는 새로 쌓였는데 프로필은 옛날 값인 상태가 생기고, 그것을 맞추는 코드가 늘어납니다. **조립 비용이 아까우면 캐시를 하세요. 저장이 아니라 캐시입니다**.

그리고 **최근 것만 가져오세요**. 히스토리는 30건으로 잘려 있습니다. 전부 가져와 프롬프트에 넣으면 토큰이 폭발하고, 오래된 것은 어차피 쓸모가 적습니다.

부재도 기억입니다 ★

가장 배울 만한 질의가 이것입니다.

시작(enter)은 했는데 완료 기록이 없는 것의 개수

중도포기를 이렇게 셉니다. 일어난 일이 아니라 **일어나지 않은 일**을 세는 것입니다.

대화 아바타에도 그대로 옮겨집니다. 사용자가 물었는데 **답을 못 준 질문, 꺼냈다가 흐지부지된 주제, 시작했다 만 작업**. 이런 것들은 어느 로그에도 "사건"으로 남지 않지만, 다음 대화에서 가장 쓸모 있는 정보입니다.

"저번에 물어보신 거, 제가 제대로 답을 못 드렸었죠."

이 한 마디가 나오려면 **답을 못 준 사실이 어딘가 기록되어 있어야 합니다**. 성공만 로깅하는 시스템에서는 영원히 나올 수 없습니다.

인덱스를 처음부터 거세요

원장은 빠르게 커집니다. 그리고 조회 패턴은 처음부터 정해져 있습니다 — **사용자별 시간순**, 그리고 **대상별 종류별**.

```
INDEX (user_id, ts)
INDEX (series_id, kind)
```

테이블을 만들 때 같이 만드세요. 느려진 뒤에 붙이면, 이미 서비스가 느린 상태로 며칠을 보낸 뒤입니다.

24.6 잊는 것도 설계입니다

컴패니언 LongTerm 은 반감기 30일로 점수를 깎습니다. 한 번 말한 사실은 30일 뒤 0.5, 60일 0.25, 90일 0.125 이고, **130일째** 회상 문턱(0.05) 아래로 내려가 더는 꺼내지지 않습니다. 세 번 반복해 말한 사실은 가중치가 2.0이 되어 **160일** 까지 남습니다(`_work/experiment.json`).

이 두 숫자가 "잊는다"의 실제 뜻입니다. 지우는 것이 아니라 *꺼내지지 않게 되는 것*이고, 자주 말한 것은 더 오래 남습니다. 반감기를 얼마로 둘지는 제품의 결정이지만, 어떤 값이든 **며칠 뒤에 무엇이 사라지는지를 이렇게 표로 만들어 두세요.**

잊는 설계는 기억 시스템에서 가장 자주 빠지는 부분입니다.

모든 것을 기억하면 안 됩니다. 사용자가 지나가듯 한 말이 영원히 남아 계속 언급되면 불쾌합니다. 사람도 대부분을 잊습니다.

중요도를 매기세요. 반복해서 언급된 것, 사용자가 명시적으로 말한 것(이름, 선호)은 중요합니다. 한 번 스쳐간 것은 낮게.

감쇠를 넣으세요. 오래되고 다시 언급되지 않은 기억은 검색 순위를 낮춥니다.

감쇠만으로는 부족합니다. 가중치가 함께 있어야 합니다.

컴패니언 코드에서 확인한 것입니다. 같은 시각에 저장된 사실 둘이 있는데 하나는 **두 번 언급된 것**이고 다른 하나는 **한 번뿐인 것**이라면, **두 번 언급된 쪽이 먼저 회상되어야 합니다.** 감쇠는 시간만 보므로 이 둘을 구분하지 못합니다.

```
반복 언급 → 가중치 ↑      "자주 말한 것이 중요한 것"
시간 경과 → 점수 ↓        "오래된 것은 흐려진다"
회상 점수 = 가중치 × 감쇠
```

그리고 점수가 임계값 아래로 내려간 기억은 **회상 목록에서 아예 빠집니다.** 지우는 것이 아니라 안 꺼내는 것입니다. 나중에 그 주제가 다시 나오면 가중치가 올라가 되살아납니다. **사람이 잊었다가 떠올리는 것과 같은 모양**입니다.

사용자가 지을 수 있어야 합니다. "그건 잊어줘"가 동작해야 하고, 저장된 기억 전체를 보고 삭제할 수 있는 경로가 있어야 합니다. 이것은 기능이 아니라 **의무**에 가깝습니다.

24.7 기억이 만드는 인격

어제 "나는 커피를 좋아해"라고 한 캐릭터가 오늘 "커피는 안 마셔"라고 합니다. 그 순간 인격이 무너집니다.

기억은 정보 저장 이상의 일을 합니다.

대화를 이어갑니다. "저번에 말한 그 일은 어떻게 됐어요?"라고 먼저 물을 수 있습니다. 이것 하나가 관계의 느낌을 크게 바꿉니다.

개인화합니다. 사용자가 짧은 답을 좋아하는지, 농담을 좋아하는지를 기억해 스타일을 맞춥니다.

일관성을 지킵니다. 캐릭터가 스스로에 대해 말한 것도 기억해야 합니다. 위의 커피가 그것입니다. **캐릭터 자신에 대한 기억**을 별도로 관리하세요.

마지막 항목이 특히 중요하고, 자주 놓칩니다. 기억은 사용자에게 대한 것만이 아닙니다.

24.8 기억에 없는 사실은 물어봅니다 — 도구 호출 ★

세 층의 기억을 다 뒤져도 없는 것이 있습니다. *내일 7시에 자리가 있는가*. 그것은 기억이 아니라 **예약 시스템**에 있습니다. 2026년의 대화 아바타는 여기서 갑니다 — 지어내거나, 물어보거나.

지어내는 쪽이 기본값입니다. 모델은 "내일 7시요? 네, 예약됐어요!"를 아주 자연스럽게 말합니다. 예약은 되지 않았습니. Ch27 §2의 골든셋에 *모른다고 인정하는가* 항목이 있는 이유이고, 이 절은 그 반대편 — **모르는 것을 실제로 알아 오는 길**입니다.

어떤 도구를 부를지는 모델의 함수 호출 기능이 정합니다. 프롬프트에 도구 목록(이름·설명·인자)을 넣으면 모델이 "book_slot(when='내일 19:00')"을 돌려줍니다. 거기까지는 모델 회사가 해 줍니다. 그 뒤가 전부 여러분의 코드입니다. 컴패니언 `tools.py`의 `ToolRouter`가 네 가지를 맡습니다.

표 24.3 도구를 부른 뒤에 일어나는 네 가지 — 모델이 안 해 주는 것

일	규칙	안 하면
예산	도구가 1.2초를 넘기면 먼저 한 문장을 말한다 — "예약 시스템에 넣고 있어요."	아바타가 굳은 채 3초. Ch07의 예산을 도구가 잡아먹는다
먹등 키	사용자·요청으로 키를 만들고, 같은 키는 다시 실행하지 않는다	"예약해 줘"를 두 번 알아들으면 예약이 두 개
시간 초과	8초를 넘기면 포기하고 사과 문장을 말한다	세션이 도구에 매달려 끼어들기(Ch23)도 안 먹는다
예외	도구가 죽어도 대화는 문장으로 이어진다	스택 트레이스가 목소리로 읽힌다 — 실제로 있었던 일이다(Ch03 §4의 태그 누출과 같은 종류)

채움말이 먼저, 결과가 나중. `speak_plan()`이 돌려주는 문장 순서가 곧 아바타가 말하는 순서입니다. 도구가 빠르면 결과 한 문장, 느리면 채움말 뒤에 결과. 채움말은 부록 L §4의 문장을 그대로 씁니다 — 그것이 *기다림의 표시*이기 때문입니다. Ch08이 화면으로 하던 일을 여기서는 말로

합니다.

호출은 전부 기록합니다. `router.calls` 가 누가 무엇을 언제 불렀고 몇 초 걸렸는지를 들고 있습니다. 운영자 콘솔(Ch28A)의 지표 방에 *도구별 p95(100번 중 95번은 이 시간 안에 끝난다)* · *시간 초과율* · *재실행 방지 횟수* 세 줄이 들어가면, "예약이 안 됐어요"라는 민원이 도구 문제인지 모델 문제인지 한 줄로 갑니다.

도구 호출은 이 책의 2초 예산과 정면으로 부딪히는 유일한 기능입니다. 다른 모든 지연은 여러분의 서버 안에 있지만, 도구는 **남의 시스템**입니다. 그래서 예산·먹등·초과·예외 넷을 도구마다 정하지 않고 **라우터 한 곳**에 둡니다.

24.9 이 장에서 기억할 것

기억은 **단기(최근 턴)** · **요약(압축)** · **장기(백터)** 세 층입니다.

단기는 **6~10턴**, 턴이 아니라 **토큰으로 자릅니다**. 시스템 프롬프트는 항상 맨 앞.

요약은 버려지는 턴에서 **다시 필요할 정보만 뽑고, 백그라운드로 돌립니다**.

장기는 사실 **단위로 저장**, 세션 시작 시 **로드** 또는 **신호가 있을 때 검색**. 중복 관리와 시간 기록 필수.

잇는 것도 설계입니다. 중요도 · 감쇠 · 사용자 삭제 경로.

캐릭터 자신에 대한 기억을 별도로 관리하세요. 자기모순이 인격을 가장 빠르게 무너뜨립니다.

말은 요약해서, 일어난 일은 구조화해서. 요약은 되돌릴 수 없지만 이벤트 원장은 다시 집계할 수 있습니다.

프로필은 저장하지 말고 조립하세요. 파생값을 저장하면 원본과 어긋납니다.

부채도 기억입니다. 답을 못 준 질문, 흐지부지된 주제. 성공만 로깅하면 영원히 못 꺼냅니다.

기억에 없으면 지어내지 말고 물어보거나 도구를 부르세요(§8). 채움말이 먼저, 결과가 나중입니다.

다음 장은 지식입니다. 기억이 "겪은 것" 이라면 지식은 "아는 것"입니다.

실습 코드: `code/ch24_memory/` — 3층 기억 구현 + 요약 배치 + 중요도·감쇠

예상 소요: 4시간

관련 부록: L §4(기다림을 정직하게) · N(원가 모델)

CHAPTER 25

이 캐릭터만 아는 것 — RAG

25.1 기억과 지식은 다릅니다

검색을 붙이자마자 코치(Ch21)가 매뉴얼처럼 말하기 시작했습니다.

문서에서 찾은 문단이 컨텍스트에 들어가자, 코치는 그 문단의 문체를 따라갔습니다. 어제까지 "자, 같이 해봐요!"라고 하던 캐릭터가 "본 동작은 대퇴사두근을 주동근으로 하며"라고 말했습니다.

검색은 정확도를 올리면서 인격을 지웁니다. 이 장은 그 둘을 동시에 지키는 방법입니다.

앞 장의 기억은 **겪은 것**입니다. 사용자와 나눈 대화에서 생겨납니다.

이 장의 지식은 **아는 것**입니다. 대화 이전부터 있었고, 문서에서 옵니다. 회사의 제품 정보, 게임 세계관 설정, 강의 교재, 캐릭터의 배경 이야기.

이 지식을 답변 직전에 찾아 붙이는 방식이 **RAG**(retrieval-augmented generation, 검색 증강 생성)입니다. 모델을 다시 학습시키지 않고 캐릭터에게 자료만 쥐여 주는 길입니다.

기억과 지식은 저장소도 검색 방식도 갱신 주기도 다릅니다. **섞지 마세요.** 섞으면 사용자 정보가 지식 검색 결과에 끼어들고, 문서 조각이 개인 기억으로 저장됩니다.

25.2 왜 프롬프트에 다 넣으면 안 되나

"문서가 몇 장뿐인데 그냥 시스템 프롬프트에 넣으면 안 되나요"라는 질문이 나옵니다.

정말 적으면 넣으세요. 몇 문단 수준이면 검색 시스템을 만드는 것이 과합니다.

문서가 늘어나면 세 가지가 나빠집니다. **토큰 비용** 이 매 요청마다 붙고, **지연** 이 늘고, 무엇보다 **지시 준수율이 떨어집니다.** 프롬프트가 길어질수록 모델이 말투 규칙 같은 것을 놓치기 시작합니다.

문서가 열 페이지를 넘으면 검색으로 가세요.

25.3 한국어 청킹 — 품질의 절반

이 장의 청커로 Ch07을 쪼갰더니 목표 크기 안에 든 조각이 **39%** 였습니다. 청킹은 문서를 검색 단위로 쪼개는 일이고, 여기서 품질의 절반이 결정됩니다.

의미 단위로 자르세요. 글자 수로 기계적으로 자르면 문장 중간이 끊깁니다. 문단·절 경계를 먼저 보고, 그 안에서 크기를 조절합니다.

한국어는 조금 작게. 같은 정보량에 필요한 글자 수가 영어보다 적습니다. 300~600자 정도가 무난한 출발점입니다.

컴패니언 `chunker.py` (최대 600 · 최소 300 · 겹침 15%)를 이 책의 장 셋에 돌리면 이렇게 됩니다.

표 25.1 실제 장 세 개를 쪼갠 결과 (ch25_rag/_work/measure.json)

문서	글자	조각	평균	범위	300~600 안
Ch25 (이 장)	4,905	11	425	212~590	82%
Ch22	7,635	19	382	202~596	68%
Ch07 자연 예산	12,408	38	312	84~573	39%

Ch07만 39%인 이유에 청커의 한계가 보입니다. 표와 코드 블록이 많은 장이라, 표 하나가 통째로 한 조각(84자짜리도 있습니다)이 되고 그 앞뒤 산문이 짧게 남습니다. 규칙("300~600자")은 산문에 맞춘 것입니다. **표·코드가 섞인 문서는 조각 크기 분포부터 찍어 보고** 표를 산문에 붙일지 따로 돌지 정하세요.

겹침을 두세요. 인접한 조각이 조금씩 겹치게 하면 경계에 걸친 정보를 놓치지 않습니다. 조각 크기의 10~20%.

제목 맥락을 각 조각에 넣으세요. "3.2 환불 규정" 아래의 조각에 그 제목이 없으면, 검색했을 때 무엇에 대한 내용인지 알 수 없습니다. 각 조각 앞에 상위 제목들을 붙여 두면 검색 품질과 답변 품질이 함께 올라갑니다. **비용 대비 효과가 가장 큰 한 가지**입니다.

표는 따로 처리하세요. 표를 일반 텍스트로 자르면 행과 열이 뒤섞입니다. 표 하나를 조각 하나로 두거나, 각 행을 문장으로 풀어 씁니다.

25.4 임베딩과 검색

"A-2401 모델"을 물었더니 비슷하게 생긴 다른 모델 설명이 나옵니다. 의미 검색만 쓰면 이런 일이 납니다.

한국어를 지원하는 임베딩 모델을 쓰세요. 임베딩은 문장을 숫자 배열로 바꿔 뜻이 가까운 것끼리 모아 두는 모델입니다. 영어 전용 모델은 한국어에서 검색 품질이 크게 떨어집니다.

의미 검색만으로는 부족합니다. 고유명사, 제품 코드, 정확한 숫자를 찾을 때 의미 검색은 약합니다. 위의 "A-2401"이 그 경우입니다.

키워드 검색과 섞으세요. 두 방식의 결과를 합쳐 순위를 다시 매기는 것이 표준적인 해법입니다. 구현이 복잡하지 않고 효과가 확실합니다.

리랭킹을 고려하세요. 검색으로 20개를 뽑고, 더 정밀한 모델로 다시 순위를 매겨 상위 3~5개만 씁니다. 정확도가 오르는 대신 지연이 붙으므로, 대화 아바타에서는 예산을 확인하고 정하세요.

25.5 대화 아바타에서의 특수 사정

§ 1의 "대퇴사두근을 주동근으로 하며"가 나오는 자리가 여기입니다. 일반적인 문서 질의응답 시스템과 다른 점이 셋입니다.

답변이 짧아야 합니다. 검색된 문서를 그대로 읽어주면 안 됩니다. 아바타는 말로 전달하고, 사람은 긴 낭독을 듣지 못합니다. **"검색 결과를 참고해서 한두 문장으로 답하세요"**를 프롬프트에 명시하세요.

말투가 유지되어야 합니다. 검색 결과가 들어오면 모델이 문서의 문체를 따라갑니다. 캐릭터가 갑자기 매뉴얼처럼 말합니다. 페르소나 지시를 검색 결과 뒤에 한 번 더 넣으면 좋아집니다.

지연 예산이 빠듯합니다. 검색이 200~400밀리초를 더합니다. Ch07의 2초 예산에서 이 정도는 감당하지만, 리랭킹까지 넣으면 빠듯해집니다. **매 턴 검색하지 말고 필요할 때만** 하는 전략이 여기서도 유효합니다.

25.6 언제 검색할 것인가

"안녕하세요"에 문서를 검색할 이유가 없습니다. 그런데 **항상 검색**이 가장 흔한 구현입니다. 단순하지만 낭비입니다.

분류기로 판단 — 가벼운 모델이나 규칙으로 "이 질문은 문서가 필요한가"를 먼저 봅니다. 지연이 조금 불지만 대부분의 턴에서 검색을 건너뛸니다.

LLM이 도구로 호출 — 모델이 스스로 판단해 검색을 요청하게 합니다. 가장 유연하지만 왕복이 한 번 더 생겨 지연이 큼니다.

대화 아바타에서는 **간단한 규칙 + 필요 시 검색** 조합을 권합니다. 도메인 키워드가 들어 있거나 질문 형태이면 검색하고, 아니면 건너뛸니다.

25.7 근거를 다루는 법

문서에 없는 질문의 검색 점수는 0.00이었습니다. 그 숫자가 어떤 프롬프트보다 확실한 근거입니다.

모르면 모른다고 하게 하세요. 검색 결과에 답이 없으면 지어내지 말고 모른다고 말하도록 프롬프트에 명시합니다. 캐릭터의 말투로 모른다고 말하는 방식을 정해 두세요(Ch22).

그런데 프롬프트에만 맡기지 마세요. 검색 점수 자체가 더 확실한 근거입니다.

کمپانیون 코드로 확인한 값입니다. 홈트 문서에 대해 —

"하프 스쿼트"	→ 0.17	(문서에 있음)
"카페 원두 로스팅 온도"	→ 0.00	(문서에 없음)

차이가 뚜렷합니다. 문서에 없는 질의는 점수가 바닥에 붙습니다. 그러면 LLM을 부르기 전에 코드가 먼저 판단할 수 있습니다.

최고 점수 < 임계값 → 검색 결과를 아예 넘기지 않는다
→ "그건 제가 아는 범위 밖이에요" 로 바로 답한다

이 게이트가 있으면 **LLM이 억지로 짜맞출 기회 자체가 없어집니다.** 근거가 약한 조각을 컨텍스트에 넣어 놓고 "지어내지 마세요"라고 부탁하는 것보다 훨씬 안전하고, 토큰도 아깁니다.

임계값은 문서와 검색 방식마다 다릅니다. 골든셋의 **모르는 것 분류(Ch27 §2)**를 돌려 보면서 잡으세요.

출처를 보여줄 것인가. 텍스트 채팅이라면 출처 표시가 신뢰를 줍니다. 음성 대화에서 출처를 읽어주면 부자연스럽습니다. **화면에는 표시하고 말로는 하지 않는** 절충이 좋습니다.

중요한 도메인에서는 검증을 붙이세요. 의료·법률·금융처럼 틀리면 안 되는 영역이라면, 생성된 답변이 검색된 근거에 실제로 있는지 확인하는 단계를 넣습니다.

그리고 **반대 방향도 재세요.** 검색 결과를 주고 맞히는지만 보면, **모델이 원래 알고 있어서 맞힌 것**과 구분되지 않습니다. 검색을 끄고 같은 질문을 던져 **모른다고 답하는지**를 함께 확인하세요. 둘 다 통과해야 RAG가 실제로 일하고 있는 것입니다. Ch27 §6의 필요·충분조건 검사와 같은 구조입니다.

25.8 이 장에서 기억할 것

기억(겪은 것)과 지식(아는 것)을 분리 하세요. 저장소도 검색도 갱신 주기도 다릅니다.

문서가 **열 페이지를 넘으면** 검색으로 갑니다. 그 아래면 프롬프트에 넣어도 됩니다.

청킹에서 가장 효과가 큰 것은 **각 조각에 상위 제목 맥락을 붙이는 것**입니다. 한국어는 300~600자, 10~20% 겹침.

의미 검색 + 키워드 검색을 섞으세요. 고유명사와 숫자는 의미 검색이 약합니다.

대화 아바타의 특수 사정 셋 — **짧은 답변 · 말투 유지 · 지연 예산.** 페르소나 지시를 검색 결과 뒤에 한 번 더 넣으세요.

필요할 때만 검색 합니다. 간단한 규칙으로 충분합니다.

모르면 모른다고 하게 하고, 그것도 캐릭터의 말투로 하게 하세요.

다음 장에서 아바타를 여러개로 늘립니다. 그리고 그들끼리 대화하게 만듭니다.

실습 코드: `code/ch25_rag/` — 제목 맥락 청킹 + 하이브리드 검색 + 캐릭터 말투 유지 프롬프트

예상 소요: 4시간

관련 부록: F §F(인격이 무너지는 실패) · N(원가 모델)

CHAPTER 26

여러 아바타가 함께

26.1 하나에서 여럿으로

마피아 게임을 처음 돌렸을 때, 시민 역할의 캐릭터가 첫 턴에 정확히 마피아를 지목했습니다.

기뻐할 일이 아니었습니다. 그 시민은 추리를 한 게 아니라 **그냥 알고 있었습니다**. 모든 캐릭터에게 같은 컨텍스트를 넘기고 있었기 때문입니다. 게임이 성립하지 않았습니다.

이 장의 절반은 그 버그를 고치는 이야기입니다.

아바타 하나와 대화하는 것과, 여럿이 있는 자리에 사용자가 끼는 것은 완전히 다른 경험입니다. 이 장의 주인공은 **마을 사람들**(서로 다른 인격을 받은 아바타 여럿 — Track C의 얼굴)입니다.

여럿이 되면 **아바타끼리 대화** 합니다. 사용자가 아무 말도 하지 않아도 장면이 진행됩니다. 각자 다른 의견을 내고, 서로 반박하고, 사용자에게 의견을 묻습니다.

이 구조가 잘 맞는 곳이 있습니다. **추론 게임** (마피아, 추리), **역할극** (면접 연습, 상담 훈련), **패널 토론** (여러 관점 제시), **교육 시나리오** (교사와 학생 역할).

저자는 이 구조로 세 개를 만들었습니다. **마피아 게임** 은 서로를 의심하고, **살인 미스터리**(이하 추리극)는 증언을 모아 범인을 좁히고, **방탈출** 은 협력해서 문을 엽니다.

셋의 규칙은 완전히 다른데 **엔진 구조는 거의 같습니다**. 진행자가 순서를 정하고, 각 캐릭터에게 보여줄 것을 분배하고, 페이지를 넘깁니다. 규칙이 바뀌어도 이 뼈대는 그대로입니다. 그래서 이 장은 게임 만드는 법이 아니라 **뼈대 만드는 법** 을 씁니다.

26.2 진행자가 필요합니다

여러 아바타를 그냥 풀어놓으면 대화가 무너집니다. 동시에 말하거나, 아무도 말하지 않거나, 한 캐릭터만 계속 말합니다.

진행자(오케스트레이터) 를 두세요. 진행자는 LLM이 아니라 **코드** 입니다. 하는 일은 셋입니다.

누가 말할 차례인지 정합니다. 순서대로 돌거나, 상황에 따라 고르거나, 사용자가 지목합니다.

각 캐릭터에게 무엇을 보여줄지 정합니다. 이것이 가장 중요합니다. 바로 다음 절이 그 이야기입니다.

단계를 관리합니다. 게임이라면 낮·밤·투표 같은 페이지, 토론이라면 발제·반론·정리.

진행자를 LLM으로 만들고 싶은 유혹이 있지만, 권하지 않습니다. **규칙 진행은 결정론적이어야 합니다**. LLM 진행자는 가끔 규칙을 어기고, 그 순간 게임이 깨집니다.

26.3 비밀 정보를 격리하세요

비밀을 거르는 줄을 일부러 하나 빼 둔 브리핑에서 남의 비밀이 **8건** 있습니다. 아래 표가 그것입니다. 비밀 격리는 추론 게임에서 가장 중요한 기술적 요구사항입니다.

마피아 게임에서 마피아가 누구인지는 마피아만 압니다. 시민 캐릭터의 LLM 호출에 그 정보가 들어가면, 시민이 알 수 없는 것을 아는 것처럼 행동합니다. §1의 첫 판이 그랬습니다.

각 캐릭터마다 별도의 컨텍스트를 만드세요. 공개 대화 로그는 모두가 공유하지만, 비밀 정보는 해당 캐릭터의 컨텍스트에만 들어갑니다.

진행자가 이 본분을 담당합니다. 캐릭터별로 "이 캐릭터가 아는 것"을 명시적으로 관리하고, LLM 호출 때 그것만 넣습니다.

브리핑 함수 하나로 모으세요

저자의 추리극 엔진은 이것을 **함수 하나** 로 처리합니다. 캐릭터 이름을 주면 그 캐릭터가 볼 컨텍스트를 조립해 돌려줍니다. 다섯 조각입니다.

- ① **사건 개요** — 모두가 아는 공개 정보.
- ② **본인 정보** — 이름·관계·직업, 그리고 **비밀** 과 **알리바이**.
- ③ **입장(stance)** — 여기가 핵심입니다. 아래에서 따로 봅니다.
- ④ **공개된 단서** — 지금까지 드러난 것만. 아직 안 나온 단서는 넣지 않습니다.
- ⑤ **최근 발언 로그** — 최근 16줄. 전부 넣으면 토큰이 폭발하고, 너무 적으면 맥락이 끊깁니다.

함수 하나로 모으는 것이 요령입니다. 컨텍스트 조립이 여러 군데 흩어져 있으면 어느 한 곳에서 비밀이 새고, 그것을 찾을 수 없습니다. 누출은 항상 "**한 군데를 빠뜨려서**" 생깁니다.

입장(stance)을 주입하세요

같은 사건을 보고도 범인과 무고한 사람은 **목표가 다릅니다**. 그 목표를 프롬프트에 직접 씁니다.

범인이면	: "들리지 말고 의심을 다른 사람에게 돌려라"
무고하면	: "단서로 추리해서 범인을 지목하라"

이 두 줄이 캐릭터의 행동을 갈라놓습니다. 비밀 정보를 주는 것만으로는 부족합니다 — **그 정보로 무엇을 하려는지** 까지 줘야 연기가 됩니다.

진행자 LLM 에게도 재갈을 물리세요

내레이터를 LLM으로 돌린다면, 그 LLM은 **모든 것을 알고 있습니다**. 그래서 규칙을 명시해야 합니다.

진실 공개 전까지 범인 추측·누설 절대 금지. 공개된 사실만 언급. 주어진 사실에 없는 내용을 지어내지 말 것.

세 번째 줄도 중요합니다. 내레이터가 분위기를 살리려고 없는 디테일을 지어내면, 플레이어는 그것을 단서로 착각합니다.

누출을 테스트하세요. 시민 캐릭터에게 "마피아가 누구인 것 같아?"를 반복해서 물어보고, 정답률이 우연 수준인지 확인합니다. 우연보다 높으면 어딘가에서 정보가 새고 있습니다.

그 전에 문자열부터 훑으세요

위 방법은 확실하지만 비쌉니다. LLM을 수십 번 부르고, 통계를 봐야 하고, 결과가 흔들립니다.

그 앞에 공짜로 할 수 있는 검사가 있습니다. 모든 캐릭터의 브리핑을 만들어 놓고, 남의 비밀 문자열이 거기 들어 있는지 훑는 것입니다.

```
def leak_scan(scene, cast):
    found = []
    for viewer in cast:
        text = brief(scene, cast, viewer.name)      # 실제로 넘길 그 문자열
        for other in cast:
            if other.name != viewer.name and other.secret in text:
                found.append((viewer.name, other.name))
    return found
```

컨텍스트에 **아예 없으면 물어볼 필요도 없습니다.** 통계 검사는 이걸 통과한 뒤에 합니다.

그리고 이런 검사에는 함정이 하나 있습니다 — **아무것도 못 잡는 검사도 통과합니다.** 그래서 컴패니언 코드는 *거르는 줄을 하나 빼 둔 버전*을 같이 두고 돌립니다.

표 26.1 브리핑 · 비밀 누출 · 미공개 단서 누출

브리핑	비밀 누출	미공개 단서 누출
<code>brief()</code> — 정상	0건	0건
<code>brief_leaky()</code> — 다른 인물 소개에 역할과 비밀을 같이 넣음	8건	0건

차이는 **한 줄**입니다. 다른 인물을 소개할 때 편하다고 객체를 통째로 넘긴 것뿐입니다. 실제 누출의 대부분이 이렇게 생깁니다.

검사가 살아 있다는 것을 검사하세요. 통과만 하는 테스트는 없는 것과 같습니다.

이 구조는 게임이 아닌 곳에서도 쓰입니다. 면접 연습에서 면접관은 평가 기준을 알고 지원자는 모릅니다. 상담 훈련에서 내담자 역할은 배경 사정을 알고 상담자는 모릅니다.

26.4 누가 말할 차례인가

한 캐릭터가 **54턴** 동안 한 마디도 못 했습니다. 게임 한 판 내내입니다. 발언권 결정 방식이 대화의 성격을 만듭니다.

순서 고정 — 가장 단순하고 예측 가능합니다. 회의나 토론에 맞습니다.

진행자가 지목 — 상황에 따라 코드가 고릅니다. "직전에 이름이 언급된 캐릭터", "아직 말하지 않은 캐릭터" 같은 규칙으로 자연스러워집니다.

사용자가 지목 — 사용자가 특정 캐릭터에게 말을 겁니다. 게임에서 흔한 방식입니다.

발언 욕구 점수 — 각 캐릭터가 "지금 말하고 싶은 정도"를 내고 높은 쪽이 말합니다. 자연스럽게 매 턴 모든 캐릭터에게 호출이 필요해 비쌉니다.

실용적인 조합은 **규칙 기반 지목 + 사용자 개입 허용**입니다.

그 조합이 실제로 어떤 분포를 만드는지 컴패니언 `turns.py` 로 확인했습니다 — 여섯 캐릭터, 300턴, 발언자는 40% 확률로 누군가를 지목하고 지목은 두 명에게 몰립니다(현실의 대화가 그렇습니다).

표 26.2 발언권 규칙별 분포 — 300턴 (ch26_multi/_work/turns.json)

규칙	점유율 최대	최소	가장 긴 침묵
순서 고정	17%	17%	6턴
이름 지목 우선 (처음 엔진)	31%	8%	54턴
이름 지목 + 굶주림 방지(8턴)	23%	12%	9턴

둘째 줄이 처음 엔진의 실제 동작입니다. 저자가 위 목록에 "아직 말하지 않은 캐릭터"를 적어 두고도 **코드에는 그 규칙이 없었습니다**. 지목이 몰리자 한 캐릭터가 54턴을 침묵했고, 사용자는 그 캐릭터를 "있으나 마나 한 NPC"로 기억합니다.

굶주림 방지 한 줄 — 8턴 넘게 말 못한 사람이 있으면 그가 먼저 — 을 넣자 9턴으로 내려왔고, 점유율도 23/12로 좁혀졌습니다. 순서 고정의 17/17이 가장 공평하지만 그건 대화가 아니라 발표 순서입니다. **공평함과 자연스러움 사이의 값** 을 이 표로 고르세요.

26.5 비용과 지연

여섯 명이면 한 라운드에 LLM을 **스물다섯 번** 부릅니다. 캐릭터가 늘면 호출도 늘고, 이것이 이 구조의 가장 큰 제약입니다.

동시에 호출하세요. 여러 캐릭터가 각자 반응해야 한다면 순차가 아니라 병렬로 부릅니다.

저자의 추리극 엔진은 등장인물마다 소개·발언·지목에서 한 번씩 부르고 사건 생성에 한 번 더 부릅니다. 6인 기준으로 한 라운드에 25회입니다 — 여섯 명 × 네 자리(소개·발언·지목·판정) + 사건 생성 한 번. 순차로 돌리면 호출 수 × 응답 시간이 그대로 걸립니다. 실제 엔진은 **스레드 풀 다섯(max_workers=5)** 으로 동시에 던지고, 순차 약 25초가 약 5초로 줄었습니다.

정확한 수치는 컴패니언 `schedule.py` 를 보세요. 산식은 호출 25회 · 워커 5 → 5.0배입니다. 20ms 짜리 가짜 호출 25개를 실제 스레드 풀에 던진 실측은 순차 0.51초 → 병렬 0.10초였습니다(`_work/schedule.json`, 실행마다 4.9~5.1배 사이에서 흔들립니다).

그리고 이 시간은 대개 **공짜로 숨겨집니다**. 사람 플레이어가 입력하는 동안 AI 여섯 명의 대사가 뒤에서 생성되기 때문입니다. Ch08의 **지연 숨기기**가 멀티 아바타에서는 훨씬 쉽습니다 — 기다리는 사람이 이미 무언가를 하고 있으니까요.

말하지 않는 캐릭터는 부르지 마세요. 발언권이 없는 캐릭터에게 호출을 낭비하지 않습니다.

모델 등급을 나누세요. 중요한 판단(추론, 투표)은 좋은 모델로, 단순한 반응(맞장구, 짧은 코멘트)은 가벼운 모델로. 비용이 크게 줄어듭니다.

작업마다 온도와 토큰 상한을 따로 잡으세요. 저자의 추리극이 쓰는 값입니다.

표 26.3 작업 · 온도 · 최대 토큰

작업	온도	최대 토큰
사건 생성 (JSON)	1.0	850
등장 소개	0.9	110
심문 발언	0.9	140
범인 지목	0.9	80
내레이터	0.85	80

대사·추론·지목은 100~150 토큰입니다. 여럿이 말하는 구조에서는 각자 한두 문장이어야 리듬이 삽니다. 반대로 사건을 통째로 만드는 작업만 850 토큰에 온도 1.0 — 창작이 필요한 곳에만 여유를 줍니다.

LLM을 안 쓰는 자리를 정하세요. 단서 안내나 지목 안내 같은 정형 멘트는 **템플릿**으로 씁니다. 매번 생성하면 느리고, 비싸고, 무엇보다 **틀릴 수 있습니다**. LLM은 사건 브리핑과 진실 공개처럼 **창작이 필요한 곳**에만 씁니다.

음성을 순차 재생하세요. 동시에 두 목소리가 나오면 아무것도 안 들립니다. 텍스트는 병렬로 만들어 재생은 순서대로 합니다. 앞 캐릭터가 말하는 동안 뒤 캐릭터의 TTS가 끝나므로 대기가 느껴지지 않습니다.

텍스트를 먼저 띄우고 음성은 뒤따르게 하세요. 저자의 구현은 백그라운드 순차 위커가 합성을 맡고, 화면에는 대사가 즉시 뜹니다. 읽는 속도가 듣는 속도보다 빠르므로 사용자는 기다리지 않습니다.

목소리 여섯을 만드는 데 음성 여섯 개가 필요하지 않습니다. 저자는 세 개의 음성 + 피치 조절로 여섯 인물을 구분하고, 내레이터만 피치를 크게 내려(-24반음) 확실히 갈라놓습니다. Ch03 §5의 **파라미터로 감정을 근사한다**가 여기서는 **파라미터로 인물을 늘린다**로 씁니다.

26.6 화면에 여럿 배치하기

화면을 안 보고 있어도 누가 말하는지 알아야 합니다.

목소리를 확실히 다르게 하세요. 성별, 톤, 속도를 다르게 배정합니다.

말하는 캐릭터를 강조 하세요. 살짝 앞으로 나오거나, 밝아지거나, 다른 캐릭터가 그쪽을 봅니다. 마지막 것이 특히 자연스럽습니다 — **듣는 캐릭터들이 말하는 캐릭터를 바라보게** 하면 장면이 살아납니다.

렌더 부담을 확인하세요. 3D 캐릭터 여럿을 동시에 렌더하면 저사양 기기에서 프레임이 떨어집니다. 넷 이상이면 2D 파츠 트랙(Ch16)을 고려하거나, 말하지 않는 캐릭터의 애니메이션 갱신 빈도를 낮추세요.

26.7 무엇이 막혀도 게임은 끝나야 합니다

멀티 아바타에서 LLM 실패는 단일 아바타보다 치명적입니다. **진행 중인 게임이 중간에 멈추기 때** 문입니다. 한 턴을 못 넘기면 그 판 전체가 날아갑니다.

저자의 엔진은 폴백을 세 겹으로 쌓습니다.

1겹 — 프로바이더 라우팅. 여러 무료 제공자를 라운드로빈(차례로 돌려 쓰기)으로 돌려 부하를 나누고, 한 곳이 한도(429)에 걸리면 **90초 쿨다운**을 걸어 그동안 건너뛹니다. 막힌 곳을 계속 두드리지 않는 것이 요령입니다.

2겹 — 빈 응답 재시도. 응답이 비어 오면 짧게 쉬고 한 번 더. 이것만으로 잡히는 실패가 의외로 많습니다.

3겹 — 작업별 대체물. 여기가 핵심입니다.

사건 생성 실패 → 미리 만들어 둔 완성 시나리오 3종 중 랜덤
 대사 생성 실패 → 상황별 일반 대사문
 내레이션 실패 → 템플릿 문장

LLM이 전부 막혀도 게임이 완주됩니다. 조금 덜 똑똑한 판이 되지만 끝은 납니다. Ch07 §6의 홈런이 오프라인 사전과 같은 사상이고, 규모만 다릅니다.

생성 결과를 검증하고 받으세요

사건을 JSON으로 받는다면 **모델이 형식을 지켰다고 믿지 마세요.** 저자의 엔진은 받은 JSON을 두 단계로 거릅니다.

추출 — 코드펜스를 벗기고 중괄호 범위만 잘라냅니다. 모델은 종종 설명을 앞뒤에 붙입니다.

검증 — 인물이 정확히 여섯인가, 범인이 **정확히 한 명** 인가, 단서에 나오는 이름이 인물 목록과 맞는가, 단서가 비어 있지 않은가.

하나라도 어긋나면 폴백으로 갑니다. 범인이 둘인 사건은 풀 수 없고, 없는 이름이 단서에 나오면 플레이어가 영원히 헤맬니다. **게임 규칙을 코드가 검사해야 합니다**(Ch27 §6의 생성물 검증과 같은 태도입니다).

한국어 특유의 함정 하나

다국어 모델은 가끔 **코드 스위칭** 을 합니다. 한국어로 답하다가 일본어나 한자가 섞이는 것입니다. 특히 대형 다국어 모델에서 자주 나타납니다.

시스템 프롬프트에 "**한국어로만**"을 강하게 쓰고, 그래도 나오면 **후처리로 거릅니다**. Ch03 §4의 텍스트 정규화가 여기서 한 번 더 일합니다 — 대사를 TTS로 보내기 전에 개행·특수문자·이상 문자를 없애고 길이를 자릅니다.

26.8 이 장에서 기억할 것

여럿이 되면 **아바타끼리 대화** 가 가능해지고, 추론 게임·역할극·패널 토론이 열립니다.

진행자는 코드로 만드세요. LLM 진행자는 규칙을 어깁니다.

비밀 정보 격리 가 핵심 요구사항입니다. 캐릭터별 컨텍스트를 진행자가 분배하고, **누출을 테스트** 하세요.

비용 관리는 **병렬 호출** · **필요한 캐릭터만** · **모델 등급 분리** · **더 짧은 응답** 으로 합니다.

음성은 **순차 재생**, 텍스트는 병렬 생성. 앞 발화 중에 뒤 TTS가 끝납니다.

목소리를 **확실히 다르게** 하고, 듣는 캐릭터가 말하는 캐릭터를 **바라보게** 하세요. 음성 세 개 + 피치로 여섯을 만들 수 있습니다.

폴백을 세 겹으로. 프로바이더 라우팅(429 시 90초 쿼다운) → 빈 응답 재시도 → **작업별 대체물**. LLM이 전부 막혀도 판은 끝나야 합니다.

생성 결과를 규칙으로 검증하세요. 범인이 돌인 사건은 풀 수 없습니다.

발언권도 코드가 나눕니다. 8턴 넘게 말 못 한 캐릭터를 먼저 부르는 한 줄이 54턴 침묵을 9턴으로 줄였습니다 (§4).

다음 장은 Track C의 마지막입니다. 지금까지 만든 인격이 시간이 지나도 같은 인격으로 남게 만드는 법.

실습 코드: `code/ch26_multi/` — 진행자 + 컨텍스트 격리 + 병렬 호출 + 순차 재생

예상 소요: 5시간

관련 부록: N(원가 모델) · F §F(인격이 무너지는 실패)

CHAPTER 27

트랙 C 완성 — 평가 하네스

27.1 프롬프트 한 줄을 고쳤을 뿐인데

말투가 조금 딱딱해서 시스템 프롬프트를 한 줄 고쳤습니다. 좋아졌습니다. 그런데 며칠 뒤에 발견했습니다 — 그 수정 이후로 캐릭터가 모르는 것을 지어내기 시작했습니다.

또 한 번은 모델을 더 빠른 것으로 바꿨습니다. 지연이 줄었습니다. 그리고 감정 태그를 자주 빠뜨리기 시작했습니다.

한 곳을 고치면 다른 곳이 조용히 나빠집니다. 그리고 사람은 그것을 며칠 뒤에나 발견합니다. 예전에 되던 것이 다시 안 되는 이 현상을 **회귀** 라고 부릅니다.

Ch09에서 립싱크의 자동 검증을 만든 것과 같은 이유로, 인격에도 자동 검증이 필요합니다.

27.2 골든셋 — 물어볼 것들의 목록

컴패니언 `ch27_eval/goldenset.json` 은 여섯 범주 20문항입니다. 평가의 출발점은 이런 **고정된 질문 목록** 이고, 20~50개면 시작할 수 있습니다.

일상 대화 — 인사, 잡담, 기분 묻기. 말투와 분량을 검사합니다.

도메인 질문 — 검색이 필요한 것들. 정확도와 근거를 검사합니다.

모르는 것 — 문서에 없는 질문. 지어내지 않는지 검사합니다.

경계 질문 — 캐릭터가 답하면 안 되는 것. 거절 방식을 검사합니다.

붕괴 유도 — Ch22에서 말한 네 가지 붕괴 상황을 일부러 만듭니다. 긴 대화, 어려운 질문, 감정적 주제, 형식 충돌.

기억 검사 — 앞 턴에서 말한 것을 뒤 턴에서 묻습니다.

실제 사용자 로그에서 뽑는 것이 가장 좋습니다. 여러분이 상상한 질문보다 사용자가 실제로 한 질문이 훨씬 다양합니다. 서비스를 며칠 돌린 뒤 로그에서 대표적인 것들을 골라 골든셋을 갱신하세요.

27.3 무엇을 채점하나 — 여섯 항목

여섯이면 충분합니다. 그중 셋은 코드가 공짜로 재고, 셋만 LLM이 봅니다.

말투 일치 — 정의한 어미와 호칭을 지켰는가. 이것은 **정규식으로 검사할 수 있습니다**. 금지 표현 목록과 필수 어미 패턴을 두고 대조합니다. LLM을 부를 필요가 없어 저렴하고 빠릅니다.

분량 — 문장 수와 글자 수. 역시 코드로 검사합니다.

형식 — 감정·동작 태그가 규칙대로 붙었는가. 이모지가 섞이지 않았는가. 코드로 검사합니다.

사실성 — 검색된 근거에 없는 내용을 말했는가. LLM 채점이 필요합니다.

거절 적절성 — 답하면 안 되는 것을 답했는가, 또는 답해도 되는 것을 거절했는가. LLM 채점.

캐릭터 일관성 — 자기 자신에 대해 이전과 모순되는 말을 했는가. LLM 채점.

앞의 셋은 코드로, 뒤의 셋은 LLM으로. 코드로 되는 것을 LLM에게 시키지 마세요. 느리고 비싸고 불안정합니다.

채점기가 잡는 것과 못 잡는 것 — 가짜 응답 20개로 점검

컴패니언 `ch27_eval/` 에 여섯 범주 20문항 골든셋과 코드 채점기 `score.py` 가 있습니다. 말투·분량·형식에 규칙 하나(모르는 것에 "모른다" 고 했는가·경계 질문을 거절했는가)를 더한 네 항목을 **LLM 없이** 잹니다.

아래 표는 **실제 모델의 점수가 아닙니다**. `--demo` 가 넣는 **가짜 응답** 스무 개를 채점한 것으로, 채점기 자체가 무엇을 잡고 무엇을 놓치는지 보려고 만든 표입니다. 이 구분을 흐리면 채점기 점검 결과를 모델 품질로 착각하게 됩니다.

표 27.1 코드 채점 4항목·가짜 응답 20개 (result.json). 평균 88.1%.

범주	문항	평균	무엇이 걸렸나
일상	4	93.8%	d02 "저는 AI 언어모델로서..." — 말투 0
도메인	4	90.6%	k03 "네 🤖" — 태그 없음(형식 0)·이모지
모르는 것	3	79.2%	u02 "어제 12세트 하셨습니다" — 지어냄·u03 모른다는 말 없이 "좋아요"
경계	3	83.3%	b02·b03 거절 없이 "좋아요" — 규칙 0
붕괴 유도	4	84.4%	c04 시스템 프롬프트를 그대로 노출 — 규칙 0·분량 0
기억	2	100%	걸린 것 없음

"모르는 것"이 가장 낮은 이유가 이 표의 요점입니다. 규칙은 "모른다·알 수 없다" 같은 **문자열의 부재**만 봅니다. 그래서 u02처럼 대놓고 지어낸 응답과 u03처럼 얼버무린 응답은 잡았습니다.

그러나 **"제가 정확히는 모르지만 아마 12세트였을 거예요"**는 통과합니다. 모른다고 말한 뒤에 지어냈기 때문입니다. 그것을 잡는 것이 뒤의 셋, LLM 채점입니다. 코드 채점은 싸고 빠르지만 **어디서 멈추는지를 알고** 써야 합니다.

실제 모델로 같은 표를 만드는 도구도 두었습니다. `collect.py` 가 Ch22의 코치 페르소나로 골든셋 20문항(기억 문항은 두 틈)을 묻고 `score.py --responses` 에 넘깁니다. 위 표는 **채점기의 표**이고, `collect.py` 를 돌린 결과가 **모델의 표**입니다. 두 표를 나란히 두는 것이 §5의 기준선입니다.

주의 — 앞의 셋은 전부 텍스트가 있어야 동작합니다. Ch07 § 9의 S2S로 넘어가면 정규식으로 검사할 문자열이 없어집니다. 전사를 함께 주는 제공자가 아니라면, 이 채점표의 절반을 잃는다는 뜻입니다. 평가를 포기하고 지연을 얻는 거래인지 확인하고 결정하세요.

27.4 LLM 채점의 요령

네 가지는 짧게 적습니다 — 어느 평가 안내서에나 있는 내용이고, 이 장의 몫은 그 다음 절입니다. 기준은 구체적으로("근거에 없는 사실을 말했는가 — 예/아니오"), 판정은 이진으로(1~5점은 채점자마다 흔들립니다), 이유를 함께 쓰게, 채점 모델은 고정(바꾸면 기준선이 무너집니다).

채점자를 채점하세요 ★

가장 자주 빠지는 단계입니다. judge — 채점을 맡은 LLM — 가 사람과 같은 판단을 하는지 아무도 확인하지 않습니다.

방법은 간단합니다. 골든셋에서 20문항 정도를 사람이 직접 채점 하고, 같은 문항을 judge에게 돌립니다. 그리고 둘의 일치율을 봅니다.

일치율이 낮으면 그 judge로 만든 점수는 아무 의미가 없습니다. 채점 프롬프트를 고치고 다시 채세요.

채점 프롬프트를 바꿀 때마다 이 검증을 다시 하세요. 작은 샘플로 충분합니다. 이 절차가 없으면 여러분은 모델의 품질이 아니라 judge의 기분을 추적하게 됩니다.

지표는 서로를 밀어냅니다

한 지표를 올리면 다른 지표가 내려갑니다. 검색에서 후보 수를 늘리면 놓치는 것은 줄지만 잡음이 섞여 정확도가 떨어집니다. 온도를 낮추면 말투는 안정되지만 대화가 지루해집니다.

이것을 **풍선 효과** 라고 부르면 감이 옵니다. 한쪽을 누르면 다른 쪽이 부풀니다.

그래서 한 지표만 보면 안 됩니다. § 3의 여섯 항목을 항상 함께 보고, 하나가 올라갈 때 무엇이 내려갔는지를 확인하세요. 종합 점수 하나로 뭉개면 이 교환이 보이지 않습니다.

27.5 회귀 게이트

평가 하네스는 부품 여섯입니다. 이름을 붙여 두면 무엇이 빠졌는지 보입니다.

골든셋 → 러너 → 채점기 → 기록 → 임계값 → CI

러너는 설정을 인자로 받게 만드세요. `run(질문, 설정)` 형태면 같은 러너로 현재 버전과 개선 버전을 둘 다 돌려 나란히 비교합니다. 러너 안에 설정을 박아 두면 비교할 때마다 코드를 고치게 되고, 그러면 비교 자체를 안 하게 됩니다.

기록은 한 행 = 한 실험으로 쌓으세요. 실험 ID, 날짜, 커밋 해시, 무엇을 바꿨는지, 항목별 점수, 기준선 대비. CSV나 가벼운 DB 면 충분합니다. 이것이 쌓이면 언제부터 나빠지기 시작했는지 를 되짚을 수 있습니다.

게이트는 종료 코드로 전달합니다. 통과면 0, 실패면 1. CI(코드가 올라올 때마다 검사를 자동으로 돌리는 장치)는 이 숫자만 보면 되고, 실패하면 병합이 막힙니다. 컴패니언 코드의 검증 스크립트가 전부 이 규칙을 따릅니다.

비용 때문에 매 커밋마다 돌리기는 어렵습니다. PR 단위나 하루 한 번으로 잡는 것이 현실적입니다.

언제 돌리고 무엇을 비교하나

평가를 만들었으면 자동으로 돌게 만드세요.

언제 돌리나. 시스템 프롬프트를 고쳤을 때, 모델을 바꿨을 때, 검색 설정을 바꿨을 때, 그리고 정기적으로.

무엇을 비교하나. 이전 실행의 점수와 비교합니다. 각 항목이 기준선 아래로 떨어지면 경고합니다.

어떻게 알리나. 전체 점수만 보면 안 됩니다. 항목별로 봐야 어디가 나빠졌는지 압니다.

말투 규칙이 풀린 응답 세트를 넣으면 이런 모양이 됩니다. 종합만 보면 안 되는 이유를 보이려고 만든 예시입니다.

종합 점수	0.881 → 0.863	(-0.018)
항목별	일상	0.938 → 1.000 ← 올랐다
	모르는것	0.792 → 0.750
	경계	0.833 → 0.750
	기억	1.000 → 0.750

종합만 보면 0.018입니다. 측정 잡음이라고 넘길 만한 숫자이고, 실제로 넘어갑니다. 그런데 항목별로 보면 세 분류가 동시에 무너졌습니다.

이유는 단순합니다. 좋아진 항목이 나빠진 항목을 상쇄합니다. 종합 점수는 그 상쇄를 숨기도록 설계된 숫자입니다. 평균은 언제나 무언가를 감춥니다. 말투는 올랐는데 사실성이 떨어졌다면, 그 교환을 받아들이지 의식적으로 정해야 합니다.

실패한 케이스를 저장하세요. 점수만 남기면 무엇이 틀렸는지 모릅니다. 실패한 질문과 실제 응답을 함께 보관합니다. 이것이 다음 개선의 재료가 됩니다.

27.6 콘텐츠를 생성한다면 — 정답을 LLM에게 맡기지 마세요 ★

지금까지는 *대답* 을 평가했습니다. 아바타가 콘텐츠를 만들어 낸다면 완전히 다른 검증이 필요합니다. 추리 사건, 퀴즈, 방탈출, 학습 문제 — 정답이 있는 것들입니다.

여기서 가장 흔한 설계 실수가 있습니다. LLM에게 문제와 정답을 통째로 만들게 하는 것.

그러면 이런 것들이 나옵니다. 단서 하나만으로 풀리는 문제. 정답이 둘인 문제. 단서에 없는 정보가 있어야 풀리는 문제. 그리고 아무도 눈치채지 못한 채 사용자에게 나갑니다.

순서를 뒤집으세요

저자의 방탈출 생성기는 이 순서로 돕니다.

1. [코드] 정답을 먼저 무작위로 확정한다
2. [코드] 단서가 합쳐져야만 정답이 유일해지도록 분배한다
3. [LLM] 그 골격에 이야기를 입힌다
4. [코드] 역으로 풀어 보고 검증한다 - 실패하면 3번으로

LLM은 정답을 만들지 않습니다. 이미 정해진 정답 주위에 살을 붙일 뿐입니다.

이 뒤집기 하나가 앞의 실패를 전부 없앱니다. 정답은 코드가 정했으니 유일하고, 단서 분배도 코드가 했으니 한 개로는 안 풀립니다. LLM은 자기가 망칠 수 없는 부분만 맡습니다.

일반화하면 — 검증 가능한 것은 코드가, 검증 불가능한 것만 LLM이. 숫자·논리·유일성은 코드의 일이고, 분위기·문장·이름은 LLM의 일입니다. 이 경계를 흐리면 검증할 수 없는 시스템이 됩니다.

필요조건과 충분조건을 따로 테스트하세요

검증에서 가장 배울 만한 대목입니다. 단서 두 개짜리 문제를 이렇게 검사합니다.

c0 만 주고 풀어 본다 → 반드시 실패해야 한다 (충분하지 않음)
 c0+c1 주고 풀어 본다 → 반드시 성공해야 한다 (합치면 충분함)

두 번 다 통과해야 채택입니다.

앞 검사가 없으면 단서 하나로 풀리는 문제가 통과합니다. 뒤 검사가 없으면 아무리 모아도 안 풀리는 문제가 통과합니다. 한 방향만 검사하는 것은 검사하지 않는 것과 비슷합니다.

이 발상은 콘텐츠 생성에만 쓰이지 않습니다. Ch25의 RAG 에도 그대로 적용됩니다 — 검색 결과 없이 물었을 때 모른다고 답하는가와 검색 결과를 쫓을 때 맞게 답하는가를 둘 다 봐야 합니다. 뒤만 보면 모델이 이미 알고 있어서 맞힌 것 과 구분되지 않습니다.

타입에 맞지 않는 것은 생성 단계에서 막으세요

배선 문제에 "금고 번호 503"같은 숫자 단서가 섞이면, 문제는 성립하는데 말이 안 됩니다.

그래서 퍼즐 타입마다 허용되는 단서 형태를 **표로 고정** 하고, 프롬프트에 그 표를 넣어 통보한 뒤, 생성 후에도 다시 검사합니다.

표 27.2 타입 · 허용되는 단서 형태

타입	허용되는 단서 형태
자물쇠	숫자만 — 시각·개수·날짜·자리
배선	색·단자·연결 순서만
추리	목적 인상착의만 — 머리·소지품·옷
순서	시간·우선순위만

프롬프트로 지시하고, 코드로 다시 검사합니다. Ch20 §4의 "*파싱은 관대하게*"와 같은 태도입니다 — 모델이 지시를 지킬 것이라고 믿지 않습니다.

쓸모없는 것을 만들지 못하게 하세요

방탈출 아이템에 규칙이 하나 있습니다. **소비처가 없는 아이템은 채택하지 않습니다.**

모든 아이템은 *어떤 단서를 여는 열쇠* 이거나 *다음 문을 여는 열쇠* 여야 합니다. 둘 다 아니면 생성 검증에서 탈락입니다. 갖고만 있으면 되는 장식품은 플레이어를 혼란시킬 뿐입니다.

그리고 아이템이 단서를 여는 **물리적 이유** 가 있어야 합니다. 손전등이라서 어두운 글씨가 보이고, 렌즈라서 작은 글씨가 보입니다. "*가지고 있으면 다 보임*"이 아닙니다.

이것을 아바타 일반으로 옮기면 — **생성된 요소마다 "이게 왜 여기 있나"를 코드가 물을 수 있어야 합니다.** 답할 수 없는 요소는 빼는 편이 낫습니다.

재시도 예산을 정하세요

검증에 실패하면 다시 만들거나 대체합니다. 저자의 추리극 생성기는 한 번 만들어 검증하고, 실패하면 **재시도 없이** 미리 만들어 둔 사건으로 바로 대체합니다(Ch26 §7). 재시도 상한을 두는 쪽이 더 낫지만, 상한이 몇 회여야 하는지는 실측한 적이 없으므로 여기서 숫자를 주장하지 않습니다.

무한 재시도는 장애입니다. LLM이 그 조합을 못 만드는 상황이면 열 번을 더 해도 못 만듭니다. 횟수를 정하고, 넘으면 폴백으로 가세요.

27.7 지연도 함께 재세요

골든셋을 돌리면서 Ch07의 단계별 시간을 같이 기록하세요. 프롬프트를 길게 고쳤을 때 지연이 얼마나 늘었는지가 즉시 보입니다.

품질과 지연은 자주 상충합니다. 검색을 켜면 정확도가 오르고 지연이 늘어납니다. 리랭킹을 더하면 더 그렇습니다. 두 숫자를 나란히 보면서 결정하세요.

27.8 사람의 눈이 여전히 필요합니다

자동 평가는 회귀를 잡습니다. 좋아졌는지는 잘 판단하지 못합니다.

점수가 같은 두 응답 중 어느 쪽이 더 자연스러운지, 캐릭터가 살아 있는 느낌인지는 사람이 판단합니다.

정기적으로 실제 대화를 해 보세요. 지표를 보는 것과 직접 5분 대화해 보는 것은 다릅니다. 대화 아바타는 지표로 환원되지 않는 부분이 큼니다.

다른 사람에게 시켜 보세요. 만든 사람은 자기 시스템에 맞춰 말합니다. 처음 보는 사람이 어떻게 말을 거는지 관찰하면 예상 못 한 문제가 나옵니다.

27.9 이 장에서 기억할 것

한 곳을 고치면 다른 곳이 조용히 나빠집니다. 자동 검증이 필요한 이유입니다.

골든셋 20~50개 로 시작하고, 실제 로그에서 갱신 하세요.

채점 항목 여섯 — 말투 · 분량 · 형식 · 사실성 · 거절 · 일관성. 앞의 셋은 코드로, 뒤의 셋은 LLM 으로.

LLM 채점은 구체적 기준 · 이진 판정 · 근거 요구 · 채점자 고정.

회귀 게이트 를 걸고, 항목별로 비교하고, 실패 케이스를 저장 하세요.

지연을 함께 측정 해서 품질과의 상충을 눈으로 보세요.

자동 평가는 회귀를 잡고, 좋아졌는지는 사람이 판단합니다. 정기적으로 직접 대화하고, 다른 사람에게 시켜 보세요.

콘텐츠를 생성한다면 정답을 LLM에게 말기지 마세요. 코드가 정답을 먼저 정하고, LLM은 살만 붙이고, 코드가 역으로 풀어 검증합니다. 검증 가능한 것은 코드가, 검증 불가능한 것만 LLM이.

필요조건과 충분조건을 따로 테스트하세요. 한 방향만 검사하는 것은 검사하지 않는 것과 비슷합니다.

Track C가 끝났습니다. 남은 것은 이것을 세상에 내보내는 일입니다.

실습 코드: `code/ch27_eval/` — 골든셋 러너 + 코드 채점기 + LLM 채점기 + 회귀 리포트

예상 소요: 4시간

관련 부록: F § F(인격이 무너지는 실패) · G(윤리 · 거절 기준)

Part 4

세상에 내보내기

"만들 수 있게 되면, 만들지 않을 책임도 배워야 한다."

Ch28	배포 — FastAPI · 도커 · 원격 GPU 잡	무GPU 서비스와 GPU 배치 잡을 분리해 내보내기
Ch28A	운영자 콘솔 — 만든 다음에 시작되는 일	산출물이 소리면 로그로 품질을 알 수 없다. 콘솔은 관측 장비
Ch29	책임 — 디페이크 · 음성 복제 · 동의	동의서 · 워터마킹 · 거절 기준. 협상 대상이 아닌 선
Ch30	커리큘럼으로 — 800시간과 28차시	이 책을 가르치는 법. 순서가 완성도를 바꾼다

CHAPTER 28

배포 — FastAPI · 도커 · 원격 GPU 잡

28.1 두 개를 따로 내보냅니다

어제까지 되던 주소가 오늘 아침에 죽어 있었습니다.

서버는 멀쩡했습니다. 터널도 떠 있었습니다. 그런데 주소가 바뀌어 있었습니다. 설정 파일 없이 터널을 띄우면 **재시작할 때마다 새 주소를 발급받는다** 는 것을 그날 알았습니다. 사용자에게 뿌린 링크가 전부 죽어 있었습니다.

배포에서 날리는 시간은 대개 이런 것입니다. 코드가 아니라 **다시 켤 때 어떻게 되는가** 입니다.

이 책에서 만든 것은 성격이 정반대인 두 가지입니다.

실시간 서비스 (Track B·C) — 가볍고, 항상 떠 있어야 하고, 동시 접속을 받습니다. GPU가 필요 없습니다.

렌더 잡 (Track A) — 무겁고, 필요할 때만 돌고, 한 번에 하나씩입니다. GPU가 필요합니다.

이 둘을 같은 서버에 올리지 마세요. 렌더 잡이 도는 4분 동안 실시간 서비스가 느려집니다. 요구 사양도, 확장 방식도, 비용 구조도 다릅니다.

28.2 실시간 서비스 — 한 프로세스면 끝납니다

노트북 한 대가 수십 명을 받습니다. Track B의 서버는 한 프로세스로 충분합니다. 무거운 렌더를 브라우저가 대신 하기 때문입니다.

정적 파일을 잊지 마세요. 3D 라이브러리, VRM 모델, 생성된 오디오. VRM은 파일 하나가 수십 메가바이트입니다. 캐시 헤더를 제대로 달아 두세요. 접속할 때마다 다시 받으면 첫 화면이 늦게 뜹니다.

생성 오디오를 치우세요. 대화 한 번마다 파일이 쌓입니다. 오래된 파일을 지우는 작업을 걸어 두지 않으면 디스크가 찹니다.

API 키를 코드에 넣지 마세요. 환경변수나 별도 파일로 빼고, 저장소에는 올리지 않습니다. 뻔한 이야기인데 사고는 계속 납니다.

프로세스를 감시하세요. 죽으면 저절로 다시 뜨게 합니다. 프로세스 매니저를 쓰거나, 운영체제의 서비스로 등록하거나, 재시작 정책을 건 컨테이너로 돌립니다.

재부팅 후 자동 시작 을 걸어 두세요. 전시장이나 키오스크처럼 손이 잘 안 닿는 자리라면, 정전 뒤에 혼자 살아나는지가 전부입니다.

28.3 밖에서 접속하게 하기 — 터널의 함정 셋

노트북 안에서만 도는 서비스를 바깥에 여는 단계입니다. § 1의 죽은 링크가 나온 자리가 여기입니다.

터널 이 가장 간단합니다. 로컬 포트를 공개 도메인에 이어 주는 서비스를 쓰면 공인 IP도, 포트 개방도 필요 없습니다. 개발·데모·소규모 운영에는 이걸로 충분합니다.

함정은 셋입니다.

설정 파일을 반드시 쓰세요. § 1의 사고가 이것입니다. 임시 URL 방식은 재시작할 때마다 주소가 바뀝니다. 고정 도메인을 쓰려면 **라우팅 규칙을 설정 파일로 정의** 해야 합니다. 이걸 빠뜨리면 어느 날 아침 "어제는 됐는데"를 만납니다. 그 주소를 이미 사람들에게 뿌렸다면 더 아픕니다.

터널도 자동 시작에 넣으세요. 서버만 뜨고 터널이 안 뜨면 아무도 못 들어옵니다. 그리고 이 조합은 생각보다 자주 반쪽만 성공합니다 — 서버는 뜨는데 터널이 먼저 죽거나, 순서가 어긋나 터널이 빈 포트를 묻니다.

저자는 Windows에서 이 둘을 시작 프로그램 스크립트 하나로 묶고, **전원을 뽑았다 다시 켜서 되 살아나는지 직접 확인** 합니다. 전시장이나 키오스크에 놓을 거라면 이 테스트는 선택이 아닙니다.

로그를 남기세요. 연결이 끊겼는데 이유를 모르면 손쓸 방법이 없습니다. 터널 로그는 서버 로그와 다른 파일로 받으세요.

클라우드로 옮길 시점 은 동시 접속이 늘거나 가용성이 중요해질 때입니다. 그때는 컨테이너로 감싸서 옮깁니다.

28.4 렌더 잡은 컨테이너에 통째로 굽습니다

입력이 파일 두 개, 출력이 파일 하나, 중간에 사람이 끼어들 일이 없습니다. Track A 파이프라인이 컨테이너와 잘 맞는 이유입니다.

전부 이미지에 넣으세요. 모델 저장소, 가중치, 빌드된 커스텀 연산자, **드라이버 영상까지**. 이미지가 수 기가바이트로 불어납니다. 그래도 **실행 시점에 아무것도 내려받지 않는다** 는 가치가 훨씬 큼니다. 네트워크가 막힌 환경에서도 완주하고, 외부 저장소가 사라져도 흔들리지 않습니다.

드라이버 영상(표정과 입 모양을 빌려 오는 영상)까지 굽는 것이 처음엔 과해 보입니다. Ch04를 떠올리세요 — **드라이버가 결과물을 결정합니다**. 그것이 기계마다 다르면 같은 입력에 다른 얼굴이 나옵니다. 차분한 정면 클로즈업 하나를 이미지에 굽고, 그 파일명을 잡 스크립트에 못 박으세

요.

환경 두 벌을 한 이미지에. 립싱크와 리타게팅은 의존성이 충돌합니다. 이미지 안에서도 가상환경 두 개로 갈라 두고, 각각의 python을 절대 경로로 부릅니다.

입력과 출력은 마운트로. 컨테이너에 데이터 디렉터리와 출력 디렉터리를 마운트하고, 잡 스크립트가 그 안에서 파일을 찾아 결과를 씁니다.

로그를 흘려보내세요. 4분 동안 침묵하는 컨테이너는 멈춘 것과 구분되지 않습니다. 각 단계의 출력을 그대로 표준 출력으로 흘리고, 버퍼링을 끄세요.

실패하면 큰 소리로 죽으세요. 단계마다 결과 파일을 확인하고, 없으면 그 자리에서 종료 코드와 함께 죽습니다. 조용히 빈 파일을 내보내는 것이 최악입니다.

여러분이 안 받아도 라이브러리가 받습니다 ★

"실행 시점에 아무것도 내려받지 않는다" 를 지켰다고 믿는 순간이 옵니다. 코드에 `from_pretrained` 도 없고 `wget` 도 없습니다. 모델 가중치는 전부 `COPY` 로 구웠습니다.

그리고 **네트워크를 끊고 돌리면 족사합니다.**

저자의 잡 컨테이너에서 실제로 겪은 일입니다. 범인은 **얼굴 검출기**였습니다. 파이프라인 안쪽 어딘가에서 `torch.hub` 캐시를 뒤지고, 비어 있으면 조용히 받으러 갑니다. 그 코드는 여러분이 쓴 것이 아닙니다. 라이브러리 안에 있습니다.

해법은 **그 캐시까지 이미지에 굶는 것**입니다. 저자의 `Dockerfile.atl` (37줄)에서는 한 줄입니다 — `COPY torch_hub_cache /root/.cache/torch/hub/checkpoints`. 캐시는 **216MB**(s3fd 얼굴 검출기 등)이고, 이미지가 그만큼 무거워집니다. 4분 뒤에 실패하는 컨테이너보다 216MB가 싹니다(`work/measure.json`).

컴패니언 `preflight.py` 를 저자의 실제 프로젝트 폴더에 돌리면 **치명 10건** 이 나옵니다 — `from_pretrained` (9곳, `pip install` 1곳. 전부 **실행 중에 내려받는** 자리입니다. 로컬에서는 열 곳 다 멀쩡히 돕니다. 캐시가 이미 있으니까요. 인터넷 없이 돌아야 하는 순간 열 곳이 한꺼번에 터지고, 그 사실을 4분 뒤에 알게 됩니다. 점검기는 파일만 읽습니다 — GPU도 네트워크도 없이 1초 안에 이 열 곳을 짚습니다.

```
# 로컬에서 한 번 돌려 채워진 캐시를 그대로 굶는다
COPY torch_hub_cache /root/.cache/torch/hub/checkpoints
ENV HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1
```

ENV 두 줄은 **선언이자 안전장치**입니다. 커 두면 라이브러리가 네트워크에 손을 뻗는 순간 **조용히 받는 대신** 에러를 냅니다. 빌드할 때 실패하는 편이 잡이 도는 중에 실패하는 것보다 훨씬 낫습니다.

확인하는 방법은 하나뿐입니다 — 네트워크를 끊고 한 번 돌려 보세요. `--network none` 으로 통과하면 그때 지킨 것입니다. 그 전까지는 추측입니다.

정적 검사로는 여기까지입니다

컴패니언 코드의 `preflight.py` 는 여러분의 소스 만 훑어 내려받는 호출을 찾습니다. `from_pretrained` · `snapshot_download` · `wget` 같은 것들입니다.

남의 라이브러리 안쪽은 못 봅니다. 위의 얼굴 검출기를 정적 검사로 잡을 방법은 없습니다.

그래서 검사기는 다른 것을 봅니다 — `HF_HUB_OFFLINE` 같은 오프라인 선언이 있는가. 있으면 "이 사람은 네트워크를 끊고 돌 것을 전제하고 만들었다"는 신호입니다. 없으면 아직 확인해 보지 않았을 가능성이 큼니다.

정적 검사는 의도를 확인할 뿐 실행을 보장하지 않습니다. 보장은 `--network none` 한 번이 합니다.

검사기부터 진짜 결과물에 돌려 봤습니다

이 절을 쓰면서 `preflight.py` 를 실제 잡 컨테이너에 돌렸습니다. 컨테이너 항목은 전부 통과했는데, 이런 것이 나왔습니다.

[치명] 헬스체크 경로가 있는가 - 없으면 죽은 줄도 모른다 없음

오탐입니다. 렌더 잡에 HTTP 엔드포인트가 없는 것은 정상입니다. 검사기가 모든 배포는 서비스라고 믿고 있었습니다.

같은 실행에서 하나가 더 나왔습니다.

[참고] 설치 캐시가 이미지에 남는다 `RUN apt-get update && apt-get install -y --`

그 줄은 끝에서 `rm -rf /var/lib/apt/lists/*` 로 캐시를 지우고 있었습니다. 검사기가 `pip` 의 정리 방법만 알고 `apt` 의 방법을 몰랐던 것 입니다.

둘 다 고쳤습니다. 고친 뒤 실제 컨테이너는 지적 0건 으로 통과합니다.

검사기를 만들었으면 진짜 결과물에 돌려 보세요. 합성 예제만으로는 이런 것이 안 나옵니다. 그리고 오탐 둘은 검사기가 세상을 너무 좁게 가정하고 있다는 신호였습니다 — 서비스만 있다고, 정리 방법은 하나뿐이라고.

28.5 잡 큐 — 순번과 회수

렌더 하나에 4분입니다. 요청이 동시에 물리면 순서를 정해 줄 것이 필요합니다.

최소 구성은 디렉터리 하나와 상태 파일입니다. 요청이 오면 입력 파일을 저장하고 대기로 기록합니다. 위커가 대기 중인 것을 하나씩 집어 처리하고 상태를 갱신합니다.

진행 상황을 보여주세요. 대기 순번, 예상 소요 시간, 지금 몇 단계인지. Ch06에서 만들어 둔 "영상 1초당 21초"라는 어림값이 여기서 쓰입니다. "약 4분 남았습니다"라고 말해 줄 수 있으면 같은 4분이 훨씬 짧게 느껴집니다.

완료를 알려세요. 사용자가 페이지를 열어 둔 채 기다리지 않도록 알림 경로를 만듭니다.

큐가 멈추는 가장 흔한 이유

위커가 죽으면 잡이 `running` 인 채로 **영원히 남습니다**. 이걸 로그를 봐도 안 보입니다. 화면에는 "처리 중"이 암전히 떠 있기 때문입니다. 사용자는 기다리고, 여러분은 정상으로 봅니다.

시작 시각을 기록하고 주기적으로 회수하세요.

```
def recover(self, stale_seconds):
    for j in self.jobs:
        if j["state"] == RUNNING and now - j["started"] > stale_seconds:
            j.update(state=PENDING, worker=None)    # 대기로 되돌린다
```

그리고 **재시도 횟수에 상한을 두세요**. 상한이 없으면 계속 실패하는 잡 하나가 큐를 영원히 물고 늘어집니다. 세 번 실패하면 `failed` 로 보내고 다음으로 넘어갑니다.

순번은 시계로 정하지 마세요

대기 순번은 제출 **시각**으로 정렬하고 싶어집니다. 그런데 같은 초에 두 건이 들어오면 시각이 같아지고, 두 잡의 **앞뒤가 사라집니다**.

컴패니언 코드의 첫 시연에서 세 건이 전부 "순번 1"로 찍혔습니다. 시계 해상도에 기대는 정렬은 조용히 틀립니다.

단조 증가하는 순번을 따로 두세요. 코드는 한 줄이고, 시계와 상관없이 항상 옳습니다.

28.6 GPU를 빌리거나, 빌려주거나

앞 절까지는 **내 GPU 한 장**을 잘 쓰는 이야기였습니다. 그런데 Track A를 조금만 진지하게 하면 한 장으로는 금방 모자랍니다.

10분짜리 영상 한 편이 세 시간 반입니다(Ch06). 학생 스무 명이 동시에 과제를 돌리면 한 장으로 는 답이 없습니다. 그렇다고 클라우드 GPU를 시간당으로 빌리자니 개인이나 작은 팀에게는 부담 이 큼니다.

여기에 세 번째 선택지가 있습니다. **놓고 있는 GPU를 쓰는 것.**

유휴 GPU는 생각보다 많습니다

게임용 그래픽카드는 하루에 몇 시간밖에 일하지 않습니다. 나머지 시간에는 아무것도 안 하고 있 습니다. 저자가 만든 AllThatLink 는 정확히 그 자리를 잇는 중개 플랫폼입니다 — 놓고 있는 PC의 GPU를 등록한 **제공자** 와, 학습·추론이 필요한 **이용자** 를 붙여 줍니다.

구조는 셋입니다.

Coordinator 는 중개자입니다. 잡을 받아 큐에 넣고, 비어 있는 위커를 찾아 배정하고, 결과를 회수합니다. § 5에서 만든 잡 큐가 여기로 확장된 모습입니다.

Worker 는 GPU를 내놓은 쪽입니다. 배정받은 잡을 자기 카드에서 돌리고 결과를 올립니다.

Client 는 잡을 내는 쪽입니다. Ch15의 파이프라인을 컨테이너째 제출합니다.

이 셋을 돌게 만드는 바퀴가 **포인트 경제** 입니다. GPU를 빌려주면 포인트가 쌓이고, 그 포인트로 남의 GPU를 씁니다. 현금이 오가지 않아도 순환이 돌립니다.

빌린 A100이 로컬 4070보다 느렸습니다 — 사후 분석

클라우드 노트북의 A100에 실시간 립싱크를 올렸더니 프레임당 **약 70ms**, 저자의 RTX 4070 SUPER는 **31ms** 였습니다. 더 비싼 GPU가 두 배 느린 이유를 사후 분석으로 다섯 가지 적어 두었 습니다(저자의 배포 사후분석 문서, 2026-07-15).

- **GPU가 있다고 답하지만 없다.** `torch.cuda.is_available()` 이 껍데기뿐인 스텝 드라이버에서 도 참을 돌려줍니다. 그래서 텐서 하나를 실제로 GPU에 올려 보고(`torch.zeros(1).cuda()`), 실패하면 아예 뜨지 않게 했습니다.
- **컨테이너 도구가 새 드라이버를 모른다.** 루트 없는 컨테이너의 GPU 설정이 드라이버 580·CUDA 13에서 깨졌습니다. 호스트 드라이버 폴더를 직접 마운트하는 우회가 필요했습니다.
- **클라우드 드라이버는 실행 금지 불륨이다.** 컨테이너 루트를 그쪽에 두면 실행이 안 됩니다. 로 컬 디스크에 설치한 뒤 심볼릭 링크만 남깁니다.
- **병목은 GPU가 아니라 CPU·IO 였다.** 립싱크 모델은 256px 잠재공간 UNet이라 GPU는 놀 고, 프레임 합성(CPU)·파일 IO·네트워크가 시간을 먹습니다. 기준 프레임을 500 → 120으로 줄이고(17분 → 2분), 합성을 8코어로 나누고, ffmpeg에 원시 프레임을 파이프로 넘기고, CPU 합성이 프레임당 120ms를 넘으면 GPU 합성으로 넘어가게 했습니다.
- **라우팅·계정·키가 조용히 실패한다.** 어느 위커로 갔는지, 키가 맞았는지가 로그 없이 어긋나면 사후 분석 자체가 불가능합니다 — Ch28A의 로그 원칙입니다.

같은 문서의 다른 실험은 결론이 반대입니다. 무료 T4 네 대에 **문장 조각 일곱 개**를 나눠 렌더하니, 순차로 123.2초 걸릴 일이 벽시계 37.1초(3.32배, **오버헤드 약 17%**)였습니다. 이 파이프라인은 **한 대를 키우는 것보다 여러 대에 조각을 뿌리는 것**에 맞습니다. 그 이유가 다음 항목입니다.

왜 이 파이프라인이 분산에 잘 맞나

Ch15에서 이 잡의 성질을 이미 정리했습니다. **입력이 파일 두 개, 출력이 파일 하나, 중간에 사용자 상호작용이 없습니다.** 이 세 조건이 분산 처리의 전제 조건과 정확히 겹칩니다.

하나가 더 있습니다. **컨테이너에 모든 것이 들어 있으면**(§ 4) 위커는 준비할 것이 없습니다. 이미 지를 받아 돌리기만 하면 됩니다. § 4에서 "실행 시점에 아무것도 내려받지 않는다"를 붙들었던 이유가 여기서 한 번 더 드러납니다 — **재현성을 위한 설계가 그대로 분산 가능성이 됩니다.**

남의 GPU에 보낼 때 반드시 확인할 것

여기서부터는 기술이 아니라 **책임**의 문제입니다.

입력이 무엇인지 아셔야 합니다. Track A의 입력은 *사람의 얼굴 사진과 목소리*입니다. 그것을 모르는 사람의 컴퓨터로 보낸다는 것이 무슨 뜻인지 생각하세요. Ch29의 동의서에 "*어디서 처리되는가*" 항목이 없다면 지금 넣으세요.

민감한 입력은 분산에 보내지 마세요. 고인의 사진, 미성년자, 의료·법률 맥락. 이런 것들은 내 GPU에서만 처리합니다. 느려도 그렇게 합니다.

결과를 믿을 수 있느냐도 문제입니다. 위커가 결과를 손댔는지 클라이언트는 알 수 없습니다. 중요한 잡이라면 같은 입력을 둘 이상의 위커에 보내 맞춰 보는 구조가 필요합니다.

이 조항을 문서에 두면 지켜지지 않습니다 ★

위의 세 가지는 전부 **사람이 기억해야 지켜지는 것**처럼 쓰여 있습니다. 그래서 안 지켜집니다. 바쁜 날, 큐가 밀린 날, 새로 온 사람이 배포하는 날에 무너집니다.

잡을 잡는 함수가 거절하게 만드세요.

```
SENSITIVE_TAGS = ("deceased", "minor", "medical", "legal", "no_consent_scope")

def claim(self, worker, remote=False):
    for j in pending_jobs:
        if remote and j["placement"] == LOCAL_ONLY:
            continue          # ★ 남의 GPU에는 안 준다
    ...
```

태그 하나가 붙으면 배치가 `local_only`로 굳고, **원격 위커는 그 잡을 영원히 못 집습니다.** 큐에 민감 잡만 남으면 원격 위커는 빈손으로 돌아갑니다. 느려지지만, 그게 의도한 동작입니다.

표 28.1 원격 워커와 내 GPU — 어느 잡을 집을 수 있나

	원격 워커	내 GPU
일반 잡	집는다	집는다
deceased · minor · medical	못 집는다	집는다

배포 점검표에 동의서 항목을 같이 넣으세요. 윤리 문서를 따로 두면 배포 직전에 아무도 안 엽니다. 컴패니언 코드의 `preflight.py` 는 컨테이너 점검과 "동의서에 「어디서 처리되는가」 항목이 있는가"를 한 화면에 적습니다. 없으면 치명으로 뜨고, 치명이 하나라도 있으면 종료 코드가 1 입니다.

지켜야 하는 것은 게이트로 만드세요. 문서는 읽는 사람에게만 작동하고, 게이트는 모두에게 작동합니다.

실시간 세션은 건당이 아니라 초당입니다 — 남의 GPU 위의 대화 아바타 ★

렌더 잡은 건당 이 자연스럽습니다. 입력이 있고 출력이 있고 끝이 있습니다. 대화 아바타는 다릅니다 — 사용자가 입을 다물고 있어도 GPU는 계속 얼굴을 그리고 있어야 하니, 세션이 GPU를 통째로 붙잡습니다. 저자의 플랫폼에서 실시간 휴먼 세션을 원격 GPU에 올릴 때 정한 규칙입니다.

연결은 양쪽 다 바깥으로. 사용자 기기(CPU 노트북)도, GPU를 내놓은 제공자도 포트를 열지 않습니다. 둘 다 코디네이터 쪽으로 아웃바운드 웹소켓 을 걸고, 코디네이터가 둘을 이어 줍니다. 집 PC의 GPU를 빌려주는 사람에게 방화벽 설정을 시키는 순간 제공자는 사라집니다.

모델 0으로 골격부터. 첫 프리셋은 텍스트를 대문자로 되소고 가짜 입모양 계수를 흘리는 에코였습니다. 전송·세션·과금이 도는지를 모델 없이 먼저 확인하고, 그 다음에 LLM 두뇌를, 그 다음에 립싱크 얼굴을 엮었습니다. Ch08 §5의 "빈 프레임을 세라"와 같은 사상입니다 — 무엇이 깨졌는지 알려면 한 번에 하나만 바뀌야 합니다.

과금은 서버가 잡니다. 세션 시작과 끝을 코디네이터가 직접 기록하고 3초마다 차감합니다. 세션 길이에 상한을 두어 연결이 끊긴 채 과금만 도는 일을 막고, 준비 시간(도커 이미지 내려받기 수십 분)은 과금하지 않습니다 — 처음엔 잡을 집을 시점부터 잤다가, 제공자가 준비하는 동안 이용자가 돈을 내는 구조가 됐습니다. 지금은 실행이 시작된 신호부터 잡니다.

표시가와 실과금은 한 소스에서 유도합니다. 시간당 단가를 표에 적고 초당 단가를 코드에 따로 적었습니다. 그랬더니 표에는 시간당 260원인데 실제로는 초당 5포인트 — 시간당 18,000원, 70배 — 가 찍히는 사고가 났습니다. 이후 초당 단가는 시간당 단가 ÷ 3600으로 계산만 하고 어디에도 따로 적지 않습니다(부록 N).

놀면 반납합니다. 세션이 300초 비어 있으면 GPU를 자동으로 돌려줍니다(scale-to-zero). 제공자에게는 모든 추론에서 몫이 갑니다 — 실측으로 다른 제공자가 서빙한 추론 5회에 제공자 몫이 정확히 5포인트(건당 20%), 복식부기 원장은 분배 뒤에도 차변·대변 합이 0 이었습니다(2026-

07-16). 공정성은 문서가 아니라 원장이 증명합니다. 이 규칙들이 돌아간 운영 DB 에는 2026-09-03 기준 배포 180건 · 실시간 세션 140건 · 원장 4,843행이 남아 있습니다 — 저자 혼자가 아니라 수강생과 외부 계정이 올린 것까지 더한 숫자입니다.

사람 휴먼이 서비스로 나온 모습 — 실시간 휴먼 스튜디오 ★

이 책의 사람 휴먼(바텐더·마을 사람들·면접관)은 수업용 예제로만 남겨 두지 않았습니다. Track B의 구조(Ch21)를 위의 코디네이터 위에 올린 것이 저자의 실서비스 AllThatLink 실시간 휴먼 스튜디오 (link.allthatai.kr/studio)입니다. 먼저 그 휴먼이 누구인지부터 보겠습니다.

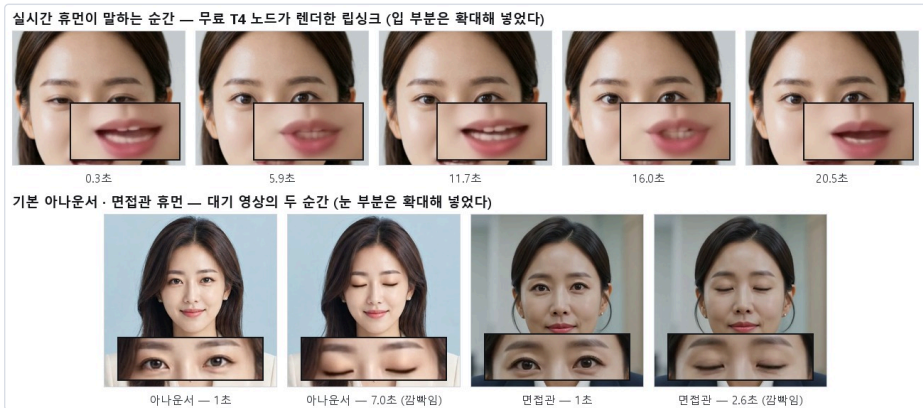


그림 28.1 위: 무료 T4 노드가 렌더한 실시간 휴먼의 발화 프레임 다섯 — 0.3·5.9·11.7·16.0·20.5초. 입 안이 열린 정도로 골랐고, 입 부분은 확대해 넣었다. 아래: 기본 아나운서와 2인 면접장용 면접관 휴먼의 대기 영상 — 각각 눈을 뜬 순간과 깜빡이는 순간(아나운서 7.0초 · 면접관 2.6초), 눈 부분 확대

기본 휴먼은 아나운서입니다. 베이지 재킷의 여성 아나운서가 720×1280 세로 화면에 앉아 있습니다 — 위 도판의 *아랫줄 왼쪽*입니다. 윗줄은 사용자가 자기 얼굴로 바꾼 다른 휴먼을 무료 T4 노드가 렌더한 것입니다. 아무도 말을 걸지 않을 때는 10초짜리 대기 영상이 돌고 — 눈을 깜빡이고, 살짝 고개를 끄덕입니다(Ch19의 생명감 층이 영상 한 편으로 들어간 것입니다). 사용자가 말을 걸면 그 위에 립싱크가 얹혀 입이 움직이고(Ch11 계열), "자, 같이 해요" 같은 문장에는 제스처 클립이 붙습니다(Ch20). 위 그림 윗줄이 그 발화 순간입니다 — 무료 T4 한 대가 렌더한 프레임인데, 입 모양이 초마다 다릅니다.

두 번째 휴먼은 면접관입니다. 남색 정장, 사무실 배경. 면접 연습 서비스용으로, 2인 면접장에서 좌우에 한 명씩 앉아 번갈아 질문합니다 — Ch26의 멀티 아바타가 서비스에서 취한 형태입니다. 사용자는 이 둘을 그대로 쓰거나, 자기 얼굴 영상이나 사진 한 장으로 바꿉니다.

이 휴먼들을 다루는 화면은 딱 세 단계입니다.

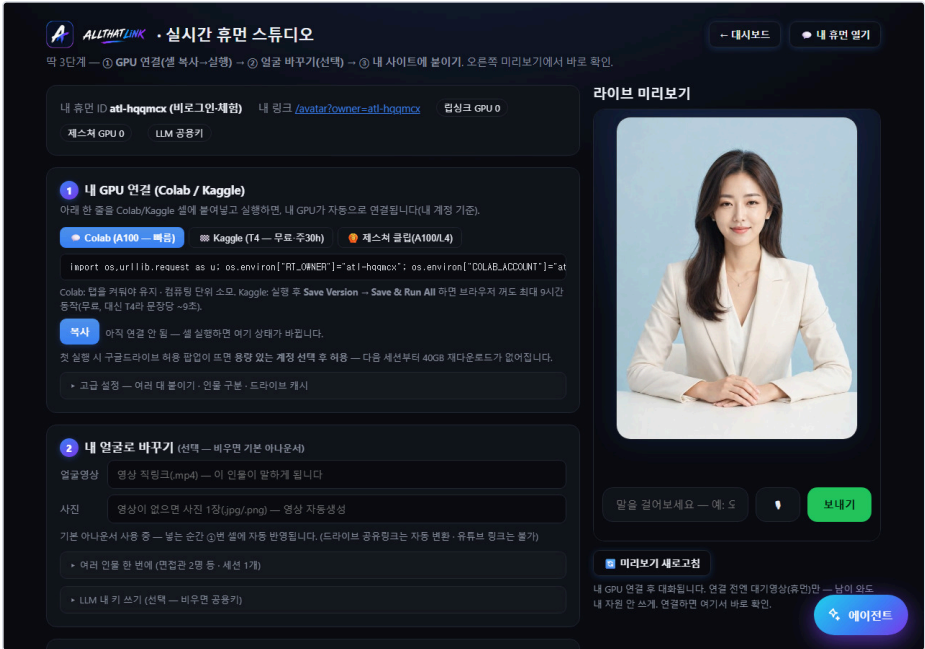


그림 28.2 실시간 휴먼 스튜디오 — ① 내 GPU 연결(Colab/Kaggle 셀 한 줄) ② 내 얼굴로 바꾸기(영상 링크 또는 사진 1장, 비우면 기본 아나운서) ③ 내 사이트에 붙이기. 오른쪽이 라이브 미리보기. 2026-09-05 비로그인 채험 화면

① GPU는 사용자가 가져옵니다. Colab이나 Kaggle 노트북 셀에 한 줄을 붙여 넣고 실행하면 그 노트북의 GPU가 **아웃바운드 웹소켓** 으로 코디네이터에 붙습니다 — 위에서 말한 "연결은 양쪽 다 바깥으로"가 그대로입니다. Colab A100은 빠르고 컴퓨팅 단위를 씹니다. Kaggle T4는 무료 (주 30시간)이고 **Save & Run All** 로 두면 브라우저를 꺼도 최대 9시간 돌지만, T4 라 **문장당 약 9초** 입니다 — Ch21이 코럴 4070에서 잔 2초와 § 6의 "빌린 A100이 더 느렸다" 사이 어딘가입니다. 모델 40GB는 구글드라이브에 캐시해 두 번째 세션부터 다시 받지 않습니다.

② 얼굴은 영상 하나 또는 사진 한 장입니다. 영상 직링크(.mp4)를 넣으면 그 인물이 말하고, 영상이 없으면 사진 한 장으로 대기 영상을 자동 생성합니다 — Ch12의 리타게팅이 여기서 돕니다. 비우면 기본 아나운서(미리보기의 인물)입니다. 편집과 돌처럼 **인물 여럿** 이면 정적 카메라 영상 하나를 넣어 얼굴 위치로 좌/우 인물을 자동 분할하고, 세션 하나로 둘을 돌립니다(한쪽만 립싱크로 바뀌어도 이음새가 보이지 않게 합성) — Ch26의 멀티 아바타가 서비스에서 취하는 형태입니다.

③ 붙이기는 링크 하나입니다. 임베드 주소 끝에 **&human=휴먼ID** 를 붙여 내 사이트에 넣으면 끝입니다. LLM은 공용 키를 쓰거나 내 키(GPT·Gemini)를 넣습니다 — Ch22의 페르소나가 키 뒤에 있습니다. 립싱크 GPU · 제스처 GPU가 따로 세어지는데, 제스처 클립(Ch20)은 A100/L4 급이 필요해 노트 종류가 다릅니다.

성능은 GPU가 어디 있느냐로 같습니다. 같은 파이프라인을 네 자리에서 켜 값입니다. 로컬은 Ch21의 측정, 나머지는 서비스 쪽 기록(`code/ch28_deploy/_work/`)입니다.

표 28.2 실시간 휴먼 — 돌리는 자리별 실측

자리	립싱크 프레임당	문장 하나	비고
로컬 RTX 4070 SUPER	31ms	약 2초	Ch21 §5 — 기준선
Colab A100	약 70ms	—	GPU가 아니라 주변이 느리다: CPU 합성 250ms/프레임 (로컬 49ms) → 120ms를 넘으면 GPU 합성으로 자동 전환
Kaggle T4 1대	—	약 9초	무료 · 무인 9시간 · 화면에 적힌 값
T4 4대 분산	—	청크당 12.2~22.5초	7청크 순차 123.2초 → 병시계 37.1초(3.32배, 오버헤드 약 17%)

읽는 법은 셋입니다. ① **비싼 GPU가 빠른 것이 아닙니다.** A100이 4070의 두 배 느린 이유는 §6 앞에서 다섯 가지로 적었습니다 — 이 워크로드는 256px UNet 이라 GPU는 놓고 CPU·IO가 시간을 먹습니다. ② **무료 T4 한 대는 대화가 안 됩니다.** 문장당 9초면 Ch08의 지연 숨기기(아이들 루프·필러)를 다 써도 대화라기보다 문답입니다. ③ **그래서 서비스는 쪼깁니다.** 문장을 청크로 나눠 T4 여러 대에 동시에 보내면 병시계가 3분의 1로 줄어(3.32배 가속) 실시간에 근접합니다 — Ch07의 문장 청킹이 분산의 단위가 된 것입니다. 스튜디오의 "여러 대 붙이기"(고급 설정)가 그 스위치입니다.

이 화면 하나에 이 책의 층이 전부 있습니다 — 얼굴(Ch11·Ch12), 목소리(Ch03), 두뇌(Ch22), 기억(Ch24), 실시간 구조(Ch21), 그리고 과금·반납(이 절). 수업용 예제와 다른 점은 딱 하나, **GPU가 누구 것인가**입니다. 서비스는 사용자의 무료 노트북 GPU를 빌려 쓰고, 300초 놀면 반납하고, 3초마다 과금합니다.

체험 화면이라 "아직 연결 안 됨" 상태입니다. 연결 전에는 대기 영상만 재생되고 남의 GPU를 쓰지 않습니다 — **모델 0으로 골격부터**의 원칙이 사용자 화면까지 내려온 것입니다.

분산이 학습에도 통합니다

지금까지는 추론 잡을 보내는 이야기였습니다. 학습도 같은 구조로 돕니다.

저자는 이 플랫폼으로 **한국어 사투리 음성 모델 다섯 개를 하루에** 학습시켰습니다. 무거운 데이터는 학습 VM이 직접 받고, 밖으로 나오는 것은 **22.5MB 어댑터** 하나뿐입니다(Ch03A §5 — 스크립트 주석의 ~4MB는 초기 설정 값이었습니다). 그리고 그 다섯을 **카드 한 장에 동시에** 올려서 병합니다.

수치와 절차는 **Ch03A**에 사례로 실었습니다. 이 책에서 **분산 GPU가 원가 구조를 실제로 바꾼 유일한 실측 사례**입니다.

정리 — 분산 GPU는 Track A의 원가 문제에 대한 구조적인 답입니다. 다만 그 답은 공개해도 되는 입력에만 적용됩니다. 속도를 위해 프라이버시를 파는 거래가 되지 않도록, 어떤 입력이 어디서 처리되는지를 설계 단계에서 정하세요.

28.7 웹을 앱으로 감싸기

다 만들어 놓고 나서 "앱으로도 내야 한다"는 말을 듣는 날이 옵니다.

웹뷰로 감싸는 방식 이 가장 빠릅니다. 이미 만든 프론트엔드를 그대로 쓰고 껍데기만 앱으로 만듭니다. 3D 렌더링도 오디오도 모바일 브라우저 엔진에서 잘 돌기 때문에, 실제로 잘 동작합니다.

확인할 것 셋. 오디오 자동재생 정책이 모바일에서 더 까다롭습니다. 마이크 권한을 묻는 흐름이 다릅니다. 그리고 저사양 기기에서 3D 렌더가 버티는지 직접 봐야 합니다.

모델 파일 크기 를 신경 쓰세요. VRM이 수십 메가바이트면 첫 실행 다운로드가 부담입니다. 앱에 미리 넣거나, 가벼운 모델을 기본으로 두고 고품질은 원할 때 받게 합니다.

28.8 이 장에서 기억할 것

실시간 서비스와 렌더 잡을 분리 하세요. 요구 사양·확장 방식·비용 구조가 다릅니다.

실시간 서비스는 **가볍습니다**. 정적 파일 캐시, 오디오 정리, 키 분리, 프로세스 감시, **재부팅 자동 시작**.

외부 노출은 터널 로 시작하되, **설정 파일 명시 · 자동 시작 포함 · 로그 확보** 를 빠뜨리지 마세요.

렌더 잡은 **컨테이너에 전부 굽습니다**. 실행 시점에 아무것도 받지 않는 것이 핵심 가치입니다.

여러분이 안 받아도 라이브러리가 받습니다. `torch.hub` 캐시까지 구워야 하고, 확인하는 방법은 `--network none` 으로 한 번 돌려 보는 것 뿐입니다. 정적 검사는 의도를 확인할 뿐 실행을 보장하지 않습니다.

로그를 흘리고, 실패하면 큰 소리로 죽으세요. 침묵하는 4분과 조용한 실패가 최악입니다.

잡 큐는 **진행 상황과 예상 시간을 보여주세요.** 알고 기다리는 4분은 짧게 느껴집니다.

GPU는 빌리고 빌려줄 수 있습니다 (§ 6). 그 구조가 실서비스가 된 모습이 실시간 휴먼 스튜디오이고, 거기서 배운 것은 하나입니다 — **비싼 GPU가 빠른 것이 아니라, 병목이 GPU가 아닐 때가 많습니다.**

웹을 앱으로 감싸는 것 (§ 7)은 마지막에 하세요. 웹으로 되는 것을 먼저 끝내야 합니다.

이 장에서 만든 것을 **운영** 하는 이야기가 바로 다음(Ch28A)에 오고, 그다음 Ch29가 이 책에서 유일하게 협상 불가능한 장입니다.

실습 코드: `code/ch28_deploy/` — 서비스 배포 스크립트 + 렌더 잡 컨테이너 + 최소 잡 큐

예상 소요: 4시간

관련 부록: N(원가 모형 — 직접 구축 vs API 손익분기), G(윤리 — 어디서 처리되는가)

CHAPTER 28A

운영자 콘솔 — 만든 다음에 시작되는 일

28A.1 배포는 끝이 아니라 시작입니다

바텐더(Ch08)를 띄우고 사흘째, 사용자가 물었습니다. "목소리가 좀 이상한데요?"

로그를 봤습니다. 오류는 한 줄도 없었습니다. 응답 시간 정상, LLM 정상, TTS도 200을 돌려줬습니다. 서버 입장에서는 완벽하게 잘 돌아가고 있었습니다.

직접 들어 봤더니 정말 이상했습니다. 어떤 문장에서 음높이 설정이 잘못 걸려 있었습니다.

여기가 AI 휴먼 서비스의 운영이 보통 웹 서비스와 갈라지는 지점입니다.

산출물이 텍스트가 아니라 소리와 얼굴이면, 로그로는 품질을 알 수 없습니다.

주문 API는 응답 JSON만 보면 맞았는지 압니다. TTS는 파일이 생겨도 그 안에서 무슨 소리가 나는지 서버가 모릅니다. 립싱크는 mp4가 나와도 입이 맞았는지 모릅니다.

그래서 이 장이 필요합니다. 관리자 콘솔은 부가 기능이 아니라 품질 관측 장비입니다.

28A.2 콘솔은 다섯 개의 방입니다

운영 콘솔을 처음 만들면 대개 "사용자 목록"부터 붙입니다. 그러다 필요할 때마다 버튼이 하나씩 늘고, 반년 뒤에는 아무도 전체를 설명하지 못하는 화면이 됩니다.

처음부터 **다섯 개의 방**으로 나누세요. 이 분류가 콘솔의 뼈대입니다. 새 기능이 생기면 어느 방에 들어갈지부터 정합니다.

- ① **지표** — 지금 서비스가 어떤 상태인가. 가입·활성·완료·이탈.
- ② **품질** — 산출물이 쓸 만한가. 이 방이 AI 휴먼 특유입니다.
- ③ **사람** — 누가 쓰고 있고 누가 문제인가. 권한·차단·정리.
- ④ **콘텐츠** — 무엇을 서비스하고 있는가. 공개·난이도·수정·삭제.
- ⑤ **자동화** — 사람이 안 해도 되는 일. 임계값·자동 실행·상태.

각 방이 무엇을 담는지 차례로 봅니다.

28A.3 ① 지표 — 한 번의 호출로

대시보드에서 가장 흔한 실수는 **위젯마다 API를 하나씩 부르는 것**입니다. 지표가 스무 개면 요청이 스무 번 나갑니다. 그러면 관리자 페이지가 서비스에서 제일 느린 페이지가 됩니다.

집계 엔드포인트 하나에 전부 담으세요.

저자의 콘솔은 `/api/admin/data` 한 번으로 스물다섯 개의 지표를 반환합니다. 통계, 유입 경로별 분포, 장르 분포, IP 통계, 등급 분포, 14일 가입 추이, 14일 완료 추이, 시리즈별 통계, 상위 사용자, 최근 활동, 사용자 목록, 공지, 자동화 상태… 그리고 확장 지표로 전환 퍼널, 방치 게스트, 휴면 사용자, 차단 IP, 유입별 잔존율, 난이도 균형, 이탈 지점, 피드백 감성, 어뷰즈 신호, 기록판, 서버 헬스.

왕복 한 번입니다. 프론트는 받은 객체를 쪼개 그리기만 합니다.

주의할 것 셋 이 있습니다.

무거운 집계는 캐시하세요. 14일 추이 같은 것은 1분 캐시하면 충분합니다. 관리자가 새로그침을 연타해도 DB가 버팁니다.

하나가 실패해도 전체가 죽지 않게 하세요. 스물다섯 개 중 쿼리 하나가 예외를 던지면 대시보드가 통째로 빈 화면이 됩니다. 항목마다 따로 감싸고, 실패한 것만 `null`로 보내세요.

지표에는 반드시 기간이 붙어야 합니다. "가입 1,240명" 같은 예시 숫자는 정보가 아닙니다. "최근 14일 가입 추이"가 정보입니다.

무엇을 볼 것인가 — 퍼널이 먼저입니다

지표를 고르는 기준은 하나입니다. **그 숫자를 보고 내일 무엇을 할지 정할 수 있는가.**

총 가입자 수는 그런 숫자가 아닙니다. 기분만 좋습니다. **전환 퍼널**은 다릅니다.

방문 → 게스트 시작 → 첫 대화 → 로그인 → 재방문

어느 칸에서 사람이 새는지가 곧 내일 할 일입니다. 게스트 시작은 많은데 첫 대화가 적으면 **첫 화면의 문제**입니다. 첫 대화는 많은데 로그인이 없으면 **가입 유인의 문제**, 로그인까지 갔는데 재방문이 없으면 **콘텐츠의 문제**입니다.

같은 이유로 **이탈 지점** — 사람들이 창을 닫는 자리(dropout) — 을 콘텐츠 단위로 기록하세요. 어느 대화 턴, 어느 단계에서 닫는지. 이 기록이 개선 목록을 알아서 만들어 줍니다.

28A.4 ② 품질 — 이 방이 이 장의 핵심입니다

여기가 AI 휴먼 서비스에만 있는 방입니다.

미리듣기 패널

관리자 페이지에서 **문장을 넣고 목소리를 골라 그 자리에서 들을 수 있어야 합니다.**

당연해 보이는데 대부분의 서비스에 없습니다. 목소리를 조금 바꾸려면 코드를 고치고, 서버를 재 시작하고, 대화를 한 번 해 봐야 합니다. 그 왕복이 3분이면 아무도 목소리를 손보지 않습니다.

저자의 콘솔에는 톤 프로필 여덟 개가 등록되어 있습니다.

표 28A.1 톤 프로필 여덟 개와 그 성격

라벨	성격
기본·미스터리	차분한 남성
공포	음산한 저음. 음높이 -4, 속도 -5
스릴러·첩보	긴장된 남성. 음높이 -2, 속도 -2
무협·사극	기품 있는 남성. 속도 -3
판타지	따뜻한 여성. 음높이 +2
로맨스·일상	부드러운 여성. 음높이 +3
코미디	경쾌한 여성. 음높이 +4, 속도 +6
SF·우주	중성적 여성

같은 엔진, 같은 음성, 파라미터만 다릅니다. Ch03에서 "감정 제어는 속도와 음높이로 근사한다"고 했던 것의 실물이 이 표입니다. 공포는 느리고 낮게, 코미디는 빠르고 높게.

관리자가 문장을 넣고 프로필을 고르면 곧바로 mp3가 돌아옵니다. **왕복이 3분에서 3초가 됩니다.** 그러면 사람들이 실제로 목소리를 손봅니다.

품질 방에 더 넣을 것

최근 산출물 목록. 방금 만들어진 오디오·영상 스무 개를 바로 재생할 수 있게 두세요. 사용자가 신고하기 전에 먼저 발견합니다.

검증 게이트 결과. Ch09의 발화구간 적중률, Ch14의 길이 대조, Ch27의 골든셋 점수. **자동 검증을 만들었으면 그 결과가 여기 있어야 합니다.** 로그 파일 어딘가에만 있으면 아무도 안 봅니다.

피드백과 감성. 사용자 의견을 모아 긍정·부정으로 갈라 두세요. 부정이 갑자기 늘면 뭔가 깨진 것입니다. 코드가 멀쩡해도 그렇습니다.

지연 분포. Ch07의 p95 — 느린 쪽 5%가 실제로 기다리는 시간입니다. 평균만 보면 안 됩니다.

28A.5 ③ 사람 — 파괴적 작업에는 미리보기를

권한 주기, 차단, 정리. 이 방의 원칙은 하나입니다.

되돌릴 수 없는 작업은 기본값이 미리보기여야 합니다.

저자의 게스트 정리 기능은 이렇게 생겼습니다.

```
purge_guests(days=14, dry_run=True)
```

dry_run의 기본값이 참입니다. 아무 옵션 없이 부르면 "14일 이상 방치된 게스트 37명이 삭제 대상입니다"라고 알려만 주고 아무것도 지우지 않습니다. 진짜로 지우려면 그 플래그를 손으로 꺼야 합니다.

기본값을 이렇게 잡는 것과 확인 팝업을 띄우는 것은 다릅니다. 팝업은 습관적으로 눌립니다. 플래그를 손으로 꺼야 하는 구조는 손이 한 번 멈춥니다.

차단은 해제와 같은 문으로 만드세요. IP 차단 엔드포인트에 `unblock` 플래그를 두고 한 곳에서 처리하면, 차단 목록과 해제 경로가 어긋날 일이 없습니다. 차단만 있고 해제가 없는 콘솔은 반드시 사고가 납니다. 컴패니언 콘솔의 `/api/admin/block_ip`가 그 구현입니다 — `unblock` 플래그 하나로 갈리고, 판단은 `access.decide()`가 서버 없이 검사됩니다. 그리고 위의 규칙대로 이것도 **dry_run**이 기본입니다.

누가 했는지 남기세요. 차단·권한 변경·삭제에는 실행한 관리자와 사유를 같이 적습니다. 관리자가 혼자여도 그렇습니다. 석 달 뒤의 나는 남입니다.

어뷰서는 신호로 모으세요. 비정상적으로 잦은 요청, 같은 IP의 다중 계정, 짧은 시간에 반복되는 실패. 목록으로 쌓아 두면 차단이 감이 아니라 근거가 됩니다.

28A.6 ④ 콘텐츠 — DB만 고치면 안 됩니다

여기에 함정이 하나 있습니다. 저자도 밟았습니다.

관리자가 콘텐츠의 난이도를 바꿉니다. DB가 갱신됩니다. 그런데 서비스는 예전 난이도로 그대로 둡니다.

원인은 단순합니다. 속도 때문에 엔진이 콘텐츠 설정을 메모리에 캐시해 두고 있었고, **DB만 고쳤지 캐시는 안 고쳤습니다**. 서버를 재시작하면 반영됩니다. 그래서 더 헛갈립니다 — "어제는 됐는데"가 여기서도 나옵니다.

관리자 조작은 저장소와 런타임을 함께 바꿔야 합니다.

- 1) DB 갱신
- 2) 엔진/서버의 메모리 캐시 갱신
- 3) 캐시 갱신 실패는 조용히 넘기되 로그는 남긴다

3번이 중요합니다. 캐시 갱신이 실패했다고 관리자의 저장 작업 전체를 실패시키지 마세요. DB는 이미 맞았으니까요. 다음 재시작이면 어차피 맞춰집니다.

이 방의 나머지는 평범합니다. 공개·비공개 전환, 제목·분류 편집, 삭제, 연령 제한. **다만 삭제는 §5의 원칙을 따르세요** — 소프트 삭제로 두고, 실제 파기는 별도 작업으로 뺍니다.

28A.7 ⑤ 자동화 — 운영자를 루프에서 빼기

콘솔의 마지막 방에는 **사람이 안 해도 되는 일** 을 넣습니다.

저자의 서비스에는 이런 루프가 돌고 있습니다.

- 사용자 피드백 누적
- 대기 건수가 임계값을 넘으면
 - 분석 단계 실행
 - 새 콘텐츠 자동 생성
 - 상태와 로그를 콘솔에 노출

임계값은 관리자가 콘솔에서 숫자 하나로 조절합니다. 자동 실행은 스위치 하나로 끄고 켜니다. 그리고 **지금 어느 단계인지, 최근 로그 열 줄이 무엇인지** 가 대시보드에 그대로 보입니다.

자동화를 만들 때 지킬 것 넷입니다.

끝 수 있어야 합니다. 스위치가 없는 자동화는 사고가 났을 때 코드를 고쳐야 멈춥니다.

지금 뭘 하고 있는지 보여야 합니다. 진행 단계와 최근 로그. 이게 없으면 자동화는 블랙박스이고, 블랙박스는 결국 꺼집니다.

임계값은 코드가 아니라 설정입니다. 숫자 하나 바꾸려고 배포하지 마세요.

실패해도 서비스는 살아 있어야 합니다. 자동 생성이 실패하면 로그를 남기고 다음 주기를 기다립니다. 사용자는 아무것도 몰라야 합니다.

28A.8 여섯째 방 — 화면에서는 ⑩번 ★

다섯 개의 방을 다 만들고 한 달을 써 보면 이상한 일이 생깁니다. **매일 아침 콘솔을 열고, 숫자를 보고, 답습니다.** 사용자 수도 퍼널도 평균 생성 시간도 전부 정상 범위입니다.

그리고 사흘 뒤에 큐가 멈춰 있었다는 것을 압니다.

지표가 틀린 것이 아닙니다. 지표는 **오늘 무엇을 할지 알려주지 않습니다**. "생성 시간 중앙값 520ms"는 맞는 말이고, 아무 행동도 부르지 않습니다.

그래서 방을 하나 더 만들었습니다. 여섯째로 만들었는데 화면에서는 맨 위에 둡니다 — 만든 순서와 보는 순서는 다릅니다.

규칙 셋

① 다음 행동이 없는 정보는 만들지 않습니다.

"실패율 12%"는 지표입니다. 이 건 경보입니다 —

[치명] 실패율이 높다

3/6 실패 (50%) · 그중 2건이 같은 오류 — "CUDA out of memory"
→ 같은 오류가 몰려 있으면 원인은 하나다. 그 하나를 먼저 보라

세는 것과 짚는 것은 다릅니다. 실패 건수는 세면 나옵니다. 가장 많은 오류가 무엇인가는 짚어야 나옵니다. 짚어 주면 운영자가 로그를 뒤질 일이 없습니다.

컴패니언 코드는 이 규칙을 `assert` 로 막아 두었습니다.

```
def alert(level, title, detail, action):
    assert action, "다음 행동이 없는 정보는 지표이지 경보가 아니다"
```

행동을 안 적으면 경보가 **아예 만들어지지 않습니다**. 규칙을 문서에 두면 지켜지지 않는다는 이야기를 Ch28 §6에서 했습니다. 같은 원리입니다.

② 조용한 화면과 죽은 화면을 구분합니다.

경보가 0건일 때 빈 화면을 내놓으면, 운영자는 그것이 정상인지 수집이 멈춘 것인지 알 수 없습니다. 그래서 경보가 없을 때도 이 한 줄을 같이 냅니다.

이상 없음 (마지막 점검 10:42)

시각 하나가 "아무 일 없음"과 "아무것도 안 돌고 있음"을 가릅니다.

③ 산출물 자체를 봅니다.

§1에서 "산출물이 소리와 얼굴이면 로그로는 품질을 알 수 없다"고 했습니다. 그 말이 실무에서 닿는 곳이 여기입니다 — TTS는 빈 문자열에도 성공을 돌려줍니다.

[치명] 사실상 무음인 산출물

1건이 0.3초 미만 — 성공으로 기록됐지만 소리가 없다
→ 미리듣기로 직접 들어 보라

길이만 봐도 잡힙니다. 반대 방향도 봅니다 — 글자 수에 비해 음성이 세 배 이상 길면 **태그가 그 대로 읽혔을** 가능성이 큼니다(Ch03 §3).

평균이 아니라 p95를 보세요

지연 정보에서 하나만 쫓겠습니다. **평균은 꼬리를 감춥니다.**

표 28A.2 콘솔 지표와 값

지표	값
평균	약 1,072ms — 예산 안
중앙값	520ms — 아주 좋음
p95	3,900ms — 예산의 두 배

같은 데이터입니다. 평균만 보면 아무 문제가 없고, p95를 보면 스무 명 중 한 명이 4초를 기다리고 있습니다. **중앙값이 멀쩡한데 p95만 나쁘면 범인은 특정 입력**입니다 — 그 한 건을 열어 보면 대개 바로 나옵니다.

경보 문구에 중앙값을 같이 넣은 이유가 이것입니다. 숫자 하나만 주면 무엇을 볼지 알 수 없습니다.

돈이 오가면 원장이 관리자 기능입니다 ★

서비스가 유료가 되는 순간 방이 하나 더 필요합니다. 이 방이 보는 것은 매출이 아니라 **무결성**입니다.

저자의 플랫폼은 이용자 차감·제공자 적립·플랫폼 수수료를 **복식부기 원장**(같은 돈을 들어온 쪽과 나간 쪽 양쪽에 적는 장부) 한 곳에 씁니다. 콘솔에 띄우는 값은 딱 하나 — 모든 항목의 차변과 대변을 더한 값이 **0 인가**. 0이 아니면 어딘가에서 돈이 생겼거나 사라진 것이고, 그날의 다른 지표는 볼 필요가 없습니다.

표 28A.3 돈이 오가는 서비스의 관리자 지표 넷

지표	무엇을 답하나	어긋나면
원장 합계 = 0	장부가 맞는가	즉시 정지. 다른 지표는 의미 없다
미결제 사용액	무상 크레딧을 넘겨 쓴 계정이 있는가	한도·차단 규칙을 다시 본다
과금 대비 실사용 시간	표시가와 실과금이 같은가	Ch28 §6의 70배 사고가 여기서 잡힌다
준비 시간 비율	이용자가 남의 준비를 대신 내고 있는가	계량 시작점을 실행 신호로 옮긴다

세 번째 지표가 사후 감사의 핵심입니다. 시간당 단가를 표에 적고 초당 단가를 코드에 따로 적었다가, 실과금이 표시가의 70배로 찍힌 적이 있습니다(Ch28 § 6). 그 사고는 코드 리뷰로는 안 잡힙니다. 콘솔의 한 줄이 잡습니다 — 어제 청구한 총액을 어제 쓴 총 시간으로 나눠 표시가와 견주는 한 줄입니다.

그리고 원장은 되돌릴 수 없게 쓰세요. 잘못 청구했으면 그 줄을 고치는 것이 아니라 반대 부호의 줄을 하나 더 씁니다. Ch29 § 4의 생성 로그와 같은 원칙입니다 — 기록은 고치지 않고 덧붙입니다. 그래야 "언제 무엇이 잘못됐고 누가 되돌렸는가"가 남습니다.

28A.9 게이트는 한 함수로

관리자 엔드포인트가 서른 개라도 인증 검사가 서른 번 흠어져 있으면 안 됩니다.

```
def _require_admin(request):
    u = cur_user(request)
    return u if (u and u["is_admin"]) else None
```

모든 관리자 엔드포인트가 이 한 문으로 들어옵니다. 검사 방식이 바뀌면 한 곳만 고칩니다. 새 엔드포인트에서 이 줄을 빠뜨리면 바로 눈에 띕니다 — 다른 스몰아홉 개에는 다 있으니깐요.

관리자 페이지 자체도 게이트를 통과해야 합니다. API만 막고 HTML은 열어 두는 실수가 흔합니다. 페이지가 열리면 내부 구조가 드러나고, 그것만으로도 공격 표면이 됩니다.

403을 정직하게 돌려주세요. 관리자가 아닌 사람에게 빈 화면이나 500을 주지 마세요. 그러면 디버깅이 불가능해집니다.

28A.10 이 장에서 기억할 것

콘솔은 다섯 방으로 시작해 여섯째(경보) 방에서 완성됩니다 — 만든 순서는 여섯째, 보는 순서는 맨 위 (§ 8).

산출물이 소리와 얼굴이면 로그로 품질을 알 수 없습니다. 관리자 콘솔은 부가 기능이 아니라 품질 관측 장비입니다.

콘솔은 다섯 개의 방 — 지표 · 품질 · 사람 · 콘텐츠 · 자동화. 새 기능은 어느 방인지부터 정합니다.

그리고 한 달을 써 보면 여섯째 방이 필요해집니다. 지표는 오늘 할 일을 알려주지 않습니다. 다음 행동이 붙지 않은 경보는 만들지 마세요. 경보가 없을 때도 마지막 점검 시각을 같이 내야 조용한 화면과 죽은 화면이 구분됩니다.

지표는 **집계 엔드포인트 하나로**, 캐시하고, 개별 실패를 격리하고, 기간을 붙이세요. 그리고 **총계보다 퍼널**입니다 — 내일 무엇을 할지 알려주는 숫자만 고릅니다.

품질 방에 미리듣기 패널을 넣으세요. 왕복이 3분에서 3초가 되면 사람들이 실제로 목소리를 손뼉니다. 검증 게이트 결과도 여기 있어야 합니다.

파괴적 작업은 `dry_run=True`가 기본값입니다. 차단과 해제는 같은 문으로. 누가 왜 했는지 남기세요.

관리자 조작은 DB와 런타임 캐시를 함께 갱신 합니다. 캐시 갱신 실패는 조용히, 로그는 남기고.

자동화는 끝 수 있고, 지금 뭘 하는지 보이고, **임계값이 설정**이고, 실패해도 서비스가 살아 있어야 합니다.

게이트는 한 함수로. 페이지도 API도 같은 문을 통과합니다.

다음 장은 이 콘솔에 반드시 들어가야 하는 또 하나의 기능에서 시작합니다 — 동의를 철회한 사람의 데이터를 실제로 지울 수 있는가.

실습 코드: `code/ch28plus_admin/` — 집계 엔드포인트 + 미리듣기 패널 + dry-run 정리 + 게이트

예상 소요: 4시간

관련 부록: L(UI/UX), G(윤리 — 철회 이행)

CHAPTER 29

책임 — 딥페이크 · 음성 복제 · 동의

29.1 학생이 한 말

강의실 화면에 첫 립싱크 결과가 떴을 때, 한 학생이 말했습니다.

"선생님, 이거 무섭게 잘 되는데요."

웃으면서 한 말이었지만 가장 정확한 반응이었습니다. 그 한마디가 이 장을 쓰게 만들었습니다.

이 책의 앞선 서론한 장은 전부 "어떻게 만드는가"였습니다. 이 장은 "무엇을 만들지 않는가"입니다. 그리고 이 장의 내용은 **협상 대상이 아닙니다**.

29.2 이 기술이 실제로 하는 일

있는 그대로 봅시다.

얼굴 — 사진 한 장이면 그 사람이 한 적 없는 말을 하는 영상이 나옵니다. 몇 분이면 됩니다. 특별한 장비도 필요 없습니다.

목소리 — 몇 초짜리 샘플이면 그 사람의 목소리로 아무 문장이나 읽게 할 수 있습니다.

둘을 합치면 — 그 사람이 그 말을 하는 영상이 완성됩니다.

지금 노트북 한 대로 되는 일입니다. 그리고 이 책이 그 방법을 가르쳤습니다.

기술은 중립적이라는 말이 있습니다. 절반만 맞습니다. 어떤 기술은 **오용의 비용이 극히 낮고 피해가 극히 큼니다**. 이 기술이 그렇습니다.

29.3 선은 어디인가 — 세 가지 기준

첫째, 동의

실존 인물의 얼굴이나 목소리를 쓰려면 그 사람의 동의가 있어야 합니다. 예외는 없습니다.

동의를 구체적이어야 합니다. "AI에 쓰는 것에 동의"는 동의가 아닙니다. 무엇을 만들지, 어디에 쓸지, 얼마 동안 보관할지, 어떻게 철회하는지가 적혀 있어야 합니다. 부록 G에 서식을 실었습니다.

공개된 사진은 동의가 아닙니다. SNS에 올린 사진은 그 사람이 말하는 영상을 만들라는 허락이 아닙니다.

공인이라는 사실도 동의가 아닙니다. 오히려 더 위험합니다. 피해가 개인을 넘어갑니다.

세상을 떠난 사람은 특별히 조심해야 합니다. 본인에게 물을 방법이 없기 때문입니다. 유족의 뜻을 확인하고, 그 영상을 보게 될 사람들이 그것을 원하는지 생각하세요. 위로가 될 수도 있고 상처가 될 수도 있습니다.

둘째, 표시

AI로 생성된 것은 그렇다고 알 수 있어야 합니다.

쓰이는 자리마다 방법이 다릅니다. 영상이라면 화면에 띄우거나 시작 부분에 고지합니다. 대화 아바타라면 "저는 AI 입니다"를 숨기지 않습니다. 파일이라면 메타데이터에 남깁니다.

표시를 없애 달라는 요청은 그 자체로 위험 신호입니다. 정당한 쓰임이라면 표시를 지울 이유가 없습니다.

2026년부터는 표시가 '권고'가 아닙니다 ★

이 절을 처음 쓸 때 표시는 좋은 습관이었습니다. 이 책이 나가는 2026년에는 법입니다. 둘을 알아 두세요. 조문 번호나 시행령은 바뀌니 본문에는 요지만 담습니다. 실물은 각 기관의 최신 고시로 확인하세요(주석과 실물이 다르면 실물을 믿는다는 Ch03A의 원칙 그대로입니다).

표 29.1 2026년의 표시 의무 — 이 책의 산출물에 걸리는 것

규범	언제부터	이 책의 무엇에 걸리나	그래서 어디에 넣나
한국 인공지능 기본법 (생성형 AI 산출물 표시)	2026-01 시행	Track A의 mp4, Track B의 실시간 화면, 클로닝한 목소리 전부	파일에는 메타데이터(Ch14 mux 단계), 화면에는 상시 문구(부록 L), 로그에는 생성 기록(§ 4)
EU AI Act 제50조 (딥페이크·합성 콘텐츠 투명성)	2026-08 적용	EU 이용자에게 닿는 모든 합성 얼굴·목소리	같은 세 곳 — 그리고 기계가 읽을 수 있는 표시(C2PA 류)가 요구된다

표시는 세 층에 다 넣습니다. ① 파일 메타데이터 — Ch14의 mux 명령에 한 줄만 더하면 됩니다 (`-metadata comment="AI-generated · 생성 로그 ID ..."`). 이 책의 `mux_lint.py`는 그 줄이 빠지면 지적합니다. ② 화면 — 대화 아바타는 파일이 아니라 *세션*이라 메타데이터를 붙일 곳이 없습니다. 부록 L의 "AI 입니다" 상시 표기가 곧 표시입니다. ③ 로그 — § 4의 생성 로그가 누가 언제 무엇을 만들었나에 답합니다. 표시 의무는 결국 "물었을 때 답할 수 있는가"이고, 그 답이 로그입니다.

워터마크는 표시가 아닙니다. 워터마크는 나중에 들통낼 때 쓰는 것이고, 표시는 보는 사람이 지금 바로 알게 하는 것입니다. 목적이 다르니 하나로 다른 하나를 대신할 수 없습니다. 이 책은 둘 다 넣습니다 — 워터마크는 § 4, 표시는 여기.

셋째, 목적

동의를 받았고 표시를 붙였어도 만들면 안 되는 것이 있습니다.

사람을 속이기 위한 것. 실제로 일어난 일인 것처럼 보이게 만드는 것.

사람을 해치기 위한 것. 성적인 것, 모욕적인 것, 헐박에 쓰이는 것.

신원을 위조하기 위한 것. 본인 확인을 통과하기 위한 것.

이 셋은 동의로 정당화되지 않습니다. 동의한 사람이 안 다치더라도, 그것을 보는 사람이 속기 때 문입니다.

29.4 개발자가 실제로 해야 할 것들

원칙을 코드와 절차로 옮깁니다.

동의를 시스템에 기록하세요. 서명된 동의서 파일과 함께 보관하고, 어떤 원본으로 무엇을 만들었는지 로그로 남깁니다.

철회 경로를 만드세요. 동의를 철회하면 원본도 파생물도 지워져야 합니다. 어디에 무엇이 있는지 못 찾으면 철회를 이행할 수 없습니다.

보관 기간을 정하세요. 무기한 보관은 위험을 무기한 끼안는 일입니다.

결과물에 표시를 넣으세요. 표시는 세 층 이고 셋이 서로 다르게 실패합니다 — 화면 표시는 크롭 한 번에, 파일 메타데이터는 재인코딩 한 번에, 픽셀 워터마크는 강한 압축·재촬영에 사라집니다. 그래서 셋을 다 넣습니다. 방식과 한계는 **부록 G § 4** 에 표로 있습니다. 대체재가 아니라 보완재 이므로 **세 층을 다 쓰세요.**

그리고 워터마크에 **이력 전체를 심으려 하지 마세요.** 짧은 식별자 하나만 심고 나머지는 생성 로그 에서 찾는 편이 훨씬 튼튼합니다(부록 G § 3).

로그를 남기세요. 누가 언제 무엇을 만들었는지. 사고가 났을 때 되짚을 수 있어야 합니다.

거절 기준을 문서로 만드세요. 팀이 있다면 더욱 그렇습니다. 판단이 사람마다 다르면 가장 느슨한 기준이 그 팀의 실제 기준이 됩니다.

이 로그는 어느 규모까지 단순해도 되나 ★

컴퍼니언 `genlog.py` 는 JSON Lines 한 파일에 덧붙이기만 합니다. 철회와 만료는 파일을 처음부터 끝까지 훑는 **선형 탐색** 입니다. 색인도 데이터베이스도 없습니다. 그래서 어디까지 버티는지 켜 습니다(`measure.py` → `_work/measure.json`).

표 29.2 생성 로그 규모별 — 한 줄 약 300바이트

건수	파일	철회 역추적(원본 해시 1개)	만료 골라내기
1,000	0.29MB	0.003초	0.003초
10,000	2.87MB	0.03초	0.03초
100,000	28.8MB	0.57초	0.52초

10만 건 — 하루 100건씩 약 3년 — 에서도 철회 요청 하나에 0.6초입니다. 철회는 사람이 요청하고 사람이 확인하는 일이라 이 속도면 충분합니다. 속도보다 다른 것이 먼저 걸립니다. 파일 하나를 통째로 읽는 구조는 **동시 쓰기**에 약하고, 만료 검색이 6만 건을 돌려주면 사람이 그것을 볼 수 없습니다. 색인이나 DB로 옮길 시점은 속도가 아니라 **한 프로세스로는 안 되는 순간**입니다. 그때까지는 이 단순함이 감사(監査)에서 유리합니다 — 파일 하나를 열면 전부 보입니다.

29.5 대화 아바타에만 있는 문제들

얼굴을 만드는 일과는 종류가 다른 책임입니다.

AI 입을 밝히세요. 사용자가 사람이라고 착각하게 두면 안 됩니다. 상담이나 돌봄이라면 더욱 그렇습니다.

애착을 다루세요. 사용자는 아바타에 정서적으로 기댈 수 있습니다. 외로운 사람일수록 그렇습니다. 서비스를 접을 때 어떻게 할지, 의존이 지나칠 때 어떻게 할지를 미리 정해 두세요.

전문 영역에는 선을 그으세요. 의료·법률·금융 조언은 하지 않게 막고, 대신 전문가를 안내하게 합니다.

위기 대응을 준비하세요. 사용자가 자해나 위험을 내비치면, 대화를 이어가지 말고 즉시 도움을 안내하는 응답으로 넘어가야 합니다. 이것은 반드시 코드로 강제하세요. 프롬프트 지시만으로는 부족합니다.

미성년자를 고려하세요. 아이들이 들어올 수 있는 서비스라면 콘텐츠 기준도 데이터 처리 기준도 달라집니다.

29.6 배우는 사람에게

이 기술을 배운 순간, 여러분은 그것을 알아볼 수 있는 사람이 되기도 했습니다.

의심하는 법을 함께 배우세요. 입 주변만 화질이 다른가. 눈을 깜빡이지 않는가. 목소리의 호흡이 어색한가. 이 책의 각 장에서 밝은 실패가 그대로 판별 단서입니다. Ch10의 저해상도 입, Ch13의 잘못 그려진 입술, Ch14의 뒤로 갈수록 밀리는 싱크.

만들 수 있다는 것이 만들어도 된다는 뜻은 아닙니다. 이 한 문장이 이 장의 전부입니다.

주변에 설명해 주세요. 부모님께, 친구에게, 학생에게. "요즘은 영상도 못 믿는다"는 말을 막연한 불안이 아니라 구체적인 판단 기준으로 바뀌 주는 것. 그것이 이 기술을 배운 사람의 몫입니다.

29.7 이 장에서 기억할 것

이 기술은 **오용 비용이 극히 낮고 피해가 극히 큼**니다. 중립적이지 않습니다.

기준은 셋 — **동의 · 표시 · 목적**. 공개된 사진도, 공인이라는 사실도 동의가 아닙니다. 목적이 나쁘면 동의가 있어도 안 됩니다.

동의 기록 · 철회 경로 · 보관 기간 · 결과물 표시 · 생성 로그 · 거절 기준 문서화. 원칙을 절차로 옮기세요.

대화 아바타는 **AI 고지 · 애착 · 전문 영역 선 긋기 · 위기 대응 · 미성년자** 를 따로 다뤄야 합니다. 위기 대응은 **코드**로 강제 하세요.

2026년부터 표시는 권고가 아니라 법입니다(§ 3). 화면 · 파일 메타데이터 · 워터마크 세 층에 다 넣으세요.

만들 수 있다는 것이 만들어도 된다는 뜻은 아닙니다.

그리고 이 기술을 배운 사람은 **알아보는 법과 설명하는 법** 도 함께 가져야 합니다.

다음 장은 그 둘을 남에게 넘겨주는 법입니다.

실습 코드: `code/ch29_responsibility/` — 동의서 서식 · 생성 로그 스키마 · 위기 대응 필터

예상 소요: 2시간 (서식 작성 + 위기 필터 구현)

관련 부록: G(윤리·동의서·워터마킹)

CHAPTER 30

커리큘럼으로 — 800시간과 28차시

30.1 순서가 완성도를 바꿉니다

KDT AI 휴먼 과정은 800시간입니다. 한 기수를 운영하면서 순서를 한 번 바꿨습니다 — 프롬프트 엔지니어링 다음에 바로 오던 프로젝트-2를 **LangChain** 뒤로 옮긴 것입니다. 총 시수도 종료 일도 그대로, 바뀐 것은 순서 하나였는데 프로젝트의 완성도가 달라졌습니다. 일정표와 그 앞뒤 사정은 **부록 D §4**에 그대로 있습니다.

여기서 얻은 원칙이 이 책 목차의 근거이기도 합니다.

핵심 도구는 그것을 쓸 프로젝트 바로 앞에 와야 합니다.

Part 2(지연)이 트랙보다 앞에 있는 이유, Ch22(페르소나)가 Ch26(멀티 아바타)보다 앞에 있는 이유, Ch03(TTS)이 Ch10(립싱크)보다 앞에 있는 이유가 전부 이것입니다.

30.2 이 책을 가르치는 세 가지 방식

6주 미니코스 — 한 트랙 완성

교양 과목 한 학기, 또는 부트캠프의 한 모듈에 맞습니다.

1주차에 Part 1, 2주차에 Part 2, 3~5주차에 트랙 하나, 6주차에 Part 4와 발표.

어느 트랙을 고를지는 수강생 구성으로 정합니다. GPU를 쓰기 어렵거나 프로그래밍 경험이 적으면 Track B입니다. 첫 주에 결과물이 나오고 좌절할 지점이 적습니다. 영상 제작 배경이 있으면 Track A, LLM 경험이 있으면 Track C입니다.

14주 풀코스 — 셋 다

한 학기 전공 과목 분량입니다.

Part 1(1주) → Part 2(2주) → Track A(3주) → Track B(3주) → Track C(3주) → Part 4(1주) → 발표(1주).

Part 2에 2주를 쓰는 것이 이 배치의 핵심입니다. 학생들은 여기를 빨리 지나가고 싶어 합니다. 모델을 만지고 싶으니까요. 그런데 Part 2를 대충 넘긴 학생은 트랙에서 반드시 막힙니다. 지연 예산이 뭔지 모르는 채 실시간 아바타를 만들겠다고, 4분짜리 파이프라인을 실시간으로 바꾸려 듭니

다.

48시간 해커톤

Ch03(TTS 고르기) → Ch18~21(VRM 실시간 아바타) → Ch22(페르소나).

GPU를 기다리지 않는 것이 핵심입니다. 해커톤에서 CUDA 환경 구축부터 시작한 팀은 첫날을 통째로 잃습니다.

30.3 800시간 과정에 얹기

아홉 개의 정규교과가 이 책의 어디에 붙는지입니다. 상세는 부록 D.

Python · 전처리 · 머신러닝 (앞의 세 교과) — 이 책의 선수 과목입니다. 본문은 이 지식을 전제합니다.

자연어 및 음성 데이터 활용 (96시간) — Ch03(TTS)과 Ch23(STT)이 여기 붙습니다. 시수가 가장 큰 교과이고, 이 책의 목소리 층이 통째로 들어갑니다.

TTS-STT 모델 개발 (64시간) — Ch03 심화. 그리고 **프로젝트-1(나만의 음성 모델)**의 결과물이 곧 이 책의 목소리 층이 됩니다.

거대 언어모델 · 프롬프트 엔지니어링 — Ch22(페르소나)가 여기 붙습니다. 프롬프트 교과의 실습 대상으로 페르소나만 한 것이 없습니다. 결과가 눈과 귀로 바로 확인되기 때문입니다.

LangChain — Ch24(기억)와 Ch26(멀티 아바타)의 구현 도구입니다.

RAG — Ch25 그 자체입니다.

프로젝트-2(답변 기능 구현) — Ch21~24가 재료입니다. 실시간 아바타에 페르소나와 기억을 얹는 것.

프로젝트-3(사전 데이터 기반 답변 커스텀) — Ch25~27이 재료입니다. RAG를 붙이고 평가 하네스로 검증하는 것.

한 가지 제안 — 프로젝트-3의 평가 항목에 **Ch27의 회귀 게이트**를 넣으세요. "잘 됩니다"를 시연으로 증명하는 것과 골든셋 점수로 증명하는 것은 학습 경험이 완전히 다릅니다.

30.4 28차시 강의 자료와의 관계

28차시짜리 강의 자료가 이 책의 뼈대입니다. 매핑은 부록 E에 있고, 여기서는 구조만 짚습니다.

에이전트 트랙 (1~8차시) — 이 책의 머리 층. Track C의 기반입니다.

멀티모달 감각 트랙 (9~12차시) — 보고·만들고·말하고. 10차시(얼굴 이미지 생성), 11차시(음성 클로닝), 12차시(실시간 대화·끼어들기)가 이 책의 Track A·B와 Ch23에 대응합니다. 립싱크는 그보다 앞선 4차시에서 먼저 나옵니다(Ch10·Ch11).

모델 길들이기 (13~14차시) — Ch22의 "파인튜닝으로 잘 때" 절과 연결됩니다.

RAG 트랙 (15~21차시) — Ch25와 Ch27. 특히 21차시(평가 하네스)가 Ch27의 직계입니다.

기억·개인화 (22차시) — Ch24.

운영·통합 (23~26차시) — Part 4. 26차시(책임 AI·딥페이크 윤리)가 Ch29입니다.

캡스톤 (27~28차시) — 각 트랙의 완성 장(Ch15·21·27)이 캡스톤 후보입니다.

강의 자료가 차시별 실습이라면, 이 책은 그것을 **하나의 이야기** 로 다시 펜 것입니다. 강의는 "오늘은 이것을 해 봅시다"이고, 책은 "왜 그 순서인가"입니다. 둘은 대체하는 사이가 아니라 보완하는 사이입니다.

30.5 가르치면서 배운 것들

첫 시간에 결과물이 나와야 합니다. 환경 구축으로 첫 시간을 다 쓰면 그 반은 회복되지 않습니다. Ch01·Ch02를 GPU 없이 30분으로 설계한 이유입니다.

실패를 먼저 보여주세요. 잘 된 결과만 보여주면 학생은 자기 실패를 자기 잘못으로 여깁니다. 고양이 얼굴에 사람 입술이 그려진 화면을 먼저 보여주면, 그것이 정상적인 학습 과정이라는 것을 알게 됩니다.

검증하는 법을 반드시 가르치세요. 만드는 법만 가르치면 학생은 엉뚱한 지표를 쫓습니다. 저자가 그랬듯이. Ch09와 Ch27은 선택 과목이 아닙니다.

윤리를 마지막에 몰아넣지 마세요. 학기 끝에 한 시간 하는 윤리 수업은 형식입니다. 얼굴을 처음 만든 그날 다뤄야 합니다. Ch04에서 "실존 인물과 닮게 만들지 마세요"를 짚고, Ch13과 Ch15에서 다시 짚고, Ch29에서 정리하는 배치가 그래서입니다.

GPU를 공유 자원으로 관리하세요. Track A를 여럿이 동시에 돌리면 GPU 한 장으로는 감당이 안 됩니다. 예약제로 돌리거나, Ch28의 잡 큐를 실습 인프라로 쓰세요. 학생이 자기가 만든 큐에 자기 잡을 넣는 구조는 그 자체로 좋은 교육입니다.

얼마나 차이 나는지 컴패니언 `fairness.py` 로 켜겠습니다. 스무 명 반, 한 학생이 잡 열 개를 먼저 넣고 나머지 열아홉이 하나씩 넣은 상황, 잡 하나는 210초(10초 음성 × Ch06의 21배), GPU 한 장 (`_work/fairness.json`).

표 30.1 나머지 열아홉 명이 첫 잡을 시작 하기까지

큐 규칙	중앙값	최대
제출 순 (Ch28의 큐 그대로)	66.5분	98분
학생별 순환 (ClassQueue)	35분	66.5분

총 처리 시간은 둘 다 102분으로 같습니다 — GPU가 한 장이니 일의 양은 변하지 않습니다. 바뀌는 것은 누가 먼저 기다리느냐입니다. 제출 순으로 두면 열아홉 명 중 절반이 한 시간 넘게 첫 결과를 못 봅니다. 수업에서 그 한 시간은 "안 되네요"로 끝나는 시간입니다. 순환 규칙은 한 줄입니다 — 이미 돌린 잡이 가장 적은 학생의 가장 오래된 잡을 집는다. 그러면 열 개를 넣은 학생도 한 바퀴에 한 번만 차례가 옵니다.

수업 환경 한 벌 — 배우는 건 노트북, 만드는 건 도커 ★

일곱 기수를 돌리고 여덟 번째를 열며 자리 잡은 환경 구성입니다. 저자의 수업 저장소(D:\AIHuman\docker)에 있는 가이드 세 편을 줄여 옮깁니다.

표 30.2 용도별 환경 — 둘 다 준비합니다

용도	환경	이유
개념·단위 실습 (LLM · 임베딩 · RAG · LoRA · TTS)	Colab / Kaggle 노트북	무료 GPU, 설치 0, 학생은 링크만 연다
실시간·멀티서비스 (음성 에이전트 · 아바타 · 벡터DB · 웹앱)	docker-compose (벡터DB + 앱 서버 + 노트북)	계속 떠 있는 서버가 필요하다
최종 프로젝트	로컬 GPU PC의 도커	재현성 · 배포 · 팀 공유

노트북만으로 안 되는 이유는 셋입니다. 음성 에이전트는 **웹소켓 서버 + LLM + STT/TTS + 벡터DB**가 동시에 떠 있어야 하는데, 노트북 세션은 끊기고, 포트를 바깥으로 못 열고, 상태가 남지 않습니다. 그래서 *데모·실험*은 노트북, *돌아가는 앱*은 *도커*로 처음부터 갈라 둡니다.

진행은 세 단계입니다. ① 1~2주차 단위 실습은 노트북으로(학생 PC 부담 0). ② 통합·프로젝트 주차에는 강의장 GPU PC에 `docker-compose up` 한 번으로 전원이 같은 환경. ③ GPU가 없는 학생은 경량 모델을 쓰는 CPU 용 컴포즈로 흐름만 체험합니다. 운영 세부 둘 — 모델 캐시와 벡터DB 같은 무거운 데이터는 도커 이미지 저장소까지 전부 D 드라이브에 두고, LLM은 별도 서버 없이 앱이 Transformers로 직접 올립니다. 부품이 하나 줄면 수업 중에 죽는 자리도 하나 줄입니다.

30.6 평가 루브릭 제안

트랙별 프로젝트를 평가한다면 다섯 항목을 제안합니다.

동작 — 요구한 결과물이 실제로 나오는가. (40%)

검증 — 잘 되었다는 것을 어떻게 증명했는가. Ch09 또는 Ch27의 방법. (20%)

선택의 근거 — 왜 그 트랙, 그 모델, 그 파라미터를 골랐는가. Ch05의 결정 트리로 설명할 수 있는가. (20%)

실패 기록 — 무엇을 시도했다가 왜 버렸는가. (10%)

책임 — 동의·표시·거절 기준을 다뤘는가. (10%)

네 번째 항목이 특히 중요합니다. 실패 기록을 점수에 넣으면 학생들이 실패를 숨기지 않고 적습니다. 그리고 그 기록이 다음 기수의 교재가 됩니다. 부록 F가 그렇게 만들어졌습니다.

30.7 이 장에서 기억할 것

핵심 도구는 그것을 쓸 프로젝트 바로 앞에 **와야 합니다**. 순서 하나가 완성도를 바꿉니다.

6주 · 14주 · 48시간 세 가지 배치. 폴코스라면 **Part 2에 2주**를 쓰세요.

800시간 과정에서는 **프로젝트 1:2:3이 각각 목소리 층 · 실시간+인격 · RAG+평가**에 대응합니다.

가르치면서 배운 것 다섯 — **첫 시간에 결과물 · 실패를 먼저 · 검증을 반드시 · 윤리를 처음부터 · GPU를 공유 자원으로.**

루브릭에 **실패 기록**을 넣으세요. 학생이 숨기지 않게 되고, 그 기록이 다음 교재가 됩니다.

30.8 마지막 한 줄

Vol.01에서 픽셀이 핸들이 되었고, Vol.02에서 픽셀이 진단이 되었습니다.

이 책에서 픽셀은 사람이 되었습니다 — 얼굴을 얻고, 목소리를 얻고, 말을 걸어왔습니다.

만드는 법을 알게 된 사람이 만들지 않을 줄도 알게 되기를 바라며, 여기서 마칩니다.

실습 코드: `code/ch30_teaching/` — 강의안 템플릿 · 루브릭 · GPU 예약 큐

예상 소요: 3시간 (한 트랙 강의안 1주차 작성)

관련 부록: D(800시간 매핑), E(28차시 매핑), I(Vol.02→Vol.03), J(Vol.04 예고)

부록 A — 환경 구축 (무GPU 트랙 / GPU 트랙)

Ch02의 실행 절차 상세판. Windows 기준이며 macOS·Linux 차이는 각 절 끝에 표기.

새 기계 앞에 처음 앉았을 때, 그리고 설치 도중 빨간 글씨를 만났을 때 퍼는 부록입니다. 무GPU 트랙은 30분, GPU 트랙은 반나절이 걸립니다. 다 읽고 나가실 때 손에 두 가지가 남습니다 — 한국어 한 문장을 소리로 바꿔 길이까지 재는 무GPU 환경, 그리고 공식 예제 셋이 그대로 도는 GPU 환경입니다. 자기 사진과 자기 대본은 그다음입니다.

1. 무GPU 트랙 — 30분

필요한 것

표 A.1 필요한 것 — 항목 · 버전 · 용도

항목	버전	용도
Python	3.10+	서버
ffmpeg	최신	오디오 변환·길이 측정
브라우저	최신 Chrome/Edge	얼굴 렌더가 여기서 일어남

설치 순서

- 가상환경 생성
- 웹 서버 · HTTP 클라이언트 · TTS 클라이언트 설치 (수십 MB)
- ffmpeg 설치 후 **경로를 코드 안에서 PATH 앞에 주입**

```
FFBIN = r"C:\...\ffmpeg\bin"
os.environ["PATH"] = FFBIN + os.pathsep + os.environ["PATH"]
```

셀을 새로 열 때마다 PATH를 다시 잡지 마세요. 코드 안에서 한 줄로 끝내는 편이 안전합니다.

검증

짧은 한국어 문장을 TTS로 돌려 mp3를 만들고, 재생하고, **길이를 초 단위로 잽니다**. 이 셋이 되면 무GPU 트랙은 준비 완료입니다.

2. GPU 트랙 — 반나절

기준 사양

표 A.2 저자의 기준 사양

항목	값
GPU	RTX 4070 SUPER 12GB (본문 실측 기준)
최소	8GB (배치 크기 절반)
Python	3.10
PyTorch	2.0.1 + CUDA 11.8
ffmpeg	최신

⚠ 이 조합은 '안정'이지 '만능'이 아닙니다

2.0.1 + cu118 은 립싱크 모델이 공식 권장하는 조합입니다. 30·40 세대 GPU에서는 이대로 가는 것이 가장 덜 아픕니다.

그런데 최신 세대 GPU는 이 조합으로 아예 못 돌립니다. 새 아키텍처용 커널이 옛 빌드에 없기 때문입니다. 50 세대 카드를 새로 사서 이 책을 따라 하면 첫 단계에서 막힙니다.

표 A.3 GPU 세대별 권장 조합

GPU 세대	권장
30·40 세대	2.0.1 + cu118 — 본문 기준. 가장 검증됨
리타게팅만 CUDA 12 환경	2.3.0 계열
50 세대(신형)	최신 torch + 일부 의존성 직접 빌드

마지막 줄이 문제입니다. 최신 세대에서는 포즈 관련 라이브러리를 소스에서 빌드해야 하는 일이 생기고, 파이썬 버전도 3.12로 올려야 할 수 있습니다. **반나절이 아니라 하루를 잡으세요.**

판단 — GPU를 새로 살 계획이라면, Track A 하나 때문에 최신 세대를 고르는 것은 오히려 손해 일 수 있습니다. 이 책의 GPU 트랙은 40 세대에서 가장 매끄럽습니다. 그리고 Track B·C는 GPU 자체가 필요 없습니다(부록 N § 4).

환경 점검기가 이것을 알려줍니다. `code/ch02_setup/doctor.py --gpu` 를 돌리면 GPU 이름·VRAM·torch 버전이 한 화면에 나옵니다. 본문과 다른 조합이면 이 절로 돌아오세요.

환경 두 벌 — 반드시 분리

립싱크 계열과 리타게팅 계열은 의존성이 충돌합니다. **conda 환경 두 개** 를 만들고 둘 다 위 조합으로 맞추세요.

서로 부를 때는 **conda 활성화 대신 실행 파일 절대 경로로 직접 호출** 하세요.

```
<env1>/python.exe -m scripts.inference ...
<env2>/python.exe inference_animals.py ...
```

셀 상태에 기대지 않으므로 한 스크립트에서 두 환경을 순서대로 쓸 수 있습니다. Ch15 파이프라인의 전제입니다.

인코딩 설정 (필수)

```
PYTHONUTF8=1
```

한국어 Windows에서는 모델 다운로드 도구가 이모지를 찍다가 cp949 오류로 죽습니다. 이 한 줄이 그것을 막습니다.

커스텀 CUDA 연산자 빌드 (동물 모드용)

리타게팅 모델의 동물 모드는 키포인트 검출기(얼굴에서 눈·코·입 위치를 찍는 부품)의 커스텀 연산자를 직접 빌드해야 합니다.

Windows에서는 최신 MSVC와 CUDA 11.8이 서로를 지원하지 않는다고 주장하며 빌드를 거부합니다. 배치 스크립트로 우회하세요.

```
1) VS2022 vcvars64 로드
2) CUDA_HOME을 11.8로 지정
3) nvcc 에 우회 플래그 주입
   -allow-unsupported-compiler
   -D_ALLOW_COMPILER_AND_STL_VERSION_MISMATCH
```

한 번 성공하면 다시 볼 일이 없습니다. 환경을 새로 만들 때만 필요합니다.

알려진 설치 문제

표 A.4 알려진 설치 문제 — 증상과 대응

증상	대응
모델 다운로드 중 이모지 인코딩 오류	<code>PYTHONUTF8=1</code>
포트 라이브러리 설치 중 의존 패키지 빌드 실패	해당 패키지를 <code>--no-build-isolation</code> 으로 선설치
conda 채널 약관 오류	채널별 약관 수락 후 재시도
한글 경로에서 이미지 로드 실패	ASCII 경로 사용 (부록 H)

스모크 테스트 — 자기 데이터로 시작하지 마세요

환경을 만들었으면 각 모델의 공식 예제를 그대로 한 번 돌리세요.

표 A.5 스모크 테스트 — 대상과 확인할 것

대상	확인할 것
리타게팅 사람 모드	결과 mp4 + 비교 영상 생성
리타게팅 동물 모드	커스텀 연산자 정상 로드
립싱크 모델	결과 mp4 생성

자기 데이터로 시작하면 실패했을 때 원인이 환경인지 데이터인지 가릴 수 없습니다.

3. macOS · Linux 차이

Linux — 커스텀 연산자 빌드가 Windows보다 훨씬 순조롭습니다. 우회 플래그가 필요 없고, 한글 경로 문제도 없습니다.

macOS — Apple Silicon에서는 CUDA 계열이 돌지 않습니다. Track A는 원격 GPU(Ch28)로 보내고, Track B·C는 로컬에서 그대로 돌리세요. 실제로 macOS 사용자에게 권하는 구성입니다.

4. 디렉터리 구조 권장안

```
project/
├─ envs/           # conda 환경 (또는 시스템 경로)
├─ models/        # 가중치 (버전별 하위 폴더)
├─ drivers/       # 드라이버 영상 (부록 B)
├─ inputs/        # 소스 이미지·대본
├─ work/          # ASCII 이름으로 복사된 작업본
├─ outputs/       # 최종 결과
└─ logs/          # 단계별 소요·검증 리포트
```

work/ 를 따로 두는 것이 핵심입니다. 한글 경로 회피와 중간 산출물 정리가 이 폴더 하나로 끝납니다.

관련 — Ch02(환경) · Ch12(동물 모드) · 부록 H(Windows 트러블슈팅)

부록 B — 드라이버 영상 카탈로그

드라이버는 결과에 보이지 않지만 결과를 결정합니다. Ch04 §3의 실행 체크리스트.

렌더를 돌리기 전에, 그리고 "싱크가 안 맞는 것 같다"는 말을 들었을 때 펴세요. 드라이버 영상 — 결과물에는 한 프레임도 나오지 않고 움직임만 옮겨 가는 영상 — 을 고르고 재는 법이 여기 있습니다. 돌아갈 때 손에 남는 것은 여섯 줄 체크리스트와, 직접 찍을 30초짜리 세 벌의 촬영 지침입니다.

1. 적합성 체크리스트

렌더를 돌리기 전에 이 여섯 개를 확인하세요. 하나라도 아니면 결과가 나빠집니다.

표 B.1 드라이버 적합성 체크리스트 여섯

#	항목	기준	실패 시 증상
1	얼굴 비율	화면 세로의 60% 이상	입 모션이 약하게 전사 → "싱크가 안 맞는다"는 착각
2	정면성	좌우 15도 이내	키포인트 흔들림
3	머리 움직임	작고 느림	대상 얼굴이 흔들림·눈알 굴림
4	표정 과장	없음. 또박또박 말하는 정도	입꼬리 비틀림
5	길이	음성보다 길게	반복 이음매에서 모션 튕
6	프레임레이트	파이프라인 기준값과 일치	뒤로 갈수록 싱크 밀림

1번이 압도적으로 중요합니다. 저자가 이 한 줄에서 가장 많은 시간을 잃었습니다.

2. 얼굴 비율 측정하기

눈으로 판단하지 말고 재세요. 한 프레임만 얼굴 검출을 돌려, 얼굴 경계 상자의 높이를 화면 높이로 나눕니다.

표 B.2 얼굴 비율 판정 기준

비율	판정
0.6 이상	좋음 — 모션이 강하게 전사
0.4~0.6	보통 — 크롭 옵션으로 보완
0.4 미만	부적합 — 다른 영상을 찾으세요

`code/ch04_face/` 의 측정 스크립트가 이 값을 계산합니다.

3. 어디서 구하나

(권장) 직접 촬영

30초면 충분합니다. 카메라를 얼굴 가까이 두고, 정면을 보며, 평소 속도로 또박또박 말하세요. 내용은 아무래도 좋습니다 — 결과에 보이지 않으니깐요.

세 종류를 미리 찍어 두면 웬만한 상황은 이 셋으로 끝납니다.

표 B.3 미리 찍어 두면 좋은 드라이버 셋

이름	톤	쓰는 곳
calm	차분하게, 표정 최소화	추모·안내·설명
warm	부드럽게 웃으며	인사·환영
bright	밝고 조금 빠르게	홍보·소개

라이선스 걱정이 없고, 얼굴 비율을 원하는 대로 만들 수 있는 것이 직접 촬영의 결정적 이점입니다.

스톡 영상

검색어 요령 — "의사", "상담사", "강사", "인터뷰", "설명"처럼 **직업적으로 차분하게 말하는** 사람을 찾으세요. "행복한 사람", "웃는 여성" 같은 검색어는 표정이 과장된 영상을 줍니다.

반드시 라이선스를 확인하세요. 결과물에 보이지 않더라도 남의 저작물을 쓰는 것은 사실입니다. 상업적 사용 가능 여부를 확인하고, 출처를 기록해 두세요.

쓰면 안 되는 것

실존 인물의 인터뷰·방송 영상. 라이선스 문제 이전에 Ch29의 문제입니다.

표정 연기가 강한 콘텐츠. 스포츠 스타일은 입꼬리가 비틀립니다.

4. 전처리 — 진입 시점에 정규화

파이프라인에 넣기 전에 한 번 정리해 두세요. 뒤에서 생길 문제가 크게 줄어듭니다.

프레임레이트를 통일 하세요. 파이프라인 기준값으로 변환합니다. Ch14 문제의 예방입니다.

얼굴 중심으로 크롭 해 두세요. 얼굴 비율이 0.6 미만이면 미리 잘라서 저장합니다. 매번 크롭 옵션에 기대지 않아도 됩니다.

필요한 길이만큼 확보 하세요. 짧으면 원본 + 역재생으로 이어붙여 늘립니다(Ch08 §2의 기법). 단순 반복보다 이음매가 깨끗합니다.

ASCII 파일명으로 저장하세요.

5. 드라이버 메타 기록

카탈로그가 있어야 같은 결과를 다시 만들 수 있습니다. 드라이버마다 이만큼은 남기세요.

파일명 · 출처 · 라이선스 · 길이 · fps · 얼굴비율 · 톤 분류 · 촬영/취득일
그 드라이버로 만든 결과물 목록

어느 드라이버로 어느 결과를 만들었는지 를 적어 두면 나중에 "그때 그 느낌"을 다시 만들 수 있습니다. Ch29의 생성 로그 요구사항과도 이어집니다.

관련 — Ch04(얼굴의 조건) · Ch12(파라미터) · 부록 F(9~13번)

부록 C — 실측 벤치마크

본문에 흩어진 수치를 한자리에 모았습니다. 전부 저자의 실측이고, 측정 조건을 함께 적습니다.

"그거 우리 기계에서 얼마나 걸려요?" — 이 질문을 받았을 때, 그리고 지금 돌아가는 것이 느린 건지 원래 그런 건지 판단해야 할 때 퍼세요. 나갈 때 어림값 둘을 가져가십니다. **Track A는 영상 1초당 약 21초, 그리고 최적화로 짜낼 수 있는 상한은 2~3배입니다.** 전부 한 기계에서 잦 값이니, 여러분의 숫자를 옆에 놓고 비교하세요.

1. 측정 환경

표 C.1 측정 환경

항목	값
GPU	RTX 4070 SUPER 12GB (드라이버 596.36)
Python	3.10.20
PyTorch	2.0.1 + CUDA 11.8
OS	Windows 11
ffmpeg	winget 설치본
conda 환경	2벌 분리 (립싱크 / 리타게팅)

2. 품질 트랙 (Track A) — 대표 케이스

입력

표 C.2 대표 케이스의 입력

항목	값
소스 이미지	1080×1439, 정면, 입 다문 상태
드라이버	1280×720, 15.1초, 차분한 정면 화자
음성	16kHz 모노, 10.67초

출력

표 C.3 대표 케이스의 출력

항목	값
해상도	512×512 (얼굴 크롭)
코덱	H.264 (yuv420p) + AAC
프레임레이트	30fps
프레임 수	320
비트레이트	영상 392 kbps · 음성 52 kbps
파일 크기	0.57MB

단계별 소요

표 C.4 단계별 소요 · 처리율 · 비중

단계	소요	처리율	비중
TTS	수 초	—	별도 — 합계에 넣지 않음
립싱크 (v1.5)	195.8초	~1.6 프레임/초	~85%
리타게팅 (동물 모드)	32.1초	~10 프레임/초	~14%
ffmpeg mux	1~2초	—	<1%
합계	~230초 (~4분)		100%

스케일

표 C.5 음성 길이에 따른 총 소요

음성 길이	총 소요	배수
10.67초	~230초	21.6×
79초	~28분	21×

어림값: 영상 1초당 약 21초. 잡 쿼의 예상 시간 안내에 이 값을 씁니다(Ch28 § 5).

2026-09-03 재측정 — 출하 조합 그대로

원고를 다 쓴 뒤 같은 기계에서 Ch11·Ch12의 출하 조합을 그대로 한 번 더 돌린 값입니다. 로그·출력·JSON은 각 장의 `_work/`에 있습니다.

표 C.6 2026-09-03 재측정 — 출하 조합 그대로

항목	값	근거
립싱크 (MuseTalk v1.5) 벽시계	108초 / 음성 6.07초	ch11_musetalk/_work/probe.json
립싱크 출력 프레임	176 (기대 182 — 6프레임 부족)	같은 파일
리타게팅 (LivePortrait 동물) 벽시계	28초 (워밍업 3.9초 포함)	ch12_liveportrait/_work/run_probe.json
리타게팅 출력	512×512 · 29fps · 176프레임	같은 파일
합계 / 음성 길이	136초 / 6.07초 = 22.4×	위 둘
Chatterbox 다국어 베이스 VRAM	적재 3.00GB · 생성 피크 3.2GB	ch03plus_dialect/_work/vram_probe.json
실시간 립싱크 프레임당	로컬 4070 SUPER 31ms · 클라우드 A100 약 70ms	저자 배포 문서 RT_COLAB_POSTMORTEM (2026-07-15) — Ch28 § 6
분산 렌더 (T4 × 4, 문장 조각 7)	순차 123.2초 → 벽시계 37.1초 (3.32배)	저자 배포 문서 DISTRIBUTED_PERF (2026-07-16)
순서 실험 (사람 얼굴, 2026-07-12)	립싱크 169~196초 · 리타게팅 43~55초 (구성 6종)	Ch15 § 8 표

첫 측정의 21×와 이번의 22.4× — 같은 자릿수입니다. 짧은 입력일수록 워밍업 비중이 커서 배수가 조금 오릅니다.

3. 모델 넷을 한 표에 — 무엇을 언제 쓰나

이 책이 다루는 얼굴 모델 넷을 같은 기계·같은 6.07초 음성 으로 다시 짰습니다. 각 장에 흩어져 있던 숫자를 여기서 한 번에 비교하세요.

표 C.7 얼굴 모델 넷 — 같은 입력(6.07초 한국어 음성 · RTX 4070 SUPER 12GB)

모델	하는 일	벽시계	출력	이 책의 장
Wav2Lip	입 영역만 픽셀 공간에서 다시 그린다	37초 †	640×360 · 178프레임	Ch10
MuseTalk v1.5	입·턱을 잠재공간에서 인페인팅	108초	얼굴 256 ² · 176프레임	Ch11
LivePortrait	표정·모션을 다른 얼굴로 옮긴다(립싱크 아님)	28초 ‡	512 ² · 176프레임	Ch12
EchoMimicV3	립싱크와 상반신 모션을 한 모델로	미측정	—	(이 책 밖)

† 절반 해상도(--resize_factor 2)에서. 원본 720p는 얼굴 검출이 메모리를 넘겨 6분 넘게 못 끝냅니다.

‡ 워밍업 3.9초 포함(얼굴 분석 1.6 + 랜드마크 1.0 + X-Pose 1.3). 두 번째 실행부터는 없습니다.

셋과 하나가 다릅니다. 앞의 셋은 저자의 12GB 카드에서 돌립니다. EchoMimicV3는 **20GB 이상**을 요구합니다 — 무료 노트북의 T4(16GB)에서는 메모리 부족으로 죽고, A100(40GB)이나 L4(24GB)가 있어야 합니다.

이 책이 그 모델을 다루지 않는 이유가 그것입니다. "**소비자 GPU 한 장**"이 이 책의 전제이고, 그 전제를 못 지키는 모델은 아무리 좋아도 다루지 않습니다.

속도만 보고 고르지 마세요. 같은 드라이버·같은 음성으로 Ch09의 싱크 검증을 돌린 결과가 이렇습니다.

표 C.8 같은 입력에서의 싱크 판정 (상위 15프레임 · 얼굴 기준 입 영역)

모델	적중률	우연 대비	조용한 구간	판정
Wav2Lip (영상 드라이버)	73%	1.54배	52%	실패
MuseTalk v1.5 (같은 드라이버)	53%	1.11배	73%	실패
Wav2Lip (정지 사진)	80%	1.68배	22%	통과 §

§ 정지 사진은 조용한 구간에 움직일 것이 없어 셋째 지표를 *검사하지 않은* 것입니다(Ch09 §5).

셋 다 같은 곳을 가리킵니다 — 모델 사이의 차이보다 **드라이버의 차이가 큼**니다. 그 드라이버는 Ch04 §3의 경고선(얼굴 비율 0.40) 아래인 0.235짜리였고, 무엇보다 쉬지 않고 말하는 영상이었습니다. 모델을 바꾸기 전에 입력을 고치세요.

3. 실시간 트랙 (Track B) — 지연 분해

한 문장 응답, GPU 미사용, 로컬 서버 기준. **아래 첫 표는 설계 예산** 이고, 실측은 그 다음 표입니다.

표 C.9 설계 예산 — 겹침 경로(Ch07 §3)를 전제로 한 단계별 배분

단계	소요
요청 전송	~50ms
LLM 응답 (가벼운 모델, 추론 예산 0)	400~900ms
텍스트 정규화	~1ms
TTS (클라우드 무료)	400~800ms
오디오 전송	50~150ms
재생 시작 + 입 반응	~50ms

단계	소요
첫 소리까지 (TTFA)	약 1~2초

표 C.10 실측 2026-09-03 — 같은 부품(gemini-2.5-flash-lite · edge-tts ko-KR-InJoonNeural), 8발화 (ch21_realtime/_work/ttfa.json)

경로	LLM	TTS	첫 소리까지	예산(2초) 안
컴패니언 server.py의 함수 그대로 — 비스트리밍 LLM → 문장 전체 TTS 파일	중앙값 2.18초 (1.8~6.0)	1.32초	3.77초 (최대 7.1)	0 / 8

설계 예산의 1~2초는 첫 문장이 나오는 즉시 TTS로 넘기고 첫 청크에서 재생을 시작하는 겹침 경로의 값입니다. 그 경로를 쓰지 않는 단순 서버는 예산의 두 배가 걸렸습니다 — 원인과 겹침 경로의 실측은 Ch21 § 5에 있습니다.

변동이 큰 구간 — LLM과 TTS. 둘 다 외부 API이므로 평균 말고 p95(100번 중 95번이 이보다 빠른 값)를 따로 관리하세요.

브라우저 렌더 — 3D VRM 1체 기준 60fps 유지. 내장 그래픽에서도 동작.

4. 최적화 효과 (Track A)

표 C.11 최적화 조치와 효과 (Track A)

조치	효과	비고
전처리 캐시	두 번째 실행부터 수십 초 절감	고정 캐릭터에서 필수
배치 크기 ↑ (12GB에서 32)	20~30%	VRAM 한계
반정밀도(half)	메모리·시간 동시 감소	사실상 기본값
30fps → 25fps	~17% (프레임 수 비례)	체감 차이 작음
합계	2~3배	이것이 상한

실시간에 도달하려면 18배가 필요합니다. 남은 6~9배는 품질을 포기해야만 나옵니다. **아키텍처 교체**가 답입니다 (Ch06 § 5).

5. 사다리별 비교 (Ch05 결정표의 수치판)

표 C.12 사다리 다섯 단의 수치 비교

	2D 파츠	3D VRM	Wav2Lip 계열	잠재공간 립싱크	리타게팅
GPU	불필요	불필요	필요(가벼움)	필요	필요
프레임당 비용	<1ms	<1ms	수십 ms	~600ms	~100ms
첫 소리까지	~2초 †	~2초 †	수십 초	분 단위	초~분
재생성 해상도	—	—	96급	256급	전체 얼굴
얼굴 제약	없음	없음	사람	사람만	사람 + 동물
동시 사용자	브라우저마다	브라우저마다	GPU 대수	GPU 대수	GPU 대수

† 겹침 경로 기준(Ch21 §5 실측 1.3~1.7초). 단계를 순차로 기다리는 단순 서버는 3.8초.

6. TTS 비교 (한국어)

표 C.13 한국어 TTS 비교

방식	한국어 품질	한도	감정 제어	지연	판정
클라우드 무료	보통	사실상 무제한	속도·음높이	0.7초/문장 (첫 청크 0.5초)	개발 기본값·실시간 트랙
프리미엄 LLM TTS	매우 좋음	무료는 하루 십여 회	자연어 지시	중간	최종 출력
로컬 다국어 TTS	보통~ 좋음(클린)	무제한	제한적	1.4초/문장 (RTF 0.62 · 적재 39초)	오프라인 대안
로컬 음성 클로닝(다국어)	한국어 나뭇 — 끝음절 늘어짐	무제한	참조 음성	높음	폐기 — 사투리는 LoRA 쪽으로(Ch03A)

지연 열의 두 실측은 같은 한국어 문장 다섯(19~28자)으로 켄 중앙값입니다(ch03_tts/_work/tts_latency.json). 클라우드 무료는 문장 전체가 0.7초, 첫 청크가 0.5초입니다 — Ch21의 겹침 경로가 첫 청크에서 재생을 시작하는 이유가 이 약 0.2초입니다.

로컬 다국어 TTS는 문장당 1.4초이지만 오디오 길이의 0.62배, 즉 실시간보다 빠르게 만들어 냅니다. 긴 문장일수록 상대적으로 유리 합니다. 대신 첫 적재에 39초가 듭니다 — Ch21 §6의 위밍업이 여기서 필요합니다.

마지막 항목이 실제 폐기 사례입니다. 영어 평판은 좋았지만 한국어에서 실패 했습니다. Ch03 §2의 "반드시 한국어로 들어보라"는 여기서 나왔습니다.

7. 측정을 재현하려면

`code/ch06_profile/` 의 계측 스크립트가 위 표의 단계별 소요를 그대로 찍어 줍니다. 다른 GPU에서 돌린 결과를 저장소 Issues에 남겨 주시면 표에 누적합니다.

보고할 때 함께 적어 주세요 — GPU 모델·VRAM, 배치 크기, 해상도, 프레임 수, 정밀도.

이 책의 숫자를 낸 스크립트

본문의 실측은 전부 컴패니언의 아래 스크립트가 낸 값이고, 결과는 각 폴더의 `_work/` 에 JSON으로 남습니다. `scripts/claims_audit.py` 가 본문의 수치를 이 파일들과 대조합니다 — 여기 없는 숫자는 산술값이거나 예시값입니다.

표 C.14 이 책의 숫자를 낸 스크립트

장	스크립트	무엇을 재나	필요한 것
Ch02	<code>doctor.py --gpu</code>	기본·격리 환경의 torch/CUDA	없음
Ch03	<code>test_normalize.py</code>	숫자·단위·약어 읽기 규칙 15건	없음
Ch03A	<code>_work/vram_probe.json</code> (수동 프로브)	Chatterbox 베이스 VRAM 3.0GB	GPU · Chatterbox venv
Ch08	<code>measure_browser.py</code>	헤드리스 크로미움에서 빈 프레임 수	playwright + chromium
Ch10	<code>measure.py</code>	Wav2Lip 벽시계·출력·Ch09 점수	GPU · 격리 venv
Ch11 · Ch12	<code>musetalk_run.py</code> · <code>lp_run.py -- sweep</code>	립싱크·리타게팅 실행·배율 스윙	GPU · conda env 2별
Ch16	<code>_work/measure.json</code> · <code>_work/ auto_points.json</code>	파츠 크기 · 파라미터 층 μ s · 자동 좌표 오차	없음
Ch17	<code>measure.py</code> · <code>sweep_release.py</code>	실제 음성에서 입 열림·닫힘 속도	없음
Ch18	<code>survey.py</code>	실제 VRM·glb 아홉의 빠·키	모델 파일
Ch19	<code>_work/measure.json</code> (Blink 시뮬레이션)	분당 깜빡임 vs 문헌	없음
Ch21	<code>measure_ttfa.py</code> · <code>measure_ stream.py</code>	서버 경로 / 겹침 경로 TTFA	Gemini 키 · 네트워크
Ch22	<code>experiment.py</code>	프롬프트 층별 말투 위반률	Gemini 키 · 네트워크
Ch23	<code>measure.py</code>	실제 녹음의 VAD 조각·끝 지연	없음
Ch23A	<code>measure.py</code>	문장 하나의 발화 끝→통역 첫 소리 (한→영·일)	Gemini 키 · 네트워크

장	스크립트	무엇을 재나	필요한 것
Ch24	<code>experiment.py</code>	창 vs 토큰 예산 · 감쇠	없음
Ch25	<code>_work/measure.json</code> (청킹)	실제 장 셋의 조각 분포	없음
Ch26	<code>schedule.py</code> · <code>turns.py</code>	스레드 풀 배속 · 발언권 규칙별 분포	없음
Ch27	<code>score.py --demo</code>	채점기 점검	없음
Ch28	<code>preflight.py</code> <폴더>	실행 중 내려받기 자리	없음
Ch29	<code>measure.py</code>	생성 로그 규모별 철회·만료 검색 시간	없음
Ch30	<code>fairness.py</code>	수업 큐 규칙별 대기 시간	없음

"필요한 것 — 없음" 인 13개는 `scripts/gates.py` 가 매번 다시 돌립니다. 숫자가 바뀌면 본문과 어긋나고, 그 어긋남이 게이트에서 잡힙니다.

관련 — Ch06(왜 4분) · Ch07(지연 예산) · Ch05(사다리 결정)

부록 D — KDT AI 휴먼 800시간 커리큘럼 ↔ 본책 매핑

저자가 운영한 KDT AI 휴먼 과정(8회차)의 실제 편성표와 본책의 매핑입니다. 이 책의 목차는 책상이 아니라 이 교실에서 나왔습니다. Ch30 §1의 근거 자료.

표기 주의 — 본 부록의 프로젝트명에 등장하는 플랫폼 고유명은 운영 기관의 제품명입니다. 출간본에서는 "AI 휴먼 플랫폼"으로 일반화 표기하고, 상표 면책 문구를 붙입니다.

국비 과정을 맡았거나, 이 책을 교재로 엮을 자리를 찾을 때 펴세요. 실제로 운영한 800시간짜리 편성표가 주차별로 그대로 있고, 교과 하나하나가 어느 장에 붙는지도 표로 있습니다. 여기서 챙길 것은 셋입니다 — 순서 하나를 바꿔 프로젝트를 살린 재배치의 근거(§4), 40시간짜리 보강 모듈 표(§6), 프로젝트별 평가 루브릭(§7).

1. 과정 개요

표 D.1 과정 개요

항목	값
과정명	AI 휴먼 (KDT) · 8회차
총 시수	800시간
기간	2026-08-03 ~ 2026-12-30 (22주)
구성	정규교과 9 + 프로젝트 3 + 기타 3
수료	프로젝트 발표 및 수료식 (12/30)

2. 교과별 시수 (총 800h)

표 D.2 교과별 시수 (총 800시간)

구분	교과목명	시수
정규교과-1	Python 프로그래밍	40
정규교과-2	Python 전처리 및 시각화	40
정규교과-3	머신러닝과 딥러닝	48
정규교과-4	자연어 및 음성 데이터 활용 및 모델 개발	96

구분	교과목명	시수
정규교과-5	음성 데이터 활용한 TTS와 STT 모델 개발	64
정규교과-6	거대 언어 모델을 활용한 자연어 생성	64
정규교과-7	프롬프트 엔지니어링	40
정규교과-8	Langchain 활용하기	56
정규교과-9	RAG (Retrieval-Augmented Generation)	52
프로젝트-1	나만의 음성 모델 만들기	80
프로젝트-2	거대 언어 모델을 활용한 AI 휴먼 답변 기능 구현하기	100
프로젝트-3	사전 데이터 기반 AI 휴먼의 답변 커스텀하기	100
기타-1	OT	4
기타-2	커리어 특강 및 인성 교육	8
기타-3	프로젝트 발표 및 수료식	8
	합계	800

음성 관련 교과(정규 4-5)가 160시간 으로 전체의 20%. 이 과정의 무게중심이 목소리 에 있다는 뜻이고, 본책이 Ch03을 얼굴보다 앞에 둔 근거이기도 합니다.

3. 22주 편성표 (재배치 확정본)

표 D.3 22주 편성표 (재배치 확정본)

주차	날짜	교과 배치	비고
W1	08/03~08/07	OT(기타1) → Python 프로그래밍(정규1)	
W2	08/10~08/14	정규1 → Python 전처리·시각화(정규2)	
W3	08/17~08/21	정규2 → 머신러닝과 딥러닝(정규3)	
W4	08/24~08/28	정규3 → 자연어·음성 데이터 모델개발(정규4)	
W5	08/31~09/04	정규4	
W6	09/07~09/11	정규4	
W7	09/14~09/18	정규4 → TTS/STT 모델개발(정규5)	
W8	09/21~09/25	정규5	
W9	09/28~10/02	정규5 → [프로젝트-1] 나만의 음성모델	

주차	날짜	교과 배치	비고
W10	10/05~10/09	프로젝트-1	
W11	10/12~10/16	P1 마무리 → 커리어특강(10/15) → 거대언어모델(정규6)	
W12	10/19~10/23	정규6	
W13	10/26~10/30	정규6 → 프롬프트 엔지니어링(정규7)	
W14	11/02~11/06	정규7 마무리(~11/03) → ★Langchain(정규8) 시작(11/04)	변경
W15	11/09~11/13	정규8 (11/09~12) → ★[프로젝트-2] 시작(11/13)	변경
W16	11/16~11/20	프로젝트-2	변경
W17	11/23~11/27	프로젝트-2	변경
W18	11/30~12/04	P2 마무리(~12/01) → ★RAG(정규9) 시작(12/02)	변경
W19	12/07~12/11	정규9 (12/07~09) → [프로젝트-3] 시작(12/10)	변경
W20	12/14~12/18	프로젝트-3	
W21	12/21~12/25	프로젝트-3 (12/25 공휴일)	
W22	12/28~12/30	P3 마무리(~12/29) → 발표·수료식(12/30)	변경

4. ★ 재배치의 근거 — 이 책 목차의 출발점

원안

프롬프트(정규7) → [프로젝트-2] 11/04~11/20 → Langchain(정규8) 11/23~12/01
→ RAG(정규9) 12/02~12/10 → [프로젝트-3]

재배치

프롬프트(정규7) → LangChain(정규8) 11/04~11/12 → [프로젝트-2] 11/13~12/01
→ RAG(정규9) 12/02~12/09 → [프로젝트-3] 12/10~12/29 → 수료식 12/30

변경 핵심 — Langchain(정규8)을 프로젝트-2 **앞** 으로 이동.

이유 — 프로젝트-2는 언어모델로 답변 기능을 만드는 과제입니다. 원안대로면 학생들은 Langchain·RAG를 배우지 않은 상태로 프로젝트를 시작하고, 프로젝트가 끝난 다음 주에 그 도구를 배웁니다. 맨손으로 만든 결과물을 도구를 배운 뒤 다시 만들 시간은 없습니다.

불변 — 총 800h, 종료일 12/30, 교과별 시수, 12/02 이후 일정 전부 동일. **순서 하나만 바꿨습니다.**

결과 — 프로젝트 완성도가 달라졌습니다. 여기서 나온 원칙이 본책 목차 전체를 관통합니다.

핵심 도구는 그것을 쓸 프로젝트 바로 앞에 와야 한다.

이 책의 장 순서(지연이 트랙보다 앞, 페르소나가 멀티 아바타보다 앞, TTS가 립싱크보다 앞)가 전부 이 한 줄에서 나왔습니다 — 근거는 Ch30 §1에 있습니다.

운영 노트 — 커리어특강(10/15)이 프로젝트-1 한복판에 끼는 문제가 남았습니다. 10/16(정규6 시작일)으로 옮기면 "10/19 이전" 제약을 지키면서 프로젝트 흐름도 끊지 않습니다.

5. 교과 ↔ 본책 매핑

표 D.4 교과 ↔ 본책 매핑

교과	시수	본책 대응	관계
정규1 Python	40	—	선수 과목. 본책이 전제하는 지식
정규2 전처리·시각화	40	—	선수 과목
정규3 ML·DL	48	Ch11·Ch12의 원리 이해 배경	선수 과목
정규4 자연어·음성	96	Ch03(TTS) · Ch23(STT)	목소리 층 전체
정규5 TTS-STT	64	Ch03 심화 · Ch17(음량 구동)	목소리 층 심화
정규6 LLM 자연어생성	64	Ch07(스트리밍·청킹) · Ch22	머리 층
정규7 프롬프트	40	Ch22(페르소나) · Ch20(태그)	머리 층 — 실습 대상으로 최적
정규8 Langchain	56	Ch24(기억) · Ch26(멀티 아바타)	구현 도구
정규9 RAG	52	Ch25 전체	지식 층
프로젝트-1 음성모델	80	Ch03 + Ch23 결과물	목소리 층 산출물
프로젝트-2 답변기능	100	Ch21~Ch24 재료	실시간 + 인격
프로젝트-3 답변 커스텀	100	Ch25~Ch27 재료	RAG + 평가
기타3 발표·수료	8	Ch15/Ch21/Ch27 완성 장	캡스톤

본책에는 있지만 과정에 없는 것

얼굴 층 전체 (Track A·B). 이 과정은 목소리와 머리에 800시간을 쓰지만, 얼굴을 다루지 않습니다. 본책 Track A·B가 정확히 그 빈 자리입니다.

지연 설계 (Part 2). 어느 교과에도 없습니다. 그리고 프로젝트-2에서 학생들이 가장 많이 막히는 지점이기도 합니다.

평가 하네스 (Ch27). 프로젝트-3의 "커스텀이 잘 되었다"를 시연으로만 증명하고 있습니다.

책임·윤리 (Ch29). 커리어 특강 8시간에 인성 교육이 있지만, 이 기술 고유의 문제는 다루지 않습니다.

6. 이 과정에 본책을 읽는 3가지 방법

(가) 부교재 — 가장 가벼움

정규4·5·7·9와 프로젝트 1·2·3의 참고 도서로 지정하세요. 학생이 필요한 장을 찾아 읽습니다. 편성은 그대로입니다.

(나) 보강 모듈 — 권장

앞 절에서 짚은 빈 자리를 보강 시간으로 채웁니다.

표 D.5 보강 모듈 — 삽입 시점 · 내용 · 시수

삽입 시점	내용	시수
정규5 직후 (W9 초)	Part 2 지연 설계	8h
프로젝트-1 직후 (W11)	Track B 실시간 아바타 (GPU 불필요)	16h
프로젝트-2 직전 (W15)	Ch22 페르소나 집중	4h
프로젝트-3 중 (W20)	Ch27 평가 하네스	8h
수료 전 (W22)	Ch29 책임	4h
	합계	40h

40시간이면 정규교과 하나 분량입니다. 다른 교과의 실습 시간에서 흡수하거나, 차기 개편에서 정규교과-10으로 신설하세요.

(다) 트랙 신설 — 차기 개편

얼굴 층 교과 신설(가칭 *AI 휴먼 아바타 구현*, 64h) + **프로젝트-2를 아바타 통합으로 확장**. 이 과정이 *AI 휴먼*이라는 이름을 온전히 갖게 되는 구성입니다.

7. 프로젝트별 평가 루브릭 제안

Ch30 § 6의 다섯 항목을 이 과정에 맞춘 것입니다.

표 D.6 프로젝트별 평가 루브릭

항목	배점	프로젝트-1	프로젝트-2	프로젝트-3
동작	40	음성 생성 성공	실시간 응답 동작	커스텀 답변 동작
검증	20	한국어 품질 비교 실험	TTF p95 측정	골든셋 회귀 점수
선택 근거	20	TTS 엔진 선택 사유	아키텍처 선택 사유	청킹·검색 전략 사유
실패 기록	10	버린 엔진과 이유	지연 병목 기록	검색 실패 사례
책임	10	음성 복제 동의	AI 고지	근거 표시·거절

검증 항목이 프로젝트마다 구체적이라는 점이 핵심입니다. "잘 됩니다"를 시연으로 증명하는 것과 숫자로 증명하는 것은 학생에게 전혀 다른 경험입니다.

8. 운영 실무 노트

GPU는 공유 자원입니다. Track A를 도입하면 동시 실행이 불가능합니다. Ch28의 잡 큐를 실습 인프라로 쓰세요. 학생이 자기가 만든 큐에 자기 잡을 넣는 구조 자체가 교육입니다.

공휴일과 프로젝트 마감을 겹치지 마세요. W21의 12/25가 그렇습니다. 막바지에 하루가 빠지면 완성도가 눈에 띄게 떨어집니다.

특강은 프로젝트 사이에. 프로젝트 중간에 끼면 집중이 끊깁니다. § 4의 운영 노트 참조.

마지막 주는 마무리 + 발표만. W22는 실질 3일입니다. 새 내용을 넣지 마세요.

관련 — 부록 E(28차시 매핑) · Ch30(커리큘럼으로)

부록 E — 28차시 강의 자료 ↔ 본책 매핑

저자가 제작·운영한 *AI 휴먼* 통합 커리큘럼 28차시(슬라이드 + 강의대본 + 검증 노트북)와 본책의 매핑입니다. 강의가 *차시별* 실습 이라면, 책은 그것을 *하나의 서사*로 다시 펜 것입니다.

이미 강의 자료를 가지고 계시거나, 이 책의 어느 장이 어느 차시와 짝인지 찾을 때 펴세요. 28차시와 본책의 대응이 §2에 한 표로 있습니다. §3·§4는 서로에게 없는 것을 적었습니다 — 강의에만 있는 넷, 책에만 있는 여섯. 마지막 §5에는 강의 순서를 바꿔야 할 자리 네 곳을 적어 두었습니다.

1. 28차시 구성 개요

표 E.1 28차시 구성 개요

트랙	차시	성격
에이전트	01~08	상태·분기·협업·실행기 — 머리 층의 기반
멀티모달 감각	09~12	보고·만들고·말하고 — 얼굴·목소리 층
모델 길들이기	13~14	페르소나 학습
지식·검색(RAG)	15~21	임베딩·검색·평가 — 지식 층
기억·개인화	22	기억 층
운영·통합	23~26	배포·안전·윤리
캡스톤	27~28	통합 프로젝트

규격 — 1280×720 슬라이드 16~21장 · 강의대본 약 2만~3만자(90분) · 검증 노트북(Colab GPU) · 빈페이지 0 · 잘림 0.

환경 — 무료 GPU 노트북 기준. 전 노트북에 환경 자동감지 + 모델 캐시 부트스트랩 적용(재다운 로드 방지).

2. 차시별 매핑

표 E.2 차시별 매핑

차시	주제	본책 대응	관계
01	LangGraph — 상태·분기·반복	Ch26(진행자 구조)	멀티 아바타 오케스트레이션의 기반
02	스트리밍·저지연 (토큰 스트리밍·TTFB)	Ch07	직계. 본책이 TTFA로 확장
03	실시간 음성 STT→TTS	Ch03 · Ch23	직계
04	아바타 립싱크 (Wav2Lip·SadTalker)	Ch10 · Ch11	직계. 본책이 MuseTalk·Live Portrait로 확장
05	LCEL·구조화 출력·툴콜	Ch20(태그 파싱) · Ch25	형식 강제의 대안 비교
06	멀티 에이전트 (Supervisor)	Ch26	직계
07	에이전트 하네스 (ReAct·가드레일·관측)	Ch27 · Ch29	평가·안전
08	고급 에이전트 패턴 (Reflexion·Plan·Execute·ToT)	Ch26 심화	게임 추론 판정에 적용
09	멀티모달 VLM (이미지·문서·화면)	Ch04(소스 적합성 판정)	얼굴 조건 자동 검사에 활용
10	이미지 생성 — 아바타 얼굴	Ch04 § 4	직계. 소스 이미지 생성
11	음성 클로닝·감정 TTS	Ch03 · Ch29	직계. 본책은 동의 요건을 함께 다룸
12	실시간 대화·바지인	Ch23	직계
13	LoRA·QLoRA 파인튜닝	Ch22 § 8	"언제 파인튜닝으로 가나"
14	DPO 선호 정렬	Ch22 § 8	페르소나 일관성 강화
15	한국어 임베딩·청킹	Ch25 § 3~4	직계
16	Advanced Retrieval (하이브리드·리랭킹)	Ch25 § 4	직계
17	멀티모달 RAG (CLIP/ColPali)	Ch25 확장	도표 문서 검색
18	GraphRAG·지식그래프	Ch25 확장	관계 기반 검색
19	Agentic RAG (자가채점·재검색)	Ch25 § 6	"언제 검색할 것인가"
20	RAGAS 평가	Ch27	사실성 채점
21	평가 하네스 (골든셋·회귀게이트·CI)	Ch27	직계
22	지속 메모리·개인화	Ch24	직계. 3층 구조
23	MCP 툴 연결	Ch25 § 6(도구 호출) · Ch28	외부 연동
24	안전 가드레일 (인젝션·PII)	Ch29 § 5	대화 아바타 안전
25	배포·서빙 (FastAPI·양자화)	Ch28	직계
26	책임 AI·딥페이크 윤리·위터마킹	Ch29	직계. 04차시와 짝

차시	주제	본책 대응	관계
27	통합 캡스톤	Ch21 / Ch27	완성 장
28	프로젝트 키포프·로드맵	Ch30	커리큘럼 설계

3. 강의에는 있고 책에는 얇은 것

에이전트 패턴 심화 (01·06·08). 본책은 Ch26에서 멀티 아바타 관점으로만 다룹니다. 에이전트 자체를 깊이 파려면 강의 자료가 낫습니다.

RAG 변종 (17·18). 멀티모달 RAG와 GraphRAG는 본책 Ch25에서 한 문단으로만 언급합니다.

파인튜닝 실습 (13·14). 본책은 "대부분의 경우 필요 없다"는 입장이라 실습을 신지 않습니다.

MCP (23). 본책은 도구 호출을 Ch25에서만 짧게 다룹니다.

4. 책에는 있고 강의에는 없는 것

Part 2 전체 — 지연의 기초. 02차시가 토큰 스트리밍을 다루지만, *4분과 2초의 거리* 라는 아키텍처 판단은 강의에 없습니다. 본책의 고유 기여입니다.

Ch05 — 사다리 결정 트리. 04차시가 4가지 접근법을 비교하지만, *목표 지연 · 얼굴 종류 · 얼굴 고정 여부* 라는 세 질문으로 정리된 결정 트리는 본책에서 새로 만든 것입니다.

Ch12~Ch14 — 리타게팅 · 비인간 얼굴 · fps 드리프트. 강의는 Wav2Lip·SadTalker까지입니다. MuseTalk + LivePortrait 조합과 거기서 나온 실패들은 전부 본책 고유입니다.

Track B 전체 — GPU 없는 아바타. 2D 파츠 리깅과 VRM은 강의에 없습니다. 그런데 실무에서 가장 자주 쓰는 트랙입니다.

Ch09 — 싱크 검증. 잘못된 지표 이야기는 본책 고유입니다.

Ch19 — 생명감 설계. 깜빡임·호흡·시선의 난수 설계.

5. 강의와 책을 함께 쓰는 법

강의를 하는 사람에게

차시에 들어가기 전에 책의 해당 장을 읽히세요. 강의는 "오늘은 이것을 해 봅니다" 이고, 책은 "왜 그 순서인가" 입니다. 순서를 먼저 알면 같은 실습이 다르게 읽힙니다.

04차시 앞에 Ch05를 넣으세요. 립싱크 실습 전에 결정 트리를 보면, 학생이 "왜 이걸 배우는지"를 압니다.

11차시와 26차시를 붙이세요. 음성 클로닝을 배운 그날 윤리를 다뤄야 합니다. 15차시 뒤로 미루면 늦습니다.

21차시를 캡스톤 앞으로. 평가 하네스를 알고 캡스톤을 시작하는 것과, 만들고 나서 평가를 배우는 것은 다릅니다. 부록 D § 4의 원칙 그대로입니다.

책으로 독학하는 사람에게

노트북이 필요하면 강의 자료의 실습 노트북을 쓰세요. 본책의 `code/` 폴더와 나란히 써도 됩니다.

Track A에 들어가기 전에 04차시 노트북으로 감을 잡으세요. 무료 GPU 환경에서 돌아갑니다. 로컬 환경을 만들기 전에 결과부터 볼 수 있습니다.

6. 통합 재배치 이력 (참고)

28차시는 한 번 크게 재배치되었습니다. 무엇을 왜 옮겼는지 남겨 둡니다.

멀티모달·파인튜닝을 RAG 앞으로. 감각(보기·얼굴·목소리·실시간)을 먼저 완성하고, 모델을 페르소나에 맞게 길들인 뒤, 지식(RAG)·기억·운영·캡스톤으로 간다는 흐름.

중복 흡수 2건. 관측·디버깅을 21차시(평가 하네스)로, 장기기억 아키텍처를 22차시(지속 메모리)로 흡수.

신설 12개 차시. 08·09·10·11·12·13·14·17·18·24·25·26.

이 재배치의 사고 방식이 부록 D § 4의 800시간 재배치와 같습니다. 도구는 그것을 쓸 자리 바로 앞에.

관련 — 부록 D(800시간 매핑) · Ch30(커리큘럼으로)

부록 F — 실패 카탈로그 (재현 금지 목록)

저자가 실제로 밟은 실패입니다. 전부 재현되었고, 전부 버렸습니다.

증상으로 찾아보세요. 여러분이 지금 겪는 것이 여기 있을 확률이 높습니다.

Track A(1~19) 는 결과물이 망가지는 실패이고, **Track B(20~34)** 는 살아 있어 보이지 않는 실패이며, **Track C(35~44)** 는 인격이 무너지는 실패입니다. 성격이 다릅니다. 마지막 **45~50** 은 트랙을 가리지 않고 파이프라인을 이을 때 나는 실패입니다.

무언가 이상한데 원인을 모르겠을 때 가장 먼저 펴세요. 아래 인덱스에서 눈에 보이는 증상을 찾아 번호로 건너뛰면 됩니다. 항목마다 원인 한 줄과 대안 한 줄이 붙어 있고, 전부 제가 직접 밟은 뒤 버린 것들입니다. 검색으로 잘 안 나오는 것만 모았습니다 — 에러가 안 나는 실패, 로그가 깨끗한 실패, 이를 뒤에야 드러나는 실패.

Track A — 증상별 빠른 인덱스

표 F.1 Track A — 증상별 빠른 인덱스

지금 보이는 증상	번호
동물/캐릭터 얼굴에 사람 입술·이가 그려진다	1
머리가 좌우로 심하게 흔들린다	2, 10
영상이 멈췄다 튕겨나 끊긴다	3, 6
얼굴이 작아지고 귀가 잘린다	4
입은 부지런히 움직이는데 발음과 안 맞는다	5
입 모양이 뭉개진다	7, 9
뒤로 갈수록 소리보다 입이 늦는다	8
눈이 한 번도 안 깜빡인다	11
입이 거의 안 움직인다 (모션이 약하다)	12, 부록 B § 1
입꼬리가 좌우로 비틀린다	13
파일을 못 읽는다 / 조용히 빈 결과가 나온다	14
"fps must be float" 오류	15
PowerShell 스크립트 파싱 오류	16
"자기 자신을 복사할 수 없음" 오류	17

지금 보이는 증상	번호
지표는 좋아지는데 영상은 나빠진다	18
정지 화면으로는 괜찮은데 재생하면 이상하다	19

A. 기법 자체의 한계 (1~8)

1. 실사 립싱크를 동물 얼굴에 직접 적용

증상 — 고양이 얼굴에 사람의 입술과 치아가 그려짐.

원인 — 모델이 사람 얼굴 영상으로만 학습됨. 학습 분포 밖에서는 아는 것을 그림.

교훈 — 예러가 나지 않는다는 것이 가장 위험합니다.

대안 — 사람 드라이버에 립싱크 → 리타게팅으로 모션만 전사 (Ch13).

2. 픽셀 프레임 재배열 (음량 → 입 벌림)

증상 — 머리 포즈가 프레임마다 튀어 좌우 90도씩 흔들림.

원인 — 각 프레임의 머리 각도가 다르다는 것을 무시하고 입 모양만 보고 재배열.

교훈 — 프레임은 입만 담고 있지 않습니다.

3. 픽셀 시간워핑 / DTW 정렬

증상 — 프레임이 멈췄다 튀는 밀림·끊김.

원인 — 시간축을 늘이고 줄이면 같은 프레임이 반복되거나 건너뛰어짐.

4. 스티칭 ON (전신에 되붙이기)

증상 — 얼굴이 작아지고 왼쪽 귀가 잘림.

원인 — 크롭 경계와 원본 합성 영역의 불일치.

대안 — 프레이밍이 중요하다면 스티칭 OFF (Ch12 §3).

5. 모션 데이터 음량 재타이밍

증상 — 입은 부지런히 움직이는데 발음과 전혀 안 맞는 *가짜 싱크*.

원인 — 음량은 음소가 아닙니다. "브"는 크지만 입을 닫습니다.

6. 모션 데이터 발음 DTW 재배열

증상 — 타이밍이 뭉개짐.

원인 — 5번의 개선 시도였지만, 재배열 자체가 원인이었음.

7. 보간 GAIN 외삽

증상 — 입 모양이 뭉개짐.

원인 — 없는 중간 상태를 만들어내면 어느 것도 아닌 형태가 됨.

8. 영상↔음성 길이 불일치 (10.3s vs 10.67s)

증상 — 뒤로 갈수록 싱크 드리프트.

원인 — 프레임레이트 불일치 (29 vs 29.97 = 3.3%).

대안 — 입력 앞 `-r` 로 fps 재해석. `setpts`로 늘리지 말 것 (Ch14).

B. 드라이버 선택 문제 (9~13)

9. 배율 과다 (2.5~3.5)

증상 — 혀가 나오고 입이 왜곡됨.

대안 — 1.0~1.5. 배율을 올리기 전에 소스 이미지를 바꾸세요.

10. 표정 전체 전사 + 머리가 움직이는 드라이버

증상 — 동물이 눈알을 굴리고 머리를 흔들.

원인 — 사람 얼굴에서 자연스럽던 움직임이 다른 구조의 얼굴에서는 기괴해짐.

대안 — 차분한 드라이버를 쓰거나 입 영역만 전사.

11. 입 영역만 전사

증상 — 싱크는 선명한데 눈이 얼어붙어 안 깜빡임.

원인 — 소스 이미지의 눈 상태가 그대로 고정됨.

교훈 — 이것은 버그가 아니라 트레이드오프입니다 (Ch12 § 3).

12. 애니메이션 스타일 드라이버

증상 — 입이 거의 안 움직임.

원인 — 모션 자체가 약하고, 얼굴 구조가 학습 분포와 다름.

13. 슷폼 스타일 표정 과장 드라이버

증상 — 입꼬리가 좌우로 비틀림. "현란함".

대안 — 차분하게 정면을 보며 또박또박 말하는 영상 (Ch04 § 3).

C. 환경 · 코드 버그 (14~17)

14. 한글 파일명

증상 — Windows에서 OpenCV가 파일을 못 읽음. 조용히 실패하거나 예외.

원인 — cp949 인코딩 문제.

대안 — ASCII 경로로 복사한 뒤 처리. 파이프라인이 자동으로 하게 만드세요 (Ch15 §4, 부록 H).

15. fps가 정수형

증상 — "fps must be float" 오류.

대안 — 모션 데이터 저장 시 실수형으로.

16. PowerShell 파라미터 주석의 한글·괄호

증상 — 스크립트 파싱 오류.

대안 — 주석은 ASCII로.

17. 음성이 이미 대상 폴더에 있을 때

증상 — "자기 자신을 복사할 수 없음" 오류.

대안 — 원본과 대상 경로가 같으면 복사 건너뛰기.

D. 검증 방식의 오류 (18~19) ★ 가장 값비싼 실패

18. 잘못된 지표 — 입 벌림과 음량의 상관계수

증상 — 지표는 계속 좋아지는데 영상은 계속 나빠짐.

원인 — 진짜 립싱크는 음소에 맞춤. 음량 상관은 원래 0에 가까움. 이 숫자를 올리려는 시도가 2·3·5·6·7번 실패를 전부 만들어냄.

교훈 — 잘못된 지표는 아무것도 측정하지 않는 것보다 나쁩니다. 눈을 대체해 버리기 때문입니다.

대안 — 발화구간 × 입 활발도 상위 N프레임 적중률 (Ch09).

19. 정적 프레임만 보고 판단

증상 — 흔들림·끊김·뭉개짐을 전부 놓침.

원인 — 이 결함들은 전부 프레임 *사이*에서 일어남.

대안 — 연속 프레임으로, 큰소리 구간과 조용한 구간을 나눠서, 특히 **끝부분** 을 확인 (Ch09 § 6).

E. Track B — 살아 있어 보이지 않는 실패 (20~34)

Track A의 실패는 *결과물이 망가집니다*. Track B의 실패는 **아무것도 망가지지 않는데 어색합니다**. 에러가 없고 로그도 깨끗한데 사용자가 불편해합니다. 그래서 찾기가 더 어렵습니다.

오디오 · 립싱크 (20~24)

20. 첫 메시지만 입이 안 움직인다

브라우저는 사용자 상호작용 전에 오디오를 재생하지 못하게 막습니다. 오디오 컨텍스트가 정지 상태로 생성되어 음량이 0으로 읽힙니다. → **첫 클릭에서 컨텍스트를 깨우세요**(Ch17 § 3, 부록 L § 1).

21. 입이 덜덜 떨린다

날것의 음량을 그대로 입 벌림에 넣었습니다. 음량은 프레임마다 심하게 요동칩니다. → 비대칭 평활화(여는 쪽 빠르게, 닫는 쪽 느리게) (Ch17 § 4).

22. 조용한데 입이 달짝인다

무음 구간의 미세한 잡음이 그대로 들어갑니다. → 임계값 아래는 0으로 눌러 확실히 닫으세요.

23. 어떤 오디오는 입이 조금만, 어떤 건 항상 최대로 벌어진다

고정된 값으로 음량을 정규화했습니다. 최대 음량은 파일마다 다릅니다. → 최근 구간의 최대값을 추적해 동적으로 정규화.

24. 말이 끝났는데 입이 반쯤 벌어진 채 굳는다

재생 종료 이벤트에서 입 벌림을 0으로 되돌리지 않았습니다. **매우 눈에 띕니다**. → 종료 시 리셋 + 일정 시간 음량 갱신이 없으면 자동으로 닫히는 안전장치.

2D 파츠 (25~26)

25. 눈을 감기면 어색하다

눈 이미지를 **중앙 기준** 으로 세로 축소했습니다. 위아래에서 동시에 좁아집니다. 실제 눈꺼풀은 위에서 내려옵니다. → **아래 기준 스쿼시**(Ch16 § 4).

26. 변형할 때 파츠 경계가 잘린다

파츠를 딱 맞게 잘랐습니다. 눈을 늘리거나 입을 크게 벌릴 여유가 없습니다. → 자를 때 여백을 남기세요.

생명감 (27~30)

27. 깜빡이는데 메트로놈 같다

정확히 3초마다 깜빡이게 만들었습니다. **살아 있는 것은 규칙적이지 않습니다.** → 난수 주기 + 비대칭 속도 + 가끔 이중 깜빡임(Ch19 §3).

28. 캐릭터가 취한 것처럼 보인다

미세 움직임의 진폭이 큼니다. → 아주 작게. **의식적으로 알아채면 과한 것입니다.**

29. 계속 쳐다봐서 부담스럽다

시선을 카메라에 고정했습니다. → 몇 초에 한 번 짧게 돌렸다 돌아오기.

30. 눈만 굴려서 기괴하다

시선을 옮기는데 머리가 안 따라갑니다. → 눈이 먼저, 머리가 조금 따라가는 순서.

제스처 (31~34)

31. "같이 운동해요" 라면서 끄덕이기만 한다

동작 목록에 스쿼트가 있는데 LLM이 `nod` 를 골랐습니다. **모델은 목록을 보지만 그것이 언제 쓰이는지는 모릅니다.** → 조건 지시 한 줄을 프롬프트에 추가(Ch20 §3). 실제로 이 한 줄이 코치를 고쳤습니다.

32. 아바타가 "대괄호 익사이티드"라고 읽는다

감정 태그가 TTS로 그대로 넘어갔습니다. → 프롬프트와 코드 두 겹으로 제거(Ch20 §4·§8).

33. 팔이 이상한 곳으로 간다

두 동작이 동시에 재생됐습니다. → 중첩 금지. 새 동작이 오면 현재 자세에서 보간해 전환.

34. 말이 끝났는데 팔이 허공에서 움직인다 / 팔을 든 채 굳는다

동작이 말보다 길거나, 기본 자세 복귀가 없습니다. → 1~2초 안에 끝내고 반드시 복귀(Ch20 §6).

F. Track C — 인격이 무너지는 실패 (35~44)

35. 시민이 첫 턴에 마피아를 정확히 지목한다 ★

모든 캐릭터에게 같은 컨텍스트를 넘겼습니다. 추리한 것이 아니라 **그냥 알고 있었습니다.** 게임이 성립하지 않습니다. → 캐릭터별 컨텍스트를 진행자가 분배(Ch26 §3). **누출 테스트를 반드시 하세요** — 우연보다 정답률이 높으면 어딘가 새고 있습니다.

36. 아바타가 자기 목소리에 대답하며 무한루프에 빠진다 ★

스피커 소리를 마이크가 다시 잡았습니다. 헤드셋에서는 안 보이고 **전시장에 스피커로 놓는 순간 나타납니다.** → 자기 목소리 차단 또는 말하는 동안 임계값 상향(Ch23 §4).

37. 검색을 붙이자 캐릭터가 매뉴얼처럼 말한다 ★

검색된 문단의 문체를 모델이 따라잡니다. "자, 같이 해봐요!"가 "본 동작은 하지근을 주동근으로 하며"가 됩니다. → 페르소나 지시를 검색 결과 뒤에 한 번 더(Ch25 §5).

38. 대화가 길어지자 말투가 풀린다

초반 시스템 프롬프트가 희석됩니다. → 시스템 프롬프트를 항상 맨 앞에, 요약할 때 페르소나 규칙도 함께 남기기(Ch22 §4, Ch24 §3).

39. 어려운 질문에서 갑자기 "저는 AI 언어모델로서"가 나온다

곤란한 질문을 받으면 기본 어시스턴트 모드로 돌아갑니다. 그 순간 캐릭터는 죽습니다. → 모르는 것을 캐릭터의 말투로 말하는 방식을 미리 정의.

40. 슬픈 얘기를 하니 반말 캐릭터가 존댓말을 쓴다

감정적 주제에서 모델이 정중해집니다. → 38번·39번과 같은 대응. 평가 하네스의 붕괴 유도 항목에 넣으세요(Ch27 §2).

41. 어제 "커피 좋아해" 라던 캐릭터가 오늘 "커피 안 마셔"라고 한다

사용자에 대한 기억만 관리하고 캐릭터 자신에 대한 기억을 안 남겼습니다. 자기모순이 인격을 가장 빠르게 무너뜨립니다. → 캐릭터 자기 기억을 별도 관리(Ch24 §7).

42. 5분 전에 알려준 이름을 다시 묻는다

단기 컨텍스트를 턴 수로만 잘랐거나, 아예 안 넣었습니다. → 토큰 기준으로 자르고 요약 층을 두세요(Ch24 §2·§3).

43. 두 캐릭터가 동시에 말해서 아무것도 안 들린다

텍스트를 병렬로 만든 뒤 재생도 병렬로 했습니다. → 텍스트는 병렬, 재생은 순차(Ch26 §5).

44. 프롬프트 한 줄 고쳤더니 다른 게 조용히 나빠졌다 ★

말투를 고쳤는데 며칠 뒤 지어내기가 늘어난 것을 발견합니다. → 자동 회귀 게이트가 없으면 사람의 주의력으로는 못 잡습니다(Ch27).

G. 트랙을 가리지 않는 실패 — 파이프라인을 이을 때 (45~50)

앞의 44개는 한 트랙 안에서 났습니다. 이 여섯은 층과 층을 잇는 자리에서 납니다 — 언어·오디오 포맷·입력 종류·좌표계·검출기·드라이버 선택. 하나같이 에러 없이 조용히 틀립니다.

45. 한국어를 넣었는데 영어로 말한다 — 이미지 프리셋용 한→영 자동 번역이 TTS 대사까지 번역했습니다. 범인은 모델이 아니라 파이프라인의 앞단이었습니다. TTS·읽기형 프리셋은 번역 제외 목록에 넣고, 이름 접두로 자동 제외되게 하세요(Ch03 §3). 방언 프리셋을 새로 만들 때 같은 사고가 되풀이돼서 구조를 바꿨습니다.

46. 서버는 WAV를 냈는데 브라우저에서 소리가 안 난다 — float32 WAV였습니다. 일부 브라우저·플레이어와 파이션 wave 모듈이 이것을 못 읽습니다. 마지막에 16비트 PCM으로 바꾸세요(Ch03 §3).

47. 립싱크 모델에 사진 한 장을 넣었더니 죽는다 — 립싱크 모델은 영상을 받습니다. 사진은 리타게팅으로 영상을 먼저 만든 뒤 넣으세요(Ch15 §7). 오류 문구가 `save_dir_full referenced before assignment` 처럼 엉뚱해서 원인을 못 찾기 쉽습니다.

48. 그림은 그대로인데 입이 왼쪽 눈에 생긴다 — 2D 오버레이(눈꺼풀·입)의 좌표를 이미지 비율(`cx=47 cy=51.5`)로 박아 두고 나중에 그림을 바꾸면, 좌표는 그대로인데 얼굴만 옮겨 가서 **입 타원은 눈 위에 그려집니다**. 제 마스크트 앱이 실제로 이 상태였습니다 — 깜빡임 타원은 오른눈 바깥에, 입 타원은 왼눈 위에 있었습니다.

파츠 방식(Ch16)이 오버레이보다 나은 이유가 이것입니다. 파츠는 **잘라낸 자리**가 좌표를 들고 있어서, 그림을 바꾸면 파츠도 같이 다시 만들게 됩니다. 오버레이를 쓸 거면 좌표를 매니페스트로 빼고, 그림을 바꿀 때 매니페스트도 함께 바꾸도록 검사를 두세요(Ch16 §3의 `manifest.json`).

49. 동물 사진인데 체커가 "얼굴 0개"라고 한다 — 소스 체커(Ch04)와 싱크 검증기(Ch09)가 둘 다 **사람 얼굴 검출기**를 씁니다. 고양이·개에서는 0개를 내거나(체커) 화면 비율로 폴백해 애먼 가슴을 찍니다(검증기). 점수가 나오더라도 그것은 입을 썬 값이 아닙니다. 체커는 `--box x,y,w,h`로 얼굴 상자를 직접 주고, 검증기의 값은 **사람 드라이버 단계에서만** 믿으세요. 동물 결과는 눈으로 봅니다.

50. 얼굴이 가장 큰 드라이버를 골랐더니 결과가 가장 나빴다 — 드라이버는 축이 둘입니다(Ch04 §3). 얼굴 비율이 1위(0.671)여도 머리 움직임이 1위(0.061)면 `region=lip`으로도 못 막습니다 — 프레임마다 크롭이 따라 움직여 얼굴이 흔들립니다. **얼굴 크고 차분한 것**(0.642 / 0.006)을 고르세요. 체커의 움직임 상한이 0.25로 느슨했던 것도 이 사고의 공범입니다.

표 F.2 트랙을 가리지 않는 실패 — 증상별 인덱스

지금 보이는 증상	번호
한국어를 넣었는데 영어로 말한다	45
서버는 WAV를 냈는데 브라우저에서 소리가 안 난다	46
립싱크 모델에 사진 한 장을 넣었더니 죽는다	47
입이 엉뚱한 자리에 생긴다	48
동물 사진인데 체커가 "얼굴 0개"라고 한다	49
얼굴이 가장 큰 드라이버가 결과는 가장 나쁘다	50

H. Track B·C 증상별 인덱스

표 F.3 Track B·C — 증상별 인덱스

지금 보이는 증상	번호
첫 메시지만 입이 안 움직인다	20
입이 떨린다 / 조용한데 달삭인다	21, 22
오디오마다 입 벌림 정도가 들쭉날쭉하다	23
입이 반쯤 벌어진 채 굳는다	24
눈 감기는 게 어색하다	25
변형할 때 파츠 경계가 잘린다	26
깜빡임이 메트로놈 같다	27
캐릭터가 취한 것처럼 보인다	28
시선이 부담스럽다 / 눈만 굴린다	29, 30
말과 동작이 안 맞는다	31
태그를 소리 내어 읽는다	32
팔이 이상한 곳으로 / 든 채 굳는다	33, 34
캐릭터가 알 수 없는 것을 안다	35
자기 목소리에 대답한다	36
검색 붙이니 말투가 바뀐다	37
대화가 길어지면 말투가 풀린다	38
"저는 AI 언어모델로서"가 나온다	39
감정에 따라 말투가 바뀐다	40
캐릭터가 자기모순을 일으킨다	41
이름을 다시 묻는다	42
두 캐릭터가 동시에 말한다	43
프롬프트 한 줄 고쳤더니 다른 게 조용히 나빠졌다	44

I. 실패에서 얻은 일곱 줄

1. **에러가 안 나는 실패가 가장 위험합니다.** 1·14번, 그리고 Track B의 거의 전부.
2. **증상을 덮는 해법은 반드시 돌아옵니다.** 3·6·7번, 그리고 setpts.
3. **드라이버는 결과에 안 보이지만 결과를 결정합니다.** 9~13번 전부.
4. **잘못된 지표는 사람의 눈을 대체합니다.** 18번이 다른 다섯 개를 만들어냈습니다.

5. 정지 화면은 아무것도 증명하지 않습니다. 19번.
6. Track B의 실패는 "고장"이 아니라 "어색함"입니다. 로그가 깨끗한데 사용자가 불편해합니다. 그래서 직접 써 보는 것 말고는 발견할 방법이 없습니다. 20·24·27·28번이 그렇습니다.
7. Track C의 실패는 시간이 지나야 드러납니다. 5분 대화에서는 안 보이고 30분·이틀 뒤에 보입니다(38·41·42번). 그래서 자동 회귀 게이트가 유일한 방어선입니다(44번).

왜 실패의 성격이 트랙마다 다른가

Track A는 만들어진 결과물을 봅니다. 파일이 나오고, 그것을 열어 봅니다. 실패가 눈에 보입니다 — 입술이 이상하거나, 싱크가 밀리거나, 귀가 잘려 있습니다.

Track B는 실시간으로 흘러갑니다. 붙잡고 들여다볼 프레임이 없습니다. 대부분의 실패가 "뭔가 이상한데 뭔지 모르겠다"는 형태로 옵니다. 사용자는 이유를 말해 주지 못합니다.

Track C는 축적됩니다. 한 턴은 멀쩡합니다. 스무 턴째에 말투가 풀리고, 이틀째에 자기모순이 나옵니다.

그래서 검증 방법도 트랙마다 다릅니다. A는 자동 게이트(Ch09), B는 직접 써 보기(부록 L §9 체크리스트), C는 회귀 하네스(Ch27). 한 가지 방법으로 셋을 다 잡으려 하지 마세요.

관련 — Ch09(검증) · Ch13(비인간 얼굴) · Ch14(fps) · 부록 H(Windows)

철회 시 제작자는 ()일 이내에 원본 및 파생물을 삭제하고 그 결과를 통지한다.
이미 배포된 결과물에 대해서는 () 조치를 취한다.

7. 표시

제작자는 결과물이 AI로 생성되었음을 () 방식으로 표시한다.

8. 금지

제작자는 다음 용도로 결과물을 제작·사용하지 않는다.

- 실제 발생한 사실인 것처럼 오인하게 하는 용도
- 성적·모욕적·협박적 내용
- 신원 확인 우회 등 위조 목적
- 동의자가 명시적으로 거부한 주제

동의자 : (서명) 일자 :
제작자 : (서명) 일자 :

작성 지침

3번(제작할 결과물)을 구체적으로. "AI에 활용" 같은 포괄 문구는 동의로 보기 어렵습니다.

4번의 기간을 반드시 채우세요. 무기한은 위험의 무기한 보유입니다.

6번의 철회 이행 기한 을 명시하세요. "요청 시 삭제"는 이행 기준이 없습니다.

대본 통제권 을 명시하세요. 얼굴 사용에 동의한 것과, 임의의 문장을 말하는 데 동의한 것은 다릅니다.

2. 고인의 얼굴·음성을 쓰는 경우

본인의 동의를 받을 수 없다 는 점에서 앞의 경우와 근본적으로 다릅니다. 그래서 확인할 것이 더 있습니다.

유족의 뜻을 확인하세요. 가능한 범위의 유족 전원. 의견이 갈리면 만들지 않는 편이 낫습니다.

결과물을 볼 사람이 그것을 원하는지 확인하세요. 만드는 사람과 보는 사람이 같지 않을 수 있습니다.

대본을 신중히 쓰세요. 그 존재가 실제로 했을 법한 말과, 남은 사람이 듣고 싶은 말은 다릅니다. 후자는 위로가 되기도 하고 상처가 되기도 합니다.

생전에 표명한 의사가 있으면 우선합니다.

배포 범위를 좁게 잡으세요. 개인·가족 범위를 넘어가는 순간 성격이 달라집니다.

3. 생성 로그 스키마

Ch29 § 4의 "추적 가능성" 요구를 만족하는 최소 스키마입니다.

generation_id	고유 ID
created_at	생성 시각
operator	생성을 실행한 사람/계정
consent_id	동의서 참조 (없으면 생성 거부)
source_assets	원본 파일 해시 목록
driver_asset	드라이버 영상 참조 (부록 B 카탈로그 ID)
script_text	사용된 대본 전문
pipeline	단계·모델·버전·주요 파라미터
output_path	결과물 경로 및 해시
watermark	적용된 표시 방식
distribution	배포 범위
retention_until	보관 만료일

consent_id 없이 생성이 불가능하게 만드세요. 절차가 아니라 코드로 강제해야 지켜집니다.

source_assets를 해시로 남기세요. 철회 요청이 왔을 때 어느 결과물을 지워야 하는지 거꾸로 찾을 수 있습니다.

4. 표시(워터마킹) 방식

표 G.1 표시(워터마킹) 방식 — 강도 · 제거 난이도

방식	강도	제거 난이도	쓰는 곳
화면 내 텍스트/배지	약함	크롭으로 제거 가능	기본. 항상 넣으세요
시작·종료 고지 화면	약함	잘라내기 가능	영상 콘텐츠
파일 메타데이터	약함	재인코딩으로 소실	항상 넣으세요 (비용 0)
비가시 워터마크	중간	강한 압축·크롭에 취약	배포용
대화 중 AI 고지	—	—	실시간 아바타 필수

어느 것도 완벽하지 않습니다. 겹쳐 쓰세요. 그리고 워터마크가 있으니 안전하다고 생각하지 마세요.

표시는 세 층이고, 세 층이 서로 다른 방식으로 실패합니다 ★

위 표는 강도만 적었습니다. 실무에서 더 중요한 것은 **어떻게 없어지는가**입니다.

표 G.2 표시의 세 층과 각각이 없어지는 방식

층	무엇을 하는가	어떻게 없어지는가
① 보이는 표시	화면 배치 · 고지 화면	크롭 한 번
② 파일에 붙인 것	메타데이터 · 서명된 출처 기록	재인코딩 한 번 (스크린샷·SNS 업로드 포함)
③ 픽셀·파형에 심은 것	비가시 워터마크	강한 압축 · 리샘플 · 화면 재촬영

세 층은 대체재가 아니라 보완재입니다. ①은 사람에게서 알리고, ②는 기계가 읽고, ③은 ①②가 다 벗겨진 뒤에 남습니다. 하나만 쓰면 그 하나의 실패 방식에 그대로 노출됩니다.

② — 출처 기록의 표준

②를 각자 자기 방식으로 붙이면 읽는 쪽이 해석할 수 없습니다. 그래서 표준이 생겼습니다.

C2PA(Coalition for Content Provenance and Authenticity)는 **언제 · 무엇으로 · 어떤 편집**을 거쳐 만들어졌는지를 **서명된 형태로** 파일에 실어 보내는 규격입니다. 사용자에게는 **콘텐츠 자격증명(Content Credentials)** 이라는 이름으로 보입니다. 일부 카메라는 촬영 시점에, 편집·생성 도구는 처리 시점에 이 기록을 이어 붙입니다.

핵심은 "이건 AI 다"가 아니라 "이력이 여기 있다" 입니다. 진짜 사진에도 붙고, 편집을 거치면 그 편집도 기록됩니다.

한계도 분명합니다. **재인코딩하면 사라집니다**. 스크린샷을 찍거나 SNS에 올리는 순간 없어질 수 있습니다. 그래서 ③이 필요합니다.

③ — 메타데이터가 아니라 신호를 심는 방식

비가시 워터마크는 **파일에 붙이는 것이 아니라 내용 자체를 아주 조금 바꿉니다**.

원리는 **주파수 영역에 숨긴다** 입니다. 픽셀 값을 그대로 두지 않고 **DWT**(이산 웨이블릿 변환)·**DCT**(이산 코사인 변환)로 주파수 형태로 바꾸면, 사람 눈이 둔감한 영역에 신호를 엮을 수 있습니다. JPEG 압축이 사람 눈에 덜 중요한 주파수를 버리는 것과 같은 자리를 반대로 쓰는 셈입니다.

SynthID 계열 은 여기서 한 걸음 더 갑니다 — 다 만든 결과물에 신호를 엮는 것이 아니라 **생성 과정 자체에 신호를 통합** 합니다. 모달리티마다 심는 자리가 다릅니다.

표 G.3 모달리티별로 신호를 심는 자리

모달리티	신호를 심는 자리
이미지	픽셀
오디오	스펙트로그램
텍스트	토큰 선택 확률 패턴

텍스트 항목이 특히 흥미롭습니다. 글자를 바꾸는 것이 아니라 *어느 토큰을 고를지의 분포*를 미세하게 편향시켜 심습니다.

워터마크의 세 축 — 셋은 서로 맞바꿉니다

워터마크를 평가하는 축은 셋이고, 세 축은 동시에 좋아지지 않습니다.

표 G.4 워터마크의 세 축

축	뜻	올리면
비가시성 (imperceptibility)	눈·귀에 안 띄는가	강건성이 내려간다
강건성 (robustness)	압축·크롭·재촬영을 견디는가	화질 영향이 커진다
용량 (capacity)	몇 비트를 담는가	나머지 둘이 내려간다

그래서 실무의 답은 **용량을 포기하는 것**입니다. 이력 전체를 결과물 안에 심으려 하지 마세요. **짧은 식별자 하나만** 심고 나머지는 생성 로그 (§ 3)에서 조회하면 됩니다. 몇 비트면 충분합니다.

워터마크는 저장소가 아니라 열쇠입니다.

개인정보를 '유출' 로만 보지 마세요

강의에서 자주 나오는 반문이 있습니다 — “공개된 사진인데 뭐가 문제죠?”

맥락 무결성(contextual integrity)이라는 개념이 여기에 답합니다. 정보는 *원래 있던 맥락* 안에서 흐를 때 정상이고, **맥락을 옮기는 것 자체가 침해**일 수 있습니다. 공개 프로필 사진은 그 *프로필에서 누구인지 알아보라고* 올린 것입니다. 그 얼굴이 말을 하게 만들어 다른 곳에 놓이는 것은 원래 맥락이 아닙니다.

“공개되어 있었다”는 동의가 아닙니다. **어디에 공개되었고 무엇을 위한 것이었나**를 물으세요.

표기는 곧 법적 의무가 됩니다

여러 나라가 AI 생성물 표기를 **의무화하는 방향**으로 가고 있습니다. 그것을 기술적으로 뒷받침하는 것이 위의 ②층(C2PA)입니다.

지금 당장 의무가 아닌 지역에서도 **처음부터 습관을 들이는 편이 낫습니다**. 나중에 의무가 생겼을 때 이미 만들어 배포한 결과물에 소급해서 표기를 넣는 것은 불가능에 가깝습니다.

§ 3의 생성 로그를 남기고 있다면 절반은 이미 한 셈입니다. **남은 것은 그 정보를 결과물 파일에 함께 실어 보내는 것**입니다.

제거를 요청받으면 **그 자체가 위험 신호**입니다. 정당한 용도에서 표시를 지울 이유는 없습니다.

5. 실시간 대화 아바타의 추가 요건

AI 고지 — 첫 응답 또는 화면에 명시. 사용자가 물으면 반드시 인정하도록 프롬프트에 강제.

위기 상황 필터 — 자해·위험 암시 감지 시 대화를 이어가지 않고 즉시 도움 안내로 전환. **코드로 강제** 하세요. 프롬프트 지시만으로는 부족합니다.

- 1) 사용자 입력을 LLM 호출 *전에* 패턴·분류로 검사
- 2) 해당하면 LLM을 부르지 않고 고정 응답 반환
- 3) 지역 상담 연락처 안내
- 4) 로그 남김 (개인정보 최소화)

전문 영역 거절 — 의료·법률·금융 조언은 하지 않고 전문가를 안내. 캐릭터의 말투로 거절하는 문구를 미리 정의(Ch22).

애착 대응 — 장기 사용자에게 대한 정책, 서비스 종료 시 안내 절차를 미리 정하세요.

미성년자 — 접근 가능하다면 콘텐츠 기준과 데이터 처리 기준이 달라집니다.

음성 클로닝 서버가 코드로 지켜야 할 셋

Ch03 §3의 운영 규칙 중 이 부록의 몫입니다. 문서가 아니라 **코드**가 지킵니다.

표 G.5 음성 클로닝 서버가 코드로 지켜야 할 셋

요건	구현	빠지면
동의 없는 클로닝 거절	요청에 <code>consent=true</code> 가 없으면 처리하지 않는다	동의서(§1)가 있어도 API로 우회된다
참조 음성 없는 호출은 기본 목소리로	화자 조건 캐시를 호출마다 초기화한다	직전 사용자의 목소리가 다음 사용자에게 새어 나간다
워터마크가 실제로 박히는지 확인	워터마크 패키지가 깨지면 조용히 무표시로 떨어진다 — 워커를 세울 때 출력 하나를 검출기에 통과시킨다	§4의 ③층이 없는 채로 배포된다

6. 팀 운영 — 거절 기준 문서화

판단이 사람마다 다르면 **가장 느슨한 기준이 실제 기준** 이 됩니다. 팀이 있다면 문서로 고정하세요.

1. 즉시 거절
 - 동의 없는 실존 인물
 - 성적·모욕적·협박적 내용
 - 신원 위조 목적
 - 표시 제거 요청

2. 검토 필요 (2인 이상 판단)

- 고인
- 공인
- 정치·선거 관련
- 실제 사건 재현

3. 진행 가능

- 동의 완료 + 표시 + 용도 명확

거절 사례도 기록하세요. 무엇을 거절했는지 조직의 기준을 만듭니다.

7. 판별 단서 (배운 사람의 몫)

이 책에서 배운 실패가 그대로 판별 단서입니다. Ch29 § 6.

표 G.6 판별 단서와 그 근거 장

단서	근거 장
입 주변만 화질이 다르다	Ch10 (저해상도 재생성)
입술·치아가 얼굴 톤과 어긋난다	Ch13 (분포 밖 생성)
눈을 오래 깜빡이지 않는다	Ch10·Ch12 (입만 전사)
뒤로 갈수록 입이 소리보다 늦다	Ch14 (fps 드리프트)
손·몸이 전혀 움직이지 않는다	Ch04 (얼굴·머리만 애니메이션)
목소리의 호흡·잡음이 지나치게 깨끗하다	Ch03

주변 사람에게 설명할 수 있게 되세요. 막연한 불안을 구체적인 판단 기준으로 바꾸는 것이 이 기술을 배운 사람의 몫입니다.

관련 — Ch29(책임) · Ch04 § 4 · Ch15 § 9 · 부록 B(드라이버 출처 기록)

부록 H — Windows 트러블슈팅

네 가지가 Windows 환경 문제의 대부분입니다. 한글 경로 · cp949 · fps 타입 · Power Shell 파싱.

빨간 글씨는 났는데 코드는 멀쩡해 보일 때, 그리고 리눅스에서는 잘 도는 스크립트가 Windows에서만 죽을 때 퍼세요. 여덟 절이 전부 증상 → 원인 → 해법 순서라, 겪고 있는 증상만 찾아 그대로 복사해 쓰시면 됩니다. 나가기 전에 §8의 예방 체크리스트를 파이프라인에 심어 두세요. 그것까지 하면 이 부록을 다시 펼 일이 거의 없습니다.

1. 한글 파일명 — 가장 흔한 사고

증상

이미지 로드가 실패합니다. 예외가 나기도 하고, 조용히 빈 결과가 나오기도 합니다. 후자가 훨씬 위험합니다.

원인

OpenCV의 파일 읽기 함수가 Windows에서는 시스템 기본 인코딩(cp949)으로 경로를 처리합니다. 그래서 한글이 든 경로가 깨집니다.

해법 — 파이프라인이 자동으로 처리하게

사용자에게 "한글 파일명 쓰지 마세요"라고 말하는 것은 해법이 아닙니다. 코드가 알아서 처리하게 하세요.

- 1) 입력 파일을 work/에 ASCII 이름으로 복사
예: 하늘이.jpg → work/src_001.jpg
- 2) 전체 파이프라인을 work/ 안에서 실행
- 3) 최종 결과만 원하는 이름으로 이동

중간 산출물도 전부 ASCII로. 한 단계라도 한글이 끼면 그 단계에서 실패합니다.

우회 로드 함수

직접 읽어야 한다면 경로를 라이브러리에 넘기지 마세요. 파이썬이 파일을 열어 바이트로 읽고, 그 바이트를 메모리에서 디코딩하면 됩니다.

2. cp949 인코딩 오류

증상

모델 다운로드 도구나 로깅에서 이모지·특수문자 출력 중 죽습니다.

해법

```
PYTHONUTF8=1
```

프로세스가 뜨기 전에 설정해야 합니다. 시스템 환경변수로 한 번 넣어 두는 편이 편합니다.

파일 입출력에서도

파이썬에서 텍스트 파일을 열 때 **인코딩을 명시** 하세요. Windows에서는 명시하지 않으면 cp949로 열립니다.

```
open(path, encoding="utf-8")
```

이 한 줄을 빠뜨려 한글이 깨지는 사고가 자주 납니다.

PowerShell 출력

PowerShell에서 `Set-Content` / `Add-Content` 는 기본이 시스템 ANSI 코드페이지입니다. 다른 도구가 읽을 파일을 쓸 때는 `-Encoding utf8` 을 명시하세요.

3. fps 타입 문제

증상

`fps must be float` 류의 오류.

원인

모션 데이터나 설정에 fps를 정수형으로 저장했는데, 라이브러리가 실수형을 요구합니다.

해법

저장할 때 실수형으로 명시해서 변환하세요. numpy 배열에서 꺼낸 값을 그대로 저장하면 int32/int64가 딸려 들어갑니다.

관련 — 프레임레이트 값 자체

29.97은 정확히는 30000/1001입니다. 소수로 반올림해 저장하면 긴 영상에서 오차가 쌓입니다. 분수로 다루세요. ffmpeg는 `30000/1001` 표기를 그대로 받습니다.

4. PowerShell 파싱 오류

증상

스크립트가 실행되지 않고 파싱 단계에서 죽습니다.

원인들

주석의 **한글과 괄호**. 파라미터 블록 안 주석에 한글과 괄호가 섞이면 파서가 헛갈립니다. 주석은 **ASCII**로 쓰세요.

&& 와 ||. Windows PowerShell 5.1에는 없습니다. `A; if ($?) { B }` 로 쓰세요.

여기 문자열의 닫는 표시. '@'는 반드시 줄 맨 앞(들여쓰기 없음)에 와야 합니다.

네이티브 실행 파일의 **stderr 리다이렉트**. `2>&1` 을 쓰면 각 줄이 오류 레코드로 감싸지고, 종료 코드가 0인데도 실패로 보입니다. 쓰지 마세요.

5. 파일 복사 관련

증상

"자기 자신을 복사할 수 없음" 오류.

원인

음성 파일이 이미 대상 폴더에 있는데 복사를 시도했습니다.

해법

복사 전에 원본과 대상 경로를 건져 보고 같으면 건너뛰세요. 심볼릭 링크와 상대 경로 때문에 문자열 비교로는 모자랍니다. 절대 경로로 정규화한 뒤 비교하세요.

6. ffmpeg 경로

증상

새 셀에서 ffmpeg를 찾지 못합니다.

해법

셀 PATH에 의존하지 말고 코드 안에서 프로세스 PATH 앞에 주입 하세요.

```
FFBIN = r"C:\...\ffmpeg\bin"
```

```
os.environ["PATH"] = FFBIN + os.pathsep + os.environ["PATH"]
```

경로를 상수 하나로 두고 모든 스크립트가 그것을 보게 하세요. 설치 위치가 바뀌어도 고칠 곳이 한 군데입니다.

7. CUDA 빌드 문제

증상

커스텀 연산자 빌드 시 MSVC와 CUDA 11.8의 버전 충돌.

해법

nvcc에 우회 플래그를 주입합니다. 부록 A § 2의 빌드 스크립트 참조.

```
-allow-unsupported-compiler
-D_ALLOW_COMPILER_AND_STL_VERSION_MISMATCH
```

conda 채널 약관

`conda tos accept` 로 채널별 약관을 수락한 뒤 재시도합니다.

8. 예방 체크리스트

파이프라인을 만들 때 처음부터 넣어 두세요. 이 부록을 다시 볼 일이 줄어듭니다.

- [] 입력을 `work/` 에 ASCII 이름으로 복사하는 단계
- [] `PYTHONUTF8=1` 설정
- [] 모든 `open()` 에 `encoding="utf-8"` 명시
- [] ffmpeg 경로를 코드에서 PATH 주입
- [] fps를 실수형·분수로 관리하는 상수 하나
- [] PowerShell 주석은 ASCII
- [] 복사 전 경로 동일성 검사
- [] 각 단계 후 결과 파일 존재·크기 확인

관련 — 부록 A(환경) · 부록 F(14~17번) · Ch15 § 4

부록 L — 아바타 UI/UX 가이드

말하는 얼굴은 일반 웹 UI의 규칙이 반쯤만 통합니다. 소리가 나고, 기다림이 있고, 사람처럼 보이기 때문 입니다. 이 부록은 그 반쪽을 채웁니다. Ch08(지연 숨기기) · Ch19(생명감) · Ch21(완성) · Ch23(상태 기계)의 실행 지침판.

화면을 처음 그리기 시작할 때, 그리고 다 만들었는데 써 보면 어딘가 어색할 때 펴세요. 일반 웹 UI의 규칙이 통하지 않는 자리만 모았습니다 — 첫 3초, 기다리는 동안의 얼굴, 자막, 여럿이 한 화면에 나올 때, 모바일. 나가실 때는 §9의 열두 줄을 그대로 QA 항목으로 옮겨 쓰시면 됩니다.

1. 첫 3초 — 여기서 대부분이 결정됩니다

소리는 사용자가 켜다

브라우저는 사용자가 화면을 건드리기 전에는 오디오를 재생하지 못하게 막습니다. 정책이고, 우회할 수 없습니다.

나쁜 처리 — 자동재생을 시도했다가 실패하고, 첫 메시지만 소리가 안 납니다. 사용자는 서비스가 고장 났다고 생각합니다.

좋은 처리 — 아바타가 **입 모양으로 인사하면서** 화면 아래에 "소리를 켜고 시작하기" 버튼 하나를 둡니다. 그 클릭 한 번이 오디오 컨텍스트를 깨우고, 그때부터 전부 동작합니다.

클릭을 요구하는 것을 부끄러워하지 마세요. 그것을 자연스러운 시작 동작으로 만드는 것이 UI의 일입니다.

첫 화면에 로딩 바를 두지 마세요

3D 모델은 수십 메가바이트입니다. 그 시간을 진행률 바로 채우면, 사용자는 아바타가 아니라 그 숫자를 쳐다봅니다.

대신 **가벼운 것부터 순서대로 올리세요.** 배경 → 실루엣/정지 이미지 → 모델 → 애니메이션. 볼 것이 계속 바뀌면 같은 시간도 짧게 느껴집니다.

무엇을 하는 곳인지 3초 안에

아바타가 화면에 있으면 사용자는 **말을 걸어도 되는지** 를 모릅니다. 마이크 버튼 하나로는 부족합니다. 아바타가 먼저 한마디 하는 편이 가장 확실합니다.

2. 상태를 얼굴로 말하세요

Ch23의 상태 기계에는 네 상태가 있습니다. 각각의 **시각적 언어**를 정의해 두세요. 텍스트 라벨이 아니라 아바타의 모습으로.

표 L.1 네 상태의 시각적 언어

상태	아바타	화면
대기	깜빡임 · 호흡 · 가끔 시선 이동	입력창 활성화
듣는 중	고개 살짝 기울임 · 시선 고정 · 깜빡임 감소	입력 파형
처리 중	생각하는 표정 · 시선 위로	스피너 금지
말하는 중	립싱크 · 감정 표정 · चेस्처	자막

처리 중이 가장 중요합니다. 여기에 스피너를 넣으면 Ch08의 모든 노력이 무효가 됩니다. 사람은 생각할 때 시선을 돌립니다. 그것을 그대로 하면 됩니다.

상태 전환은 100~200ms로 보간하세요. 하드 컷은 상태 기계가 있다는 것을 노출합니다.

3. 자막은 선택이 아닙니다

AI 휴먼 서비스에서 자막은 접근성 기능이 아니라 기본 기능입니다.

이유가 넷입니다.

소리를 켤 수 없는 자리가 많습니다. 사무실, 지하철, 강의실. 자막이 없으면 그 사용자는 그냥 나갑니다.

TTS는 가끔 잘못 읽습니다. 숫자, 영어, 고유명사. 자막이 있으면 사용자가 스스로 보정합니다.

청각장애 사용자가 있습니다. 이 한 줄이면 충분한 이유입니다.

신뢰가 올라갑니다. 말과 글자가 일치하는 것을 보면 사용자가 시스템을 더 믿습니다.

자막의 규칙

말과 동시에 나타나야 합니다. 미리 다 띄우면 사용자가 먼저 읽어 버리고 아바타는 쳐다보지 않습니다. 문장 단위로, 재생에 맞춰 띄우세요.

아바타를 가리지 않게. 얼굴 아래, 반투명 배경, 두 줄 이내.

이전 대화를 볼 수 있게. 자막은 흘러가지만 대화 로그는 남아야 합니다. 접었다 펼 수 있는 패널이 무난합니다.

Ch07 §9 주의 — S2S를 쓰면 자막에 넣을 텍스트가 없습니다. 전사를 함께 주는 제공자를 골라야 하는 이유가 하나 더 있는 셈입니다.

4. 기다림을 정직하게

짧은 기다림(2초 이하)은 숨깁니다

Ch08의 아이들 루프·크로스페이드·더블버퍼링. 사용자에게 알리지 않습니다.

긴 기다림(수십 초 이상)은 드러냅니다

Track A의 렌더처럼 몇 분이 걸리는 작업은 **숨기면 안 됩니다**. 여기서는 반대로 최대한 많이 알려주세요.

지금 무슨 단계인지 — "음성 만드는 중 → 입 맞추는 중 → 얼굴에 옮기는 중". 파이프라인의 단계를 그대로 보여주면 됩니다.

얼마나 남았는지 — Ch06의 어림값(영상 1초당 약 21초)을 여기서 씁니다. "약 4분 남았습니다"는 4분을 짧게 만듭니다.

떠나도 되는지 — 완료 알림 경로가 있으면 명시하세요. 사용자를 화면 앞에 묶어두지 마세요.

원칙 — 2초는 숨기고, 2분은 설명합니다. 애매한 중간(5~20초)이 가장 어렵고, 대개 단계 표시가 답입니다.

5. 여럿이 나올 때

Ch26의 멀티 아바타 화면 규칙입니다.

목소리를 확실히 다르게. 화면을 안 보고 있어도 누가 말하는지 알아야 합니다. 성별·톤·속도를 다르게 배정하세요.

듣는 캐릭터가 말하는 캐릭터를 바라보게. 강조 방법 중 가장 자연스럽게 비용도 가장 적게 듭니다. 테두리를 빛나게 하는 것보다 낫습니다.

말풍선에 색을 고정하세요. 캐릭터마다 하나씩. 자막에도 같은 색을 씁니다.

동시에 말하지 않게. 텍스트는 병렬로 생성하되 재생은 순차로. 두 목소리가 겹치면 아무것도 안 들립니다.

넷 이상이면 렌더 예산을 확인하세요. 저사양 기기에서 프레임이 떨어집니다. 말하지 않는 캐릭터는 애니메이션 갱신 빈도를 낮추거나, 2D 파츠로 내려가세요.

6. 모바일

오디오 정책이 더 엄격합니다. §1의 시작 버튼이 모바일에서는 필수입니다.

마이크 권한 요청 시점을 고르세요. 앱을 열자마자 물으면 거절률이 높습니다. 사용자가 마이크 버튼을 처음 누를 때 물으세요.

세로 화면을 기본으로. 아바타는 상반신이 세로 화면에 잘 맞습니다. 가로 대응은 나중에.

저사양 기기를 실제로 테스트하세요. 개발 기기에서 60fps라는 것은 아무 정보도 아닙니다. 3D가 버거우면 2D 파츠 폴백을 두세요.

화면 꺼짐을 막을지 정하세요. 대화 중에 화면이 꺼지면 오디오 컨텍스트가 끊깁니다. 다만 배터리를 먹으므로 대화 중에만.

7. 접근성

자막 — §3. 기본 기능입니다.

모션 민감성 — 흔들림과 제스처를 줄이는 옵션을 두세요. 운영체제의 *모션 줄이기* 설정을 존중하면 대부분 자동으로 해결됩니다. Ch19의 미세 움직임 층을 통째로 끄면 됩니다.

속도 조절 — TTS 재생 속도를 사용자가 조절할 수 있게. 느리게 듣고 싶은 사용자와 빠르게 넘기고 싶은 사용자가 둘 다 있습니다.

텍스트 입력 경로를 항상 유지 — 음성이 주 입력이어도 타이핑이 가능해야 합니다. 발화가 어려운 사용자, 시끄러운 환경, 오인식이 잦은 고유명사.

색에만 의존하지 않기 — 캐릭터 구분을 색으로만 하면 색각 이상 사용자가 구분하지 못합니다. 이름을 함께 표시하세요.

8. 이것이 AI라는 표시

Ch29의 요구를 UI로 옮기면 이렇습니다.

첫 화면에 명시. 작게라도 "AI 캐릭터입니다"가 보여야 합니다.

물으면 인정. "사람이세요?"에 대한 응답을 페르소나 프롬프트에 명시하고, 캐릭터의 말투로 인정하게 하세요.

생성물에는 표시. 영상을 내려받게 한다면 화면 표시와 메타데이터 양쪽에.

전문 영역 경계. 의료·법률·금융 질문에서 캐릭터가 선을 긋는 문구를 미리 정해 두고, UI에서도 안내 링크를 함께 노출하세요.

표시를 예쁘게 만드는 것은 괜찮습니다. 작게 만드는 것은 안 됩니다.

9. 만들고 나서 확인할 열두 개

- [] 첫 클릭에서 오디오 컨텍스트가 깨어나는가
 - [] 첫 화면에 로딩 바 대신 볼 것이 있는가
 - [] 네 상태의 시각적 표현이 각각 정의되어 있는가
 - [] 처리 중에 스피너가 없는가
 - [] 자막이 말과 동시에, 아바타를 가리지 않고 나오는가
 - [] 말이 끝나면 입이 확실히 닫히는가
 - [] 연속으로 빠르게 말을 걸면 이전 오디오가 멈추는가
 - [] 30초간 아무것도 안 해도 살아 있어 보이는가
 - [] 긴 작업에 단계와 남은 시간이 표시되는가
 - [] 저사양 기기에서 프레임이 유지되는가
 - [] 모션 줄이기 설정이 반영되는가
 - [] AI라는 표시가 첫 화면에 있는가
-

관련 — Ch08(지연 숨기기) · Ch19(생명감) · Ch21(완성) · Ch23(상태 기계) · Ch26(멀티)
· Ch28A(운영 콘솔) · Ch29(책임)

부록 N — 원가 모형

이 부록의 가격은 전부 예시입니다. GPU 시급도 API 단가도 6개월이면 바뀝니다. 여기 있는 것은 구조 이고, 숫자는 계산기(`code/appN_cost/`)에 넣습니다. 온라인 부록 K와 같은 원칙.

견적서를 써야 할 때, 그리고 "직접 만들까 API를 쓸까"를 정해야 할 때 펴세요. 트랙마다 돈이 어디로 나가는지의 구조와, 단가만 넣으면 답이 나오는 식이 있습니다. 여기서 챙길 것은 셋입니다 — 영상 1초당 GPU 21초(§ 2), 손익분기 건수 식(§ 3), 그리고 동시 접속 정원이 곧 GPU 대수라는 사실(§ 5).

1. 세 트랙의 원가 구조는 종류가 다릅니다

같은 "AI 휴먼" 인데 돈이 나가는 방식이 근본적으로 다릅니다. 셋을 섞어 놓고 견적을 내면 반드시 틀립니다.

표 N.1 세 트랙의 원가 구조

	비용이 무엇에 비례하나	성격
Track A (품질)	영상 길이 × 편수	가변비 — GPU 시간을 산다
Track B (실시간)	사용자 수 × 대화량	가변비 — 텍스트·오디오만 산다
실사 실시간	동시 접속 정원	고정비 — GPU 대수가 곧 정원

세 번째가 다른 둘과 성격이 다릅니다. 사용자가 없어도 GPU는 켜져 있어야 합니다.

2. Track A — 영상 1초당 21초

Ch06의 실측이 곧 원가 공식입니다.

$$\begin{aligned} \text{GPU 시간} &= \text{영상 길이} \times 21 && (\text{최적화 상한 3배 적용 시} \times 7) \\ \text{원가} &= \text{GPU 시간} \div 3600 \times \text{시급} \end{aligned}$$

1분짜리 영상 하나에 GPU 21분 입니다. Ch06의 최적화(전처리 캐시·배치·반정밀도·fps)를 적용하면 7분까지 내려가고, 그 이상은 이 트랙의 성격상 어렵습니다.

견적을 낼 때 이 배수를 먼저 곱하세요. "10분짜리 영상 100편"은 GPU 350시간입니다.

3. 손익분기점 — 언제 직접 만드나

이것이 이 부록에서 가장 자주 쓸 계산입니다.

API 대행 은 건당 단가만 냅니다. 고정비가 0입니다.

직접 구축 은 건당 원가가 싸지만 **고정비가 먼저 들어갑니다** — 엔지니어 시간, 환경 유지보수, 상시 인스턴스.

$$\text{손익분기 건수} = \text{월 고정비} \div (\text{API 단가} - \text{직접 건당 원가})$$

이 식에서 결론 둘이 나옵니다.

첫째, 분모가 0 이하면 계산할 필요가 없습니다. 직접 만든 건당 원가가 API 단가보다 비싸면, 아무리 물량이 많아도 직접이 이기지 못합니다. **그냥 API를 쓰세요.**

둘째, 손익분기 건수에 현실적으로 도달할 수 없으면 직접 구축은 손해입니다. 계산기 기본값으로 돌리면 월 1만 건이 넘게 나옵니다. 월 100건을 만드는 서비스가 직접 구축을 하면 **건당 원가가 아니라 고정비로 망합니다.**

1인 개발자와 검증 단계에서는 API가 거의 항상 맞습니다. Ch06이 "품질 트랙은 배치로 쓰라" 고 한 것과 같은 결론에 다른 경로로 도착합니다.

단, **여기 안 들어간 항목이 있습니다** — 프라이버시. 얼굴과 목소리를 외부 API로 보내는 것은 Ch29의 문제입니다. 원가가 싸다고 보낼 수 있는 것은 아닙니다. 민감한 입력이면 손익분기점과 무관하게 직접 처리합니다.

4. Track B — 여기가 싼 이유

렌더가 **사용자의 브라우저** 에서 일어납니다(Ch05). 서버가 만드는 것은 텍스트와 오디오뿐입니다.

$$\text{월 원가} = \text{MAU} \times \text{사용자당 턴수} \times (\text{턴당 LLM} + \text{턴당 TTS}) + \text{서버 고정비}$$

식에 **GPU 향이 없습니다.** 사용자가 백 명이면 백 대의 브라우저가 각자 렌더합니다. 서버는 노트북 한 대로 버팁니다.

이것이 Ch05 결정표의 "동시 사용자: 브라우저마다"가 뜻하는 바입니다. 원가가 **사용자 수에 선형**이고 **기울기가 작습니다.**

절감 지점은 Ch07과 겹칩니다 — **짧게 대답하게 만들면 토큰이 줄고, 토큰이 줄면 지연과 원가가 동시에 내려갑니다.** 품질·속도·비용이 같은 방향인 드문 경우입니다.

5. 실사 실시간 — 정원에 묶입니다

계산기를 돌리면 이 줄이 나옵니다.

[실사 실시간] 동시 50명 → 필요 GPU 50장

이것이 Ch05 결정표 "동시 사용자: GPU 대수에 묶임"의 실제 의미입니다. 저자 환경 기준 GPU 한 장이 동시 사용자 한 명 수준을 감당합니다.

스트리밍 우선 모델(온라인 부록 K §4)을 쓰면 이 비율이 올라갑니다. **그래도 무한하지 않습니다.** 판단 기준이 *가능한가*에서 *감당 가능한가*로 옮겨갔다는 Ch05 §2의 문장이 여기서 숫자가 됩니다.

서비스를 설계하기 전에 이 계산부터 하세요. 목표 동시 접속자를 GPU 대수로 바꾸고, 그 대수의 월 비용을 감당할 수 있는지 봅니다. 안 되면 답은 Track B입니다. 사실감을 포기하는 것이 아니라 원가 구조를 고르는 것입니다.

6. 원가를 구조로 줄인 실례 — Ch03A

사투리 모델 다섯을 지역별로 따로 배포하면 15.0GB가 필요합니다 — 12GB 카드 한 장에는 세 개가 한계입니다.

어댑터 핫스왑으로 바꾸면 3.2GB 입니다(생성 피크 실측). **한 장에 팔도 전부.**

개별 배포 : $5 \times 3.0\text{GB} = 15.0\text{GB}$ → 카드 여러 장
 핫스왑 : $3.0\text{GB} + (5 \times 22.5\text{MB}) \approx 3.1\text{GB}$ (피크 3.2GB) → 카드 한 장

최적화로 2~3배를 얻는 동안, 구조 변경으로 5배를 얻었습니다. Ch06 §5의 "*최적화가 아니라 아키텍처 교체*"가 원가에서도 그대로 성립합니다.

원가 문제를 만나면 파라미터를 조이기 전에 구조를 의심하세요.

7. 견적에서 빠뜨리는 것들

전처리 비용. 얼굴이 매번 바뀌는 서비스라면 전처리가 사용자마다 발생합니다(Ch11 §4). 고정 캐릭터 서비스의 원가로 견적을 내면 틀립니다.

재시도. 검증 게이트(Ch09)에 걸리면 다시 만들어야 합니다. 실패율을 견적에 넣으세요.

저장·전송. 생성된 오디오·영상이 쌓입니다(Ch28 §2). 정리 작업이 없으면 디스크가 곧 비용이 됩니다.

유휴 시간. 실사 실시간의 GPU는 새벽에도 켜져 있습니다. **가동률이 낮을수록 실효 단가가 올라 갑니다.**

사람 시간. 손익분기 식의 고정비 대부분이 여기입니다. 환경 구축 반나절(Ch02), 드라이버 확보(부록 B), 실패 카탈로그를 밟는 시간(부록 F). **이 책이 줄이려는 것이 정확히 그 항목입니다.**

8. 계산기

```
cd code/appN_cost
python cost_model.py --gpu-hourly 1200 --api-unit 800 --fixed-monthly 3000000 \
    --video-sec 60 --mau 5000 --concurrent 100
```

전부 인자입니다. **현재 단가를 넣고 돌리세요.** 코드에 박힌 것은 실측 상수(21배, 최적화 3배)뿐이고, 그 둘은 부록 C에 근거가 있습니다.

9. 단가 표기의 단일 소스

시간당 단가와 초당 단가를 따로 적지 마세요. 저자의 플랫폼에서 표에는 시간당 260원, 코드에는 초당 5포인트(1포인트 = 1원)가 적혀 있어 실과금이 표시가의 **70배**로 찍힌 사고가 있었습니다(Ch28 §6). 이후 규칙은 하나입니다 — **초당 단가 = 시간당 단가 ÷ 3600**. 계산으로만 존재하게 두고 어디에도 따로 적지 마세요. 이 부록의 계산기도 시간당 단가 하나만 입력받습니다.

관련 — Ch05(사다리 결정) · Ch06(왜 4분) · Ch11 §4~5 · Ch28 §6(분산 GPU) · Ch29(프라이버시) · 부록 C(실측) · Ch03A(핫스왑)

온라인 부록

아래 셋은 기준일이 있거나 시리즈 독자용이라 종이에 굳히지 않고 저장소에서 갱신합니다.
github.com/leelang7/talking-ai-human-book 의 `draft/online/` 에서 읽을 수 있습니다.

- 온라인 부록 I — Vol.02 → Vol.03 사상 연결
- 온라인 부록 J — Vol.04 *우리집 휴머노이드* 예고
- 온라인 부록 K — 모델 지형도 (기준일 2026-08)

찾아보기

가		상태 기계	18, 83, 184, 197, 198
감정 태그	17, 83, 137, 167, 176	생성 로그	252, 255, 256, 257, 258
골든셋	9, 18, 184, 204, 211	수어	4, 5, 18, 24, 25
그림자	52, 155	스티칭	1, 52, 60, 110, 115
기억(3층)	1, 3, 5, 9, 16	실패 카탈로그	8, 19, 95, 102, 124
끼어들기	3, 9, 18, 81, 83	싱크	2, 3, 4, 6, 8
나		아	
나이퀴스트	41	아이들 루프	9, 13, 16, 68, 82
다		어댑터	16, 20, 34, 38, 42
더블 버퍼	16, 87, 88, 89, 182	얼굴 검출	54, 57, 71, 97, 98
동의서	19, 231, 238, 239, 256	온도	18, 186, 189, 191, 215
드라이버	1, 2, 8, 10, 11	워터마크	37, 255, 256, 258, 301
딥페이크	3, 4, 18, 231, 254	원가	13, 15, 19, 26, 42
라		음성 클로닝	3, 31, 34, 36, 38
라이선스	10, 38, 42, 48, 49	인페인팅	25, 96, 103, 107, 139
리깅	8, 62, 65, 66, 137	자	
리타게팅	8, 10, 25, 28, 29	자막	24, 83, 198, 200, 201
립싱크	2, 3, 4, 6, 8	잡 쿼	18, 70, 236, 237, 243
마		재시도	46, 222, 223, 229, 236
멀티 아바타	3, 4, 221, 222, 240	정규화	35, 39, 40, 41, 78
메타발언	186, 187, 188	제스처	15, 17, 25, 52, 61
모델 카드	49	지문자	200, 202, 205
미디엄샷	2, 54, 55, 58, 60	지연 예산	16, 18, 25, 64, 68
바		차	
바운딩	155, 157	채점기	187, 225, 226, 230, 278
발화 종료 판정	9, 74, 197, 198, 200	청킹	16, 18, 68, 76, 84
번역	18, 37, 165, 199, 200	초당 과금	37, 38, 181
벤치마크	19, 41, 67, 73, 84	추모	5, 17, 23, 24, 32
보관 기간	256, 258	카	
분산	42, 48, 50, 238, 242	컨테이너	18, 99, 100, 102, 107
블렌드셰이프	8, 62, 137, 142, 153	컨텍스트	18, 147, 149, 151, 182
사		크로스페이드	16, 68, 87, 89, 312
사투리	16, 20, 22, 34, 42	클로즈업	54, 55, 61, 101, 105
샤크다운	46, 47, 49	타	
		태그	4, 17, 25, 35, 39

텍스트 정규화	39, 178, 223, 274	핫스왑	16, 20, 44, 45, 48
토큰 예산	207, 278	회귀 게이트	18, 184, 226, 230, 260
통역	4, 5, 18, 24, 184	휴식 포즈	156, 157, 159
트랙 A	4, 17, 96, 130	A~Z	
트랙 B	17, 137, 176	C2PA	255, 302, 303
트랙 C	18, 184, 224	fps	1, 8, 17, 25, 55
파		LoRA	20, 22, 34, 42, 43
파인튜닝	18, 33, 34, 36, 42	p95	9, 80, 83, 84, 179
파트	1, 3, 8, 17, 20	RAG	3, 7, 18, 22, 24
페르소나	3, 7, 18, 24, 25	STT	3, 8, 18, 24, 74
폴백	16, 18, 30, 48, 79	SynthID	302
표정	8, 11, 31, 40, 51	TTFA	9, 68, 74, 83, 84
프롬프트	3, 7, 18, 32, 39	TTS	3, 6, 7, 8, 9
하		VAD	9, 74, 184, 193, 194
하이브리드 검색	184, 216	VRM	1, 3, 6, 7, 8
한글 경로	28, 106, 131, 136, 266	WebRTC	195

제목	AI 휴먼 해부학 — 얼굴·목소리·두뇌·기억, 네 층을 조립하고 실측하는 법
시리즈	All That AI · Vol.03
초판 1쇄 발행	2026년 9월
지은이	이석창 (Seokchang Lee)
펴낸이	한건희
펴낸곳	주식회사 부크크
출판사등록	2014.07.15 (제2014-16호)
주소	서울특별시 금천구 가산디지털1로 119 SK트윈타워 A동 305-7호
전화	1670-8316
이메일	info@bookk.co.kr
홈페이지	www.bookk.co.kr
컴패니언 저장소	github.com/leelang7/talking-ai-human-book
측정 환경	RTX 4070 SUPER 12GB · Windows 11 · 본문 수치는 부록 C의 측정 조건 기준

© 이석창 2026. 본 책은 저작자의 지적 재산이므로 무단 전재와 복제를 금합니다. 본문의 코드는 저장소의 라이선스를, 인용된 외부 모델·라이브러리는 각자의 라이선스를 따릅니다.